Malware Analysis and Incident Forensics

Study book for the course by Daniele Cono D'Elia and Leonardo Querzoni

Sapienza University of Rome, A.Y. 2026/27

How to read this book

This book organises the substance of the course by topic and prerequisites. Repeated explanations are merged in one main place. It can be read from start to end without opening the slides. Diagrams and boxes are additional ways into the same explanations. The written text always keeps the full reasoning.

The reading path is recognisable and flexible.

1. Read the opening of the chapter and its **Chapter structure** diagram: they state the goal, the prerequisites and the order of the topics. Each chapter closes with a **Concept map** that shows how the ideas connect.
2. Study one section at a time. Tables and diagrams appear when they clarify a comparison, a relation or a procedure.
3. Use the summary or the points to remember to rebuild the thread of the chapter.
4. When there are review questions, try to answer without looking. Then check the answers and follow the references.

These signals say where a piece of information comes from.

> **Correction**
>
> An error or inconsistency in the material. The original text stays reported, next to the correction and the reason.

> **To verify**
>
> A point that the course sources do not settle. Check it before relying on it at the exam.

> **From the book**
>
> Information that the notes attribute to the textbook Practical Malware Analysis (PMA). Page numbers are carried over as cited in the notes.

> **From the lecture (date and time in the recording)**
>
> Content from a lecture recording rather than from the slides, such as a demonstration in IDA.

> **From the lab (date and time in the recording)**
>
> The same, for the recording of a lab.

> **Simone's note (original wording)**
>
> A comment that Simone wrote during the lessons. The wording is the original one, typos included. It is not checked material: where it needs a remark, a Correction or To verify box stands next to it. Only the Obsidian markup, such as the double brackets of links and the image embeds, was removed.

Source: class notes only (no slide)

This line marks a part reconstructed from the class notes, with no slide behind it.

[**Addition, not from the course material:** this marks general knowledge added by this book, not coming from the course material.]

> **In plain words (explanation added by this book)**
>
> An explanation added by this book after a difficult passage. It is not a sentence of the lecturer and does not replace the main text.

At the end of the volume you find review cards, a glossary and an analytic index. The PDF has bookmarks and selectable text. Text extraction was spot-checked; this is not a PDF/UA certification.

Contents

Map of the course **1**

1 Why malware analysis: course, cases and the attack model **4**

1.1 The course in practice 5

1.2 Disclaimer: dual use and risk 8

1.3 What are we talking about? Five cases 9

1.4 Relevant for the job market? 13

1.5 Anatomy of an attack: the kill chain 13

2 The analyst's method: goals, safety and expected behaviours **19**

2.1 The goal of malware analysis 20

2.2 Warning: handle samples safely 23

2.3 Static versus dynamic analysis 24

2.4 General rules for malware analysis 26

2.5 Expected behaviours 27

3 The x86 machine: CPU, registers and process memory **34**

3.1 Why read binary code 36

3.2 ISA and x86 38

3.3 Word size and endianness 39

3.4 Registers 43
3.5 The instruction cycle 47
3.6 Memory map of a Windows process 48
3.7 Memory protection 52
3.8 Memory addressing 52

4 Reading x86 code: instructions, functions and tricks 57

4.1 Basic x86 instructions 58
4.2 Program flow control 63
4.3 Intel versus AT&T syntax 66
4.4 Functions and stack frames 66
4.5 Calling conventions 74
4.6 LEA 83
4.7 REP and string instructions 85
4.8 The loop instruction 90
4.9 Anti-disassembly 91

5 Reading real code in IDA 96

5.1 Gearing up your machine 97
5.2 Using IDA 98
5.3 CONSTRUCTS.EXE 102
5.4 CALLS\CDECL.EXE 109

6 Basic static analysis 118

6.1 An obvious first step: antivirus scanning 121
6.2 Hashing samples 122

6.3 Strings 124
6.4 Obfuscation through packing 126
6.5 File execution in Windows and the PE format 127
6.6 Clues in libraries: imports 138

7 Basic dynamic analysis: foundations 146

7.1 What dynamic analysis is 147
7.2 Approaches to behavioural analysis 149
7.3 Limits of dynamic analysis 151
7.4 Instrumentation levels 151
7.5 What to observe: system calls and Windows APIs 153
7.6 What an analysis report contains 153
7.7 Sandboxes 154
7.8 Malware classification 154
7.9 Where to run: real and virtual machines 156

8 Basic dynamic analysis: tools, registry and persistence 162

8.1 Launching DLLs 163
8.2 Process Monitor 164
8.3 Process Explorer 166
8.4 Detecting malicious documents 169
8.5 The Windows Registry 170
8.6 Regshot 171
8.7 Persistence on Windows 173
8.8 Network analysis 175
8.9 Using the tools together 181

9 Windows from the analyst's point of view 186

9.1 From disassembly to malware behaviour . . . 187

9.2 The privilege boundary . . . 189

9.3 The Native API . . . 191

9.4 Windows API (Win32) . . . 193

9.5 Windows registry functions . . . 197

9.6 Network API . . . 203

9.7 Execution mechanisms . . . 205

10 Recognising C constructs in assembly 219

10.1 Global and local variables . . . 220

10.2 Switch . . . 222

10.3 Arrays . . . 228

10.4 Structs . . . 229

11 Advanced dynamic analysis: the debugger 234

11.1 Why we need a debugger . . . 235

11.2 What a debugger offers . . . 236

11.3 Following instructions . . . 237

11.4 Breakpoints . . . 239

11.5 How breakpoints are implemented . . . 239

11.6 Exceptions . . . 244

11.7 Debugging and ASLR . . . 247

11.8 Popular debuggers . . . 249

11.9 The debugger of IDA . . . 250

11.10 Common IDA shortcuts . . . 260

12 Malware evasion and anti-debugging 263

12.1 Why evasion matters . . . 265

12.2 Discrepancies . . . 266

12.3 Virtualization artifacts . . . 267

12.4 Hardware characteristics . . . 273

12.5 Windows installation . . . 277

12.6 Applications . . . 281

12.7 User artifacts . . . 284

12.8 Timing attacks . . . 284

12.9 What could one monitor? . . . 286

12.10 In-guest agents . . . 290

12.11 Virtual machine introspection . . . 290

12.12 Anti-debugging . . . 296

13 Hands-on exercises and Lab 03 305

13.1 Assembly exercises . . . 307

13.2 Lab 03: method . . . 309

13.3 LAB_03-1.EXE . . . 311

13.4 LAB_03-2.EXE . . . 340

13.5 LAB_03-3.EXE . . . 343

13.6 LAB_03-4.EXE . . . 348

13.7 Technical references for the lab . . . 352

14 Labs 04 and 05: reversing and debugging 356

14.1 Lab 04: the logic bomb . . . 358

14.2 Lab 05: debugging . . . 370

Review sheets **383**

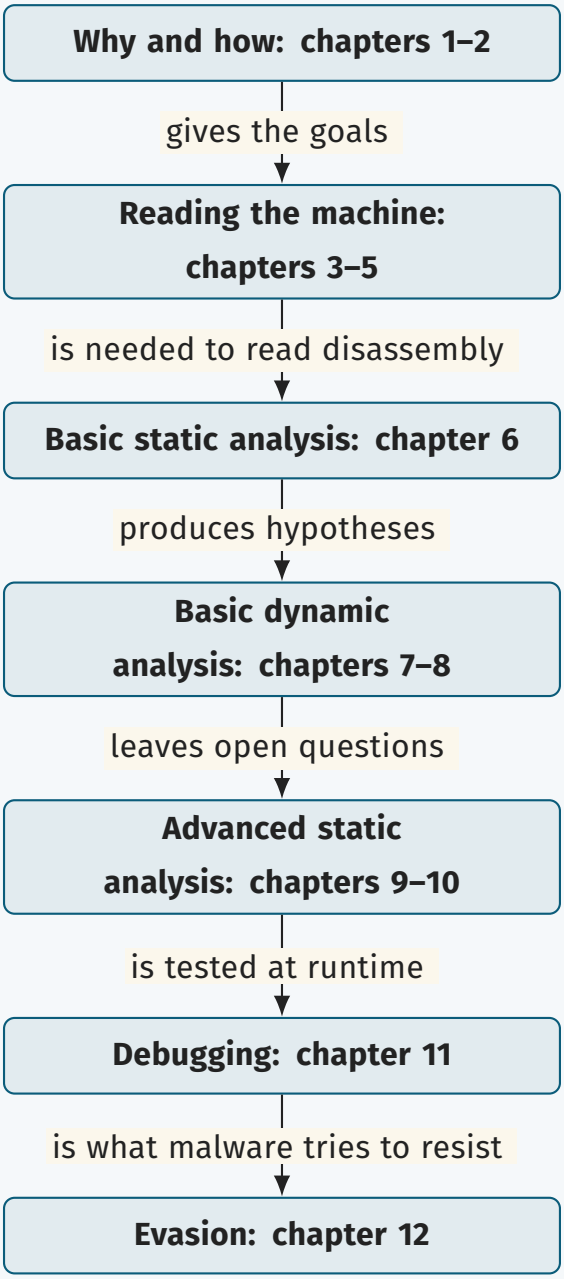
Glossary **387**

Analytic index **390**

Map of the course

The book follows the order of prerequisites, not the order of the lessons. The diagram shows how the parts depend on each other.

Diagram: The course in one page



Read the diagram from top to bottom. The first chapters explain why we analyse malware and how to do it safely. The x86 chapters give the language needed to read code, and the IDA chapter shows it on real programs. Basic static and dynamic analysis then produce and test hypotheses without reading much code. When they leave questions open, advanced static analysis and the debugger answer them. Evasion comes last because it attacks the tools described before it.

Chapters 13 and 14 are the hands-on part. You can use them as soon as their

prerequisites are clear.

Chapter	Topics
1	The course, the exam, real cases, the kill chain
2	Goals of analysis, safety, static and dynamic, expected behaviours
3	ISA, registers, instruction cycle, process memory, addressing
4	Instructions, flags, stack frames, calling conventions, anti-disassembly
5	Using IDA, control structures and function calls in real binaries
6	Antivirus scanning, hashes, strings, packing, the PE format, imports
7	Approaches, limits, instrumentation, sandboxes, machines
8	Process Monitor, Process Explorer, registry, persistence, network tools
9	Native API, Win32, files, registry, network, processes, threads, services, COM
10	Variables, switch, arrays and structs in assembly
11	Stepping, breakpoints, exceptions, the IDA debugger
12	Discrepancies, artifacts, timing, monitoring, anti-debugging
13	Assembly exercises and Lab 03
14	Lab 04 (logic bomb) and Lab 05 (debugging)

Chapter 1

Why malware analysis: course, cases and the attack model

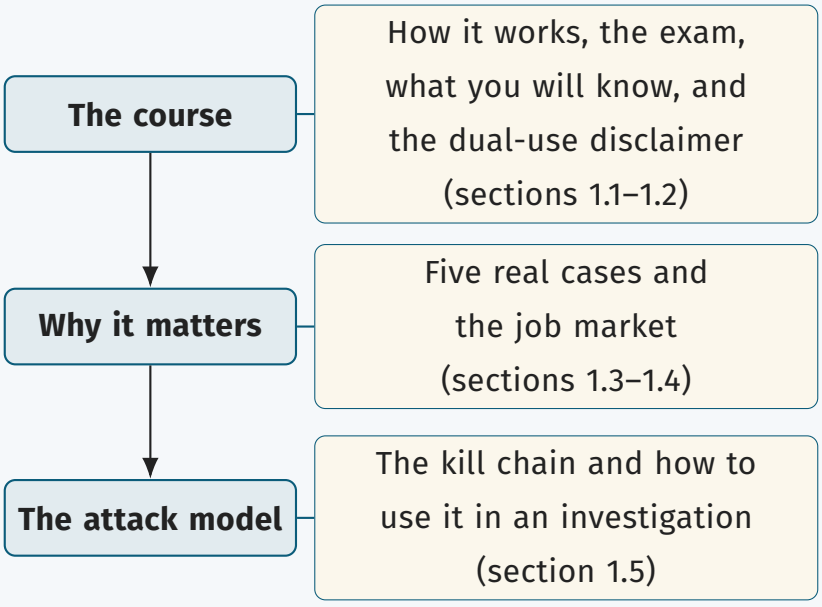
Before you start

Central question: why do we study malicious programs, and where does one sample fit inside a real attack?

Prerequisites: none.

Where it leads: the expected behaviours and the method of chapter 2, then all the technical chapters.

Chapter structure: what comes, and in which order



The chapter has three parts, read from top to bottom. It first explains how the course works. It then shows why the topic matters. It ends with the model that places a sample inside an attack.

The course connects three things: what an attacker is trying to accomplish, what a malicious program actually does on a host, and what evidence an analyst can collect to explain an incident. The historical cases of this chapter motivate the subject. The attack model gives the wider context. The emphasis is on Windows malware, although the analytical questions apply more broadly.

> **Simone's note (original wording)**
>
> back in the old days, Malware previously was something fun to create to play with collegues, now is is used to attack big coorporations. we will see sattic and dynamic tools, like sanbox services.

1.1 The course in practice

1.1.1 Courses, credits and term

There are two courses with shared teaching.

- **Malware Analysis and Incident Forensics (MAIF):** 9 CFU, made of the 6-CFU Malware Analysis component and a further 3-CFU incident-forensics component.
- **Malware Analysis (MA):** 6 CFU. Students of both courses share the malware-analysis teaching. The additional MAIF component has its own activities.

A CFU is a university credit, not a count of lectures. The 6-CFU material is the shared foundation: examining a program and its behaviour. The extra 3 CFU in MAIF extend the perspective from a sample to an incident, including threat intelligence and indicators of compromise.

The course runs in the first semester of the academic year 2026/2027. The dates given in the introduction are **25 September to 22 December**.

Google Classroom holds the course resources. The enrollment link is `https://tinyurl.com/maif26enroll` and the enrollment code is `ylnd7xxb`. If the short link is unavailable, the slides also give the direct address `https://classroom.google.com/c/MTYyNzI0MDYwNDkz?cjc=ylnd7xxb`.

1.1.2 Schedule

Day and time	Who attends	Room
Tuesday, 14:00–16:00	MAIF and MA	Aula 101, Building D, Viale Regina Elena 295
Thursday, 14:00–17:00	MAIF only	Aula 201, Building D, Viale Regina Elena 295
Friday, 14:00–17:00	MAIF and MA	Lab 16, Via Tiburtina 205

The Friday laboratory is especially relevant to hands-on analysis. The table records the introductory material: use the course communication channel for later changes of room or timetable.

1.1.3 Lecturers

Lecturer	Contact	Office hours
Daniele Cono D’Elia	`delia@diag.uniroma1.it`	Online, by email appointment
Leonardo Querzoni	`querzoni@diag.uniroma1.it`	Online, by email appointment

The course diagram associates the shared 6-CFU Malware Analysis portion with D’Elia and the additional 3-CFU MAIF portion with Querzoni.

1.1.4 Reference material

The reference textbook is Michael Sikorski and Andrew Honig, Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software (No Starch Press). In this book it is abbreviated as **PMA**. Classroom also holds teaching slides, scientific papers, malware samples and code, tools, and a virtual machine for hands-on analysis.

The textbook supplies a practical foundation. The papers and samples expose you to techniques and evidence that may not fit a single textbook example. A **virtual machine (VM)** provides a controlled guest environment for examining software: the point is to observe a sample and preserve evidence while limiting what the sample can affect. The precautions are in section 2.2.

1.1.5 Exam format

1. **Practical test:** analyse a supplied sample and answer questions about it. Expect to explain observations and conclusions, not merely to name a tool or identify a file.
2. **Questionnaire, for MAIF students only:** true/false questions. It is mandatory and assessed **pass/fail**. It does **not** determine the numerical grade.

A useful study habit is to record the chain **observation → interpretation → supporting evidence**. For example, the existence of an autostart entry is an observation. Calling it persistence is an interpretation. The entry's target, the responsible process and the behaviour after a reboot are supporting evidence. This is an analytical example of the notes, not an additional exam rule.

> **Simone's note (original wording)**
>
> notice that the professor said that we will have to analyse an malware at the exam in 3 hours, which is a very short time so they will teach us when we have to stop about anolysing a specific way and change way.

1.1.6 Expected knowledge after the course

You should be able to:

- describe the basic characteristics of a cyberattack and place a malicious sample within that wider sequence;
- recognise the basic structure of a malware sample: file format, code, data, imports, strings, resources, and possible hidden or packed content;
- perform basic **static analysis** (examining a sample without running it) and **dynamic analysis** (observing what it does when executed), combining automated tools with manual reverse engineering;
- recognise how **anti-analysis** techniques can make a sample harder to inspect or can change its behaviour in an analysis environment;
- **MAIF only:** explain basic threat-intelligence concepts and extract **indicators of compromise** from incidents to characterise a threat.

An **indicator of compromise (IoC)** is an observable trace that can help identify potentially related activity: a malicious file hash, a persistence entry, a domain, a network destination. An indicator needs context. An address used by a sample at one moment is not automatically a unique identifier for an attacker. The absence of one known indicator does not prove that a system is clean.

Scope. The hands-on focus is Windows malware. The questions “How does it start?”, “How does it survive?”, “What does it change?” and “Where does it communicate?” remain useful on other platforms. The exact artifacts and operating-system mechanisms differ.

> **Simone’s note (original wording)**
>
> Whyb windwos malware? simply because are the most common ones.

1.2 Disclaimer: dual use and risk

The material is **dual use**: techniques learned to recognise malicious activity can also be used to build it. Apply them responsibly and follow the ethical and legal

boundaries of security work. Developing and spreading malware is a crime.

Analysing software from unknown authors can put the analyst and other systems at risk. The course teaches basic protections and states that **the samples assigned for this course are harmless**. That statement is specific to the supplied teaching samples: do not generalise it to arbitrary files found elsewhere. A controlled environment is needed because analysis often involves execution, system changes or network activity, whose effects must be contained and interpreted.

1.3 What are we talking about? Five cases

The introduction uses several incidents to move from the **scale of cyber harm**, to the **availability of attack tools**, to the **organisation of modern cybercrime**. They are historical illustrations. They do not claim that every breach is caused by a malware sample or follows the same path.

When	What
2015	Equifax internal audit reveals issues that are not acted on effectively
August 2016	Shadow Brokers offer an attack tool package at auction
14 April 2017	The package is released publicly
May 2017	WannaCry spreads using EternalBlue
September 2017	Equifax breach
October 2017	Report that all three billion Yahoo! accounts were probably affected
2019	LockBit operation dates from this year

1.3.1 Yahoo!: the scale of an account breach

In October 2017 it was reported that **all three billion Yahoo! accounts** had probably been affected by an earlier breach. The example is about breadth of exposure: a single incident can involve an enormous user population, even if discovery and public understanding evolve after the original compromise.

The analytical lesson is to distinguish **when an intrusion happened**, **when it was detected** and **when its full scope became known**. These can be different dates. Also distinguish account exposure from proof that each account was then misused. Incident forensics tries to establish the timeline and the actual extent of compromise from evidence.

1.3.2 Equifax: vulnerability and organisational response

The slides describe the September 2017 Equifax breach as exposing highly sensitive data of **more than 160 million consumers**. Attackers exploited a **known vulnerability in an unpatched server**. The introduction also notes a 2015 internal audit that had revealed issues on which Equifax did not act effectively.

An exploit is one part of a larger failure chain. A known software weakness, delayed remediation, exposure of a valuable service and the handling of sensitive data all affect the outcome. Malware analysis can explain what code did. Incident forensics asks how access was gained, what systems were touched, what data may have left, and which evidence supports those conclusions. The audit point shows that security is also an operational and governance problem.

1.3.3 Shadow Brokers: advanced capabilities can spread

In August 2016 a group calling itself **Shadow Brokers** offered a software package at auction. It claimed the package contained advanced penetration tools associated with the **Equation Group**, allegedly linked to the NSA. The auction did not succeed. The software was later released publicly on **14 April 2017**.

The package contained executable attack code, including techniques described

in the slides as previously unknown, or **zero-day**, at the relevant time. Its structure made the group's claim about its source appear credible, but attribution should still be treated as a claim requiring evidence.

A **zero-day vulnerability** is one for which defenders have had no effective advance warning or patch opportunity at the time of discovery or use. Once details and fixes become available the practical situation changes: "zero-day" is not a permanent property of a piece of code.

The analytical lesson: a tool developed in one setting can escape into a much wider ecosystem. An analyst therefore studies the capability and observed behaviour of a sample, and keeps claims about its origin separate from what the sample itself proves.

1.3.4 WannaCry: reused exploit, large-scale propagation

In May 2017 **WannaCry** spread across thousands of PCs and used **EternalBlue**, an exploit associated with the leaked tool set. The slides stress that the malware spread widely despite relatively careless design.

An **exploit** is a way to use a vulnerability to achieve an attacker goal, such as code execution. A **worm-like propagation mechanism** can use that opportunity to reach other machines without each new victim taking a separate action. Widespread impact does not imply sophisticated engineering in every part of the malware: the size of the vulnerable population, ease of propagation and operational conditions can matter greatly.

> **Simone's note (original wording)**
>
> after an encryption, not only the hacker copuld possibly extort you, but may extor t you because he could possibly publish online ypur sensitive data. also, if you are storing other people sensitive data, they could possibly extort also them.

1.3.5 Capabilities become accessible

The central claim of the slide is that very advanced technologies can come within reach of criminals with sufficient budget. Public leaks, reuse and commercialised criminal services can separate the original developer of a capability from the actor who deploys it. This helps explain why identifying a technique does **not**, by itself, identify the person or organisation behind an incident.

1.3.6 LockBit: ransomware as a service

The introduction presents **LockBit** as a ransomware-as-a-service operation dating from 2019, and describes a third iteration of its software platform. This describes the case as presented in the course material, not its current operational status.

- **Ransomware as a service (RaaS)**: an organised model in which a platform and its affiliates or operators can divide the work of developing, accessing victims, deploying malware and pursuing payment. The exact division varies by operation.
- **Triple extortion**, as in the slides: encrypt data to deny access; threaten to publish stolen sensitive data; apply pressure through third parties connected to the victim.
- **Business-oriented features:** a bug-bounty program and customer-care functions are cited as examples of how a criminal operation can imitate service-provider processes.

This example comes after WannaCry for a reason: malware is not only a program. Its consequences depend on an ecosystem of operators, access methods, infrastructure, negotiation and incentives. Encryption is only one visible part of a ransomware incident. Data theft and third-party pressure can continue to matter even if a victim can restore files from backups.

1.4 Relevant for the job market?

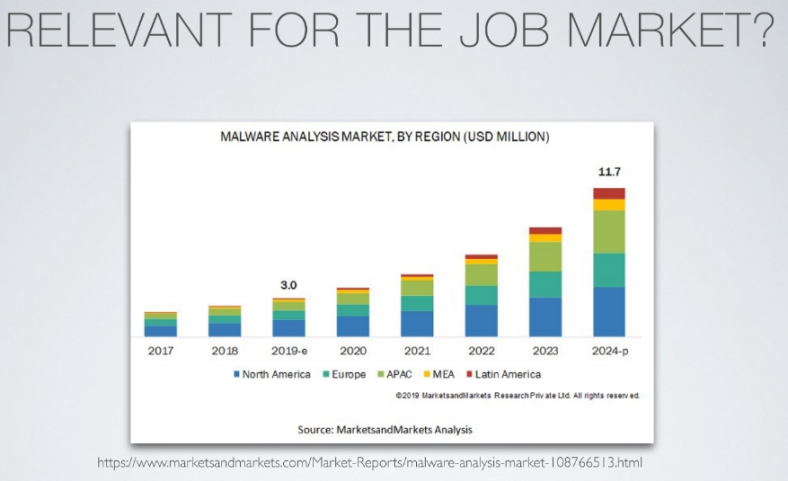


Figure 1.1: Regional malware-analysis market chart, attributed to a MarketsandMarkets report, with figures and projections up to 2024 (slides, Introduction).

The slide uses this chart as a period illustration of demand. It is **not a current market estimate or a career forecast**. Its substantive point is that organisations need people who can interpret malicious software and incident evidence.

Relevant work may be framed as malware analysis, reverse engineering, incident response, digital forensics, threat intelligence or security operations. These roles overlap but ask different questions. A sample analyst may determine what a binary can do. An incident responder also has to determine whether it ran in a particular environment, what it affected, and what action the available evidence justifies. The skills of this course connect those viewpoints.

1.5 Anatomy of an attack: the kill chain

The lecture introduces a model of how perpetrators carry out an attack, the intrusion (or cyber) **kill chain**, attributed to **Lockheed Martin**. The stages, in the displayed order, are in the table. The guiding questions and meanings are the framing of the notes.

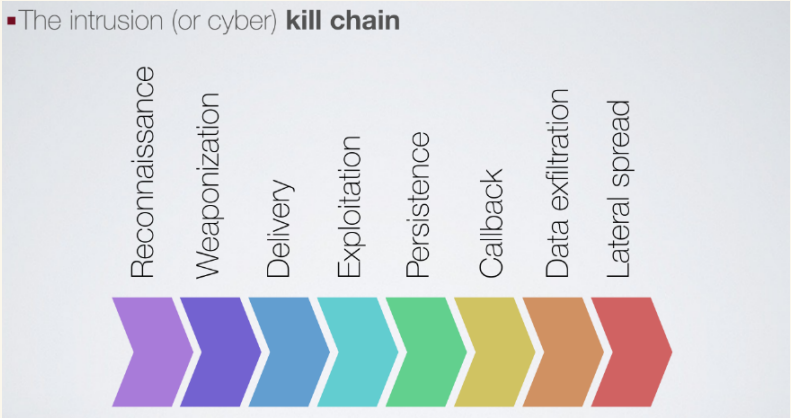


Figure 1.2: The intrusion kill chain as displayed in the slides (Introduction).

Stage	Guiding question	Meaning in this model
Reconnaissance	What can the attacker learn?	Gather information about the target and opportunities for access.
Weaponization	What capability will be prepared?	Put together a malicious payload or exploit suited to the objective.
Delivery	How does it reach the target?	Bring the payload or access attempt to the victim environment.
Exploitation	How does it gain execution or access?	Use a weakness or trigger the delivered content to cross a security boundary.
Persistence	How does access survive?	Arrange for continued or renewed execution, often across reboots.
Callback	How does it contact the operator?	Establish an outbound channel or report that it is active.
Data exfiltration	What information leaves?	Transfer selected data out of the victim environment.
Lateral spread	How does it reach other systems?	Move from one compromised machine or account to others.

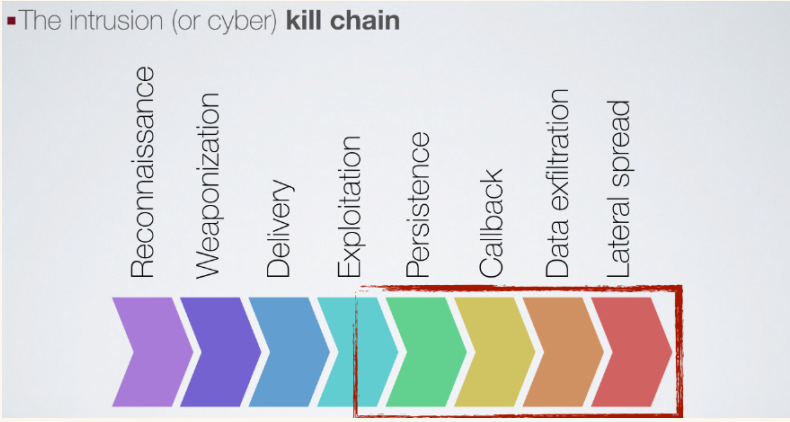


Figure 1.3: The same chain with the later stages highlighted: persistence, callback, data exfiltration, lateral spread (slides, Introduction).

The second visual draws attention to **persistence, callback, data exfiltration and lateral spread**. These later actions are especially relevant when interpreting an installed sample and the traces it leaves.

The diagram is a **reasoning aid, not a mandatory timeline**. Real attacks may skip stages, repeat them, or move laterally before exfiltrating.

> **Do not confuse**
>
> **Delivery** and **exploitation** are distinct. Receiving a file does not establish that it executed or compromised a host.

1.5.1 Using the model in an investigation

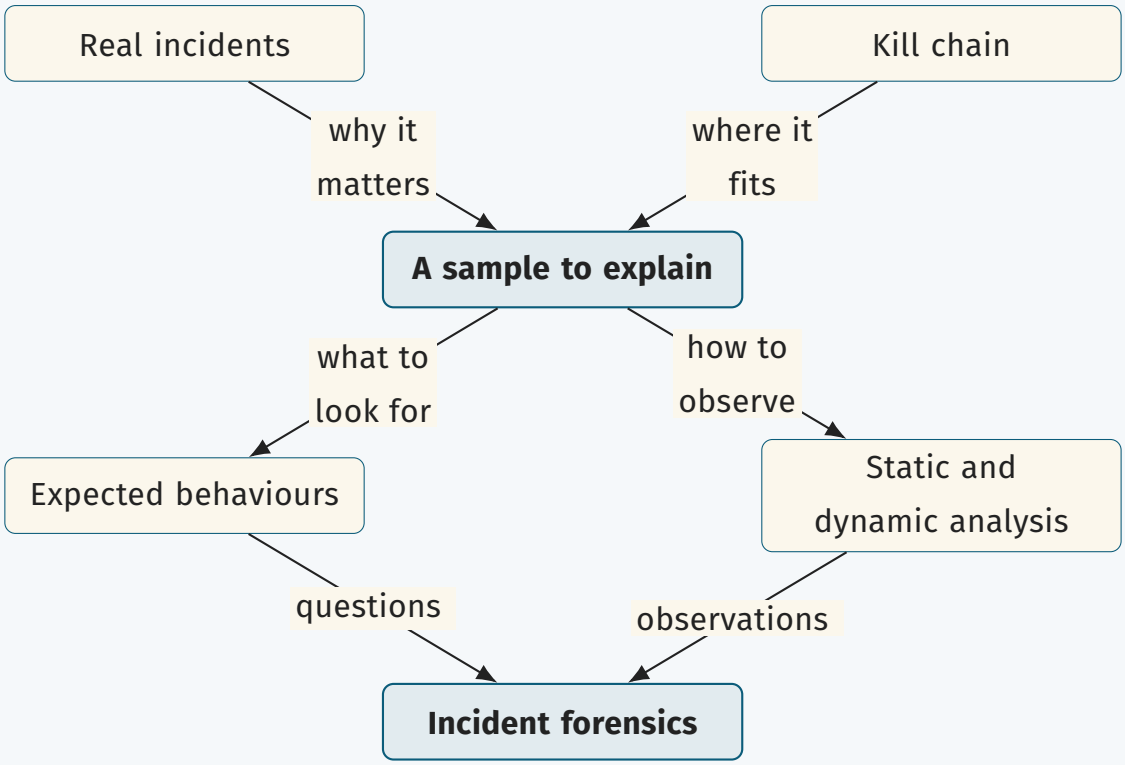
Ask what evidence would support each stage.

- An attachment may support a delivery hypothesis.
- A process execution trace or an exploit artifact may support execution.
- A newly created autostart mechanism may support persistence.
- Repeated outbound connections may support callback.
- File access plus outbound transfer may support exfiltration.

Each conclusion has a different evidentiary threshold. A malware sample may reveal possible behaviour, while host and network records help establish what

actually happened in one incident.

Concept map: how the ideas connect



Start from the sample. The incidents say why it matters, and the kill chain says where it fits in a wider attack. The expected behaviours say what to look for, and static and dynamic analysis provide the observations. Incident forensics tests how those observations apply to a real environment.

In short

The incidents explain **why** the topic matters. The kill chain asks **where** a sample fits in a wider attack. The expected behaviours of chapter 2 ask **what to look for** on a host and in network records. Static and dynamic analysis provide observations; incident forensics tests how those observations apply to a real environment. Keep capability, observation and conclusion distinct whenever you explain a sample.

Review: try to answer without looking

1. Why are the dates of intrusion, detection and full scope kept separate in the Yahoo! case?

2. What does triple extortion add to plain encryption?
3. Does finding a delivered file prove exploitation?

Answers, with the section where to find them

1. They can differ, and incident forensics has to establish the timeline from evidence (section 1.3).
2. The threat of publishing stolen data and pressure through third parties connected to the victim (section 1.3).
3. No: delivery and exploitation are distinct stages (section 1.5).

Chapter 2

The analyst’s method: goals, safety and expected behaviours

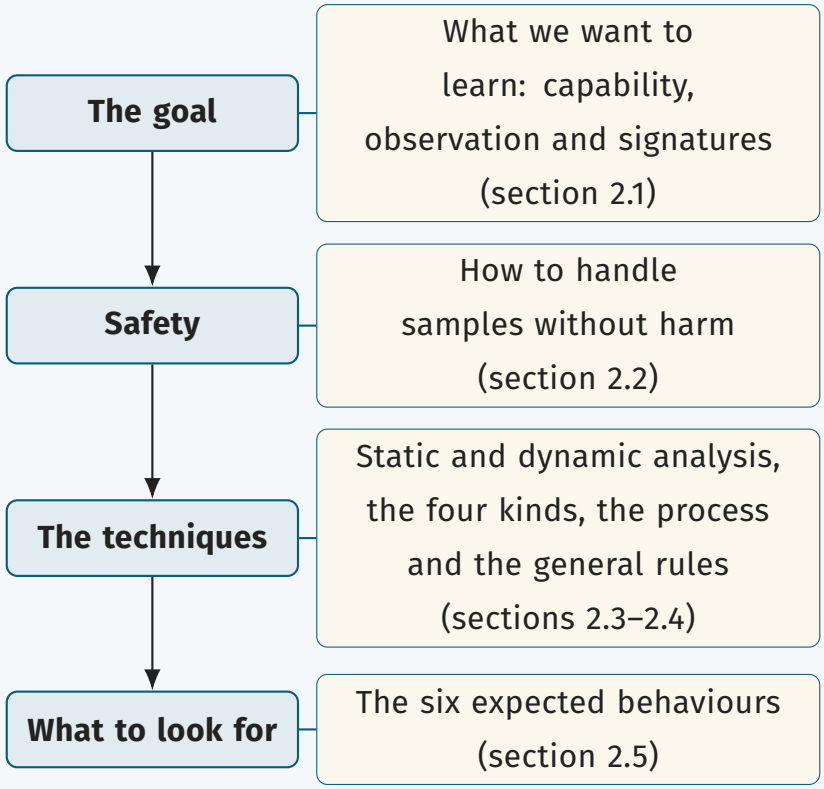
Before you start

Central question: what are we looking for in a sample, how do we stay safe, and in which order do we use our techniques?

Prerequisites: the kill chain and the idea of an indicator of compromise (chapter 1).

Where it leads: every later chapter applies one quadrant of the static/dynamic, basic/advanced scheme introduced here.

Chapter structure: what comes, and in which order



The chapter moves from the goal to the practice. It defines what we want to learn and how to stay safe. It then gives the techniques and, last, the behaviours they should look for.

2.1 The goal of malware analysis

Malware analysis means dissecting malware to turn an unknown program into actionable knowledge. The slide lists three goals:

- **How it works:** know your enemy's strategies.
- **How to identify it:** extract signatures and indicators of compromise (IoCs), and prevent further similar infections.
- **How to defeat or eliminate it:** roll back its effects if possible, and clean the infected system.

The slide calls this a critical part of incident response. The resulting knowledge serves two connected purposes.

- **Monitoring and prevention:** recognise recurring attack patterns, write detection rules, and design systems that are less susceptible to similar attacks.
- **Incident response:** reconstruct what happened, determine which machines, files, accounts and other assets were affected, contain propagation, remove the threat, and recover safely.

> **Simone's note (original wording)**
>
> The idea is to extract information from malwares in a way to create better monitoring systems.

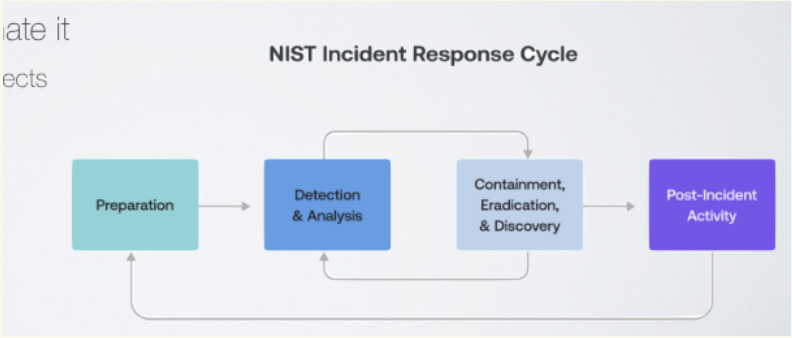


Figure 2.1: NIST incident response cycle, shown on the slide about the goal of malware analysis: preparation; detection and analysis; containment, eradication and "discovery"; post-incident activity (slides, Basic static analysis).

The cycle shows four phases. Preparation leads to detection and analysis. Detection and analysis alternate with the containment phase: the arrows go both ways, because what you contain teaches you what to look for next. Then comes post-incident activity, which feeds back into preparation.

> **To verify**
>
> The image reads "Containment, Eradication, & Discovery". [**Addition, not from the course material:** the NIST guideline SP 800-61 names this phase "Containment, Eradication & Recovery".] The note in the course files places this image under "Signatures"; here it is moved next to the slide text it comes from. Check the wording of the phase against the original source if you need to quote it.

2.1.1 Capability versus observation

The distinction between **capability** and **observation** is essential. A suspicious imported API suggests what a program might do. Logs, memory, network captures or controlled execution are needed to establish what it actually did in a particular incident.

Analysis also produces IoCs, such as hashes, filenames, paths, registry locations, domains or network patterns. An IoC is useful only with context: a common filename alone is weak evidence, while a coherent combination of indicators can be much stronger.

2.1.2 Signatures

A **signature** is a description of something that identifies an infection. The slides list three types.

Type	What it describes	Example and limitation
Host-based	Files, processes, registry keys, services or scheduled tasks on a victim computer that indicate an infection.	A particular autorun key may indicate persistence, but legitimate applications also use autoruns.
Network	Patterns in traffic: destinations, protocols, requests, timing. Does the malware inspect its surroundings? Does it contact an external Command & Control (C&C or C2) server?	Repeated contact with a C2 endpoint can suggest beaconing; IPs and domains may change or be shared.
Behavioural	What the malware does to the system, not the malware itself. Different from a standard antivirus signature.	A process writing an autorun key and injecting another process resists minor file changes better, but needs careful tuning.

The slides add that network signatures are more effective when coupled with malware analysis. In general, host and network signatures become more meaningful when analysis explains why they occur. A traditional file signature can be defeated by small changes to the binary. Behavioural detection focuses on activity that the attacker must perform to reach an objective. No single signature proves an entire infection story.

2.2 Warning: handle samples safely

The slide is direct: be conscious of the risks. You will dissect software designed to harm computers, possibly your own. You may need to run it to understand how it works. Some of the code was designed by criminals. Some IP addresses referenced in malware may be live, and you must assume that a criminal is behind them.

Even a sample that looks inert can contain a destructive payload, exploit an analysis tool, or communicate with a live attacker-controlled service. The rules are:

- **Keep a safe offline backup of your important data.** The slide repeats this line four times on purpose: isolation can fail and mistakes happen.
- **Always analyse on a virtual machine isolated from your main host.** Use a dedicated, disposable VM with snapshots before experiments. Avoid shared folders, clipboard integration and other paths back to the host when they are unnecessary.
- **Disconnect the VM from the network unless network communications are necessary.** If networking is required, use a controlled lab arrangement and observe its traffic.
- **If you find references to external IP addresses, never try to contact them directly.** Treat embedded IPs and domains as untrusted evidence. They may still be active, and the contact itself may expose the investigation. The slide's image: if someone robs your house and you find the robber's home address, would you go there to ask how they got in?

- **Distinguish tools that merely parse bytes from tools or plugins that execute the sample.** A graphical "inspection" interface does not guarantee passive analysis (see the PEiD warning in section 6.4.1).

2.3 Static versus dynamic analysis

Static analysis inspects the file without running it: hashes, strings, file format, imports, resources and, at a more advanced level, disassembly.

Dynamic analysis executes it in a controlled environment and observes processes, filesystem and registry changes, network activity and memory. Snapshots make the environment reproducible. The lecture names tools such as RegShot, Process Monitor, Process Hacker, CaptureBAT, debuggers, and memory analysis tools such as Redline and Volatility. Tool choice depends on the question and the environment.

> **Simone's note (original wording)**
>
> The question is the following: Why should we use dynami cwhile we could possibly simply reverse engeenrineg the malware? simply because not everuthing is readable in the reversed engeneeringed code: we are not considering the actual content of the memory (stack, heap etch...), enviroment, and also the static code could be offuscated. In tat last case, when we dynamically analyse a malware the obvuscation could disappear.

The two approaches form an **iterative process**, not two isolated phases. Initial static clues guide what to watch during execution. Observed behaviour then tells the analyst which strings, imports, functions and data to inspect more closely. Neither view is complete: a file can contain dormant branches, and a single run may miss a trigger.

2.3.1 The four kinds of analysis

> **Simone's note (original wording)**
>
> even tho they are "basic", doesn't mean that they are useless: they are extremelly powerfull but not complex enough. so when you have to start to analyse a software, you have to start with basic analysis and then move to the advanced one when things becomes harder.

Figure 2.2: Slides on basic analysis (left) and advanced analysis (right), Basic static analysis.

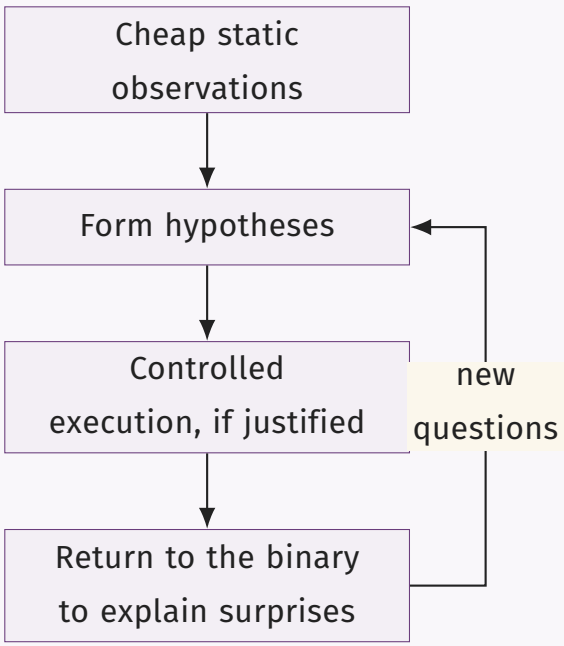
	Static	**Dynamic**
Basic	Quick triage of observable file properties without closely reading the machine code. Can expose obvious clues, but packing, encryption and runtime resolution can hide them.	Execute and compare observable effects. Needs a safe test environment. Can miss behaviour gated by time, configuration, location, privileges, user interaction or anti-VM checks.
Advanced	Disassemble or decompile code and reason about control flow, data, operating-system interfaces and hidden logic. Requires assembly and OS knowledge.	Debug a running process and inspect how registers, memory, calls and internal state change. Can expose decrypted or unpacked material, but debugging can itself alter behaviour.

> **Simone's note (original wording)**
>
> Using advanced analysis have a cost: time and computational power.

2.3.2 The malware analysis process

Begin with cheap, nonexecuting observations. Form hypotheses. Use controlled execution when justified. Return to the binary to explain surprising observations. Record what is **confirmed**, what is **suggested**, and what remains **unknown**.

Flowchart: the iterative cycle behind the lecture's static/dynamic diagram



The diagram is a loop. Each return to the binary produces new questions, which become new hypotheses to test.

2.4 General rules for malware analysis

- **Do not get trapped in details.** The goal is to answer the investigation's questions, not to understand every instruction. Prioritise execution path, persistence, payload, communications and indicators.
- **Try several tools.** Parsers and disassemblers disagree or fail on malformed and intentionally confusing files. Check findings against raw bytes and other tools where the conclusion matters.
- **Move on when blocked.** Record a question and seek another piece of evidence. Return when new information makes it tractable.

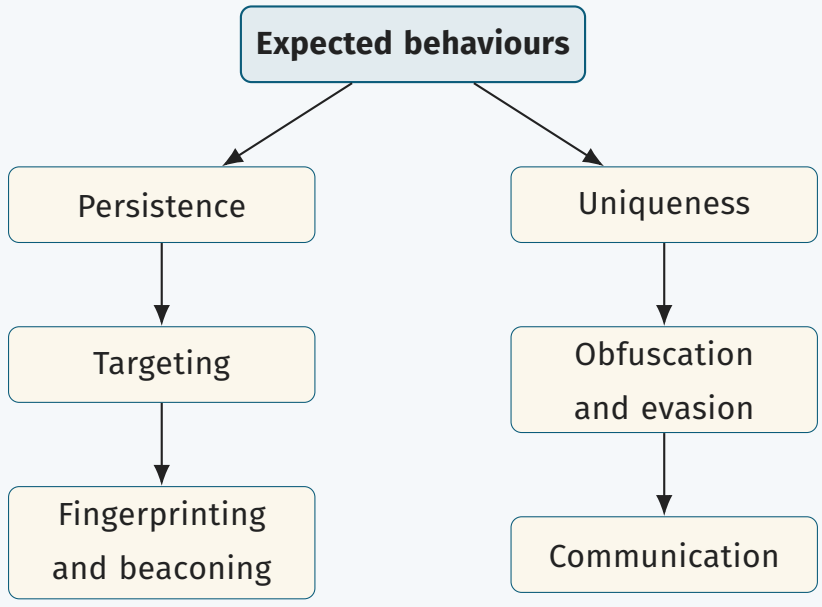
- **Expect adaptation.** Attackers change their methods, so maintain uncertainty instead of forcing every observation into a familiar family label.

2.5 Expected behaviours

This section translates the attack model into behaviours an analyst might observe. They appear both in the Introduction and in the Basic static analysis lecture and are merged here. They are **questions to investigate, not a checklist** that every sample must satisfy. The purpose of the sample, the victim, the execution conditions and the stage of the operation determine what becomes visible.

For each behaviour, separate **the program's capability**, **the conditions under which it activates**, and **the evidence that it activated on this host**.

Diagram: The six expected behaviours



The map simply gathers the six behaviours of this section. Read top to bottom on each side: the arrows give the reading order of the subsections, not a sequence that every sample follows.

2.5.1 Persistence

Persistence means continuing to run after a reboot, a user logoff, a removal attempt, or another change. Malware may install itself, copy itself, modify operating-system configuration or create an automatic launch mechanism. It may hide those changes. On Windows, places to inspect include autostart configuration, services and scheduled tasks; these are examples, not a complete list.

Why does the attacker want it? Initial execution can be temporary, and persistence turns a brief opportunity into durable access.

The analyst asks: what launches it again? Where is the installed copy? What permissions are needed? Which account runs it? When was the change made? Are the changes concealed? Finding a persistence-related string or API is a lead; finding the actual configuration change establishes more. An autostart mechanism is more informative when correlated with a suspicious file and execution history. Some malware does not require persistence: absence of a persistence artifact does not rule out compromise. The concrete Windows mechanisms are in section 8.7.

> **Simone's note (original wording)**
>
> in practice to enstablish persistance an malware produce always some traces, but not only becausae of persistance. There also exists a case in which the malware survives only in memory and not in the storage, but this requires to be infected many times also after the reboot.

2.5.2 Uniqueness (single-instance behaviour)

A sample may check at startup whether another copy is already running, and exit if it finds one. Simultaneous instances would waste resources, conflict over files or disrupt the operation. Two ransomware instances, for example, could compete for the same files and tasks.

Startup checks may look for an existing process, a named mutex, a lock file or another marker. A run that exits quickly may therefore not be inert: a previous

instance or a leftover marker may have altered its path. The existence and name of a marker can be a useful indicator. Its presence alone does not prove maliciousness, nor that all other behaviours executed.

> **Simone's note (original wording)**
>
> a peace if malware is not that elegant: has to work and doesnt care about being efficent. It is funny because an malware wants to be alone in that infected system: if there exists already another malware in that victim machine, then me (the new malware) will try to kill that old malware in a way that i have all by myself the resources,

2.5.3 Targeting and environment checks

Malware is designed to work in an appropriate environment. It checks at startup whether specific execution conditions are met, or whether it is on its intended victim. The two lectures list these signals:

- region, language or localisation;
- time and date;
- hardware or software properties, installed applications;
- a particular configuration, privileges, network state;
- availability of specific resources, external devices, known documents;
- presence of other processes.

It may also check for a virtual machine or a debugger (chapter 12).

The operator may want to affect a particular organisation, avoid unrelated systems, or reserve a capability for a valuable environment. For the analyst, this means a sample can appear quiet in a lab because the expected conditions are absent. **A quiet sandbox run does not establish that the binary has no payload.** Distinguish the code path that checks a condition from evidence that the condition was met in the real incident.

2.5.4 Obfuscation and evasion

Malware may try to hide its presence, effects or purpose. The lecture gives three broad mechanisms.

- **Scrambling or concealing code**, so that readable clues are harder to find. Obfuscation changes how understandable a program is. It does not by itself tell you what the program ultimately does.
- **Injecting code into another process**, so that malicious activity appears under a different process identity. Attributing an action to a file can therefore require more than checking the process name.
- **Hampering analysis** by changing behaviour or making tools less effective, for example when an expected trigger or environment is missing.

> **Simone's note (original wording)**
>
> obfuscation means that the code of the malware should not be readble by an analyser: so the attacker will actuate some technics to make the code way more random. Also the attacker wants to let them think that it is doing a thing but in practice is doing something else.

The usual aim is to remain undetected for longer while acting on the infected system. A static view of a packed file may show mostly the unpacking stub: separate the evidence of concealment from any claim about the precise function of the hidden payload.

Ransomware is a useful exception. Its final extortion step is deliberately visible to the victim, although preparation, intrusion and data theft may have been concealed. The key habit is to avoid equating "I did not see the behaviour in this run" with "the behaviour is impossible".

> **Simone's note (original wording)**
>
> as an Ransomware you have to be as fast as possible in compare to other possible malwares, which tipically those works really slowly to avoid beaing detected by an IDS.

2.5.5 Fingerprinting and beaconing

Malware may inspect the infected machine and collect facts about it, such as OS version, configuration, user or installed software. It can use that information to tell a remote operator that a new host is available. The lecture calls this notification or repeated check-in **beaconing**.

> **Do not confuse**
>
> **Fingerprinting** is learning about the host. **Beaconing** is communicating status or presence, periodically or on an event. A program can inspect a system without sending the result, and a beacon can carry very little host detail. System discovery does not automatically imply exfiltration: examine the data flow and traffic to establish what leaves the machine.

Periodic outbound connections can be a clue, but timing and destination need context, because legitimate software also checks in with remote services.

2.5.6 Communication and control

Malware often communicates with external **command-and-control (C2)** infrastructure to receive instructions, update components or **exfiltrate** data. The lecture stresses that the transport can vary: direct network links, cloud storage, social networks, or ordinary network services such as **DNS**. Data exchanged over these channels is often obfuscated, encoded or encrypted.

C2 is the means by which an operator directs infected systems; it does not have to be one dedicated server. Exfiltration is data leaving the victim's control. DNS normally resolves names, so suspicious DNS activity has to be interpreted against normal network use. Obfuscation may conceal meaning without hiding that a connection occurred.

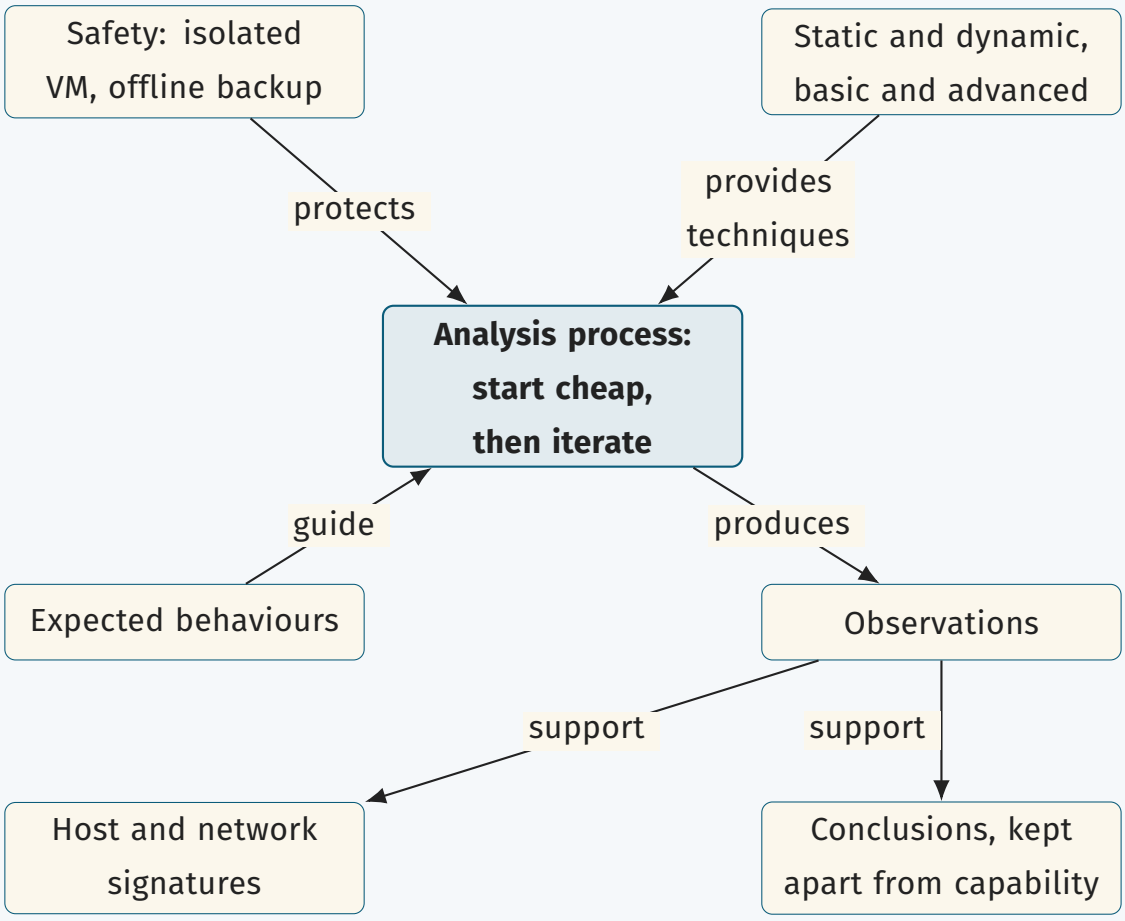
For an investigation, record the destination, timing, frequency, protocol, data volume, and relationship to the host's other actions. The question is not merely whether networking functions are present, but which endpoints, protocols, requests and data are involved. An import for networking may be benign or unused. A domain or IP can be an IoC, but destinations can change or be shared

with legitimate traffic. Correlating network evidence with process activity and file access is stronger than treating one isolated connection as a complete explanation.

Simone's note (original wording)

now we have a modern approach: just store things in a online storage instead of directly create a communication: for instance twitter with comments, ora whatever online service which could store informations for you.

Concept map: how the ideas connect



Safety protects the process, and the four kinds of analysis provide its techniques. The expected behaviours guide what the process looks for. The process produces observations, and these support both signatures and conclusions. A conclusion stays distinct from what the program is merely capable of doing.

Remember

- Capability, observation and conclusion are three different things.
- Offline backup, isolated VM, no network unless needed, never contact embedded addresses.
- Start cheap and static, then iterate with dynamic analysis; do not get trapped in details.
- A quiet run does not prove the absence of a payload.

Review: try to answer without looking

1. What do behavioural signatures describe, compared with standard antivirus signatures?
2. Give a limit of basic static and a limit of basic dynamic analysis.
3. Why does ransomware count as an exception among evasive malware?
4. What is the difference between fingerprinting and beaconing?

Answers, with the section where to find them

1. What the malware does to the system, not the malware itself (section 2.1.2).
2. Packing and runtime resolution hide static clues; dynamic runs miss gated behaviour (section 2.3.1).
3. Its final extortion step is deliberately visible (section 2.5.4).
4. Learning about the host versus reporting status (section 2.5.5).

Chapter 3

The x86 machine: CPU, registers and process memory

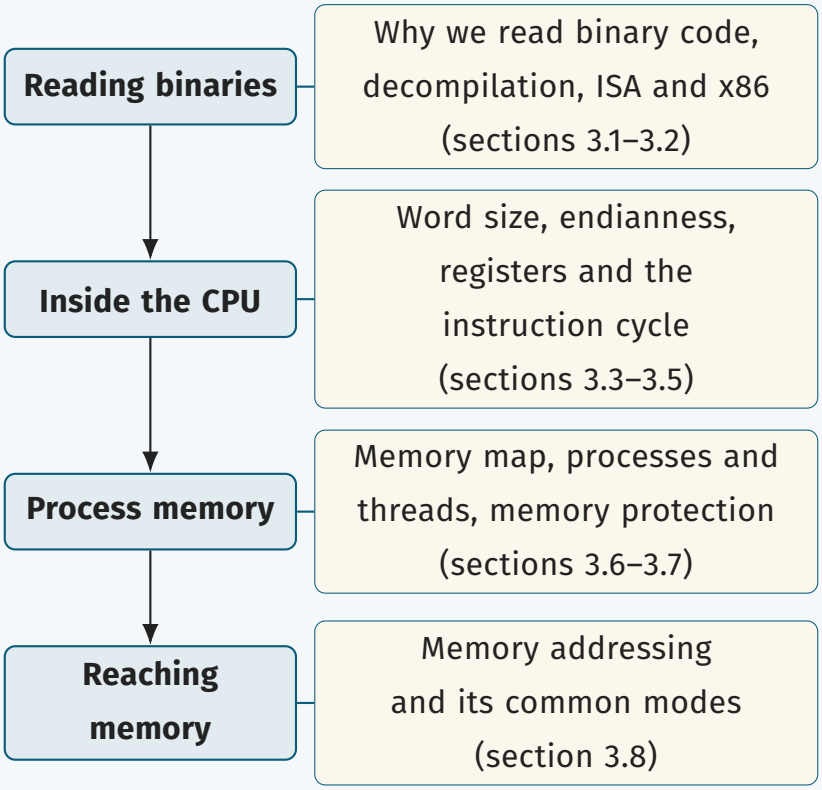
Before you start

Central question: what does the machine look like to someone who reads a binary instead of source code?

Prerequisites: the static/dynamic scheme of chapter 2. A previous course on computer architecture or C compilation helps but is not required.

Where it leads: chapter 4 uses these registers and addressing modes to read instructions and functions. Chapter 9 returns to process memory from the Windows side.

Chapter structure: what comes, and in which order



The chapter starts from why we read binaries at all. It then describes the CPU, then the memory of a process, and finally how an instruction reaches that memory.

Examples use **Intel syntax** and generally assume a **32-bit Windows process**. Addresses and stack layouts are illustrative unless tied to a named exercise.

Simone's note (original wording)

it may be interesting to compare the binary ETHL 0x07 — Binary Exploitation p1 with this note.

Connections with other courses. Sistemi di Calcolo studies how C becomes machine code; here the same concepts are used in reverse, to reconstruct behaviour from a binary. Sistemi Operativi connects registers and stacks to the execution state managed by the operating system. Sistemi di Calcolo often uses **AT&T syntax** (`source, destination`), while this book uses **Intel syntax** (`destination, source`). Its calling-convention material describes **System V i386**: the Windows conventions of chapter 4 must be compared with it, not assumed identical.

3.1 Why read binary code

The slide calls this the daily routine of malware analysts. Reverse engineering binary code is difficult, but only some portions of a sample really require it. Source code is usually unavailable. So an analyst first identifies the behaviour that matters (entry points, suspicious calls, unpacking, persistence, data handling) and then inspects the relevant machine instructions. It is rarely necessary to understand every instruction in a large sample.

The slide states a fact: malicious code is not like compiler-generated code. The differences it lists are these.

- **Obfuscated control and data flow:** branches, indirection, encoded strings or unusual data transformations hide an otherwise simple operation.
- **Subtle semantics and instruction boundaries:** x86 instructions have variable lengths, so a jump can enter bytes that another disassembly path interpreted as the middle of an instruction.
- **Handwritten sequences:** for example, a custom API-name hash may locate Windows functions without explicit import names.
- **Self-modifying code:** bytes executed later can be rewritten at runtime.
- **Decoys:** plausible-looking instructions or unreachable paths can distract static analysis.

> **Simone's note (original wording)**
>
> it is interesting because for obdfuscation reasons, the attacker might use a lot of jump" operators to make things harder to read. compilers obviously do not use this kind of stuff.

The practical question is not merely "What does this instruction say?" but **"Which bytes actually execute, under what state, and what behaviour do they implement?"**

From compilation to reverse engineering. The normal compiler patterns for branches, loops, arrays and structures are the starting point (chapter 10).

Compiler optimisations such as inlining, register allocation and eliminated code can obscure those patterns in the binary.

3.1.1 Decompilation

A **decompiler** such as Ghidra or Hex-Rays attempts to turn machine instructions into a C-like representation. This is a useful **hypothesis** about the code, not a recovery of its original source. The process is far from perfect:

- compilation discards information: names, many types, comments and source-level structure;
- optimisations rearrange operations;
- decompiling obfuscated code returns an obfuscated source;
- anti-analysis techniques make reconstruction harder.

The slide adds that decompilers are nevertheless very helpful for reverse-engineering professionals. Use the pseudocode to orient yourself. Then check the assembly when a branch condition, pointer operation, calling convention, instruction boundary or side effect matters. The assembly is the closer description of the bytes being executed, although even static assembly must be checked against runtime behaviour when code changes itself.

> **Simone's note (original wording)**
>
> If you already know how things works in assebly and how it is structured a binay file, you could possibly just use a decompiler to speed-up your work. notice that when you move from C to assembly, types are gone: we will deal only with memory.

3.2 ISA and x86

> **Simone's note (original wording)**
>
> A cpu is a colletion of cells/memory and there exists also a lot of well specified circuites that execute specifics task for optimization reasons. We have a lot of kinds of registries, many of them are the so callde ”general-purpose” ones.

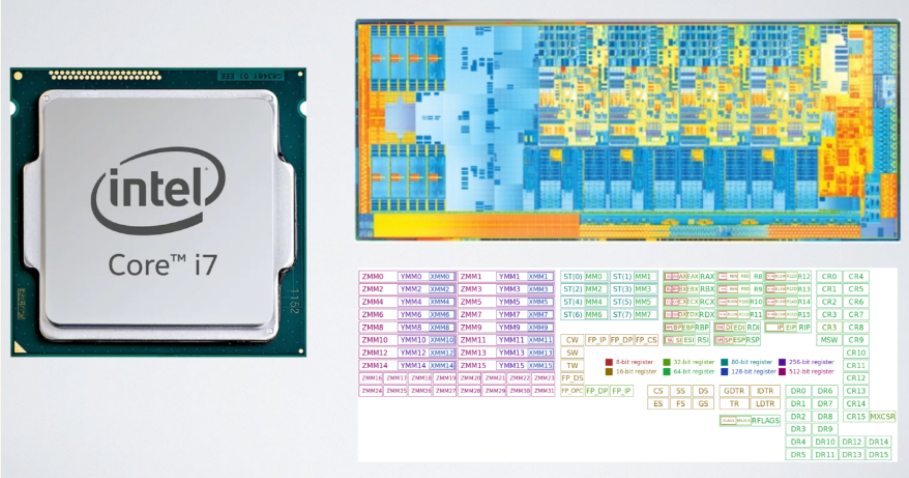


Figure 3.1: A processor, its die, and the full map of modern x86 registers from 8 to 512 bits (slides, Assembly).

The figure gives a sense of scale: the modern register file is much larger than the part this course needs. The next sections concentrate on the eight general-purpose registers, the instruction pointer and the flags.

An **instruction set architecture (ISA)** specifies the programmer-visible machine: instructions, registers, operand forms and their effects. It is the contract between software and a compatible processor. A compiler or a human produces assembly in that ISA, and an assembler encodes it into bytes. Different hardware designs can implement the same ISA.

Intel defined the x86 ISA for its 8086 processor (1976–1978). Later extensions preserved backward compatibility. This course focuses on **IA-32**, the 32-bit x86 environment. **x86-64** (x64 for short) extends the architecture with 64-bit registers, a different default calling environment and other changes. The 32-bit Windows conventions in this book should therefore not be carried over

unchanged to 64-bit Windows.

The slide motivates the choice with the claim “Today most malware is still 32-bit”. Treat this as context from the lecture, not as a timeless statistic.

> **Simone’s note (original wording)**
>
> we willl focus more on 32 bit assembly. why? because most of malwares are in 32bit. In windows the backward compatibility works better, so designing 32bit programs, it works flawlessy in compare to linux x64, which it may have so problems with 32 bits. notice that, over time, obviously we will get more 64bits malware in the future.

An ISA defines instruction semantics. An ABI defines the conventions used by compiled programs on top of it. The two must be kept separate: the same instructions can be used under different calling rules.

3.3 Word size and endianness

The slide defines a **word** as the natural data unit of a CPU design, and for x86 processors that size is 32 bits. However, for backward compatibility, x86 instructions assume a `word` operand to be 16 bits long and a `dword` operand to be 32 bits long.

Assembly name	Size
`byte`	8 bits
`word`	16 bits
`dword` (doubleword)	32 bits; a 32-bit general-purpose register holds one
`qword` (quadword)	64 bits

> **Do not confuse**
>
> “32-bit architecture” does **not** mean that the assembly keyword `word` means 32 bits.

The two statements in the slide do not contradict each other, because “word” has two different meanings:

Meaning	What it is	On 32-bit x86
Architecture concept	The CPU’s natural data unit, such as the size of its registers	32 bits
Assembly type name	A fixed label for an operand size	16 bits

The 8086 was a 16-bit CPU, so the name `word` became tied to 16 bits. When x86 moved to 32 bits, the natural size changed but the name did not, because changing its meaning would have caused confusion with existing code and documentation. The new size got a new name, `dword` (“double word”). x86-64 repeated the pattern: the natural size is 64 bits, `word` is still 16 bits, `dword` is still 32 bits, and 64 bits is called `qword`.

When reading assembly, only the type name matters:

```
mov ax,  word ptr [esi]    ; reads 2 bytes (16 bits)
mov eax, dword ptr [esi]   ; reads 4 bytes (32 bits)
```

Windows types follow the same names: `WORD` is 16 bits and `DWORD` is 32 bits, even on a 64-bit system.

> **Simone’s note (original wording)**
>
> word size is the most elemental size that a cpu can crunch. a word is 16bit long, if it is a double word (dword) is 32bits. we have qword for 64 bits. byte=8-bit. Notice that a memory could be intendet as an infinite array of bytes.

Memory always operates at byte level: locations are byte-addressed. **Endianness** determines the order of the bytes of a multi-byte value in memory. Big endian stores the most significant byte first. Little endian stores the least significant byte first. x86 uses little endian. If the dword `0xABADCAFE` starts at `0x1000`:

Address	`0x1000`	`0x1001`	`0x1002`	`0x1003`
Little endian (x86)	FE	CA	AD	AB
Big endian	AB	AD	CA	FE

Figure 3.2: The dword 0xABADCAFE stored at 0x1000 in both byte orders (slides, Assembly).

The numerical value is still `0xABADCAFE`. Only its byte representation in memory is reversed relative to the usual left-to-right hexadecimal writing. This matters when you inspect dumps, parse structures, reconstruct immediate operands from instruction bytes, or interpret four bytes as an address.

> **Simone's note (original wording)**
>
> Big-Endian is like writing in memory like humans, while Little-Endian is the starting with the least significant one and this is what intel processors (but i suppose also amd) and it is beign choosen for optimization reasons.

> **Correction**
>
> The personal note above says little endian was chosen for optimisation reasons. The notes themselves qualify this. Little endian is **not inherently faster** for a CPU: once the CPU has loaded the value into a register, addition and comparison work the same way either way. A big-endian CPU can perform those operations efficiently too. Intel documents x86 as little endian, while Arm supports designs that handle more than one byte order.

Little endian does make a few tasks convenient. For `0x12345678`, memory holds `78 56 34 12` at increasing addresses, so the lowest address holds the least significant byte.

- Reading a byte or a 16-bit value from the same starting address gives the

low 8 or low 16 bits.

- Software doing arithmetic on a number too large for one register can walk through its pieces from low to high addresses, following the direction in which carries propagate.

To see the first point, suppose the dword 5 is stored at `0x1000`:

Address	`0x1000`	`0x1001`	`0x1002`	`0x1003`
Little endian (x86)	`05`	`00`	`00`	`00`
Big endian	`00`	`00`	`00`	`05`

All the reads below **start at `0x1000`**. Only the number of bytes taken changes, and the remaining bytes are ignored:

Read from `0x1000`	**Little endian**	**Big endian**
4 bytes	`05 00 00 00` = 5	`00 00 00 05` = 5
2 bytes	`05 00` = 5	`00 00` = 0
1 byte	`05` = 5	`00` = 0

In little endian the `05` is the first byte, so every read includes it. In big endian it is the last byte, so a shorter read must start from a higher address: `0x1002` for 2 bytes, `0x1003` for 1 byte.

```
mov eax, dword ptr [esi]   ; 5
mov ax,  word ptr [esi]    ; 5
mov al,  byte ptr [esi]    ; 5
```

Moving the starting address does exclude the earlier bytes: in little endian, 1 byte read from `0x1001` gives `00`, because the `05` is left behind.

The result stays 5 only because the number is small. A shorter read returns the **low part of the value**, which equals the whole value only if it fits in the

bytes read. With `0x12345678`, stored as `78 56 34 12`, reading from the same address gives `0x78` (1 byte), `0x5678` (2 bytes), or `0x12345678` (4 bytes).

For the second point, take a 64-bit number added on a 32-bit CPU. The low half must be added first, because the carry moves toward the high half. In little endian, the low half is at the lower address, so the code simply moves forward through memory:

```
add eax, dword ptr [esi]     ; low 32 bits, first in memory
adc edx, dword ptr [esi+4]   ; high 32 bits, plus the carry
```

Here `adc` means “add with carry”.

The benefit lies in particular ways of accessing memory, not in the speed of the arithmetic itself.

3.4 Registers

Coders use many variables, while CPUs work on few registers. **Registers** hold data with fast access time.

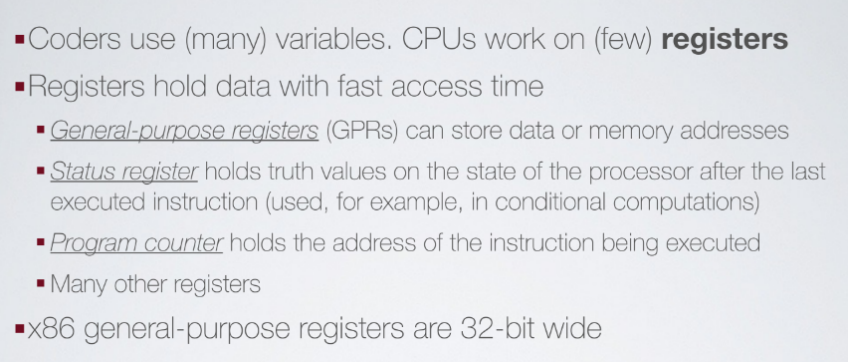


Figure 3.3: Slide on registers: general-purpose, status and program counter (slides, Assembly).

- **General-purpose registers (GPRs)** store data or memory addresses. x86 GPRs are 32 bits wide.
- **EIP**, the instruction pointer or program counter, tracks where execution continues. The slide says it holds the address of the instruction being executed.

- **EFLAGS**, the status register, records condition bits on the processor state after the last executed instruction, used for example in conditional computations. Many instructions affect it.

The conceptual distinction is essential: a register can contain an address, but an operation accesses memory only when its operand actually dereferences that address.

Dereferencing means using an address to go to memory and read or write what is stored there. A register only holds a number, and the CPU uses it as an address only if the instruction asks for it. In Intel syntax, **square brackets** express this. Suppose `EBX = 0x1000` and memory at `0x1000` holds `42`:

```
mov eax, ebx      ; copies the number: EAX = 0x1000
mov eax, [ebx]    ; reads memory there: EAX = 42
```

Holding or computing an address is not a memory access:

```
add ebx, 4        ; only computes: EBX = 0x1004
mov eax, [ebx]    ; the access happens here, at 0x1004
```

The CPU does not know whether EBX holds “an address” or “a number”: they are just 32 bits, and they act as an address only when an instruction uses them as one. In C, this is the `*` operator:

```
int *p = &x;   // p holds the address of x
int *q = p;    // like mov eax, ebx: copies the address
int b = *p;    // like mov eax, [ebx]: reads the value of x
```

`lea` is the exception: it has square brackets but **does not access memory** (see LEA in the next chapter).

```
mov eax, [ebx+4]  ; reads memory at 0x1004 (EBX = 0x1000)
lea eax, [ebx+4]  ; only computes: EAX = 0x1004
```

This matters for analysis. Dereferencing an invalid address can crash the program, while copying it or adding to it cannot. A computed address also does not prove that data was read or written: to establish that, find the instruction that dereferences it.

> **Simone's note (original wording)**
>
> An interesting thing about the program counter is: when the CPU starts to eecute the instruction that the program counter is pointing at, then automatically moves to the next memory-address: so even if the cpu is still finishing to executing the instruction, the IP is ahead.

3.4.1 Accessing registers

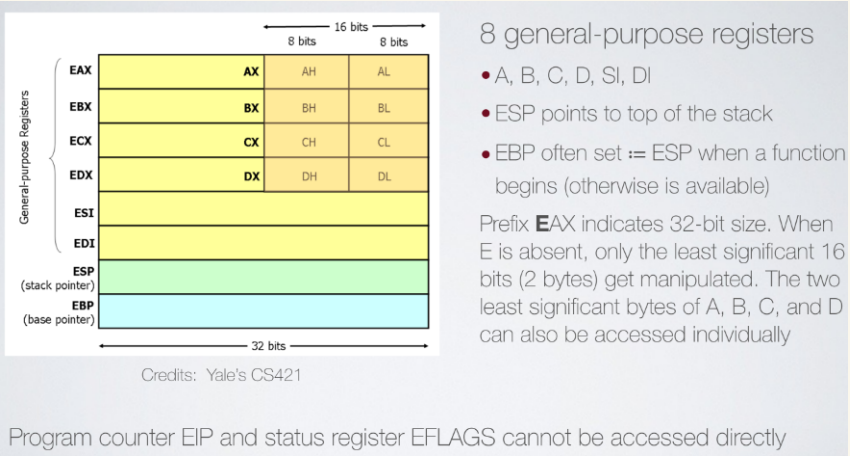


Figure 3.4: Register subdivision: EAX contains AX, which contains AH and AL. Credits on the slide: Yale's CS421 (slides, Assembly).

The eight 32-bit GPRs in the IA-32 setting of the course are `EAX`, `EBX`, `ECX`, `EDX`, `ESI`, `EDI`, `ESP` and `EBP`.

- **ESP** points to the top of the stack.
- **EBP** is often set to the value of ESP when a function begins, and anchors its stack frame. Otherwise it is available for other purposes.
- **ESI** and **EDI** are frequently source and destination pointers in string operations, though they are general-purpose in many contexts.

The E prefix denotes the 32-bit version of a legacy register. When E is absent, only the least significant 16 bits are manipulated. EAX contains AX as its low 16 bits, and AX contains AH (high byte) and AL (low byte). The same byte subdivision applies to EBX, ECX and EDX. ESI, EDI, ESP and EBP have low 16-bit forms in the IA-32 terminology of these slides.

A write to AL changes only that low byte of EAX: it does not replace the entire register.

> **Correction**
>
> The slide says that EIP and EFLAGS "cannot be accessed directly". Read this in a narrow sense. EIP is not an ordinary `mov` destination and EFLAGS is not used as a GPR, but both have **indirect or dedicated** ways of being changed or observed: branches and calls affect EIP, arithmetic and comparisons affect flags, and instructions such as `pushfd` can expose flags.

> **Simone's note (original wording)**
>
> notice: we have rax, eax, ah, al. l=lower, h=high, e=extended and r= i dont know. ESP and EBP are considered as general-purpose registries, but in practice it is being used for stack and base stack pointers, for optimizations reasons we could possibluy use them to get a speed-up if we do not care about the stack. one interesting thing is that, if we use in a such a way, then we could possibly break the decompiler or something similiar, because the interpreter doesn not understand the fact that ESP AND EBP are being used in a differetnwaym, and chrashes.

In Sistemi Operativi these register values are part of the context of a process, preserved across a context switch and associated with the process through its control block.

3.5 The instruction cycle

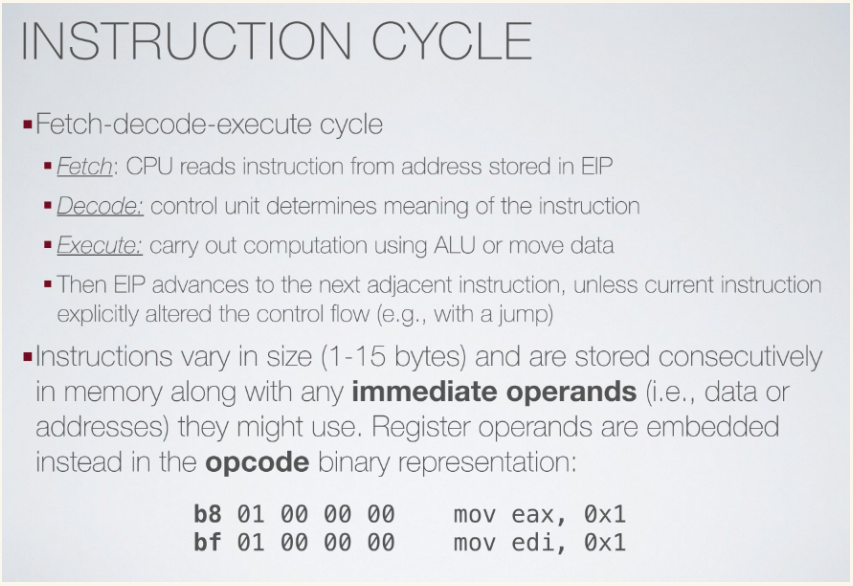


Figure 3.5: Slide on the fetch-decode-execute cycle, with the encoding of two mov instructions (slides, Assembly).

For each step the CPU works through the **fetch-decode-execute cycle**:

1. **Fetch:** read instruction bytes from the address in EIP.
2. **Decode:** the control unit determines the meaning of the instruction.
3. **Execute:** carry out the computation using the ALU (arithmetic logic unit), or move data.

In ordinary sequential execution, EIP then advances past the entire instruction. A branch, call, return or exception changes that path.

x86 instructions range from **1 to 15 bytes** and are stored consecutively in memory along with any immediate operands they use. So the next instruction is not found by adding a fixed width.

Register operands are embedded in the opcode's binary representation.

```
b8 01 00 00 00      mov eax, 0x1
bf 01 00 00 00      mov edi, 0x1
```

The first byte differs because the destination register is encoded in the instruction. The four-byte immediate 1 appears in little-endian order. This is why choosing the wrong starting byte can produce an entirely different disassembly.

> **To verify**
>
> Simone's note at this point marks something to recover from the recorded lessons. The wording of the note is: "QUI IL PROF HA DETTO UNA COSA DA RECUPERARE DALLE LEZIONI REGISTARTE". The recordings are not among the sources of this book, so this content is missing and the point is still to verify.

3.6 Memory map of a Windows process

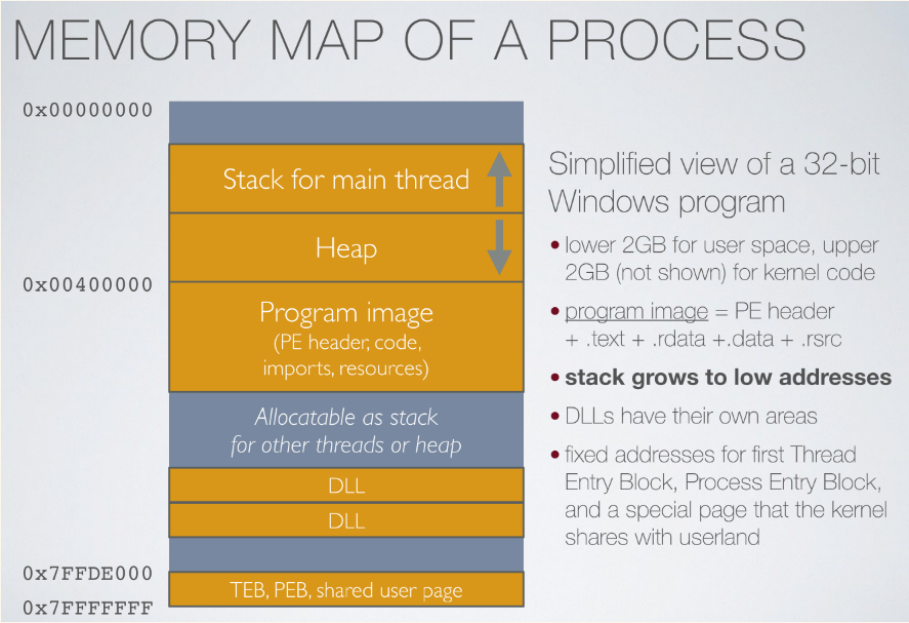


Figure 3.6: Simplified view of a 32-bit Windows program: from 0x00000000, the main thread's stack and heap, the program image at 0x00400000, DLLs, TEB, PEB and shared user page near 0x7FFDE000, up to 0x7FFFFFFF (slides, Assembly).

The slide depicts a **simplified, illustrative** 32-bit Windows virtual address space. In its example, the lower 2 GiB is user space and the upper 2 GiB (not shown) is reserved for kernel code. Do not infer a universal or fixed position for the heap, stack, image or DLLs from the drawing: actual mappings vary with system

configuration, loader behaviour and ASLR.

- The **program image** is a mapped PE executable (section 6.5), with headers and sections such as `.text` (code), `.rdata` (often read-only data), `.data` (writable initialised data) and `.rsrc` (resources), plus imports and other structures.
- **DLLs** map into their own regions and contribute code and data to the process.
- The **heap** supplies dynamic allocations.
- Each thread has a **stack** for call state and local storage. In these examples the stack grows toward lower addresses.
- Windows maintains thread and process metadata: the **TEB** (Thread Environment Block, one per thread), the **PEB** (Process Environment Block, shared by the threads of the process) and a shared user-data page. These user-space structures can matter when malware inspects its environment or resolves APIs. They are distinct from the kernel objects that represent processes and threads.

> **Correction**
>
> The slide expands these names as “Thread Entry Block” and “Process Entry Block”, and says the first of them have fixed addresses. The structures are named Thread **Environment** Block and Process **Environment** Block, as used in the rest of this book. Their fixed addresses belong to the slide’s simplified picture.

An address in a disassembly is a **virtual address in a process**, not automatically a physical RAM address or a file offset. Mapping it back to a PE file requires the image layout and relocation context.

> **Simone’s note (original wording)**
>
> the professor noticed that we stop at 0x7fffffff, bcause from 0x8fffffff and 0xffffffff are reserved for the kernel.

Correction

In the personal note, the kernel area should begin at `0x80000000`, the address right after `0x7FFFFFFF`. This matches the 2 GiB split of the slide. The value `0x8fffffff` would leave the range from `0x80000000` to `0x8FFFFFFE` unaccounted for.

Simone's note (original wording)

notice that glibc is not here: in windows we have virtual-alloc instead of malloc, and also instead of mmap again heap-alloc. or others.

Compare this map with process images in Sistemi Operativi: code, data, libraries, heap and stacks serve analogous roles, while their placement and the user/kernel split depend on the system. Executable loading and shared libraries in the Linux/ELF setting are a useful comparison for PE images and DLLs, whose formats and loading details differ.

3.6.1 Processes and threads: Windows versus Linux

A **thread** is a flow of execution inside a process. In both systems, threads in the same process share its virtual address space (code, data, heaps and mapped libraries). Each thread has its own CPU register state, stack and thread-local storage (**TLS**). Both kernels schedule individual threads.

On **Windows**, processes and threads are **distinct kernel objects**. The process acts as a resource container, providing the virtual address space and handle table. Each thread object represents an execution within that process and holds its execution and scheduling state. "Distinct objects" describes the organisation of the kernel. It does **not** imply that threads of the same process have separate memory or separate handle tables.

On **Linux**, each thread is represented by a `task_struct`. These structures contain references to resources: threads in the same process point to the same memory-management and file-descriptor structures. The flags supplied to `clone()` or `clone3()` determine which resources are shared. Saying "Linux threads are processes" is therefore a simplification: the kernel uses a common

task representation, but distinguishes thread groups and their shared resources.

Aspect	Windows	Linux
Kernel representation	A process object and separate thread objects	One `task_struct` per thread, referencing shared resources
Virtual address space	Shared through the process	Tasks refer to the same address space
Open files	Shared process handle table	Shared file-descriptor table
CPU registers and stack	Own state and stack per thread	Own state and stack per thread
Identification	Process PID and individual thread TIDs	Individual TIDs and a shared thread-group ID (TGID), returned by `getpid()`
Typical creation path	`CreateThread()`; native API `NtCreateThreadEx`	`pthread_create()`, implemented using `clone()` or `clone3()`
User-space thread metadata	TEB and TLS	TLS and runtime thread structures

> **Do not confuse**
>
> **A thread's own stack is not isolated from other threads.** It lies in the shared virtual address space of the process, so another thread can access it if it knows a valid address. "Own stack" means that each thread uses its own stack for calls and local storage, not that memory protection prevents other threads from accessing it.

3.7 Memory protection

Paging determines whether a virtual address is valid and enforces read, write and execute permissions, with regions as the granularity on the slide. Pages in kernel memory are accessible only from kernel code.

Operating systems may enforce higher-level security mechanisms to hinder the exploitation of memory vulnerabilities:

- **DEP (Data Execution Prevention)** blocks execution from non-executable memory.
- **ASLR (Address Space Layout Randomization)** makes the placement of mapped regions less predictable.
- Heap allocation randomisation and other protections exist in Windows 10 and later.

When investigating a sample, ask both **what address is computed** and **whether that region is currently readable, writable and executable**. For example, self-modifying code may change the permissions of a code region before writing bytes, and a pointer into an unmapped region would fault if dereferenced.

In the operating-systems course, paging and the MMU connect virtual addresses to physical memory and access checks, dual mode explains the user/kernel privilege distinction, and exceptions connect invalid accesses to faults handled by the OS.

3.8 Memory addressing

An **addressing mode** describes where an instruction gets its operands. Expressions can take immediate operands, registers or both, so they can compute addresses that depend on the program state.

In Intel syntax, square brackets usually mean **read or write memory at the computed address**:

- `mov eax, [esi]` loads a dword from the address in `ESI`;
- `mov eax, esi` copies the register value itself;
- `mov eax, 0x1000` loads a literal;
- `mov eax, [0x1000]` loads from address `0x1000`.

Effective addresses can combine a base register, an index register times a scale, and a displacement:

$$\text{address} = \text{base} + \text{index} \times \text{scale} + \text{displacement}$$

Symbol	What it means	Unit
base	register holding a starting address	
index	register counting elements	
scale	size multiplier of the index: 1, 2, 4 or 8	
displacement	constant offset, can be absent	

Most ordinary two-operand instructions allow at most one explicit memory operand, so a direct `mov [dest], [src]` is generally invalid. Specialised string instructions are a separate case (section 4.7).

3.8.1 Common addressing modes

Mode	Intel syntax
Immediate	mov eax, [0x1000]
Register	mov eax, [esi]
Register + offset	mov eax, [esp-8]
Register*width + offset	mov eax, [ebx*4+0xff]
Base + Reg*width + offset	mov eax, [edx+ebx*4+8]

Figure 3.7: Slide listing the common addressing modes in Intel syntax; the first row is labelled “Immediate” (slides, Assembly).

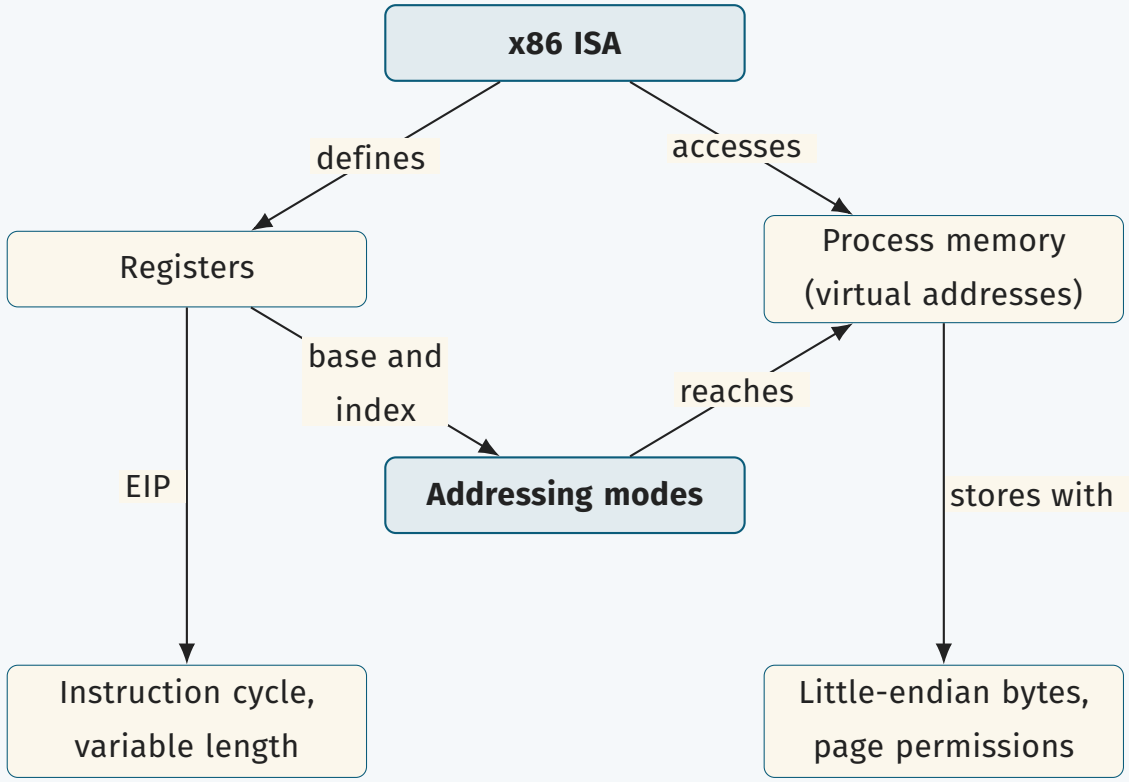
Form	Example	Meaning
Absolute or direct memory address	`mov eax, [0x1000]`	Load from address `0x1000`.
Register indirect	`mov eax, [esi]`	Load from the address held in `ESI`.
Register + displacement	`mov eax, [esp-8]`	Load eight bytes below `ESP`.
Scaled index + displacement	`mov eax, [ebx*4+0xff]`	Use `EBX` as an index into 4-byte elements, then add `0xff`.
Base + scaled index + displacement	`mov eax, [edx+ebx*4+8]`	Use a base, array index and field offset.

> **Correction**
>
> The slide labels `mov eax, [0x1000]` as “Immediate”. The brackets make it a **memory operand**: the value `0x1000` is an absolute address, not the operand itself.

For the last two rows the slide states that the offset is optional. An offset such as `[ebp-0Ch]` often denotes a local variable, but register use alone does not prove the type or meaning of a variable.

Comparing syntaxes. Sistemi di Calcolo expresses the same calculation in AT&T syntax. Intel `[edx+ebx*4+8]` corresponds to AT&T `8(%edx,%ebx,4)`. These expressions correspond to array elements and structure fields, as chapter 10 shows.

Concept map: how the ideas connect



The ISA defines the registers and how instructions access memory. Registers provide base and index, and the addressing modes use them to reach a location in memory. EIP drives the instruction cycle. Memory stores multi-byte values in little-endian order, and its pages have permissions.

Remember

- In assembly, `word` means 16 bits and `dword` 32 bits. x86 is little endian.
- Writing AL changes only one byte of EAX.
- Instructions have variable length (1–15 bytes): a wrong starting byte gives a wrong disassembly.
- Disassembly addresses are virtual addresses, and the memory map on the slide is illustrative.
- Brackets mean memory access, with `lea` as the exception seen in chapter 4.

Review: try to answer without looking

1. How is `0x12345678` laid out in x86 memory, at increasing addresses?
2. After `mov al, 1`, what happens to the upper 24 bits of EAX?
3. Do two threads of the same Windows process have separate handle tables?
4. What is the difference between `mov eax, 0x1000` and `mov eax, [0x1000]`?

Answers, with the section where to find them

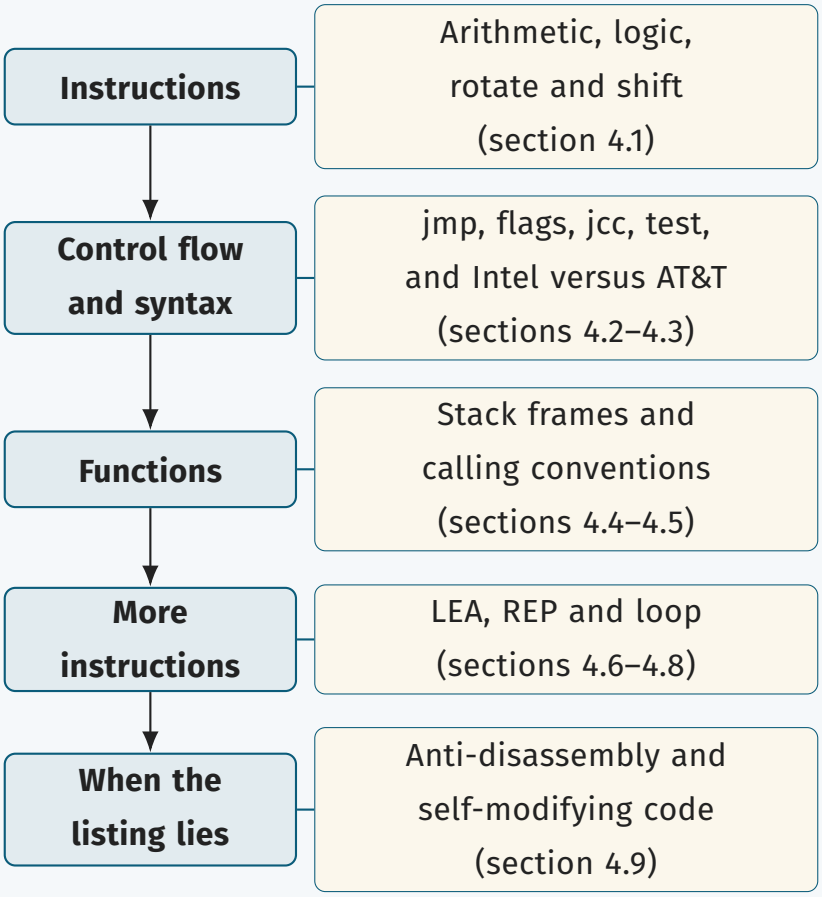
1. `78 56 34 12` (section 3.3).
2. They stay unchanged (section 3.4.1).
3. No: the handle table belongs to the process (section 3.6.1).
4. The first loads a literal, the second reads memory at that address (section 3.8).

Chapter 4

Reading x86 code: instructions, functions and tricks

> **Before you start**
>
> **Central question:** how do we follow data, control flow and function calls through a listing, and when can the listing itself mislead us?
>
> **Prerequisites:** registers, flags and addressing modes (chapter 3).
>
> **Where it leads:** real programs in IDA (chapter 5), the C constructs of chapter 10, the debugger of chapter 11, and the guided exercises of section 13.1.

Chapter structure: what comes, and in which order



The chapter builds up from single instructions to branches and then to whole functions. It adds a few special instructions. It ends with cases in which the listing itself cannot be trusted.

4.1 Basic x86 instructions

The course groups instructions by purpose: **data movement**, **arithmetic and logic**, and **program flow control**.

`mov dest, src` copies a register, memory content or immediate to a register or memory location, without erasing the source. Operand widths must be compatible. `push` and `pop` move values through the stack (section 4.4.2).

An instruction can serve more than one analytical role: arithmetic may calculate an address, alter flags, or obscure a simple data movement.

4.1.1 Arithmetic instructions

The slide says these instructions matter for code analysis for several reasons: computing offsets and function addresses, modifying the stack pointer, and also moving data or updating EFLAGS.

Instruction	Effect on destination	Useful interpretation
`add dest, src`	`dest += src`	Accumulate a value, offset or stack adjustment.
`sub dest, src`	`dest -= src`	Difference; local stack allocation with `sub esp, N`.
`inc dest`, `dec dest`	Increment or decrement by one	Counters and loop conditions.
`not dest`	Invert every bit (ones' complement)	Unlike most arithmetic here, it does not update arithmetic flags.
`neg dest`	Replace with `0 - dest` (two's complement)	Negation.

Arithmetic usually affects the relevant EFLAGS bits, and the exact flags differ by instruction. In particular, `inc` and `dec` leave **CF unchanged**, so do not treat them as identical to `add 1` and `sub 1` in every context. For ESP, `sub esp, N` reserves stack space and `add esp, N` releases it, under the usual downward-growing stack model.

4.1.2 Logic instructions

`and`, `or` and `xor` operate **bit by bit**. Common uses are these:

- zeroing registers: `xor eax, eax` produces zero in EAX and is a common compact idiom;

- extracting bits and checking conditions: an `and` with a mask selects bits, and `test` performs an AND solely to set flags;
- encoding, decoding and obfuscation, especially with XOR.

High-level Boolean `&&` and `||` have different semantics, including short-circuit evaluation. A bitwise `and` or `or` on machine words is not automatically a source-language logical operation. Determine the meaning from the values, flags and subsequent branches.

4.1.3 Rotate and shift instructions

Family	Operation	What to watch
`shr`, `shl`	Unsigned shift right or left, with zero fill	`shr` suits unsigned right shifts; shifted-out bits can affect CF.
`sar`, `sal`	Signed shift: arithmetic right shift preserving the sign bit, and left shift	`sal` is an alias of `shl`; there is no separate sign-preserving left-shift behaviour.
`ror`, `rol`	Rotate within the operand width	Bits wrap around rather than disappearing.
`rcr`, `rcl`	Rotate through the carry flag	CF is part of the rotation state.

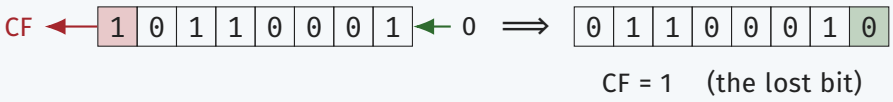
All take the form `op dest, src`. When `src` is absent, a default of 1 is assumed.

> **In plain words (explanation added by this book)**
>
> A shift and a rotate both move the bits of the operand left or right. The difference is what happens to the bits that fall off one end:
>
> - **shift**: the bit that leaves is lost (the last one ends up in CF), and a 0 enters from the other side;
> - **rotate**: the bit that leaves **re-enters from the other side**. Picture the bits on a ring: the rotate turns the ring, so no information is lost;
> - **rotate through carry** (`rcl`, `rcr`): the ring has one extra position, CF.

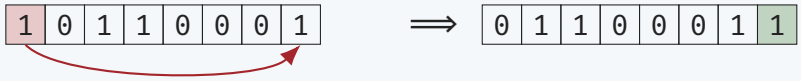
The bit that leaves goes into CF, and the old CF enters from the other side.

Diagram: Before and after: shift and rotate on the 8-bit value 10110001 (AL)

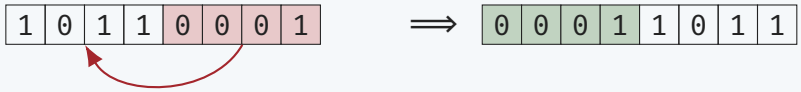
`shl al, 1`: the left bit is lost (goes to CF), a 0 enters on the right



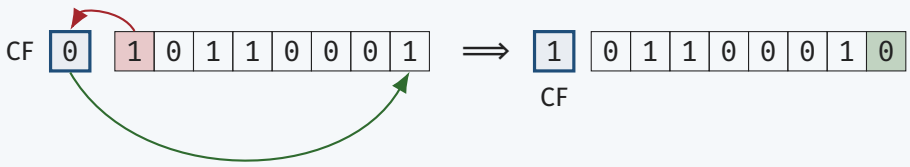
`rol al, 1`: the left bit re-enters on the right



`ror al, 4`: the four right bits re-enter on the left



`rcl al, 1` with CF = 0: the left bit goes to CF, the old CF enters on the right



Red: the bits that leave. Green: the bits that enter. With `rol` nothing is lost: `01100011` still contains all the original bits, only in a different position. With `shl` the leftmost 1 is gone from AL. With `rcl` the 9 positions (CF plus 8 bits) form the ring: a second `rcl` would bring the 1 back from CF into bit 0.

Example: ROR13 API hashing (added by this book)

Many shellcodes and malware samples do not store API names such as `"LoadLibraryA"` as strings. They compute a hash of each exported name and compare it with a constant. For every character the classic loop does:

```
ror edx, 13      ; rotate the hash right by 13 bits
add edx, eax     ; add the current character
```

A loop that applies `ror` or `rol` together with `add` or `xor` to the bytes of a string is a strong hint of name hashing, not random noise.

These instructions commonly occur in hashing, bit packing, cryptographic routines and obfuscation. A sequence such as rotate, add and XOR may be computing a name hash rather than arbitrary noise.

4.1.4 The nop instruction

What nop does (explanation added by this book)

nop (no operation, opcode `90`) does nothing except move EIP to the next instruction: no register, no memory and no flag changes. It still occupies bytes, and that is the point: it fills space in the code without changing what the program computes.

Why it is used for alignment (explanation added by this book)

The CPU does not read code one instruction at a time: it fetches blocks of bytes starting at aligned addresses, commonly 16 bytes at a time. If a loop starts near the end of a block, the first fetch brings in only a few useful bytes. So compilers often place the start of a loop at a multiple of 16, and fill the gap with padding.

The padding must be harmless because execution **runs through it** on its way into the loop, and that is why it is made of nops. Here the first instruction ends at `0x40100E`, and 2 bytes are needed to reach `0x401010`:

```
0x401009  B9 64 00 00 00  mov ecx, 100
0x40100E  66 90           xchg ax, ax      ; padding
0x401010  03 06           add eax, [esi]   ; L: loop start
0x401012  83 C6 04        add esi, 4
0x401015  E2 F9           loop L
```

Alignment only affects speed: the result is the same without it.

Reading nop in the disassembly (explanation added by this book)

- **Longer forms:** one long instruction is decoded faster than many `90`s, so gaps of several bytes use multi-byte encodings. They look like real instructions, but do nothing: `xchg ax, ax` (`66 90`) and `nop dword ptr [eax]` (`0F 1F 00`), which does not even read that address.
- **Between functions:** no one runs through this padding, so it may be made of `int3` (`CC`) instead, as in MSVC builds.
- **Patching:** overwriting an unwanted check or call with `90` bytes removes it without moving the code that follows.
- **NOP sled:** in an exploit, a long run of `90`s before the shellcode means a jump can land anywhere in it and still slide into the payload.
- **Junk code:** obfuscators insert nops and equivalents to change the bytes and hide what the code does.

4.2 Program flow control

Execution proceeds sequentially unless control changes in one of three ways:

- **unconditional branch:** EIP is replaced with a desired address;
- **conditional branch:** EIP is replaced depending on one or more bits of EFLAGS;
- **function calls and returns:** a special type of unconditional branch.

Two primitives are needed: a way to specify the **target** of a branch (static or dynamic), and a way to evaluate a **condition** on the program state. Assembly labels name code locations for humans and assemblers. They are not necessarily symbols retained in a stripped executable.

A useful analysis habit is to draw the incoming and outgoing edges of each basic block. If a target is computed from a register, or points into bytes already disassembled differently, a linear listing may be misleading.

4.2.1 Unconditional branching

An unconditional branch can take as destination an offset, or an absolute address provided as a register or memory operand.

```
jmp 0x4
jmp L
sub eax, 0
jmp eax
L:
add eax, 0
jmp [eax]
```

`jmp L` transfers execution to a labelled location. Direct jumps are commonly encoded using a displacement relative to the address **after** the jump instruction. So a 5-byte `jmp` with displacement `+7` from address `0x1000` reaches `0x100C`, not `0x1007`: the slide's annotation is `7 + 5 = c` (hexadecimal). A disassembler usually displays the resolved destination rather than the raw displacement.

`jmp eax` transfers to the address in EAX. `jmp [eax]` reads a target address from memory at EAX and then transfers there. That pair illustrates the difference between an **indirect register target** and a **memory-indirect target**. Intel's encodings also have near, far and segment-related distinctions: for ordinary flat 32-bit Windows code, focus first on whether the target is direct, register-indirect or memory-indirect.

4.2.2 Conditional branching: flags and cmp

Conditional jumps inspect EFLAGS. The course highlights four **flags**:

- **CF** (carry flag): carry or borrow, meant for errors in unsigned arithmetic;
- **OF** (overflow flag): meant for errors in signed arithmetic;
- **SF** (sign flag): set when the result of an operation is negative;
- **ZF** (zero flag): set when the result of an operation is zero.

`cmp D, S` calculates *D* – *S* for flag-setting purposes, without storing the result or altering the operands. The same bit pattern can be interpreted as signed or unsigned, so the jump selected after `cmp` is crucial.

4.2.3 jcc conditions

A **jcc** instruction transfers control if the condition named by its `cc` suffix is true. Its target is encoded as a relative displacement, not as an absolute address. After `cmp D, S`:

Relation	Signed jump	Opposite	Unsigned jump	Opposite
Equal	`je`, `jz`	`jne`, `jnz`	Same	Same
Greater than	`jg`, `jnle`	`jle`, `jng`	`ja`, `jnbe`	`jbe`, `jna`
Greater than or equal	`jge`, `jnl`	`jl`, `jnge`	`jae`, `jnb`	`jb`, `jnae`

“Above” and “below” denote **unsigned** order. “Greater” and “less” denote **signed** order. For example, after `cmp eax, ebx`, `jg` tests `ZF=0` and `SF=OF`, while `ja` tests `CF=0` and `ZF=0`. Equality uses ZF and does not depend on signedness.

Examine the instruction that last wrote the relevant flags: it may not be the nearest `cmp`, if other flag-setting instructions intervene.

4.2.4 Other comparisons

`test D, D` computes `D & D` for flags without changing D, so `jz` afterward asks whether D was zero.

Malware will not use only `cmp` and `test`. A compiler or malware author can use the flags left by `sub`, `dec` or another operation (the slide notes that optimising compilers do it too). For instance, `dec ecx; jnz L` is both a decrement and

a loop test. Conversely, do not assume every flag was set by every instruction: `lea`, for example, does not set flags.

The operating-systems and computer-systems courses connect comparisons and flags to conditional jumps, and to the `if`, `while` and `for` structures they implement.

4.3 Intel versus AT&T syntax

Both syntaxes describe the same x86 machine instructions but write operands differently. Intel syntax is dominant in the Windows world and is the primary notation in this book.

Aspect	Intel	AT&T
Operand order	`dest, src`	`src, dest`
Register	`eax`	`%eax`
Immediate	`1`	`$1`
Memory	`[ebp-4]`	`-4(%ebp)`
Width when explicit	`byte ptr`, `word ptr`, `dword ptr`; otherwise derived from the register	Suffixes such as `b`, `w`, `l`

For instance, Intel `mov eax, [ebp-4]` corresponds to AT&T `movl -4(%ebp), %eax`. Watch for explicit width when neither operand tells the assembler how many bytes are read or written.

4.4 Functions and stack frames

A **function** is a logical unit that accepts inputs, computes and eventually returns. The slide describes it as a unit of code that controls register values and its own

portion of stack independently of other units: if one unit calls another, the latter should not clobber registers in use.

The CPU does not know C function boundaries. It knows instructions and addresses. To cooperate, caller and callee follow a calling convention and use the stack.

CPUs have few general-purpose registers, so a value can be **spilled** (saved) to the stack and restored later. The stack is conceptually divided into **frames**, one per active function invocation. Recursive calls create distinct frames of the same function. A frame usually includes:

- the arguments of the invocation;
- local variables and other storage, including saved registers;
- the return address, where execution resumes upon function exit.

A normal-looking prologue or epilogue is useful evidence of a function boundary, but optimised or malicious code need not follow the pattern.

4.4.1 Local storage

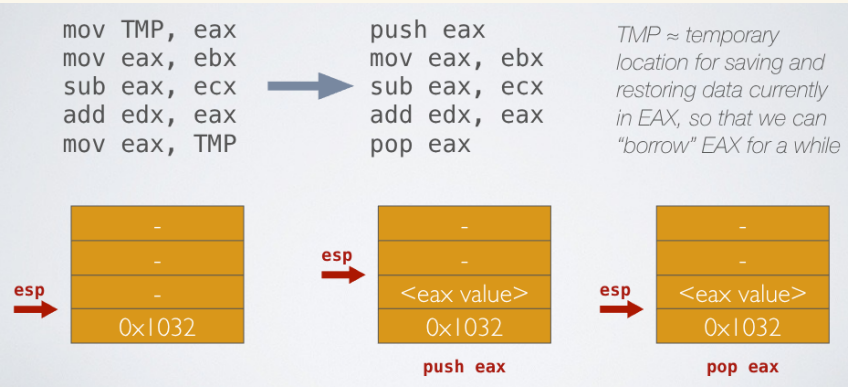


Figure 4.1: Slide on local storage: push eax before borrowing EAX and pop eax afterward, with the stack before and after each (slides, Assembly).

Suppose EAX holds a value that must survive, no register is free, and we need to add to EDX the difference between EBX and ECX. A temporary location TMP could be used, or the stack:

```
push eax
mov  eax, ebx
sub  eax, ecx
add  edx, eax
pop  eax
```

The push saves the original EAX value and pop restores it. EDX retains the result. This is an example of **register preservation and temporary storage**, not proof that a source-level variable named TMP existed. Track stack depth as well as register values while reading such sequences.

4.4.2 Push and pop

For a 32-bit push, `push src` first decrements ESP by 4 and then writes four bytes at `[esp]`. `pop dest` first reads the four bytes at `[esp]` and then increments ESP by 4. Conceptually:

```
push ebp          ; sub esp, 4  /  mov [esp], ebp
pop  edx          ; mov edx, [esp]  /  add esp, 4
```

These are **effect** equivalences for the illustrated operands, not a claim that the replacement instructions always have identical flags or edge-case behaviour. The stack grows toward lower addresses on push, so the numerical value of ESP decreases. Values at old stack addresses may remain in memory after a pop, but are no longer part of the active stack content.

4.4.3 Using the base pointer

A common frame setup uses EBP as a stable anchor.

```
mov ebp, esp
sub esp, 8
mov [ebp-8], 0x1111
mov [ebp-4], 0xabadcafe
```

```
...
mov eax, [ebp-8]
```

The function accesses two 4-byte locals at `[ebp-4]` and `[ebp-8]` while ESP changes for further pushes or calls. The slide's image: accessing local variables with ESP is like chasing a moving target. These offsets make sense only in the context of this chosen frame layout. Arguments often appear at **positive** EBP offsets once the return address and saved EBP are accounted for.

If code logic uses EBP differently, dereferencing ESP plus an offset is the only option left. Optimised functions may omit EBP as a frame pointer and address locals relative to ESP, or use registers instead. The ESP offset of a fixed local can then change as the stack depth changes.

4.4.4 Call and ret

`call target` pushes the address of the **next instruction**, called the return address, and transfers control to `target`. `ret` pops that return address into EIP. If the callee corrupts the stack, `ret` can jump to an unintended address. A tail call may use `jmp` instead of `call` followed by `ret`, so a logical function call need not look exactly like the basic pattern.

> **`ret` is, in practice, a `pop eip` (explanation added by this book)**
>
> `ret` reads the address on top of the stack, puts it in EIP and does `esp += 4`; execution continues from that address. Formulas like "`pop eip` + `jmp eip`" describe the effect, but should not be taken literally:
>
> - **`pop eip` does not exist:** EIP cannot be used as an operand of `pop` or `mov`, so this is only a description of the behaviour.
> - **`jmp eip` would be redundant:** writing to EIP already is a jump, because the CPU always executes the instruction at the address held in EIP.
>
> A nearly real equivalent, which however overwrites a register, is:

```
pop ecx      ; ecx = return address, esp += 4
jmp ecx      ; eip = ecx
```

Instruction	What it does
`call addr`	pushes the address of the next instruction, then jumps to `addr`
`ret`	`eip = [esp]`, then `esp += 4`
`ret n`	like `ret`, then `esp += n` (also removes the arguments, as in `stdcall`)

On x86-64 the same holds with RIP and RSP, and the increment is 8. Flags are not affected.

`ret`, `retn` and `retf` (explanation added by this book)

`retn` (return near) is another name for the plain `ret`: same instruction, same bytes. Which one you read depends on the tool: IDA, for example, often prints `retn`. The name exists to set it apart from `retf` (return far), the only real difference.

Instruction	Opcode	Effect
`ret` / `retn`	`C3`	pops EIP
`ret n` / `retn n`	`C2 imm16`	pops EIP, then adds `n` to ESP (`stdcall`)
`retf`	`CB`	pops EIP and CS
`retf n`	`CA imm16`	pops EIP and CS, then adds `n` to ESP

In malware analysis. `retf` is rare in ordinary 32-bit code, so stop and check it. It may be a mode switch, as in the Heaven's Gate technique, where the code loads segment `0x33` to pass from 32 to 64 bits. Or the disassembly may be wrong and those bytes are data, in which case the nearby instructions usually make no sense either.

What `ret` does beyond a `pop eip` (explanation added by this book)

- **Variants:** `ret n` also adds n to ESP, and `retf` (far return) pops CS as well as EIP.
- **Branch prediction:** the CPU pairs each `call` with its `ret` using the return stack buffer, an internal stack that predicts the return address. This changes performance only, not the result. It is why `pop ecx; jmp ecx` is usually slower: the return is not paired with the `call`, and the predictions of later `ret`s can go wrong.
- **Hardware protection:** with Intel CET enabled, `ret` compares the address on the stack with a copy kept in a shadow stack, and raises an exception if they differ. This is a defence against ROP.

`ret` trusts the stack (explanation added by this book)

`ret` jumps to whatever it finds on top of the stack, and some techniques rely on this:

- **Disguised jump:** `push addr` followed by `ret` is equivalent to `jmp addr`. It is used to obfuscate the control flow and confuse disassemblers.
- **Buffer overflow and ROP:** overwriting the return address on the stack hijacks execution.

Tail call: what it is (explanation added by this book)

A **tail call** is a call that is the **very last thing** a function does before returning:

```
int f(int x) {
    ...
    return g(x);   // last operation of f
}
```

The plain translation would be `call g` followed by `ret`: after `g` returns, `f` has nothing left to do except return in turn. The compiler can therefore

replace both with a single `jmp`:

Without optimisation

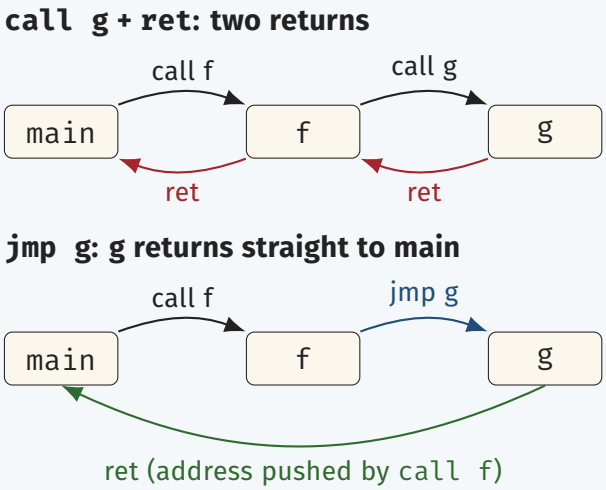
```
f:
    ...
    call g
    ret
```

With a tail call

```
f:
    ...
    jmp g
```

This works because, after `jmp g`, the top of the stack still holds the **return address of whoever called f**. When g executes its `ret`, it goes back **directly to the caller of f**, skipping one step. One `call`, one `ret` and one stack frame are saved. In recursive functions this also avoids filling the stack.

Diagram: Where each `ret` goes, with and without a tail call



Remember

Why a tail call matters in malware analysis:

- a `jmp` to the start of another function can be a **logical call**, not a branch inside the same function;
- the disassembler can get function boundaries wrong and merge g into f;
- cross-references that look only at `call` instructions miss callers that reach g with `jmp`.

> If a function ends with a `jmp` to another function instead of a `ret`, it is very likely a tail call.

Functions exist only at a logical level. Code is laid out in memory without any separators, so function start and end addresses are inferred from control flow and calling behaviour. Adversaries can scramble those cues, overlap instructions, or direct a disassembler to a false boundary.

4.4.5 Function call: an example

```
push eax            myfun:
add eax, 1          mov ebp, esp
call myfun          add esp, 8
sub eax, 1          ...
                    sub esp, 8
                    ret
```

The slide traces this code and shows how the call places its return address on top of the caller's saved EAX. The key invariant is:

1. Immediately before the call, ESP points at the caller's current top-of-stack value.
2. `call` changes ESP by -4 and puts the continuation address at `[esp]`.
3. The callee may move ESP for its own storage, but restores the necessary stack position before `ret`.
4. `ret` reads the continuation address and changes ESP by +4 (plus any explicit immediate cleanup).

The abbreviated drawing is meant to show the transfer of control and the saved address. Do not treat its sample prologue as a universal template for all functions.

> **To verify**
>
> As transcribed, the slide's `myfun` uses `add esp, 8` on entry and `sub esp,`

> 8 before `ret`. This is balanced, so `ret` still finds the return address, but under the downward-growing stack it does not reserve local space: section 4.1.1 uses `sub esp, N` for that. The sequence would normally be expected the other way round. Check the slide if this example is needed in detail. This remark is mine; it is not in the slides or in the notes.

Saving registers for a function call follows an ABI contract. The OS preserves an execution context during a context switch. These are related uses of registers and stacks, with different purposes.

4.5 Calling conventions

A **calling convention** answers how arguments are passed and in which order, which registers survive the call, where a result is returned, and who realigns ESP. It is a contract between caller and callee.

Internal malware routines can use custom rules, but a direct call into a standard Windows API has to satisfy that API's ABI. A mismatch can produce wrong parameters or a misaligned stack, and may become evident only after returning.

Different conventions can be used in the same program. For these 32-bit examples, stack arguments are typically placed right to left: pushing `10` and then `30` gives a function invoked as `subtract(30, 10)` access to its first argument nearer the return address. The return address is pushed by `call`, not by the source code's argument setup.

4.5.1 cdecl and stdcall

Feature	**cdecl** (C programs)	**stdcall** (Win32 API)
Stack arguments	Right to left	Right to left
Volatile (caller-save) registers	EAX, ECX, EDX	EAX, ECX, EDX
Typically preserved (callee-save) registers	EBX, ESI, EDI, EBP	EBX, ESI, EDI, EBP
Who removes stack arguments?	Caller, often `add esp, N`	Callee, often `ret N`
Typical 32-bit return value	EAX	EAX

The callee needs to save a nonvolatile register only if it actually changes it. `ret 8` pops the four-byte return address **and then** advances ESP by eight more bytes, consuming two dword arguments. Its total effect on ESP is **+12**: in general, `ret N` adds *N* + 4 bytes to ESP. These conventions are for 32-bit Windows, not for the 64-bit Windows ABI. The System V i386 material of Sistemi di Calcolo is a useful comparison, keeping the ABI distinction explicit.

4.5.2 Example: cdecl

```
; caller                    ; int __cdecl__ subtract(int arg1,
  int arg2)
mov ecx, 20                 ; { return arg1-arg2; }
...                         subtract:
push ecx                    push ebp
push 10                     mov ebp, esp
push 30                     mov eax, [ebp+8]
call subtract               mov edx, [ebp+12]
add esp, 8                  sub eax, edx
pop ecx                     pop ebp
```

```
cmp eax, ecx                ret
jnz ErrorLabel
```

The caller saves ECX because it will compare the return value with ECX after the call. It then pushes `10` (second argument) and `30` (first argument). In `subtract`, the familiar prologue `push ebp; mov ebp, esp` makes `[ebp+8]` the first argument and `[ebp+12]` the second. The function computes 30 - 10 = 20 in EAX, restores EBP and returns with plain `ret`.

The caller's `add esp, 8` discards the two arguments, and `pop ecx` restores the previously saved `20`. `cmp eax, ecx` tests whether the returned `20` matches.

After	Change in ESP	Stack, from the top
`push 30`	-12 overall (ecx, 10, 30)	30, 10, stored ecx
`call subtract`	-4	return address, 30, 10, stored ecx
`push ebp`	-4	stored ebp, return address, 30, 10, stored ecx
`pop ebp`	+4	return address, 30, 10, stored ecx
`ret`	+4	30, 10, stored ecx
`add esp, 8`	+8	stored ecx
`pop ecx`	+4	back to the start

Separate the **8-byte argument cleanup** from the **4-byte register restoration** when tracing ESP.

4.5.3 Example: stdcall

Argument passing, register preservation and the EAX result are the same as in the previous example. The difference is the callee's `ret 8`: on return, it discards both 4-byte arguments. The caller then restores its saved ECX with `pop ecx`, **without** an additional `add esp, 8`.

The same example in stdcall (example added by this book)

This is the code of section 4.5.2 rewritten for `stdcall`. There is only one difference, who removes the arguments, and it shows up in two places, marked in **red**:

```
; int __stdcall subtract(int arg1, int arg2)
; { return arg1-arg2; }

; caller                    ; callee
mov ecx, 20                 subtract:
...
push ecx                    push ebp
push 10                     mov ebp, esp
push 30                     mov eax, [ebp+8]
call subtract               mov edx, [ebp+12]
; no add esp, 8             sub eax, edx
pop ecx                     pop ebp
cmp eax, ecx                ret 8
jnz ErrorLabel
```

ESP ends up in the same place in both conventions: `ret 8` adds +12, exactly what `ret` (+4) and `add esp, 8` (+8) add together in `cdecl`. Only the instruction that does the cleanup changes.

Correction

The `stdcall` slide still shows `add esp, 8` in the caller, even though the slide's `ret 8` has already cleaned up the arguments. Executing both in this simple example would over-adjust ESP. Use the stated `stdcall` rule and the sequence described above when reasoning about the stack.

4.5.4 Caller-save versus callee-save

Caller-save means the caller must preserve a volatile register **if it needs the old value after the call**: usually EAX, ECX and EDX here. **Callee-save** means that

a function that changes a nonvolatile register must restore its original value before returning: usually EBX, ESI, EDI and EBP. The slide lists EDI, ESI and EBX as the typical callee-saved registers pushed in the prologue; EBP is handled by the frame setup itself.

"Save" does not require a literal `push`: a compiler may use other storage, and a register that is never changed needs no restoration.

The slide adds that compilers often prefer callee-save registers for performance, and use caller-save ones too when register pressure gets high. In custom code you can find any assortment. When reconstructing a function, track which values are guaranteed across a call and which might be overwritten.

4.5.5 More Windows conventions

The slides describe two less popular 32-bit Microsoft conventions.

Convention	Argument placement	Stack cleanup	Typical use
fastcall	First two arguments (left to right) in ECX and EDX; others on the stack, right to left	Callee, as in stdcall	Register-based calls in 32-bit code
thiscall	`this` pointer in ECX; arguments on the stack, right to left	Callee with a fixed number of arguments (as stdcall); caller with a variable number (as cdecl)	Visual C++ non-static member functions

These are **typical patterns**, not a promise that every compiler, variable-argument case or custom binary uses exactly the same register allocation. EAX, ECX and EDX are generally volatile in this 32-bit context, and the caller saves them when necessary. For a suspected member function, ECX carrying an object pointer is a clue: look for field accesses such as `[ecx+offset]` to support it.

Example: fastcall (example added by this book)

A function with three arguments, called as `sum3(30, 10, 5)`. In **red** is what changes compared with `stdcall`: the first two arguments travel in registers, so `ret` removes only the third.

```
; int __fastcall sum3(int a, int b, int c)
; { return a + b + c; }

; caller
push 5                  ; c: third argument, on the stack
mov edx, 10             ; b: second argument, in EDX
mov ecx, 30             ; a: first argument, in ECX
call sum3               ; no add esp: the callee cleans up

; callee
sum3:
push ebp
mov ebp, esp
mov eax, ecx            ; eax = a
add eax, edx            ; eax += b
add eax, [ebp+8]        ; eax += c, the first stack argument
pop ebp
ret 4                   ; removes only c
```

The result in EAX is 30 + 10 + 5 = 45. Two consequences:

- `[ebp+8]` is c, not a: arguments passed in registers take no stack slot.
- With at most two arguments, nothing is on the stack and the function ends with a plain `ret`, so it can look as if it took no arguments at all.

Example: thiscall (example added by this book)

A member function of a class with two 4-byte fields, `id` at offset 0 and `value` at offset 4, called as `counter.add(5)` on a local object. In **red** is what changes compared with `stdcall`: the hidden `this` pointer travels in ECX.

```
; class Counter { int id; int value; ... };
; int Counter::add(int n) { value += n; return value; }

; caller
push 5                ; n: on the stack
lea ecx, [ebp-8]      ; this: address of the object
call Counter_add      ; no add esp: the callee cleans up

; callee
Counter_add:
push ebp
mov ebp, esp
mov eax, [ebp+8]      ; eax = n
add eax, [ecx+4]      ; eax += this->value
mov [ecx+4], eax      ; this->value = eax
pop ebp
ret 4                 ; removes n
```

`this` never appears among the arguments in the C++ source, but in assembly it is a real parameter. In the disassembly, the `lea ecx` just before the `call` and the `[ecx+4]` accesses are what suggest an object and one of its fields.

With a variable number of arguments, the caller cleans up as in `cdecl`. In Visual C++, `this` is then also pushed on the stack instead of being passed in ECX.

Telling them apart in the disassembly (added by this book)

What you see	Points to
Callee reads ECX and EDX before writing them; caller loads both just before the `call`	`fastcall`
Callee reads only ECX before writing it, as a base: `[ecx+offset]`; caller loads it with an object address	`thiscall`
Arguments only on the stack; callee ends with `ret N`	`stdcall`
Arguments only on the stack; caller does `add esp, N` after the `call`	`cdecl`

These are clues, not proof: optimised or custom code can break these patterns, so check both the callee and its callers.

4.5.6 EBP, enter and leave

A typical **prologue** is:

```
push ebp
mov ebp, esp
sub esp, N       ; optional, for locals
```

A typical epilogue is:

```
mov esp, ebp
pop ebp
ret
```

To leave a function, ESP must point to the caller's return address, and EBP comes to the rescue. `leave` abbreviates the first two epilogue instructions. `enter` can construct a frame in a single instruction, with an optional immediate subtracted from ESP for locals (and an operand for nesting), though modern compiler output often uses the simpler explicit prologue.

`leave` is unnecessary when ESP has never moved away from the position that lets `pop ebp` reach the saved value. Conversely, `pop ebp` alone is unsafe if locals or other live stack allocations still sit above that saved value. Trace the actual stack depth rather than assuming an epilogue from its appearance.

enter and leave: what they replace (explanation added by this book)

`enter` and `leave` are shorthands: each one does the work of several instructions. The same function with 8 bytes of locals, written both ways, with the shorthands in **red**:

```
; explicit              ; with enter and leave
push ebp                enter 8, 0
mov ebp, esp
sub esp, 8
...                     ...
mov esp, ebp            leave
pop ebp
ret                     ret
```

- `enter N, 0` equals `push ebp; mov ebp, esp; sub esp, N`, where `N` is the number of bytes for the locals. The second operand is the nesting level, used by languages with nested functions; C code uses `0`.
- `leave` equals `mov esp, ebp; pop ebp`. It takes no operands.
- `leave` does **not** return: `ret` is still needed after it.

enter and leave, step by step (example added by this book)

Suppose that on entry ESP is `0x1000`, the address of the return address pushed by `call`. The function body also leaves one value on the stack, with a `push` that is never popped:

After	ESP	EBP	What happened
function entry	`0x1000`	caller's	ESP points at the return address
`enter 8, 0`	`0x0FF4`	`0x0FFC`	caller's EBP saved at `0x0FFC`; 8 bytes reserved below it
`push eax`	`0x0FF0`	`0x0FFC`	ESP moves, EBP does not
`leave`	`0x1000`	caller's	ESP back to `0x0FFC`, then the pop restores EBP
`ret`	`0x1004`	caller's	EIP = return address

This shows why `leave` is convenient: EBP never moved during the body, so `mov esp, ebp` throws away locals and leftovers at once, however much ESP has moved in the meantime.

Without locals, as in `subtract` in section 4.5.2, ESP is still at the saved EBP at the end of the function: `pop ebp` is enough and `leave` is not needed.

In practice, you will meet `leave` often in compiled code and `enter` rarely, because compilers prefer the explicit prologue.

4.6 LEA

LEA (Load Effective Address), written `lea dest, [expression]`, computes the effective address expression and stores the result in a register. It does not dereference memory and does not change flags. Data movement and arithmetic instructions dereference a given address and operate on the content of the location; LEA simply computes an address. So it has two uses: pointer arithmetic and ordinary arithmetic.

Example 1. Suppose EBX=0x1000, and memory at 0x1004 contains 0xABADCAFE.

- `mov eax, [ebx+4]` loads 0xABADCAFE into EAX.
- `lea eax, [ebx+4]` places 0x1004 in EAX.

Example 2. Operands may also not be addresses. With EAX=10 and EBX=4, compute $C = A + 2B - 1$ in one instruction: `lea ecx, [eax+2*ebx-1]` gives $10 + 2 \cdot 4 - 1 = 17$, even if address 17 is never meant to be dereferenced.

Correction

The slide says that the multiplication width for LEA has to be 1, 2 or 4. The encoded index scale of x86 can also be **8**. LEA is restricted to its address-expression form rather than arbitrary multiplication: it can combine a base, a scaled index and a displacement, subject to register and encoding constraints.

Why use `lea` instead of `mov` + `add`? (explanation added by this book)

`mov eax, ebx+4` does not exist, while `mov eax, ebx` followed by `add eax, 4` gives the same value in EAX. So `lea` does nothing impossible: it is a shortcut with two concrete advantages.

1. Several operations in a single instruction. With `ebx+4` the gain is small, but it shows with the slide's $C = A + 2B - 1$:

```
; with lea
lea ecx, [eax+2*ebx-1]

; without lea
mov ecx, eax
add ecx, ebx
add ecx, ebx
sub ecx, 1
```

With EAX=10 and EBX=4, both give `ECX = 17` and leave EAX and EBX intact, but `lea` takes one line instead of four.

2. It does not change the flags. `add` and `sub` update EFLAGS, `lea` does not. This matters when a computation sits between a `cmp` and its jump:

```
cmp eax, ebx
lea ecx, [ecx+4]   ; the flags of cmp stay intact
je  equal          ; still depends on cmp

cmp eax, ebx
add ecx, 4         ; overwrites the flags of cmp
je  equal          ; now depends on add: bug
```

What `lea` is used for in practice (explanation added by this book)

Computing an address without reading memory, like `&` in C. The two instructions have the same operand: `mov` reads the value at that address, `lea` gives the address itself.

```
int *p = &arr[i];     // lea eax, [edx+ebx*4]
int  v =  arr[i];     // mov eax, [edx+ebx*4]
```

Fast arithmetic, even on values that are not addresses. Compilers use it often, for example to multiply by 5:

```
lea eax, [ebx+ebx*4]   ; eax = ebx * 5
```

When analysing malware, remember that the square brackets of `lea` **do not indicate a memory access**, and that the result is not necessarily a pointer: it can be a plain number.

4.7 REP and string instructions

String instructions compactly process buffers. Their names include an element size.

Instruction	Operation	C analogue
`movsb`	Copy one byte from `[esi]` to `[edi]`	`memcpy`
`stosb`	Store AL to `[edi]`	`memset`
`cmpsb`	Compare bytes at `[esi]` and `[edi]`	`memcmp`
`scasb`	Compare AL with the byte at `[edi]`	`memchr`

EDI holds the first buffer (destination), ESI the second (source), AL the byte to set or search, and ECX the repetition count. ESI and EDI update after each element, with direction controlled by **DF**. `cld` commonly selects increasing addresses.

`rep` repeats while ECX is nonzero, which suits `rep movsb` (copy) or `rep stosb` (fill). For comparison and search, **`repe/repz`** and **`repne/repnz`** additionally test ZF, allowing early termination.

The slide says these instructions can implement `memcmp`, `memset`, `memcpy` and `memchr`. But the exact count, stopping condition, return value and treatment of overlap must be checked before claiming equivalence to a C library function.

What the prefix does (explanation added by this book)

A string instruction alone handles **one** element. The prefix turns it into a loop, in this order:

```
while (ecx != 0) {
    run the instruction     // movsb, stosb, cmpsb, scasb
    ecx--
    repe/repz:   if (ZF == 0) stop     // they differ
    repne/repnz: if (ZF == 1) stop     // they are equal
}
```

`movsb` and `stosb` do not compare anything, so they use plain `rep` and stop only at ECX = 0. `cmpsb` and `scasb` compare and set ZF, so they use the variants that can stop early. The names read as “repeat while equal” (`repe`) and “repeat while not equal” (`repne`).

Instruction	C	Data flow	Stops when
`rep movsb`	`memcpy`	memory to memory: `[esi]` to `[edi]`	ECX = 0
`rep stosb`	`memset`	register to memory: AL to `[edi]`	ECX = 0
`repe cmpsb`	`memcmp`	compares `[esi]` with `[edi]`	ECX = 0 or first difference
`repne scasb`	`memchr`	compares AL with `[edi]`	ECX = 0 or byte found

Copy and fill, with 16 bytes (examples added by this book)

Copy, `memcpy(dst, src, 16)`: each byte can be different.

```
mov edi, dst
mov esi, src
mov ecx, 16
rep movsb        ; [edi] = [esi], esi++, edi++, ecx--
```

Fill, `memset(dst, 0, 16)`: the same byte, written 16 times. Only EDI moves, because AL does not change and there is no source in memory.

```
mov edi, dst
xor eax, eax     ; al = 0
mov ecx, 16
rep stosb        ; [edi] = al, edi++, ecx--
```

Compare and search, with 16 bytes (examples added by this book)

Compare, `memcmp(a, b, 16)`: after the loop, ZF tells how it ended.

```
mov esi, a
mov edi, b
mov ecx, 16
repe cmpsb       ; compare; esi++, edi++, ecx--
jne different    ; ZF = 0: stopped on a difference
```

Search, `memchr(buf, 'x', 16)`: EDI ends one byte **after** the match, because it advances after every comparison.

```
mov edi, buf
mov al, 'x'
mov ecx, 16
repne scasb      ; compare al, [edi]; edi++, ecx--
jne not_found    ; ZF = 0: count exhausted, no match
dec edi          ; edi = address of the match
```

Larger elements (explanation added by this book)

The suffix gives the size: `b` is 1 byte, `w` is 2 and `d` is 4. Compilers often use `rep movsd` and `rep stosd`, which handle 4 bytes at a time; `stosd` writes EAX. ECX counts **repetitions, not bytes**: for 16 bytes it is 4.

DF and cld: the direction (explanation added by this book)

DF (Direction Flag) is a bit of EFLAGS that decides which way ESI and EDI move in string instructions. **`cld`** (Clear Direction Flag) sets it to 0, **`std`** (Set Direction Flag) sets it to 1.

DF	Set by	ESI and EDI after each element
0	`cld`	increase: towards higher addresses
1	`std`	decrease: towards lower addresses

The step is the element size: 1 byte for `movsb`, 2 for `movsw`, 4 for `movsd`.

Why go backwards? When source and destination overlap and the destination is further ahead. Copying 16 bytes from `0x1000` to `0x1004` forwards, the byte written at `0x1004` is also the fifth to be read, and it would already have changed. Starting from the end avoids this, and it is the case `memmove` has to handle. ESI and EDI must then point at the **last** byte of each area:

```
mov esi, src+15
mov edi, dst+15
mov ecx, 16
std                 ; DF = 1: go backwards
rep movsb           ; [edi] = [esi], esi--, edi--, ecx--
cld                 ; restore DF = 0 before returning
```

In the disassembly. Standard calling conventions expect DF = 0 on entry to and exit from a function, which is why `rep movsb` often appears with no `cld` before it. An `std` is rare: if you meet one, read ESI and EDI as end-of-area addresses.

4.8 The loop instruction

Loops are normally encoded using jump instructions, and compilers avoid the special `loop` instruction for efficiency. A reverse engineer must still recognise the compact family and its implicit use of ECX.

`loop L` decrements ECX and branches to L when the resulting ECX is nonzero.

```
mov ecx, 10
L: add eax, 1
loop L
```

Here the addition executes ten times if execution enters at L. The instruction does not itself update the arithmetic flags.

- `loope` / `loopz` jumps when ECX is nonzero and ZF is 1.
- `loopne` / `loopnz` jumps when ECX is nonzero and ZF is 0.

The flags in those variants must come from a relevant earlier instruction, such as a comparison in the loop body.

Why loopz wants ZF = 1 (explanation added by this book)

The trap. ZF means “the result was zero”, and when that is true its value is 1. So the z of `loopz` means “keep looping while the result is zero”, which is ZF = 1. The e of `loope` says the same thing: `cmp` subtracts, so equal values give a result of 0 and set ZF to 1.

Last result	ZF	After a `cmp`
zero	1	the values are equal
not zero	0	the values differ

When each variant stops (example added by this book)

Instruction	Stops when
`loop`	ECX = 0
`loope` / `loopz`	ECX = 0, or ZF = 0 (the values differ)
`loopne` / `loopnz`	ECX = 0, or ZF = 1 (the values are equal)

Example. Scan up to 10 bytes while they are equal to 5:

```
mov esi, buf
mov ecx, 10
L: lodsb          ; al = [esi], esi++
cmp al, 5         ; equal to 5? then ZF = 1
loope L           ; ecx--, repeat if ecx != 0 and ZF = 1
```

This is the same logic as `repe` and `repne` in Section 4.7.

4.9 Anti-disassembly

Anti-disassembly means hindering the static reconstruction of instructions. The two sample programs of the course show why instruction **boundaries** matter.

4.9.1 antidisassembly.exe: a jump between instructions

The sample uses the **`call $+5`** pattern. This call targets the address immediately after its own five-byte encoding, so it effectively pushes that next address. The following `pop eax` retrieves it, which gives the code a position-relative reference to itself. The slides say that we will meet this pattern again in a few weeks.

Step by step: Following the jump statically

1. `call $+5` pushes the address of the following `pop eax`.

2. `pop eax` writes that address to EAX: its value is `0x401468`.
3. The code adds `0x0A`: `0x401468 + 0x0A = 0x401472`.
4. The code jumps there, and execution continues at `0x401472`.
5. Older versions of the IDA disassembler interpret those bytes as part of the instruction starting at `0x401470`.

The static interpretation therefore conflicts with the actual jump target: the disassembler chose a "wrong" initial position to look for valid instructions.

The repair is to examine the raw bytes, undefine the incorrect instruction in the analysis database, mark `0x401472` as code, and rebuild the interpretation; once the code is displayed you can also create a function there. A debugger provides execution evidence, but static arithmetic on the leaked address is enough in this example.

The broader lesson is that a disassembly listing is an **analysis of bytes from chosen entry points**, not a ground-truth segmentation of the executable. Follow computed targets, and check whether they enter overlapping or previously misclassified bytes.

4.9.2 smc.exe: self-modifying code

Self-modifying code takes this to the next level. Instructions and data reside in the same memory; whether code bytes can actually be modified or executed depends on the mappings and permissions of section 3.7.

Step by step: What smc.exe does

1. As before, `call $+5` leaks a position.
2. The code computes two addresses and stores them in EAX and EDX.
3. It has called `VirtualProtect` so that the relevant code bytes can be edited.
4. A loop swaps memory contents in four-byte chunks, with byte order

> affected by little-endian representation.
>
> 5. Once the loop reaches the nop, the bytes about to execute no longer match the original static disassembly.

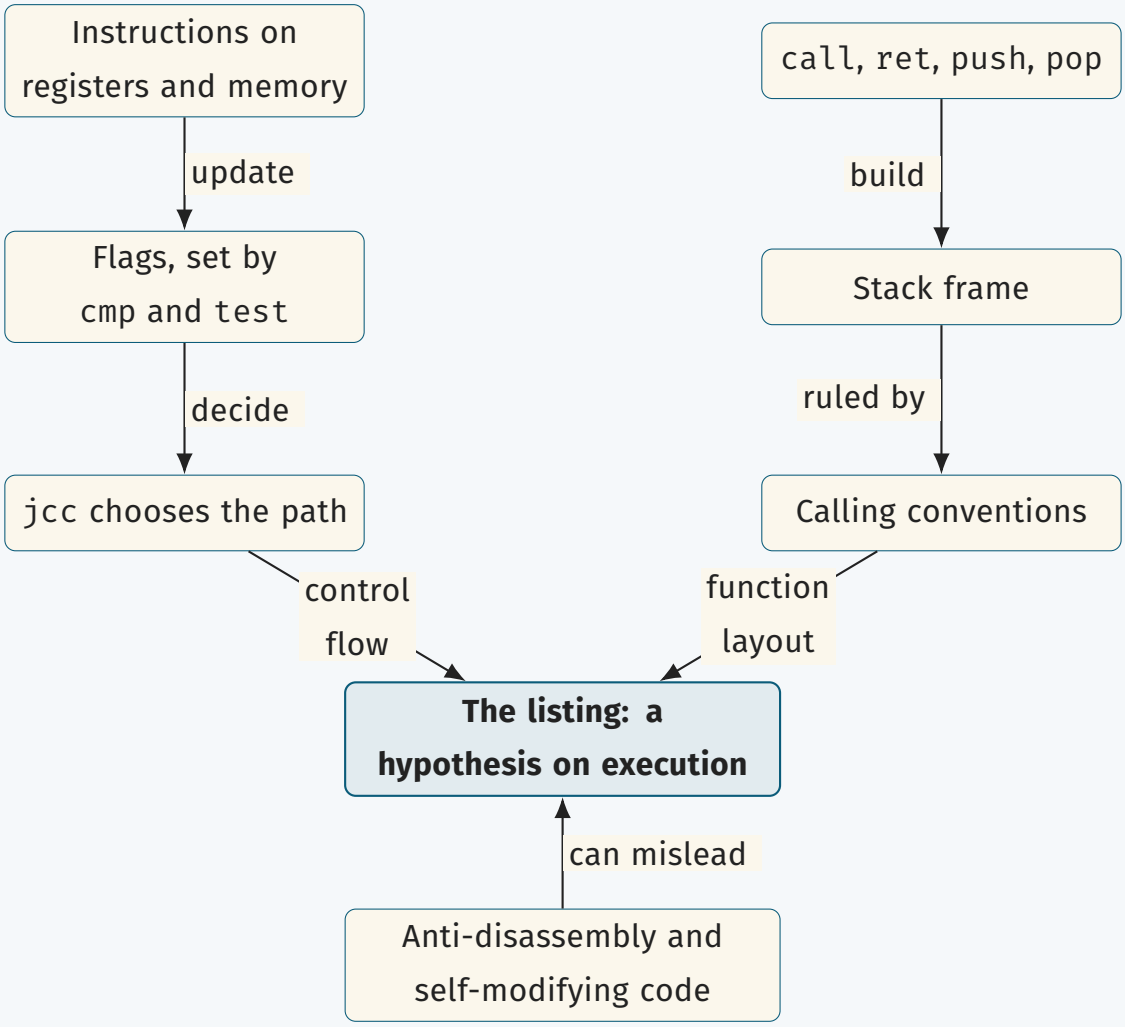
The slide suggests using a debugger, or working by hand if you prefer. It reports the outcome:

- the apparent `pop ebp; ret` sequence is gone;
- undefine the dword `1400EB02h` at address `0x401488` to inspect how the bytes are reinterpreted: the two bytes `02 eb` become, after the swap, a `jmp $+2` placed next to the nop;
- this jump skips the swapped `ret; pop ebp` sequence and reaches the real behaviour;
- the code that gets executed is the same as in `antidisassembly.exe`.

In encoding terms, the jump is the **`EB 02`** short-jump form: opcode `EB` jumps by a signed 8-bit displacement measured from the end of the two-byte instruction, so `02` skips the next two bytes. Keep the displayed dword value separate from the order in which bytes are fetched as instructions: the swap and little-endian storage affect that order.

To reconstruct this confidently for the actual binary, compare bytes **before and after** the write loop, track each swap and address, and disassemble from the runtime target. Static control-flow analysis alone is insufficient when instructions are rewritten in memory. The slide provides the intended outcome, but the full byte-by-byte reconstruction requires the accompanying executable, which is not in the slides.

Concept map: how the ideas connect



On the left, instructions update the flags, and the flags decide which path a conditional jump takes. On the right, call, return, push and pop build the stack frame, and calling conventions set its rules. Together they let you read control flow and function layout from a listing. That reading stays a hypothesis, because anti-disassembly and self-modification can mislead it.

In short

The ISA tells you what instructions mean. Registers and memory addressing tell you **which values** they use. Flags, branches, calls and the stack tell you **where execution goes and how functions cooperate**. Disassemblers and decompilers turn bytes into readable hypotheses. Overlapping instruction boundaries and self-modification show why those hypotheses must be checked against bytes and runtime state. Chapter 5 applies this reasoning

to real function layouts.

Review: try to answer without looking

1. After `cmp eax, ebx`, which jump treats the values as unsigned: `jg` or `ja`?
2. What is the total effect of `ret 8` on ESP?
3. What does `lea eax, [ebx+4]` read from memory?
4. What stops `repne scasb`?
5. What does `call $+5` followed by `pop eax` give the code?

Answers, with the section where to find them

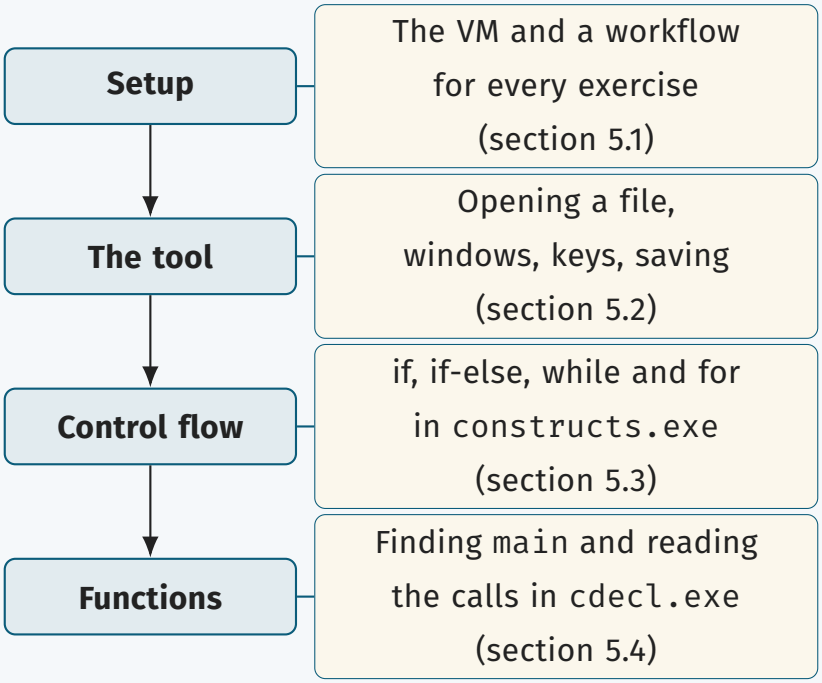
1. `ja` (section 4.2.3).
2. +12 (section 4.5.1).
3. Nothing: it only computes the address (section 4.6).
4. A match or an exhausted count (sections 4.7 and 13.1).
5. Its own position in memory (section 4.9.1).

Chapter 5

Reading real code in IDA

> **Before you start**
>
> **Central question:** what do the registers, the memory layout and the instructions of the previous two chapters look like in a real compiled program, opened in a real tool?
>
> **Prerequisites:** registers, process memory and addressing (chapter 3); instructions, flags, stack frames and calling conventions (chapter 4).
>
> **Where it leads:** the C constructs of chapter 10, the debugger side of IDA in chapter 11, and the remaining guided exercises of section 13.1.

Chapter structure: what comes, and in which order



The chapter goes from the tool to the code. It first prepares the machine and IDA, then reads the control structures of one program, and finally follows the function calls of a program with no debug information.

Chapters 3 and 4 give the theory, and this chapter shows it on real binaries. The registers, the process memory and the addressing modes of chapter 3 come back, together with the flags, jumps, stack frames, `lea` and calling conventions of chapter 4. The content comes from the exercise slides and from the IDA demonstrations in the lectures of 2 and 6 October 2026.

Remember

What the material contains. The parts marked **From the lecture** report what the professor showed in IDA, with the addresses visible on screen. The exercise slides themselves give no executables, so they do not provide these values.

5.1 Gearing up your machine

The course uses IDA Freeware in its provided VM, where it comes pre-installed. For a personal VM, the slides mention IDA Freeware 8.3 or later and, if needed,

the appropriate Microsoft Visual C++ runtime redistributable:

- `https://out7.hex-rays.com/files/idafree83_windows.exe`
- `https://aka.ms/vs/17/release/vc_redist.x64.exe`
- `https://aka.ms/vs/17/release/vc_redist.x86.exe` (optional)

These are course setup notes, and they do not guarantee that the same URLs or versions remain current. Work on the exercise binaries in an isolated analysis VM, and keep the C source alongside the executable when provided.

> **Step by step: A reliable workflow for every exercise**
>
> 1. Identify the function entry.
> 2. Map the stack and register state at each key instruction.
> 3. Follow branch targets and calls.
> 4. Compare the inferred behaviour with the accompanying C source.
> 5. Revisit any unexplained compiler choice.

Record addresses from the actual supplied binary. The slides do not provide the executables, or enough bytes to derive the requested addresses here.

5.2 Using IDA

> **From the lecture (2 October 2026, 02:24–23:36)**
>
> The course VM runs **IDA Freeware 8.4** (window title `IDA v8.4.240527`). The professor stresses that the course teaches methods rather than tools. IDA is used for two things: **disassembling** a compiled program, and **debugging** it, which means running it in a controlled way while inspecting memory, registers and flags. The decompiler stays off for most of the course, because the goal is to learn the baseline method first.

Opening a file. Use New (disassemble a new file) or drag the executable onto the IDA icon. The load dialog shows IDA's guess: a Portable Executable

(section 6.5) for the 80386. Keep the defaults and the `metapc` processor type, which accepts anything valid in the Intel instruction set.

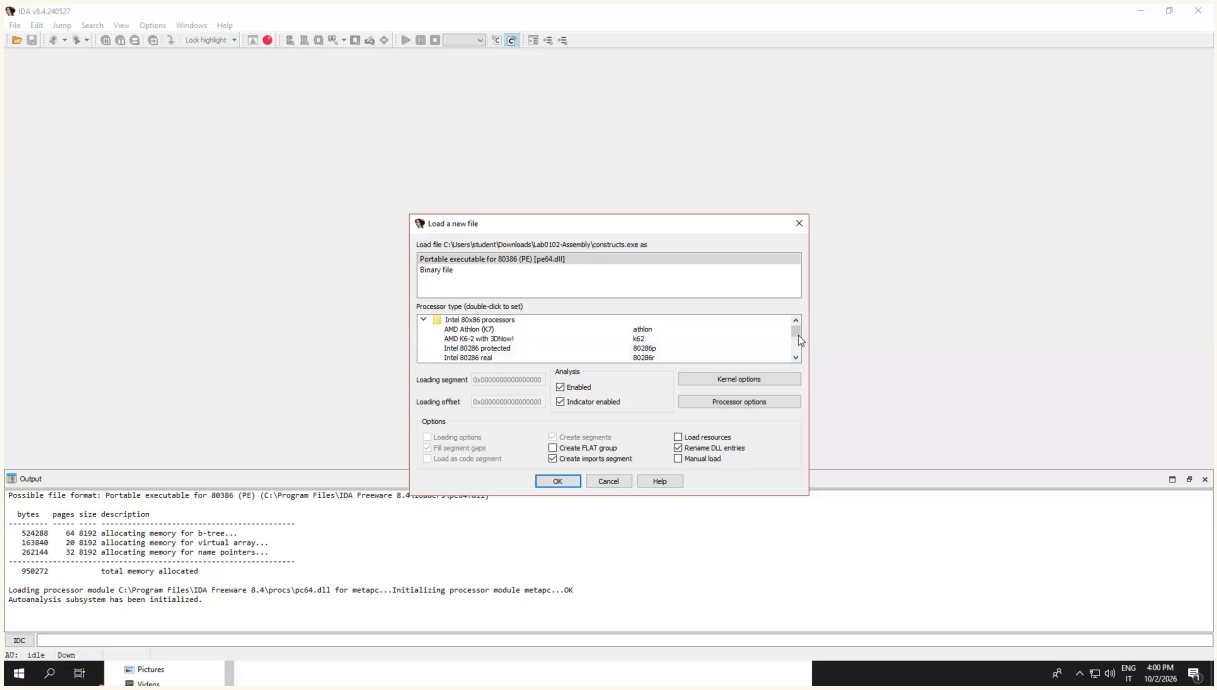


Figure 5.1: The load dialog of IDA: file type, processor type and the default options (lecture of 2 October 2026).

Debug information. `constructs.exe` was compiled with MinGW, the GCC port for Windows, so it carries DWARF information inside the binary and IDA offers to load it. That is why functions and globals already have names such as `_fun_if` and `_A`. The professor warns this is “pretty much the one and only time” you will have it, since a malware sample shows only names like `sub_401373`. MSVC keeps debug information in a separate PDB file instead. A careless author may leave the PDB path in the binary, and IDA’s warning then reveals that path from the author’s machine.

Windows. The Functions window on the left lists the recognized functions, normally sorted by address. The Output window at the bottom is a status console and can be shrunk. Here IDA lands directly on `main`, but the professor says this does not always happen: later examples will require finding it by hand, starting from the entry point.

Reading main. The header lines are IDA’s annotations, not assembly. Here IDA infers three arguments (`argc`, `argv`, `envp`), although the source does not use the third.

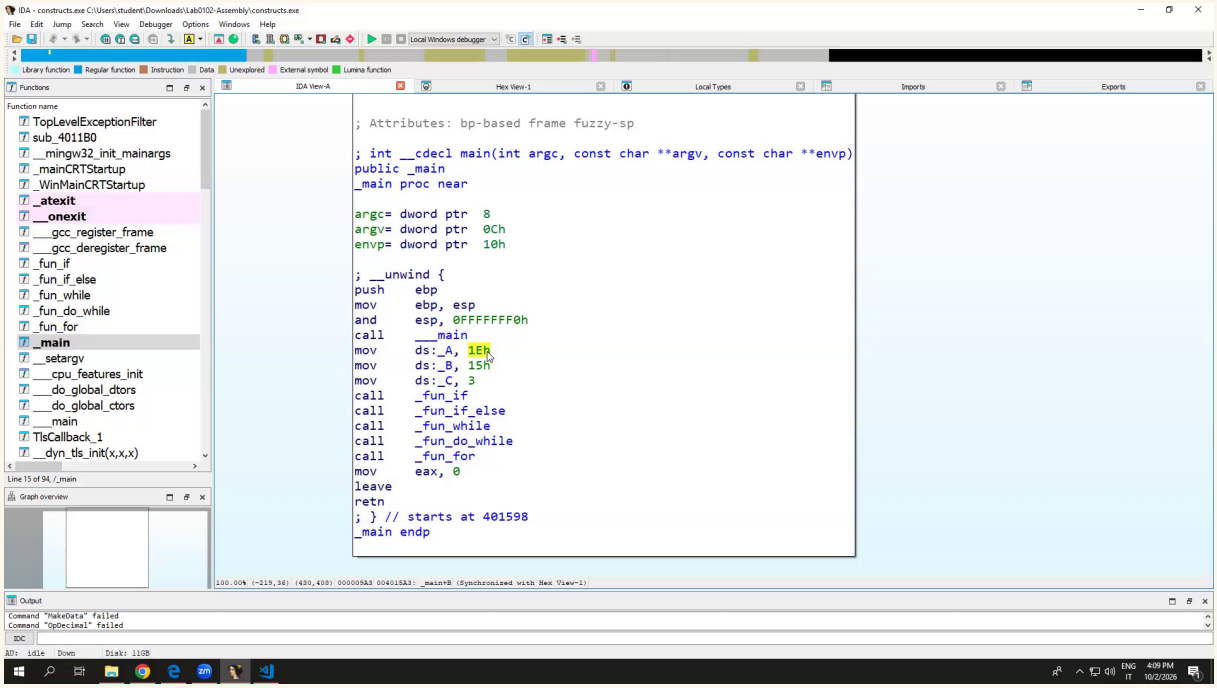


Figure 5.2: `main` of `constructs.exe` in IDA, with its header annotations and the Functions window (lecture of 2 October 2026).

Two things in this `main` can be ignored once recognized:

- `call ___main` is support code generated by the compiler, not something the author wrote.
- `and esp, 0FFFFFFF0h` clears the last 4 bits of ESP, so the stack becomes 16-byte aligned for compatibility with some code.

Following a reference. Clicking _A jumps to its definition. A, B and C have no initial value, so they live in `.bss` (the PE sections are described in section 6.5). They appear in the order _B, _A, _C, not as declared, because the compiler is free to arrange them. Initializing one in the source would move it to `.data`. Press `Esc` to go back.

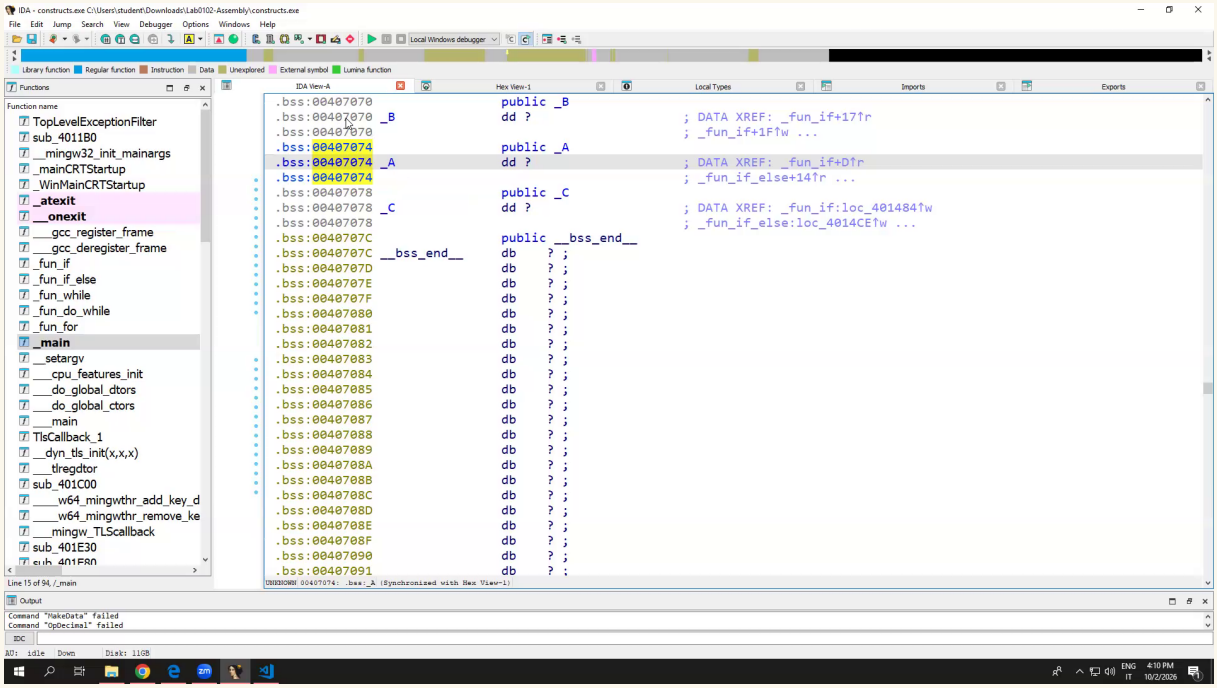


Figure 5.3: The globals in `.bss`, reached by clicking _A (lecture of 2 October 2026).

Key	Action	Key	Action
Esc	Go back	:	Enter or edit a comment
G	Go to an address	N	Rename
X	Cross-references	Space	Switch graph and flat view
H	Show the constant in decimal	F5	Decompiler
Q	Show the constant in hex		

The professor lists X, G and Esc as the ones used most in this course. X and G are not explained in this lecture. The debugger side of IDA is in chapter 11.

5.2.1 Saving your work

> **From the lecture (6 October 2026, 60:00–61:00)**
>
> Closing IDA opens the Save database dialog, which stores your comments and renames. The default packs the database, while DON'T SAVE the database discards the session. The professor discards a throwaway test

here, but normally keeps the database because it holds the notes.

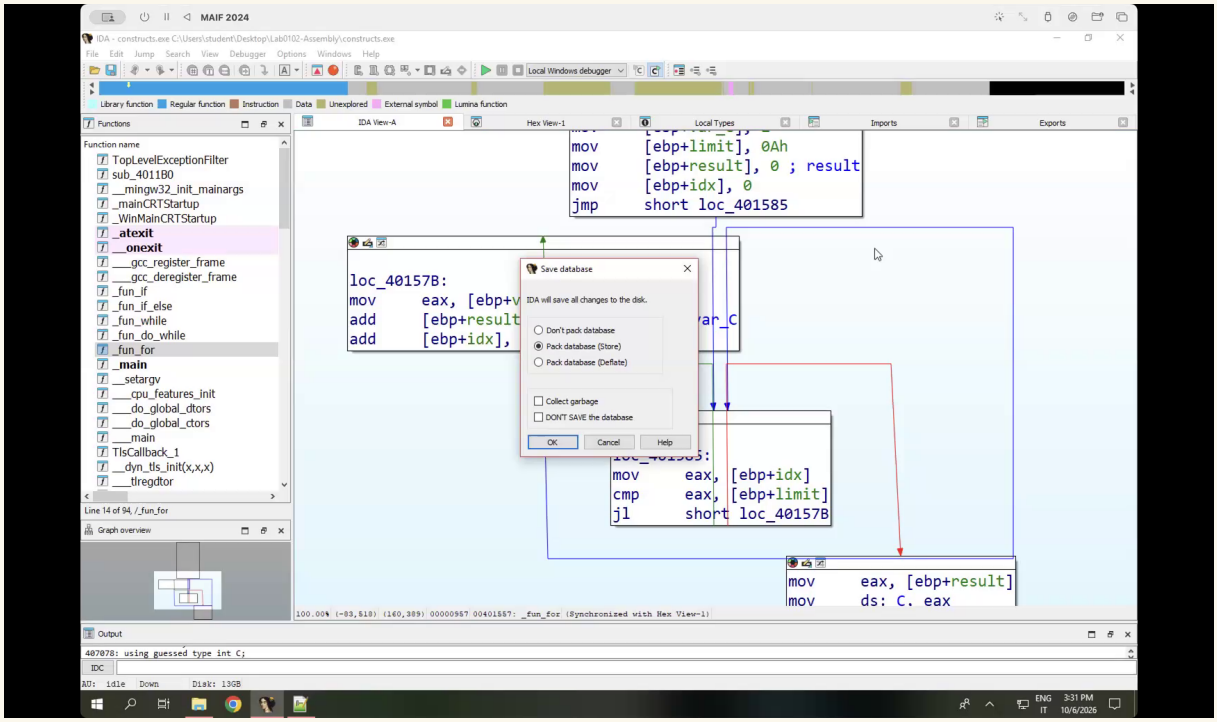


Figure 5.4: The Save database dialog shown when closing IDA (lecture of 6 October 2026).

5.3 CONSTRUCTS.EXE

From the lecture (2 October 2026, 23:54–62:30)

All addresses below are visible on screen. The comments in the screenshots were typed by the professor during the lecture. `fun_do_while` and `fun_for` were postponed to the next lecture.

5.3.1 fun_if (starts at 401460)

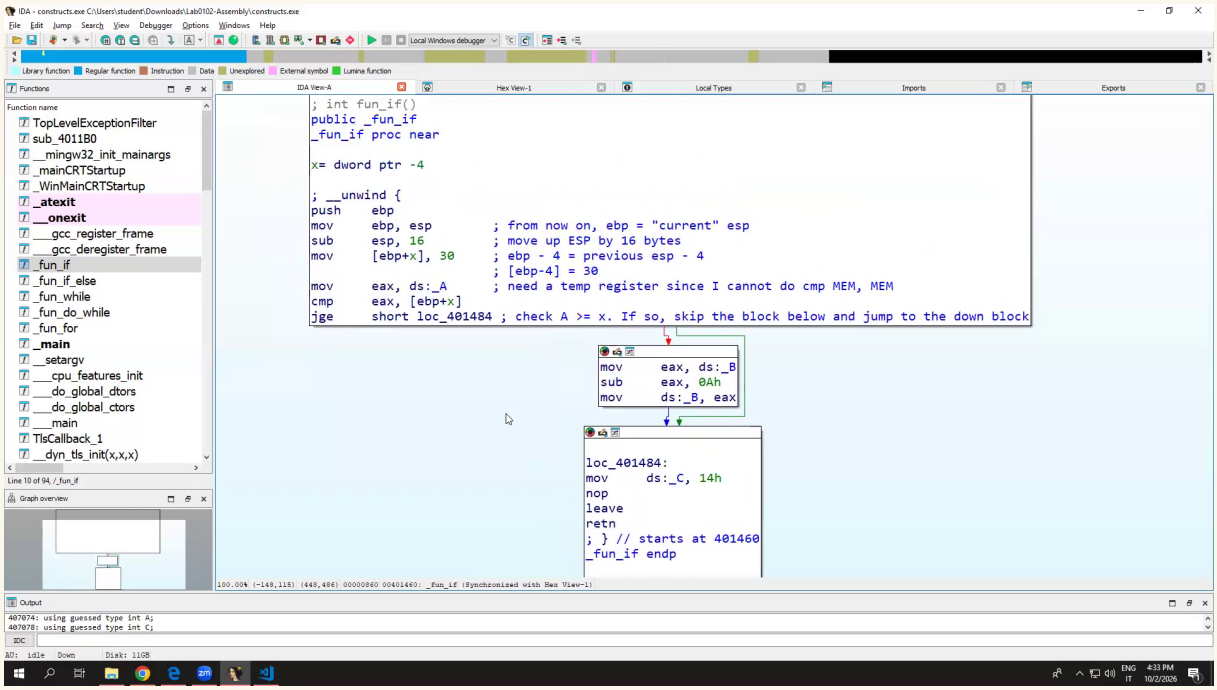


Figure 5.5: `fun_if` in IDA, with the comments typed by the professor (lecture of 2 October 2026).

- **`var_4`:** this name is not assembly. It is an IDA abstraction for `dword ptr -4`, so `[ebp+var_4]` means `[ebp-4]`. Right-click shows the plain form, and the professor then goes back to the friendlier one.
- **Why EBP:** after `mov ebp, esp`, EBP holds the ESP value of that moment. So `ebp-4` still means "previous ESP minus 4", even after `sub esp, 16`. See section 4.4.3.
- **Renaming:** once you understand a variable, press `N`. Here `var_4` becomes `x` everywhere in the function.
- **Temporary register:** `cmp` cannot compare two memory operands, so `A` is first copied into `EAX`.
- **Inverted condition:** the source says `if (A < x)`, but the code jumps with `jge` when the opposite holds, skipping the body. The body is the path taken when the jump is not taken. In C terms: `if (A >= x) goto L; B = B - 10; L: C = 20;`.

> **Correction**
>
> In the audio the jump sounds like “JG”, described as “greater or equal”. The instruction on screen is `jge`, and that is what the professor’s comments use.

Decompiler. `F5` gives a friendlier view, but the professor warns that it is semantically equivalent rather than truly accurate. The screenshot shows this: the source declares `void fun_if()`, while the output is `int fun_if()` with an extra `result` variable.

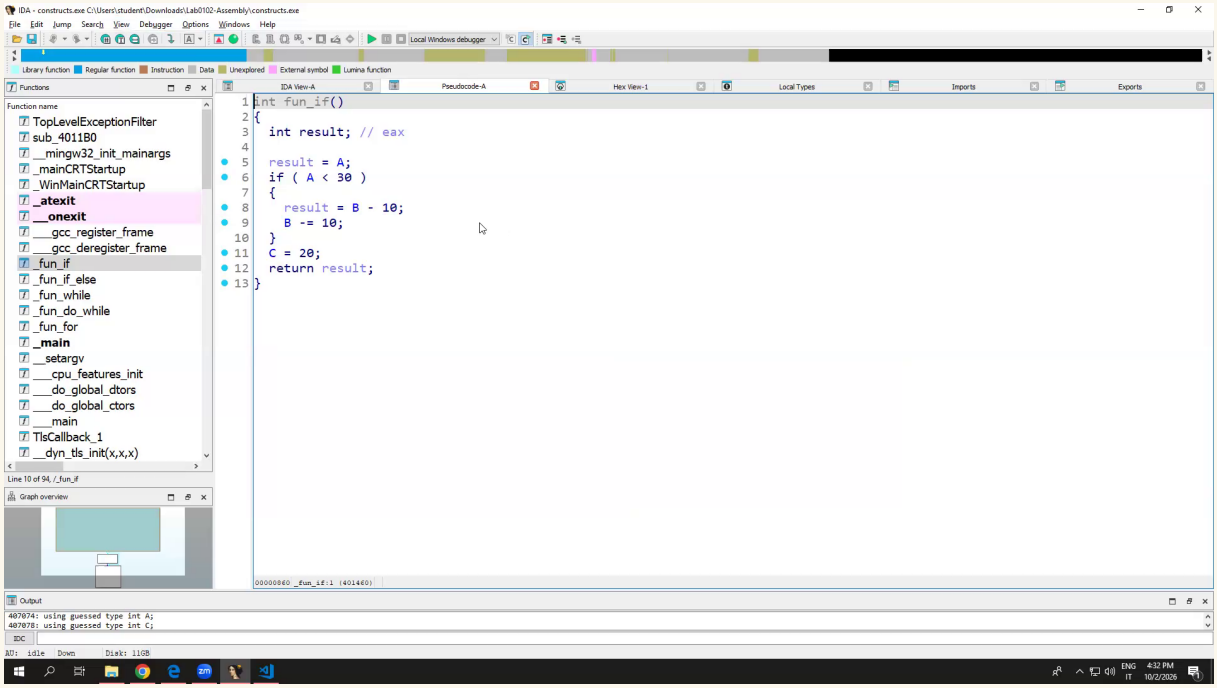


Figure 5.6: The decompiler output for `fun_if` (lecture of 2 October 2026).

5.3.2 fun_if_else (starts at 401491)

An if-else needs **two jumps**. The conditional one selects the block, and an unconditional one stops the first block from running into the second.

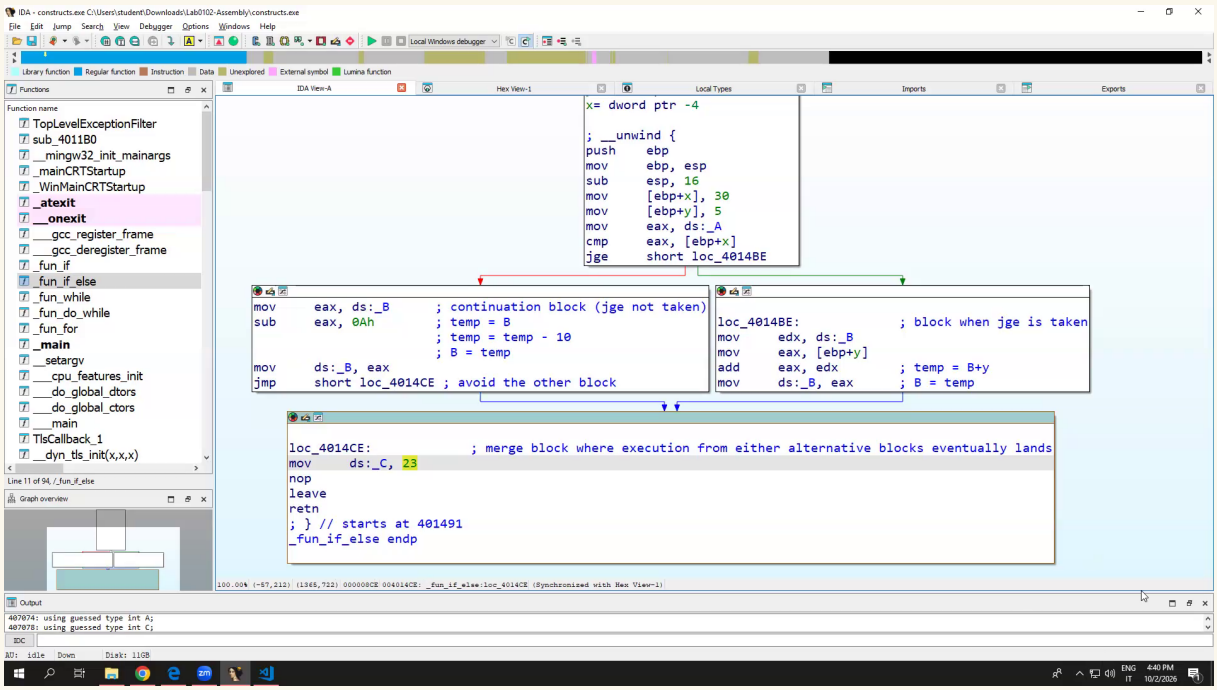


Figure 5.7: `fun_if_else` in graph view (lecture of 2 October 2026).

The graph is IDA's drawing: the CPU has no left or right, only a sequence of addresses. Pressing Space shows the flat view, where the same `jmp short loc_4014CE` is what skips the other block.

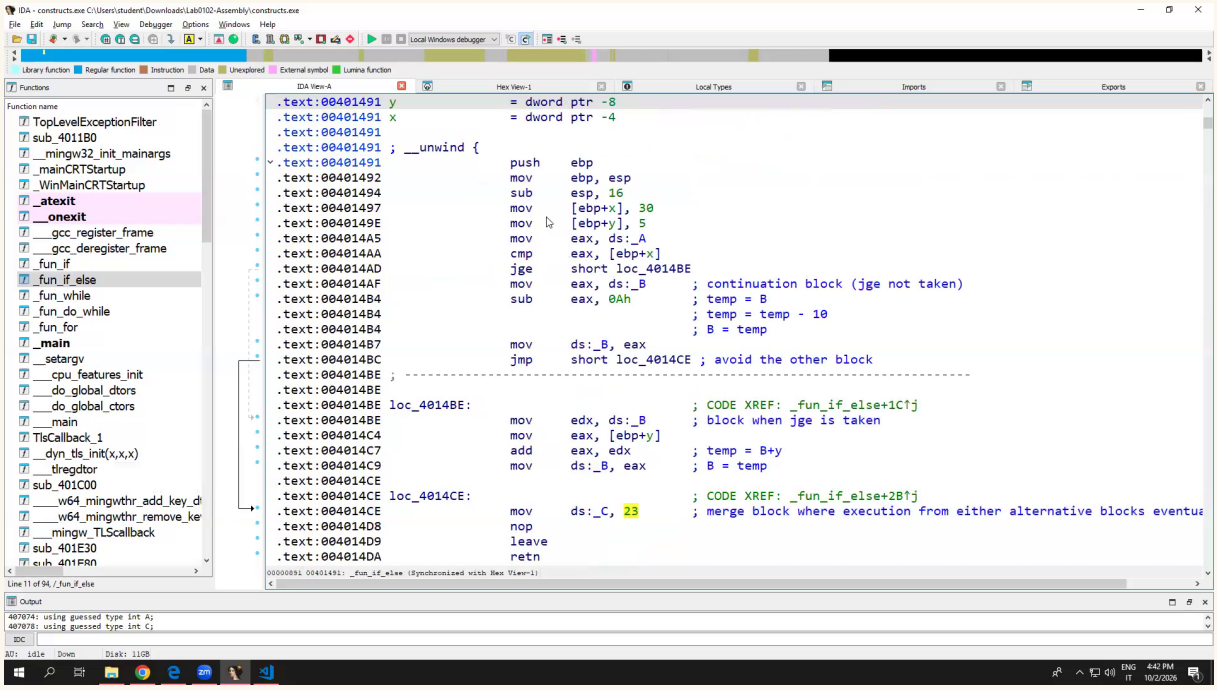


Figure 5.8: `fun_if_else` in flat view, with the `jmp` that skips the other block (lecture of 2 October 2026).

The code reads B into a register, changes it and writes it back, instead of using one direct memory operation. The professor attributes this redundancy to compiling without optimization.

5.3.3 fun_while (starts at 4014DB)

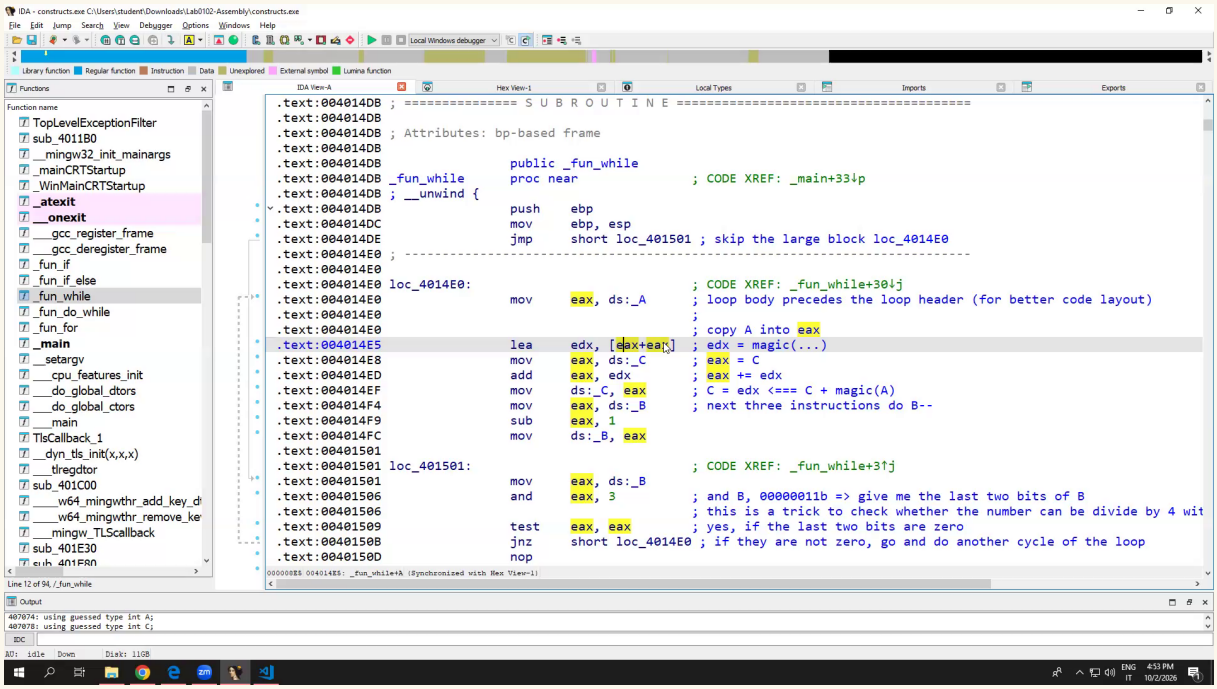


Figure 5.9: `fun_while` in graph view, with the check drawn at the higher address (lecture of 2 October 2026).

- **Body before check:** the function immediately jumps over the body (`jmp short loc_401501`) to the check at the higher address. In the graph this looks swapped, with higher addresses above lower ones. The professor connects this layout to CPU efficiency, at least on older architectures.
- **Remainder trick:** `and eax, 3` keeps the last two bits of B, which are zero exactly when B is divisible by 4. The hardware remainder instruction is slow, so the compiler avoids it. `test` and `jnz` then repeat the loop while the bits are not zero.
- **The "magic":** when this screenshot was taken, the `lea edx, [eax+eax]` line was still commented as `magic(...)`. The professor then reveals that it computes `A * 2` without touching memory. See section 4.6.

Correction

The on-screen comment at `4014EF` reads `C = edx`, but the instruction is `mov ds:_C, eax`, and the previous `add eax, edx` left the sum in EAX. So the comment should say `C = eax`. The expression it adds, `C + magic(A)`, is correct.

5.3.4 fun_for (starts at 401557)

From the lecture (6 October 2026, 50:09–59:45)

The professor reads this function as a reverse engineer, guessing each variable's role from what the code does before checking the source.

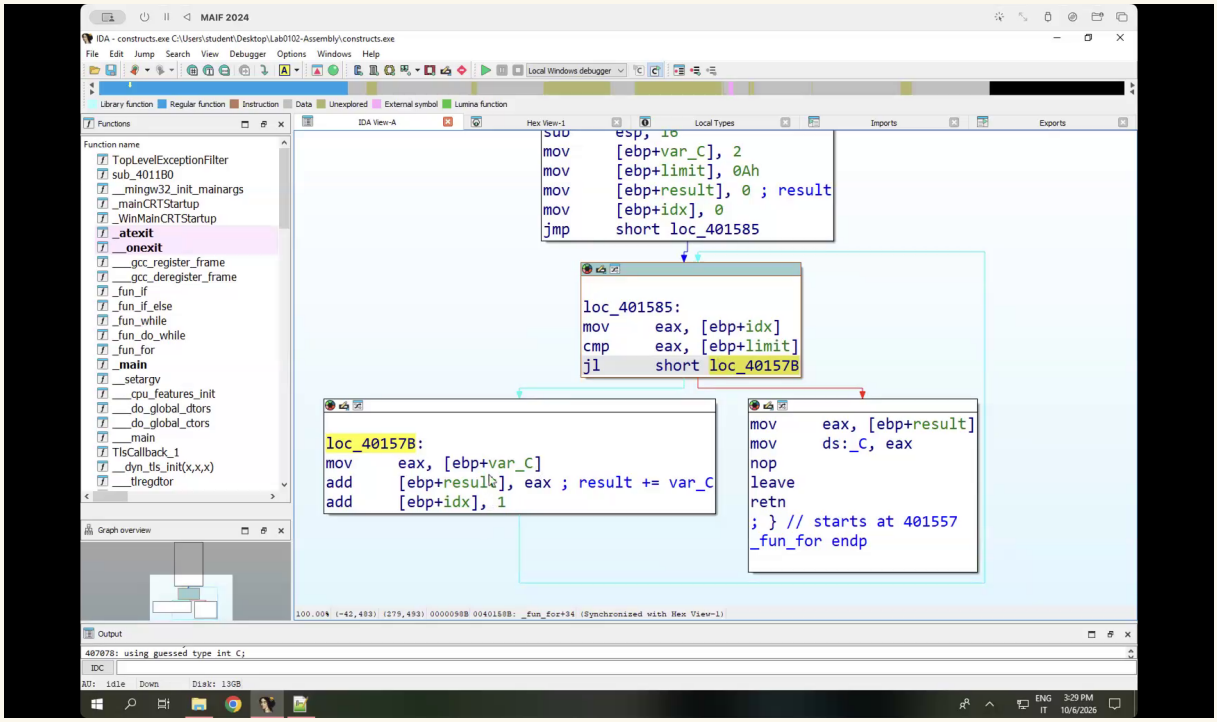


Figure 5.10: `fun_for` in graph view, with the variables renamed by role (lecture of 6 October 2026).

- **Spotting a loop:** a `cmp` followed by a jump whose target eventually leads back to the same check, with a variable compared against another and incremented by the program, suggests a counter and a limit.
- **Renaming by role:** `var_10` becomes `limit`, `var_4` becomes `idx` and `var_8` becomes `result`. The professor stresses that these are guesses.

In the source, `limit` was actually b.

- **Stale comments:** comments are not renamed automatically when a variable is, so those need fixing by hand.
- **Unknown values:** `var_C` holds 2, and from the code alone it could be any constant. `result += var_C` repeated `limit` times is a multiplication.
- **Decompiler:** `F5` does not simplify this into a multiplication, so the professor sees no benefit here.

The graph can mislead. The graph draws the path into `loc_401585` as if there were a back edge where no jump exists, and the real backward jump looks like a forward one. The flat view (`Space`) shows the true order: the entry `jmp` skips the body, the check follows the body, and `jl` jumps backward.

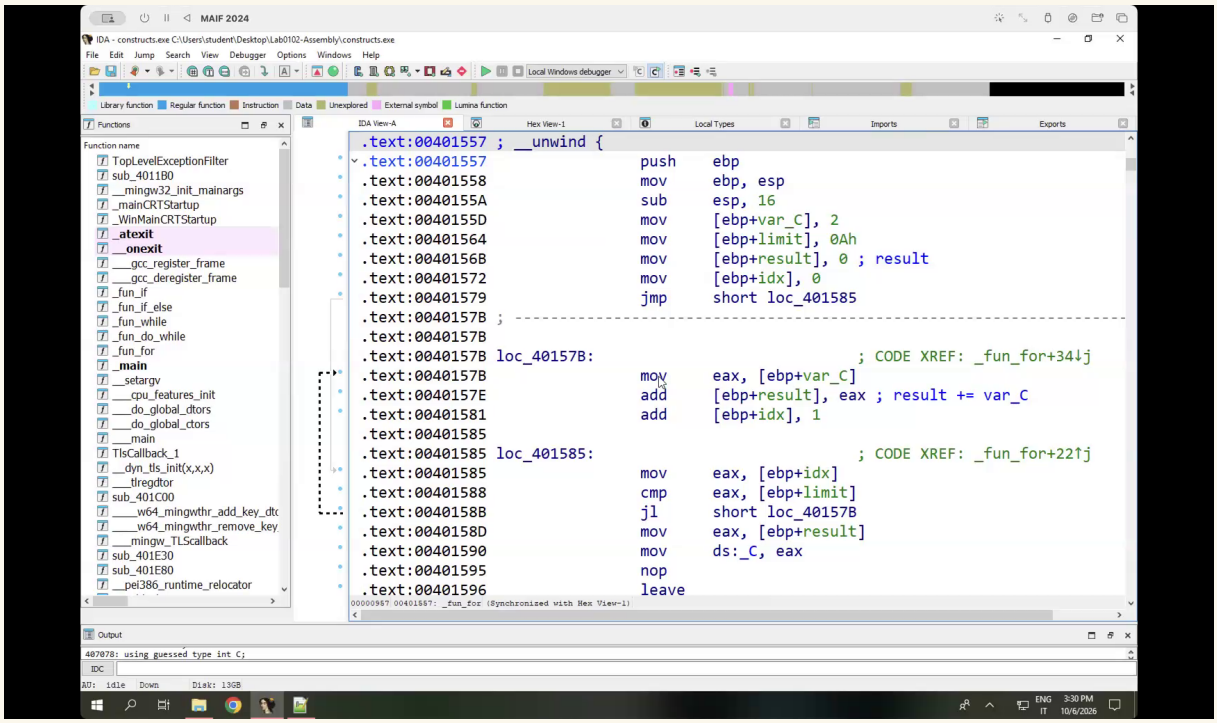


Figure 5.11: `fun_for` in flat view, which shows the true order of body and check (lecture of 6 October 2026).

`fun_do_while` is left as an exercise. The professor notes that a compiler may translate a `for` into something resembling a do-while, with the body followed by the check.

Correction

The comparison needs one caveat: `fun_for` is not a true do-while. The entry `jmp short loc_401585` reaches the check first, so the body can run zero times, as a `for` requires. Only the layout is similar.

Section 14.2.2 runs these same functions step by step in the debugger, including `fun_do_while`.

5.4 CALLS\CDECL.EXE

The exercise asks for the addresses of `main()`, `test()` and `mysum()`, then concentrates on `test()` and `mysum()`. Read the executable to obtain the exact addresses, because a function name in C does not itself determine a virtual address.

What you see	How to reason about it
`sub esp, 18h`	Reserves `0x18` = 24 bytes of stack space. Determine which local values, temporary slots or alignment requirements occupy it, by tracing the later uses of `[ebp-offset]` and ESP.
`push esi`, `push ebx`	They likely preserve nonvolatile registers that `test()` plans to modify. Match each with its restoration on every return path.
`lea eax, [esi+ebx]`	Computes the sum of the two register values, without reading the memory at that sum (section 4.6).
`[ebp-0Ch]`	An EBP-relative slot 12 bytes below the frame anchor. Follow its writes and reads to infer its role. The offset alone does not prove its source-level name or type.
`mysum()` using only `pop ebp`	This can work if ESP already points to the saved EBP at that point. Confirm that there was no local allocation or other push, or that it was already released. `leave` would be redundant in that frame.

5.4.1 Finding main

> **From the lecture (6 October 2026, 63:20–68:50)**
>
> `cdecl.exe` has no debug information, so functions appear as `sub_...` and IDA opens at `start`, far from `main`. The code the professor walks through on the way to `main` was generated by the compiler, not written by the programmer.

The professor follows the only call in `start`, then scrolls through the runtime setup looking for familiar landmarks:

1. `__p__environ` fetches environment variables and command-line data, so the setup is still preparing `main`.
2. `_cexit` and `ExitProcess` end the program, so `main` must have returned by then.
3. The only program call between them is `call sub_4014A1`.

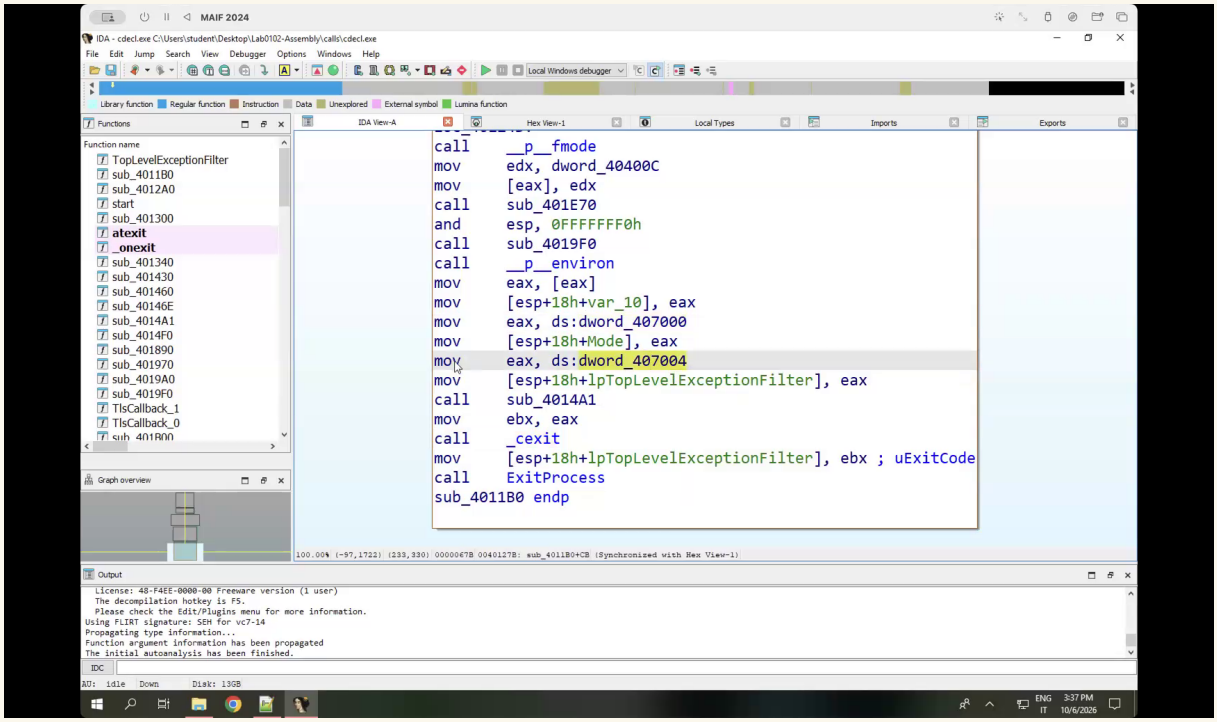


Figure 5.12: The runtime setup of `cdecl.exe`, with the call to `main` between the landmarks (lecture of 6 October 2026).

The candidate is confirmed by opening it and recognizing the `printf` format strings `"Result: %d\n"` and `"Expected: %d\n"`. N then renames it to `main`, which IDA accepts here even though it refuses some other reserved names.

Function	Address
`main()`	`sub_4014A1`
`test()`	`sub_40146E`
`mysum()`	`sub_401460`
Compiler stub called first by `main`	`sub_4019F0`

5.4.2 test() (sub_40146E)

IDA first shows the outgoing arguments in its own notation, `[esp+20h+var_1C]`. The professor prefers the plain form (right-click), where this is simply `[esp+4]`.

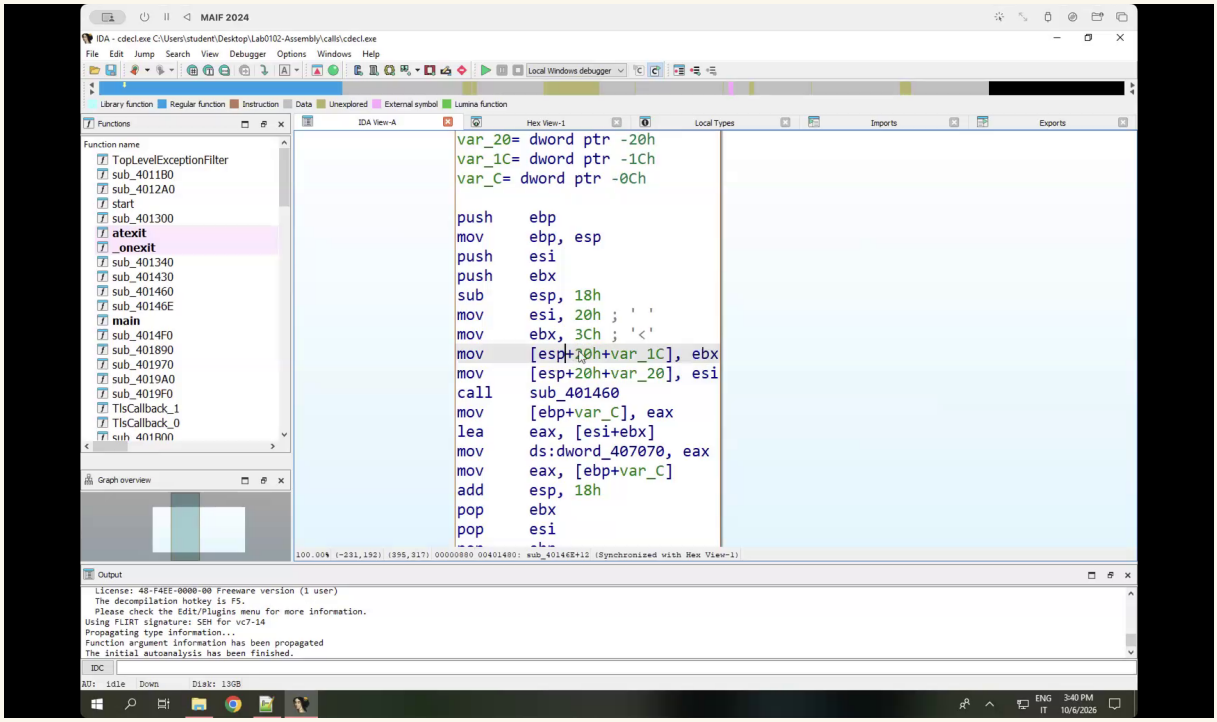


Figure 5.13: The first part of `test()`, with the outgoing arguments (lecture of 6 October 2026).

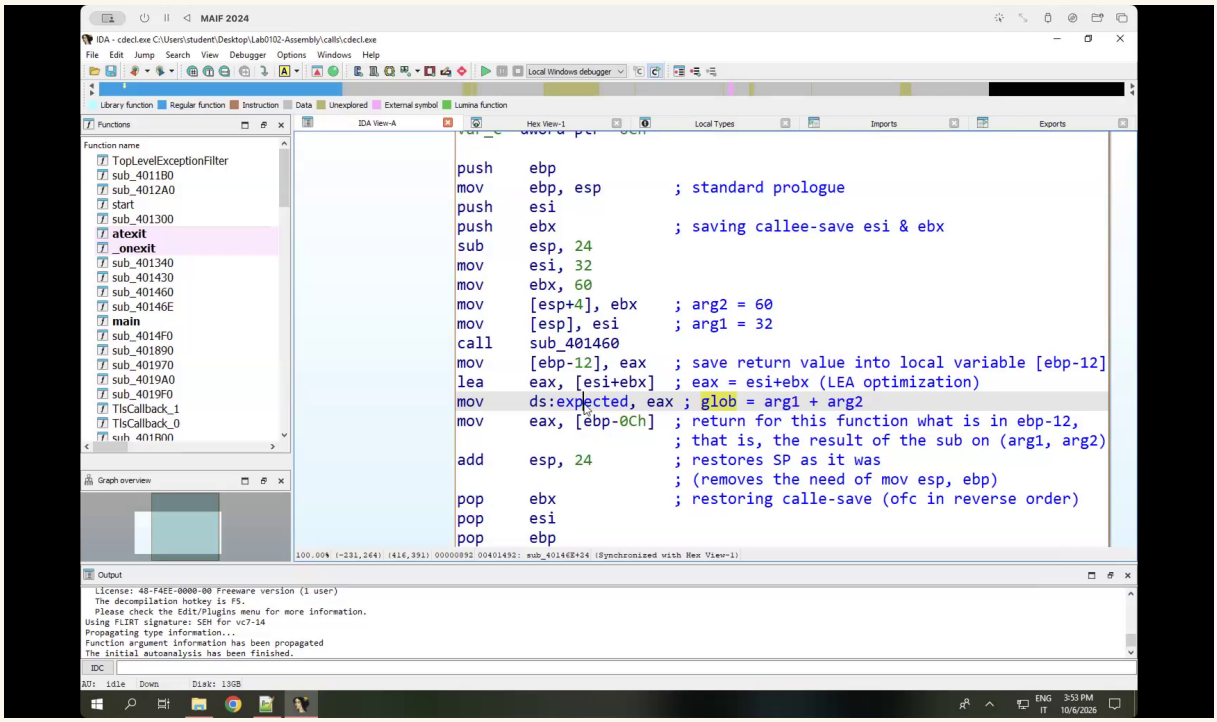


Figure 5.14: The second part of `test()`, with the comments typed by the professor (lecture of 6 October 2026).

- **`sub esp, 18h`:** the professor estimates about 12 bytes are needed, but 24 are reserved for alignment. One large subtraction prepares both the local variable and the outgoing arguments, replacing separate `push` instructions.
- **`mov esi, 20h` and `mov ebx, 3Ch`:** IDA offers the ASCII characters `' '` and `'<'`, but these are simply the numbers 32 and 60.
- **`[ebp-0Ch]`:** this saves the return value of `mysum`, because the following `lea` overwrites EAX.
- **`lea eax, [esi+ebx]`:** this is the source's `expected = a + b`, and it relies on the callee preserving ESI and EBX.
- **`dword_407070`:** clicking it leads to `.bss`, so it is a global. The professor renames it `glob`, then `expected` after seeing the source.
- **`add esp, 24`:** the frame size is known, so this restores ESP without needing `mov esp, ebp`.
- **Pop order:** registers come back in reverse order. Popping in the same order would swap them.

Reading the screenshot (explanation added by this book)

sub in the comment “the result of the sub on (arg1, arg2)” means subroutine, namely `mysum`, not a subtraction. The comment `glob = arg1 + arg2` still says `glob` while the operand already reads `expected`: that is the stale-comment effect described in section 5.3.

5.4.3 mysum() (sub_401460)

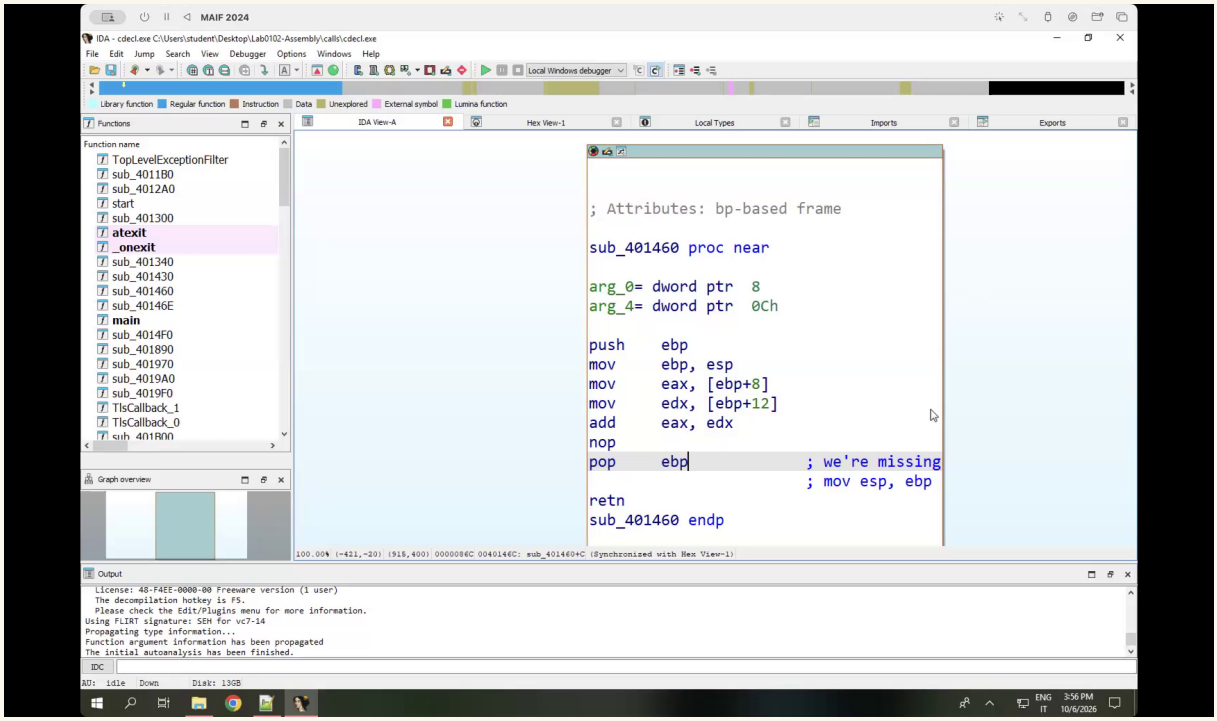


Figure 5.15: `mysum()` in IDA, with `arg_0`, `arg_4` and `retn` (lecture of 6 October 2026).

- **`arg_0` and `arg_4`:** IDA numbers arguments from zero, so these are the first two arguments, at `[ebp+8]` and `[ebp+12]`.
- **`retn`:** a student asks whether this is the special `ret N`. It is not: IDA just shows a plain `ret` this way. See sections 4.4.4 and 4.5.1.
- **Missing `mov esp, ebp`:** ESP never changes in this function, so the instruction would be unnecessary. The compiler can drop it with this quick optimization while keeping the standard prologue, hence the asymmetry.

5.4.4 main() (sub_4014A1)

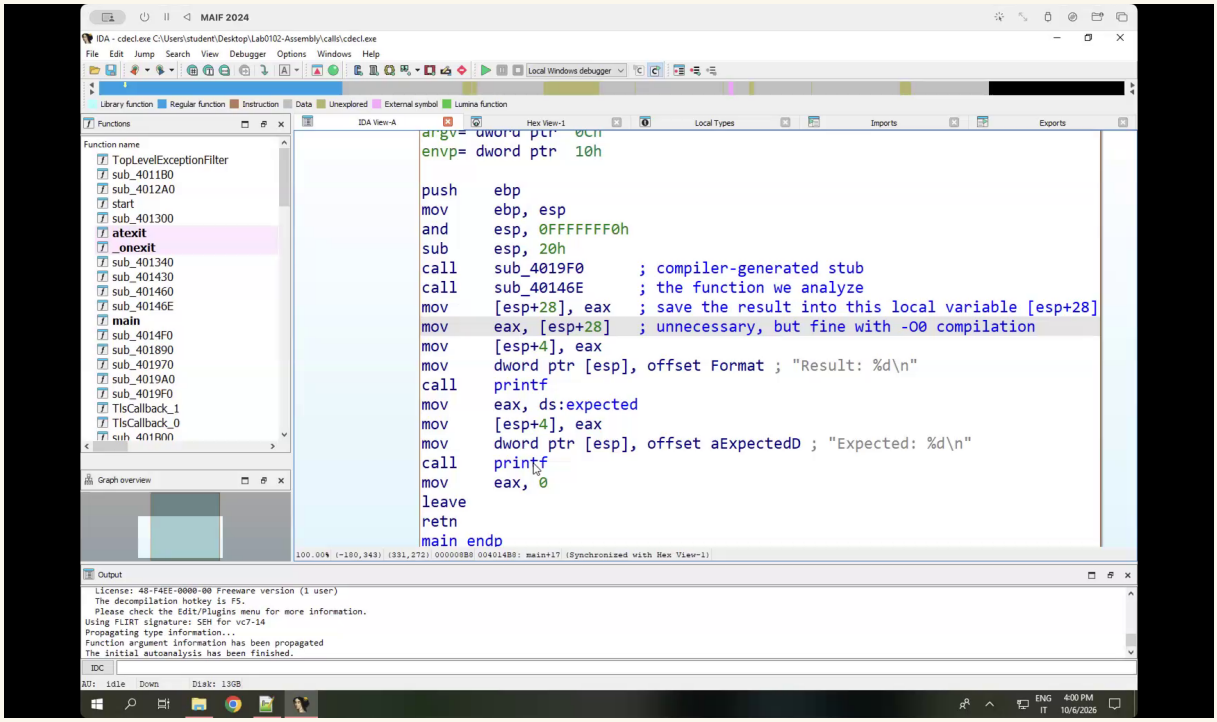
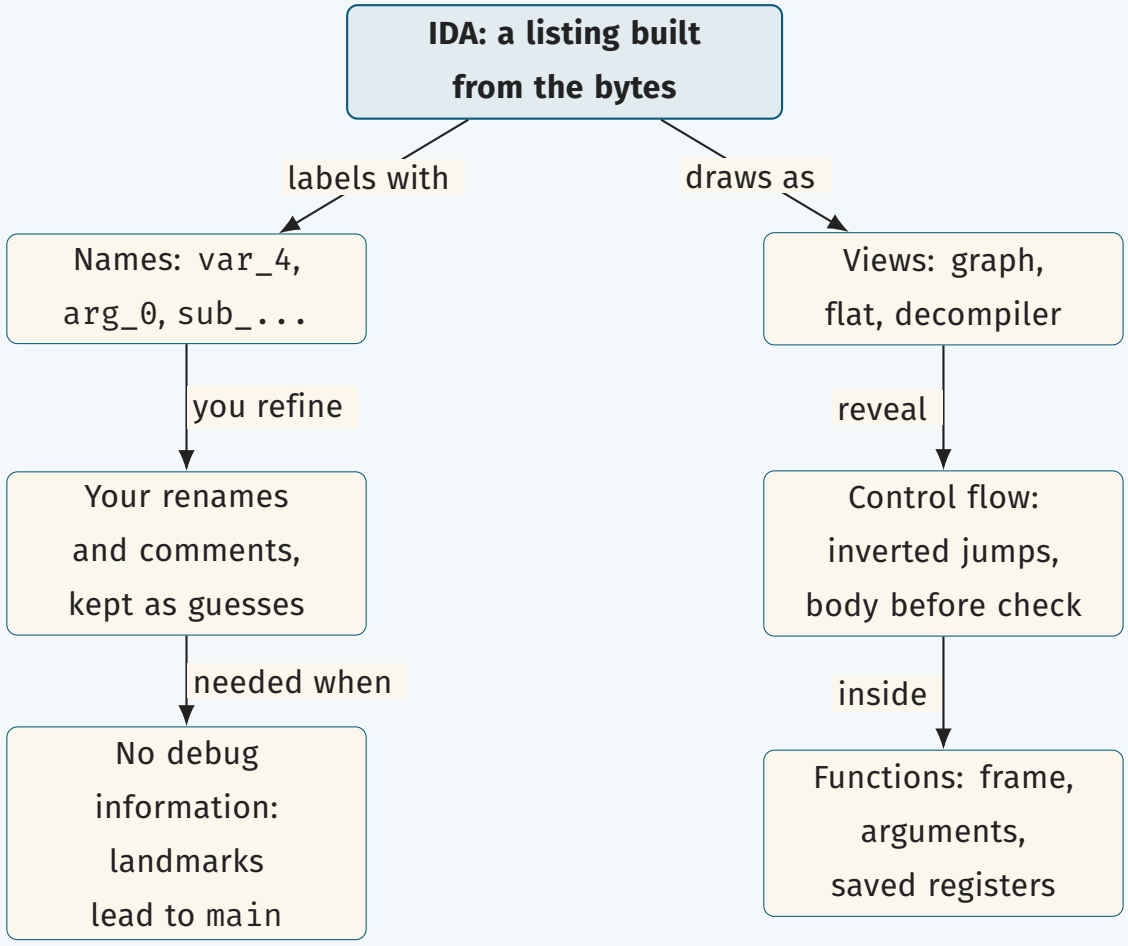


Figure 5.16: `main()` of `cdecl.exe` in IDA (lecture of 6 October 2026).

The result of `test()` is saved to `[esp+28]` and immediately read back. The professor calls this unnecessary: without optimization (`-O0`) the compiler translates one statement at a time and does not notice the value is already in EAX. As in `constructs.exe`, the first call is a compiler stub to ignore, recognizable because it is the same code seen in the previous lecture.

Concept map: how the ideas connect



IDA builds a listing from the bytes. On one side it gives names, which you refine as you understand the code, and these guesses matter most when the binary has no debug information and `main` must be found through landmarks. On the other side it gives views, which reveal how the compiler laid out control flow and functions. The flat view shows the true order of the code when the graph misleads.

In short

- IDA's names such as `var_4` and `arg_0` are annotations for plain stack offsets.
- Compiled conditions are often inverted, an if-else needs two jumps, and loops may place the body before the check.
- The graph is IDA's drawing and the decompiler is only semantically equivalent, so check the flat listing when they seem odd.

- Without debug information, find `main` through the landmarks of the runtime setup, then confirm it from its content.
- Renames and comments are hypotheses saved in the database, and comments do not follow renames.

Review: try to answer without looking

1. What does `[ebp+var_4]` mean in IDA?
2. Why does an if-else need two jumps?
3. Why does `and eax, 3` followed by `test eax, eax` check divisibility by 4?
4. Without debug information, which landmarks did the professor use to find the call to `main` in `cdecl.exe`?
5. Is IDA's `retn` the same as `ret N`?

Answers, with the section where to find them

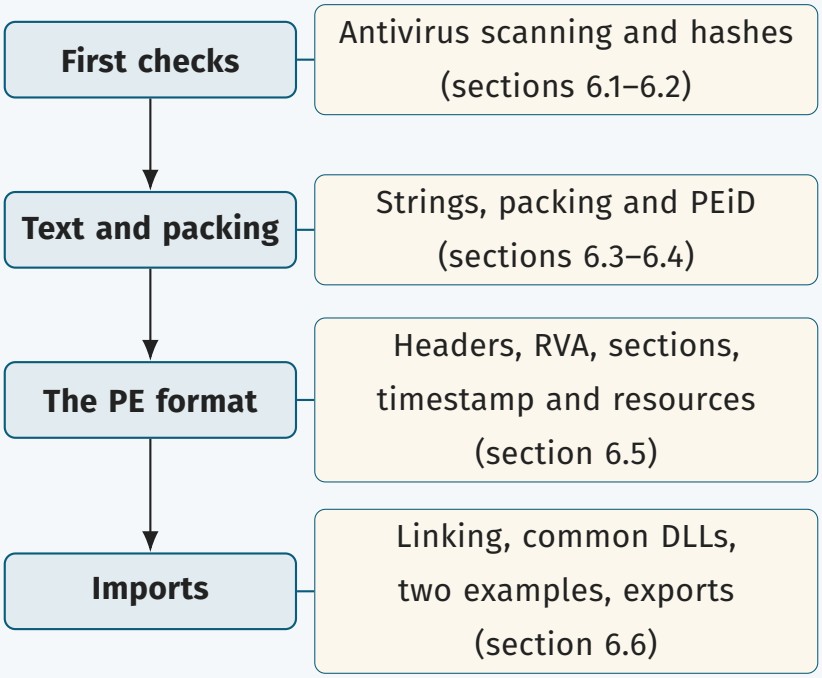
1. `[ebp-4]`: `var_4` is IDA's name for the offset -4 (section 5.3).
2. The conditional one selects the block, and the unconditional one stops the first block from running into the second (section 5.3).
3. The last two bits of a number are zero exactly when it is divisible by 4 (section 5.3).
4. `__p__environ` before it, and `_cexit` and `ExitProcess` after it (section 5.4.1).
5. No: IDA just shows a plain `ret` this way (section 5.4).

Chapter 6

Basic static analysis

> **Before you start**
>
> **Central question:** what can we learn about a sample without running it and without reading every instruction?
>
> **Prerequisites:** the distinction between capability and observation, and the safety rules (chapter 2); the memory map and little-endian order (chapter 3).
>
> **Where it leads:** each clue becomes a question for controlled execution (chapter 7) or for code reading (chapter 9).

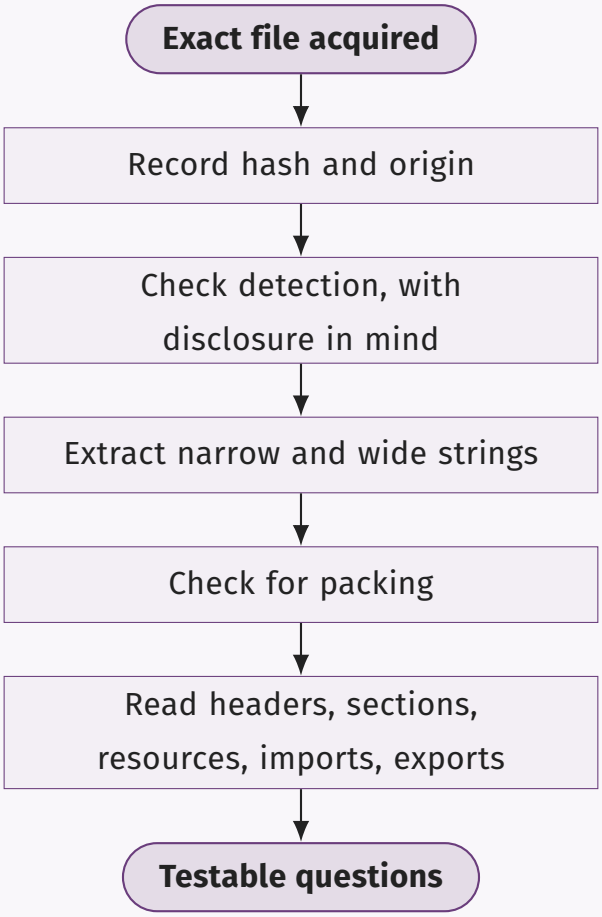
Chapter structure: what comes, and in which order



The chapter follows the order of a first look at a sample. It starts with the quickest checks, then reads the text in the file. It then opens the file structure and ends with the libraries the program uses.

The lecture's initial triage consists of **antivirus scanning, hashes, strings, functions and headers**. Together these identify the sample, suggest its purpose, expose possible concealment and guide later analysis. They do not by themselves establish every behaviour the file will exhibit.

Flowchart: Triage of an unfamiliar sample



The diagram summarises this chapter as one path. Record the exact file and its hash, look at detection results without disclosing the sample carelessly, extract strings, check for packing, and read the file structure. The output is a list of questions for deeper static work or controlled execution. Keep possible capability, confirmed execution and incident impact separate in your notes.

6.1 An obvious first step: antivirus scanning

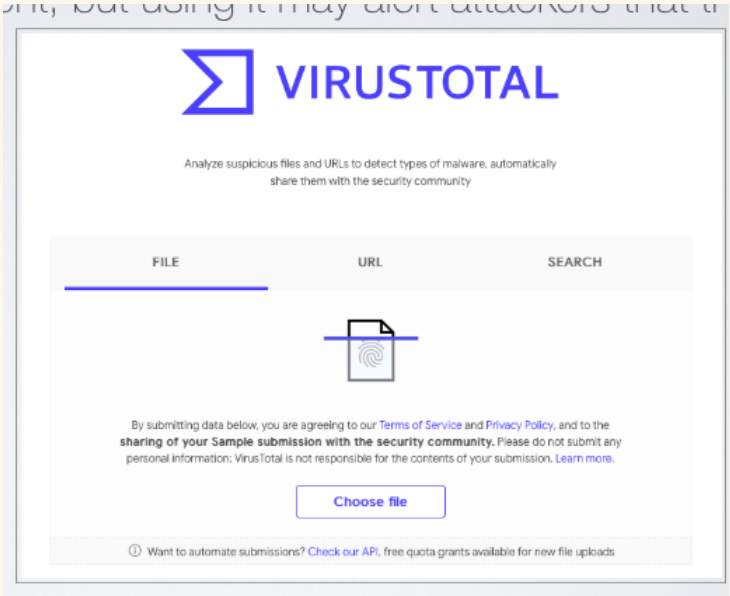


Figure 6.1: The VirusTotal upload page. Its small print says that submissions are shared with the security community (slides, Basic static analysis).

An antivirus result is a quick clue, not a ground truth. Small changes, packing or a previously unseen sample can change detection. Conversely, a label can be generic or mistaken. A “clean” result is not proof of safety.

Multi-engine services such as VirusTotal may provide useful context, but **uploading a private sample may disclose it to others** and could alert an attacker who monitors submissions. If the case is sensitive, follow the applicable handling policy and consider checking an existing hash reputation before uploading bytes.

> **Simone’s note (original wording)**
>
> in the backend of virustotla or in genera of an antivirus scanning, there is behind exactly what we are going to learn in this course.

> **Simone’s note (original wording)**
>
> the attacker could possibly just check periodically if it’s own malware or url is actually being stored by virustotal website and labelled as malware and in that case could possibly change the malware itself

6.2 Hashing samples

> **Simone's note (original wording)**
>
> a malware could have multiple samples: each sample is basicallly the exact same malware but with an useless thing being added to change the MD5 hash in a way to change its own fingerprint.

A cryptographic **hash** maps file bytes to a fixed-length digest. Equal bytes produce equal digests, and changing the bytes will usually change the digest. This makes a hash an exact-file identifier for labelling and comparing samples.

The lecture discusses **MD5** and **SHA-1** because they are common in older malware collections. **SHA-256** is a sensible additional identifier today. MD5 and SHA-1 are not collision-resistant, and no hash is literally unique. For ordinary triage, accidental equality is unlikely, but do not make an adversarial authenticity claim from MD5 alone.

The hash describes the **sample**, meaning the exact acquired file, not the entire malware family or all its variants. Two builds of the same malware may have different hashes. Preserve the original bytes and record where and when the sample was obtained.

6.2.1 Obtaining a digest

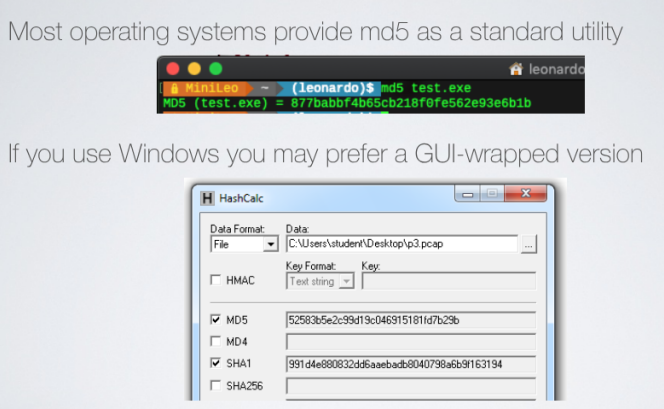


Figure 6.2: Slide on hash tools: a terminal md5 command and a Windows graphical tool, HashCalc (slides, Basic static analysis).

The slide illustrates a terminal `md5` command and a Windows graphical hashing tool. Equivalent commands for a local file named `sample.exe` are these:

```
Linux:    sha256sum sample.exe
macOS:    shasum -a 256 sample.exe
Windows:  certutil -hashfile sample.exe SHA256
```

These compute an identifier: they do not inspect the program's behaviour. Quote the path correctly when it contains spaces, and hash the original file rather than a modified working copy.

6.2.2 What hashes are for

1. Label the exact sample in notes and evidence records.
2. Share a compact identifier with another analyst for an exact-file comparison.
3. Search existing reports or reputation data for prior knowledge, subject to the disclosure rules of the case.

An **import hash** (imphash) summarises the imported libraries and functions of a PE file. Similar import sets can help cluster related binaries even when their whole-file hashes differ. It is a **similarity heuristic**, not a family proof: recompilation, a change in import order, runtime API resolution or packing may change or erase the visible imports.

6.3 Strings

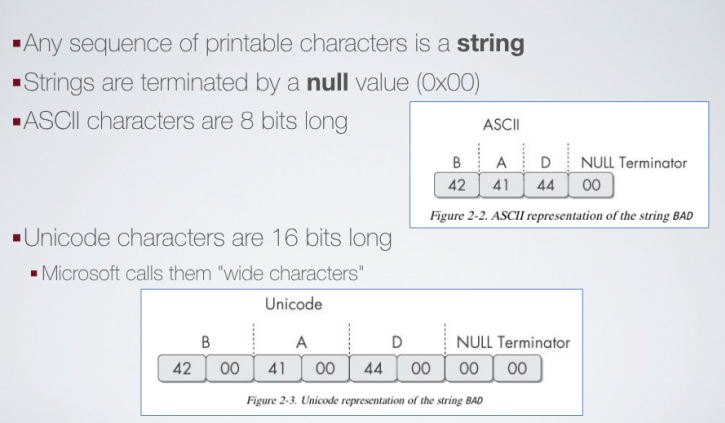


Figure 6.3: Slide on strings: the word BAD as ASCII bytes (42 41 44 00) and as two-byte characters (42 00 41 00 44 00 00 00). Figures from the book (slides, Basic static analysis).

A **string** is a sequence of characters encoded in the file. C-style strings often end at a zero byte. However, not every printable run is a meaningful or null-terminated program string. The slides distinguish single-byte ASCII-style text from Windows “wide” text, as the two encodings of BAD in the figure show.

> **Correction**
>
> The slide says that Unicode characters are 16 bits long. More precisely, Windows often uses **UTF-16 little-endian** for wide strings, usually two bytes per basic character. Unicode itself is not inherently a 16-bit encoding.

Strings can reveal paths, URLs, IP addresses, error messages, commands, library names and configuration. A string may also be dead code, a decoy or part of a benign library. Encrypted, compressed or constructed-at-runtime strings may be absent from raw file scans.

6.3.1 The strings command

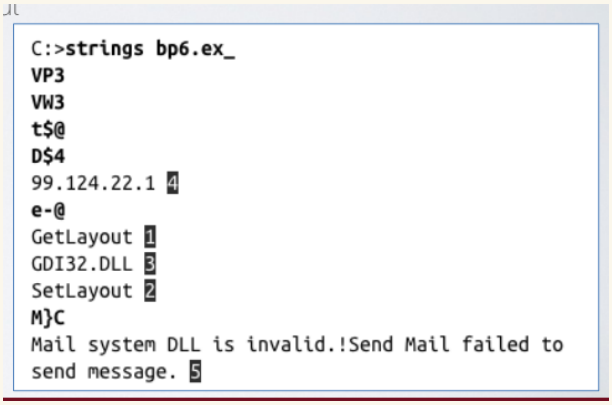


Figure 6.4: Output of strings on bp6.ex_, with numbered markers on five entries (slides, Basic static analysis).

The `strings` command extracts printable sequences meeting a minimum length, commonly three or four characters depending on the tool and options. Linux and macOS variants and Windows versions differ. Inspect both narrow and UTF-16LE text when appropriate.

In the slide's example:

- `GetLayout` and `SetLayout` are Windows functions;
- `GDI32.DLL` is a graphics DLL;
- `99.124.22.1` has the shape of an IP address;
- `Mail system DLL is invalid.!Send Mail failed to send message.` suggests an error path;
- short gibberish such as `VP3`, `VW3` or `t$@` can be ignored initially.

None of these observations alone proves that the host sent mail or contacted that address.

> **Simone's note (original wording)**
>
> This it may looks like trivial, but it can be extreemley usefull: you might be interested why there exists that ip address in that malware, and you just search inside the malware assembly code where that ip address is and reconstruct the logic behind of it.

6.4 Obfuscation through packing

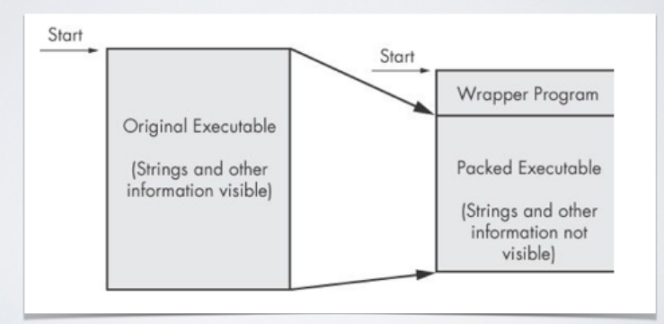


Figure 6.5: The original executable, with strings and other information visible, becomes a wrapper program around a packed executable whose strings are not visible (slides, Basic static analysis).

A **packer** wraps the original code and data of a program in a compressed or encrypted representation, plus a small unpacking stub. On execution, the stub reconstructs the payload in memory and transfers control to it. Static strings and instructions may therefore describe mostly the wrapper.

Packing is used by both legitimate software and malware. It is evidence of an analysis obstacle, not a verdict on intent.

6.4.1 Detecting packers with PEiD

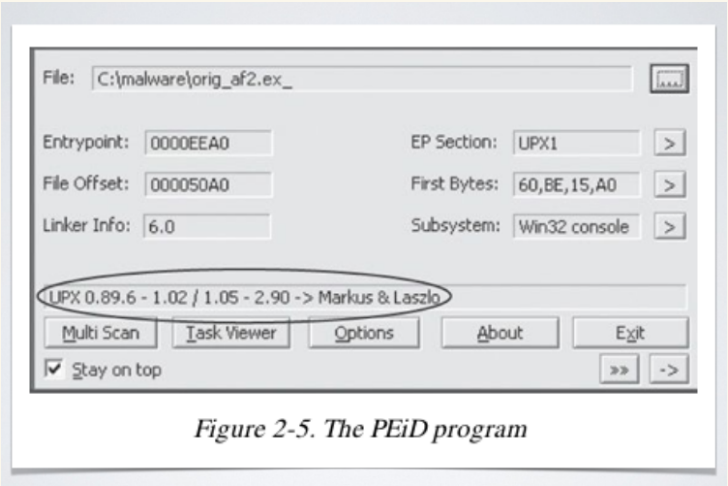


Figure 6.6: The PEiD program identifies UPX on a sample, with UPX1 as entry-point section (book figure 2-5 on the slide).

The illustrated **PEiD** result identifies **UPX** and shows the entry-point section as UPX1. This is a useful hypothesis to check against sections, entry point, entropy and other evidence. Signatures can miss modified packers or misidentify a stub.

> **Do not confuse**
>
> The lecture warns that **some PEiD plugins may execute the file silently**. Run unfamiliar plugins only inside an appropriately isolated VM. Do not assume that every operation in a static-analysis tool is passive.

6.5 File execution in Windows and the PE format

> **Simone's note (original wording)**
>
> it is a standard format that it is being used by windows file in a way that the dynamic loader called by the kernel could possibly knwo where to insert stuff in the correct place.

When an executable is launched, the Windows loader interprets its file format to map code, data, resources and metadata into a process and prepare execution. The bytes on disk cannot simply be copied as one unstructured blob: the loader needs the location, size, permissions, dependencies and entry point of each component. A DLL uses related structures, but is usually loaded into a process rather than launched on its own.

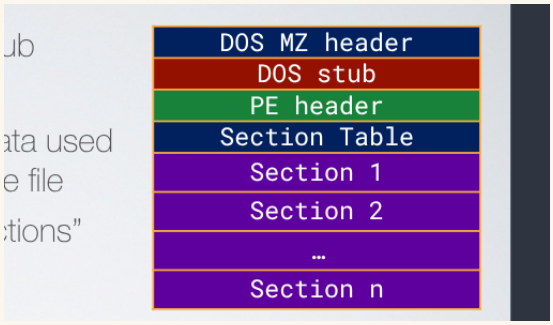


Figure 6.7: The PE layout: DOS MZ header, DOS stub, PE header, section table, then sections 1 to n (slides, Basic static analysis).

PE stands for **Portable Executable**. The PE/COFF family covers executable images, DLLs and object files, with differences between these file types. For an image, the broad order is:

```
DOS MZ header -> DOS stub -> PE signature -> COFF file header
              -> optional header -> section table -> section
  data
```

> **Correction**
>
> The slide writes “Portable Execution”. This is a wording error: the format is the Portable **Executable**.

The headers tell the loader how to map the file. Sections conventionally separate executable code, writable data, read-only data, resources and other material. Typical names are `.text` or CODE, `.data`, `.rdata`, `.bss`, `.rsrc`, `.edata`, `.idata` and `.debug`.

Section names are labels, not authoritative semantics. Windows uses header fields and directories rather than trusting the names, and packers often choose unusual names. Some content is in headers or overlays instead of a named section. Many sections are not writable or executable.

> **Simone’s note (original wording)**
>
> Notice that those section are named to helop humans to understand easilly how to deubug, but can be renamed: this thing is being done by the attacker.

> **Simone’s note (original wording)**
>
> tipiccally we have something like 3-6 sections.

6.5.1 DOS header and stub

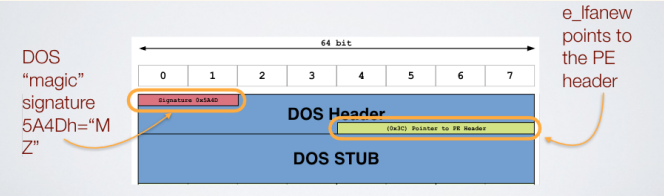


Figure 6.8: The DOS header, with its magic signature 5A4Dh (“MZ”) and e_lfanew pointing to the PE header (slides, Basic static analysis).

The first bytes usually contain the MZ signature (`4D 5A` in byte order). The DOS header includes `e_lfanew` at file offset `0x3C`, a little-endian pointer to the PE signature. The old DOS stub often prints “This program must be run under Microsoft Windows.”

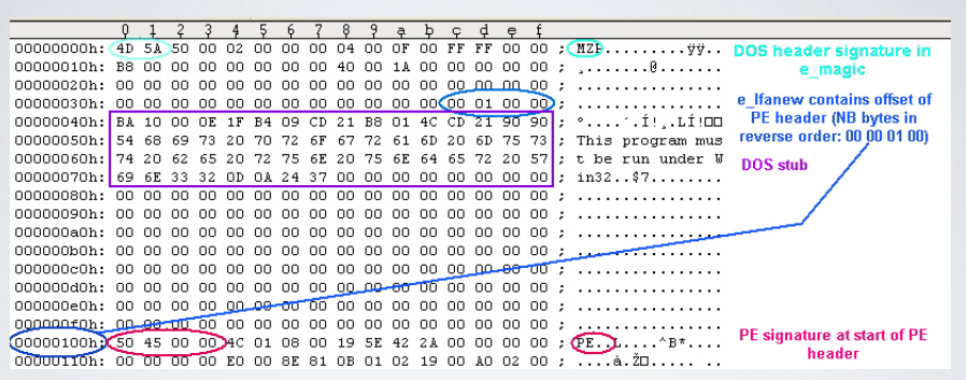


Figure 6.9: Hex view of the header: MZ, e_lfanew equal to 00 01 00 00 in reverse order, the DOS stub, and the PE signature at the start of the PE header (slides, Basic static analysis).

In the lecture’s hex view, `e_lfanew` contains `00 01 00 00`, giving offset `0x100`. At that offset the signature is `50 45 00 00`, that is `PE\0\0`. This is a concrete example of **following a field rather than guessing where a structure begins**.

> **Simone’s note (original wording)**
>
> if the first 2 bytes are being interpreted as MZ then it will be intepreted as an PE file.

> **Correction**
>
> A recognisable MZ alone does not prove a valid PE. The parser must follow `e_lfanew` and find the PE signature and consistent headers. Malformed offsets are worth investigating.

6.5.2 PE header

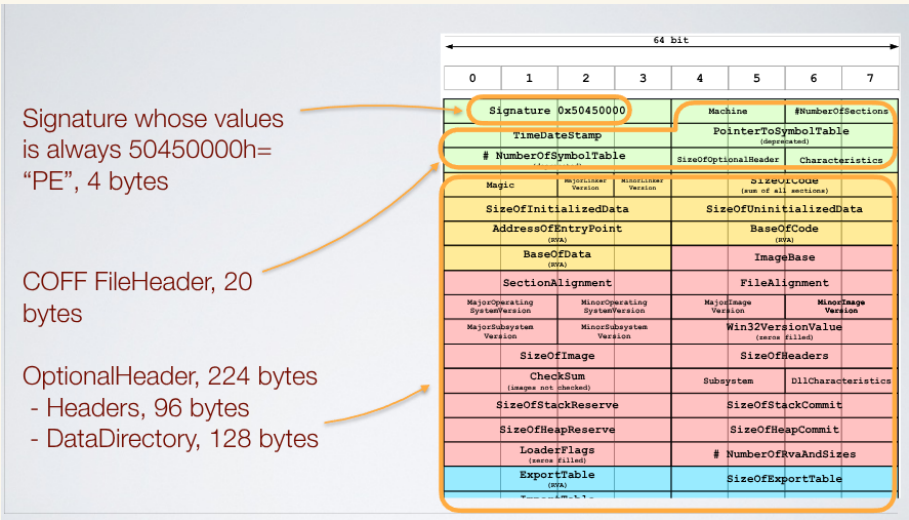


Figure 6.10: PE header layout: signature 50450000h (“PE”), 4 bytes; COFF file header, 20 bytes; optional header, 224 bytes, of which headers 96 and data directory 128 (slides, Basic static analysis).

The PE signature occupies four bytes. It is followed by a **20-byte COFF file header**, an **optional header**, and one **40-byte section header per section**. “Optional” is historical terminology: an executable image needs this header.

The slide’s 224-byte optional-header illustration is a common **PE32** layout with 16 data directories. **PE32+** (64-bit) differs, and the actual size is specified by `SizeOfOptionalHeader`. Always parse the declared variant and size rather than applying one fixed diagram to every binary.

6.5.3 Signature and file header

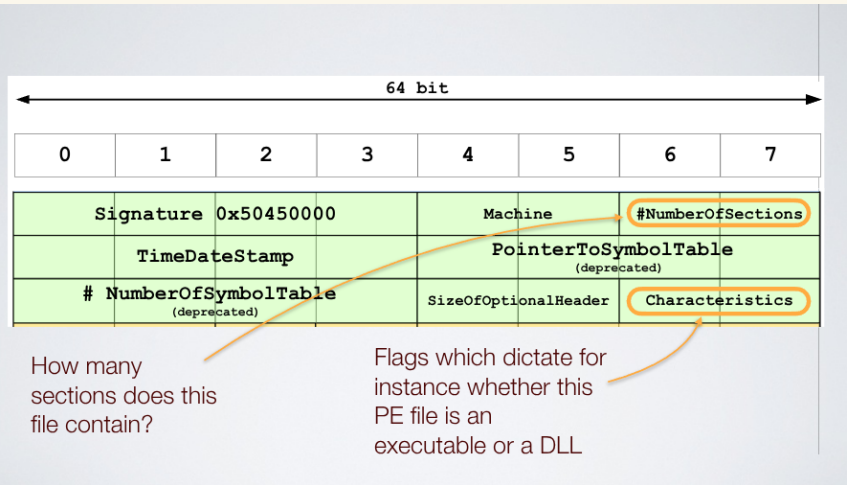


Figure 6.11: The signature and the file header, with NumberOfSections and Characteristics highlighted (slides, Basic static analysis).

- `Machine` identifies the target architecture.
- `NumberOfSections` tells the parser how many section records follow.
- `SizeOfOptionalHeader` locates the section table.
- `Characteristics` contains flags such as executable-image or DLL status.
- `TimeDateStamp` is a clue about build time, not reliable proof of it (section 6.5.9).

The COFF symbol-table fields shown in the diagram are generally unused for executable images.

6.5.4 Optional header: entry point, base and alignment

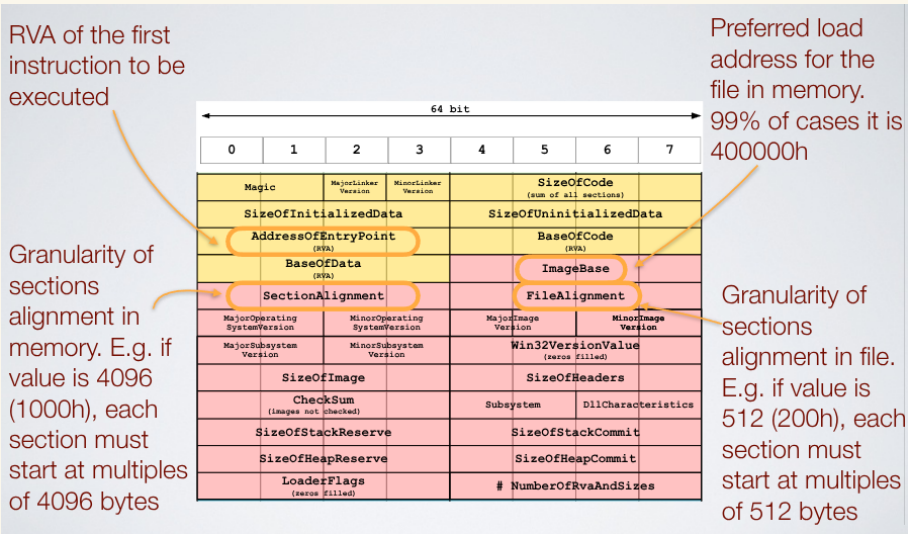


Figure 6.12: Optional header with AddressOfEntryPoint, ImageBase (“99% of cases it is 400000h”), SectionAlignment and FileAlignment (slides, Basic static analysis).

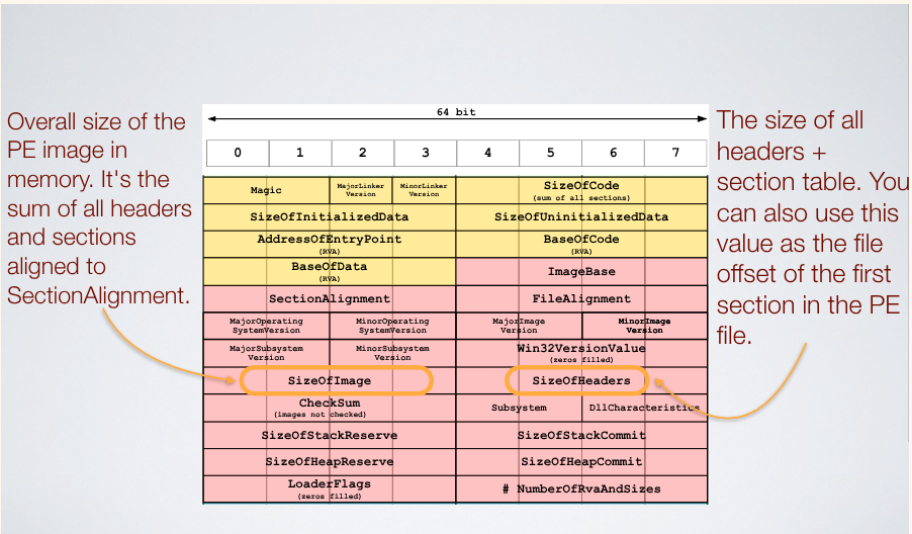


Figure 6.13: Optional header with SizeOfImage and SizeOfHeaders (slides, Basic static analysis).

- **`AddressOfEntryPoint`** is a relative virtual address (RVA): the location where execution starts, relative to the image base. The first code reached may be a runtime stub or packer rather than the application payload.
- **`ImageBase`** is the preferred memory base. `0x400000` is a familiar older 32-bit example from the slide, **not a universal value**: relocation and address-space layout randomization (ASLR) can move the image.

- **`SectionAlignment`** controls section placement in memory, and **`FileAlignment`** controls section placement on disk. Illustrative values are `0x1000` (4096) and `0x200` (512).
- **`SizeOfImage`** is the aligned extent of the mapped image in memory, including headers and sections.
- **`SizeOfHeaders`** describes the aligned on-disk header area. It often coincides with the raw offset of the first section in ordinary files, but `PointerToRawData` of the section is the field to use as its actual offset.
- **Data directories** point to structures such as imports, exports, resources and relocations. They are part of why a PE parser can find useful evidence without disassembling code.

> **Simone's note (original wording)**
>
> remember it is Relative not absolute. notice that in practice the starting point of the first instruyction is randomized by the winows operative system for security reasons.

> **Correction**
>
> What ASLR randomises is the **load base of the image**, not the entry-point field. The RVA in the header stays the same. The in-memory address of the first instruction is actual image base + RVA, so it changes only because the base changes.

6.5.5 RVA versus file offset

A **file offset** counts bytes from the beginning of the on-disk file. An **RVA** counts bytes from the mapped image base. For an RVA inside a section, a typical conversion is:

$$\text{file offset} = \texttt{PointerToRawData} + (\text{RVA} - \texttt{VirtualAddress})$$

This holds provided the RVA refers to bytes backed by the raw data of that

section. The in-memory virtual address is actual image base + RVA. Alignment and zero-filled memory mean that not every RVA has corresponding file bytes.

6.5.6 Section table

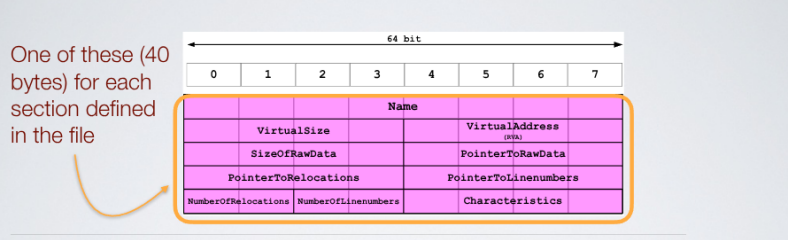


Figure 6.14: One 40-byte header for each section defined in the file (slides, Basic static analysis).

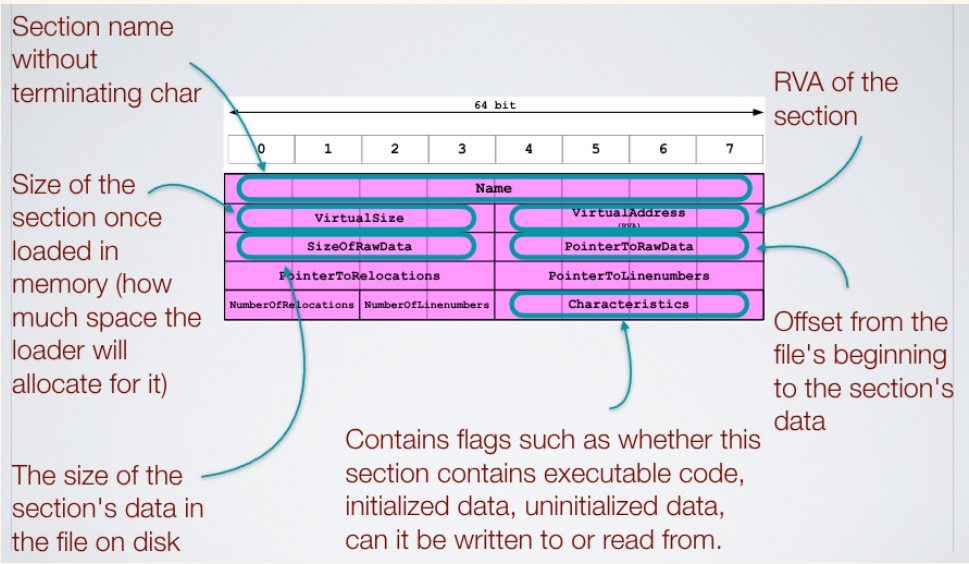


Figure 6.15: Annotated IMAGE_SECTION_HEADER (slides, Basic static analysis).

Each 40-byte `IMAGE_SECTION_HEADER` describes one section.

Field	Meaning	Why it matters
`Name`	Up to eight bytes of label; need not be null terminated	A clue, not proof of the section's purpose
`VirtualAddress`	RVA of the section in memory	Locates data after mapping
`VirtualSize`	Unrounded section size in memory	Compare with disk size and map bounds
`SizeOfRawData`	Size of initialised section bytes in the file, usually file aligned	Large differences from memory size can merit investigation
`PointerToRawData`	On-disk starting offset	Locates the section's bytes
`Characteristics`	Code or data, and read, write, execute flags	Intended mapping; unusual permissions

Older fields concerning relocations and line numbers also appear in the structure, but are not the primary clues for a typical image. The section table describes **where** the data is. The actual section contents follow in the file.

6.5.7 Sections on disk and in memory

On disk, raw section data is normally placed at offsets aligned to `FileAlignment`, and padding may appear between regions. In memory, sections are mapped at RVAs aligned to `SectionAlignment`, often a page-sized value.

The slide gives 4 KB as the x86 page size and 8 KB for IA-64. **Read the alignment fields of the file** instead of relying on an architecture-specific constant. Padding in the file and gaps or zero-filled space in memory are different things.

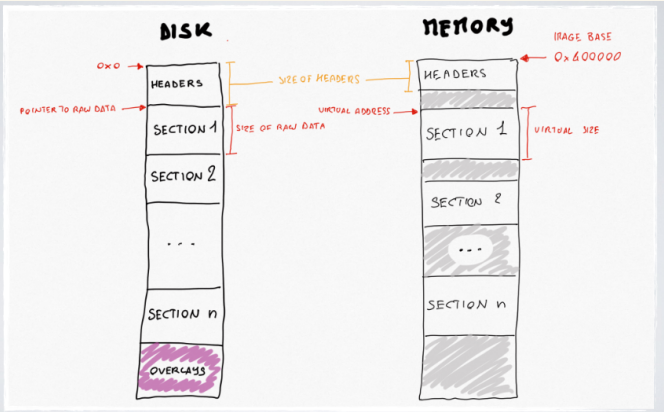


Figure 6.16: Hand drawing on the slide: linear disk file on the left, mapped image on the right. PointerToRawData, SizeOfRawData, VirtualAddress and VirtualSize are marked, and the overlay exists only on disk (slides, Basic static analysis).

The diagram contrasts a linear disk file with its mapped memory image. Headers start at file offset zero. The raw bytes of each section begin at `PointerToRawData`, but the loader maps the section at actual image base + `VirtualAddress`. `SizeOfRawData` and `VirtualSize` need not match.

An **overlay** is extra on-disk data after the mapped sections. It is not automatically mapped as a PE section, though code can read it explicitly. This distinction explains why a position in a hex view of the file cannot be treated directly as a process memory address.

6.5.8 Virtual size versus raw size

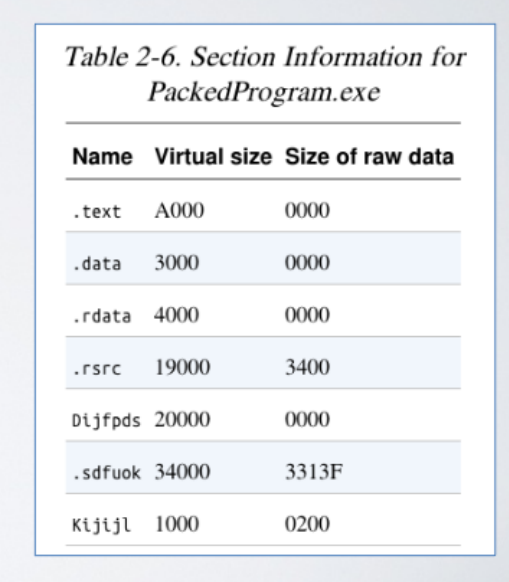


Table 2-6. Section Information for PackedProgram.exe

Name	Virtual size	Size of raw data
.text	A000	0000
.data	3000	0000
.rdata	4000	0000
.rsrc	19000	3400
Dijfpds	20000	0000
.sdfuok	34000	3313F
Kijijl	1000	0200

Figure 6.17: Section information for PackedProgram.exe (book table 2-6 on the slide).

The lecture compares `VirtualSize` (memory) and `SizeOfRawData` (disk). In an ordinary code section they may be close. A packed image may have a virtual size much larger than its raw size, because the stub expands data into memory. The table on the slide reads:

Name	Virtual size	Size of raw data
.text	A000	0000
.data	3000	0000
.rdata	4000	0000
.rsrc	19000	3400
Dijfpds	20000	0000
.sdfuok	34000	3313F
Kijijl	1000	0200

Several sections have memory space but no bytes on disk, and three section names are meaningless. This is only a **heuristic**: uninitialised data, zero filling,

file alignment and unusual but legitimate layouts also produce size differences. Correlate with the entry point, import table, section permissions and content.

6.5.9 Time date stamp

The COFF `TimeDateStamp` can be useful for chronology and clustering, but it can be wrong, manipulated or produced by a reproducible build. The slide notes a historical artifact: all Delphi programs show June 19, 1992. Treat the timestamp as a clue to corroborate with other evidence, never as proof of when malware was created or deployed. Age alone does not determine antivirus detection.

6.5.10 Resource Hacker

The `.rsrc` area may contain icons, dialogs, menus, version data, embedded strings or other resources. A resource viewer such as **Resource Hacker** helps reveal disguise, branding, decoy content and embedded payload material. Resources can be absent, renamed or stored outside a conventional `.rsrc` section. Viewing them does not execute the file if the tool is strictly parsing it.

6.6 Clues in libraries: imports

PE **imports** name DLLs and functions that the loader may resolve. An imported function is implemented outside the importing module, often in a DLL. For example, `URLDownloadToFile` suggests download capability.

An imported function is evidence of a dependency or possible capability, **not proof that the relevant call is reached**. Conversely, missing imports do not rule out a behaviour: functions can be linked statically, resolved at runtime or invoked through other components.

Linking specifies how a program obtains external code. The lecture distinguishes three practical cases. In common terminology “dynamic linking” can include both load-time and runtime resolution, so the last two categories below are best understood as **load-time versus explicit runtime** use of DLLs.

Case	What happens	Effect on triage
Static linking	Library code is copied into the built executable	Use of library functions may no longer appear as named imports
Dynamic linking (load time)	The import table names DLLs and functions; the loader resolves them as the module is loaded	Import inspection is a useful first pass, though it does not show transitive dependencies or every future path
Runtime linking	The program loads a DLL when needed, often with `LoadLibrary`, and gets an address with `GetProcAddress`	Can obscure capabilities: investigate strings, code and runtime behaviour

Static linking gives a larger and more self-contained result. It is possible on Windows as well as on Unix-like systems, and its prevalence depends on the build and library rather than on a hard platform rule. “Self-contained” does not necessarily mean that the program has no OS dependencies.

Runtime linking is common in plugins and optional features. When the import table lists only these resolver functions, do not conclude anything about the other APIs in use before investigating further.

6.6.1 Dependency Walker

Dependency Walker visualises imported modules and functions, and can show exports. The lecture contrasts a normal `services.exe` view with a suspicious `services.ex_` sample whose displayed imports are mainly `KERNEL32.DLL` and `WS2_32.DLL`.

The **similar filename** is not identity: hash and inspect the actual file. Few static dependencies can signal packing or runtime resolution, but a small utility can also legitimately import little. The large dependency tree in the screenshot can include **indirect dependencies**, so distinguish what the analysed file imports directly from what its DLLs import in turn.

The import and export panes answer different questions: **which external functions does this module request?** and **which functions does this module expose to others?** Legacy tools may misinterpret newer Windows API-set mappings, so cross-check surprising results with another PE parser or the raw import table.

6.6.2 Common DLLs

DLL	Typical role	Interpretation in triage
`Kernel32.dll`	Common process, memory, file and other core Win32 functions	Extremely common: inspect **which functions** are imported
`Advapi32.dll`	Registry, services, security and other advanced interfaces	Could point to persistence or privilege-related activity, but has many benign uses
`User32.dll`	Windows UI and user input	Hooks and hotkeys can be relevant to input monitoring; UI software also uses it
`Gdi32.dll`	Graphics and display operations	May relate to display manipulation; common in graphical programs
`Ntdll.dll`	Low-level NT interfaces and runtime support	Direct imports can be interesting, but are not automatically malicious; indirect dependency is normal
`WSock32.dll`, `Ws2_32.dll`	Socket and network interfaces	Suggest network-capable code; inspect actual functions and traffic
`Wininet.dll`	Higher-level Internet protocol APIs	May support HTTP or FTP communication; imports do not reveal the endpoint or prove transmission

These are **orienting clues**, not a classification system. DLL names say less than the particular functions, their call sites and the observed effects.

Simone's note (original wording)

Ntdll is interesteing because microsoft didn't created a documentation on that because was not meant to be public, but programmers by reversee enegeneering it they found out how to use it.

Simone's note (original wording)

wininet works on top WSock32 i suppose

To verify

The second personal note is marked as a supposition by its author, and the slides do not confirm it. Chapter 9 only states that WinINet is a higher-level API than Winsock (section 9.6.1).

6.6.3 Example: a keylogger

The slide combines three clues:

- import of `User32.dll` and `SetWindowsHookEx`;
- exports called `LowLevelKeyboardProc` and `LowLevelMouseProc`;
- use of `RegisterHotKey`.

A hook can arrange callbacks for input events, and a hotkey can trigger an action. Together these motivate closer examination of keyboard and mouse monitoring and of a possible collection workflow.

The exported procedure names alone do **not** demonstrate that data is sent elsewhere: that requires evidence of storage or transmission. `SetWindowsHookEx` and hotkeys have legitimate uses too. Trace the callback registration, the recorded data and the subsequent data flow before labelling the behaviour.

6.6.4 Example: a packed program

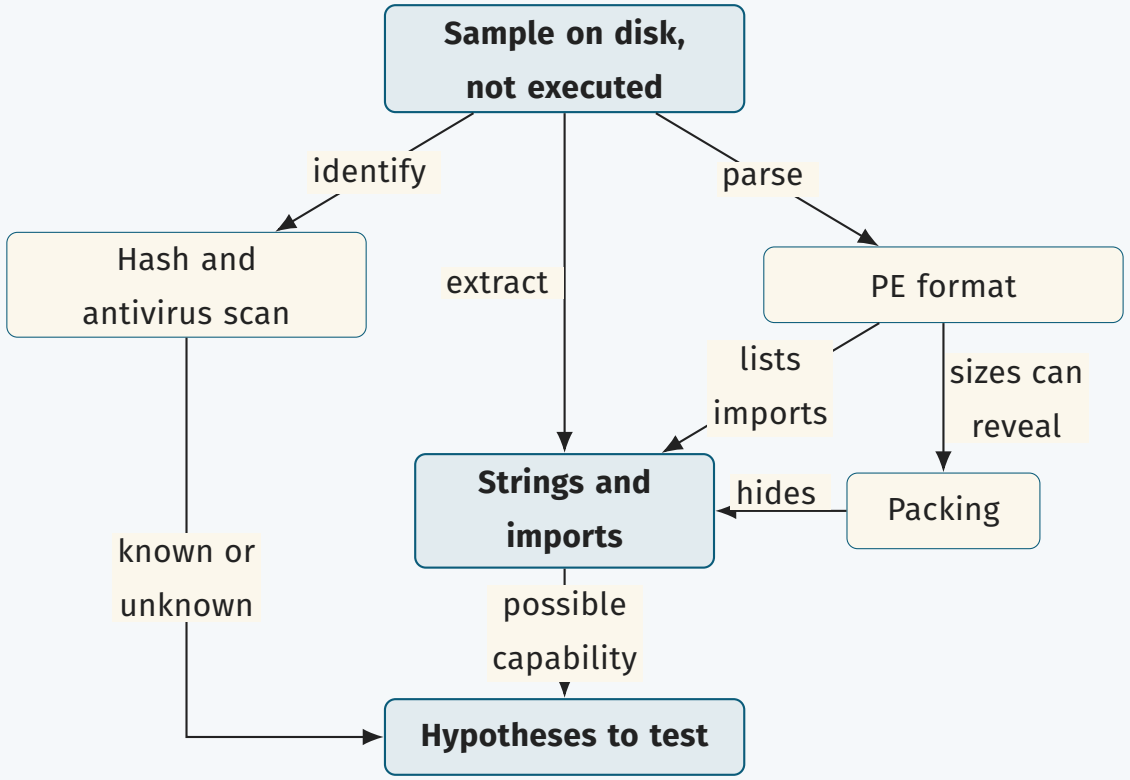
The slide's compact import list includes `GetModuleHandleA`, `LoadLibraryA`, `GetProcAddress`, `ExitProcess`, `VirtualAlloc`, `VirtualFree` and `MessageBoxA`. This is consistent with a stub that allocates memory and resolves additional APIs after startup.

Static imports in such a case primarily describe the **unpacker**. The absence of filesystem, registry or network APIs in that list cannot rule out those capabilities in the unpacked program.

6.6.5 Exports

DLLs commonly **export** named or ordinal-addressed functions, and EXEs commonly import functions. Both directories are described by PE data directories, although either may be absent. EXEs can export functions too, and DLLs can import from other DLLs. Thus "DLL exports, EXE imports" is a useful default mental model, not a format rule. An export tells you a function is made available, not whether another module invokes it.

Concept map: how the ideas connect



From the sample on disk, the hash and the scan tell whether the exact file is known. The PE format lists the imports, and its section sizes can reveal packing. Packing hides strings and imports. What stays visible shows only possible capability, so every clue is a hypothesis to test.

Remember

- A hash identifies one exact file, not a family.
- Uploading a sample is disclosure, and a clean scan is not proof of safety.
- Packing hides strings and imports, so what you see may describe only the stub.
- Follow PE fields rather than guessing positions, and do not trust section names.
- An RVA is relative to the actual image base, and it is not a file offset.
- An import shows possible capability, not an executed behaviour.

Review: try to answer without looking

1. Where is `e_lfanew`, and what does it give?
2. A section has `VirtualAddress` `0x1000` and `PointerToRawData` `0x400`. Which file offset corresponds to RVA `0x1234`?
3. The import table lists only `LoadLibraryA` and `GetProcAddress`. Can the program use the network?
4. What should you check before saying that a PEiD plugin is safe to use?

Answers, with the section where to find them

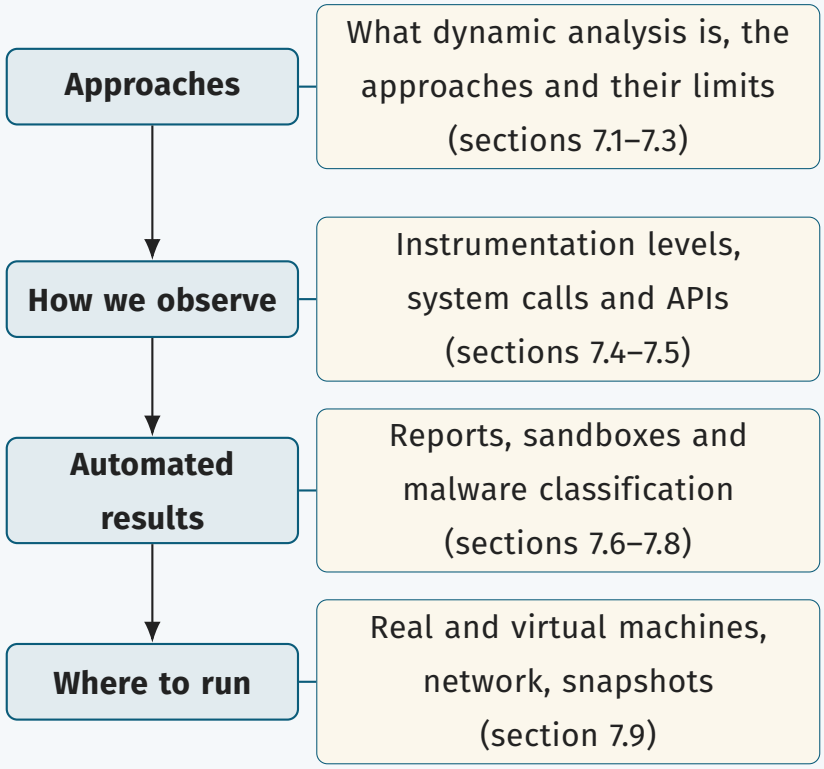
1. At file offset `0x3C`; it gives the offset of the PE signature (section 6.5.1).
2. `0x400` + (`0x1234` − `0x1000`) = `0x634`, if the bytes are backed by raw data (section 6.5.5).
3. Yes: it can resolve network functions at runtime (section 6.6).
4. Whether it executes the file; if unknown, use it only in an isolated VM (section 6.4.1).

Chapter 7

Basic dynamic analysis: foundations

> **Before you start**
>
> **Central question:** if we run the sample on purpose, what can we see, how is the observation possible, and how do we keep the run safe?
>
> **Prerequisites:** static triage (chapter 6) and the safety rules (section 2.2).
>
> **Where it leads:** chapter 8 presents the tools. Chapter 12 shows how a sample notices the analysis, and chapter 11 develops the debugger, only named here.

Chapter structure: what comes, and in which order



The chapter first asks what we can learn by running a sample. It then explains how observation works and what an automated system returns. It ends with the machines on which to run the sample safely.

7.1 What dynamic analysis is

Dynamic analysis means deliberately running a malware sample while monitoring the results. Executing hostile code has an obvious consequence:

- it **requires a controlled, safe environment**;
- it **must not spread to production machines**;
- real machines can be **air-gapped**, that is, without a network connection to the Internet or to other machines.

Why bother, when static analysis does not need all this? Because static analysis can reach a **dead end**, due to obfuscation, packing (section 6.4), or the examiner having simply exhausted the available static techniques.

The idea is that dynamic analysis will show you exactly what the malware does. The slide immediately adds: “Not really...”. You only see what happens in **this** run, in **this** environment (section 7.3). The **main goal** is to understand the behaviour of the malware.

7.2 Approaches to behavioural analysis

Approach	What you do	Pro	Con
Diffing	Snapshot a clean state and a compromised state, then compare	Artifacts can be observed easily	Misses evidence created and purposely erased during the activity
System monitoring	From a clean state, record every change in the system and in network traffic after execution	Collects all manifested changes, including those diffing misses	Often too much information; irrelevant data must be weeded out
API tracing	Hook and record the important API calls of the process	Visibility beyond the file, process, registry and network view; a little closer to static interpretation	Again too much information; interpretation can take a lot of time, but less than static analysis
Debugging	Set breakpoints to stop at a given location and inspect state	A superset of the functionality of an API monitor	Usually needs basic static analysis and assembly reading; malware often changes behaviour or refuses to run when debugged

A temporary dropper that deletes itself, for example, never appears in the "after" snapshot of a diff. The tools named on the slides are these:

- diffing: `regshot`, `autoruns`;
- system monitoring: `procmon`, Wireshark;

- API tracing: Rohitab API Monitor, WinApiOverride;
- debugging: IDA, OllyDbg, Immunity Debugger, WinDbg.

7.2.1 Choosing a technique

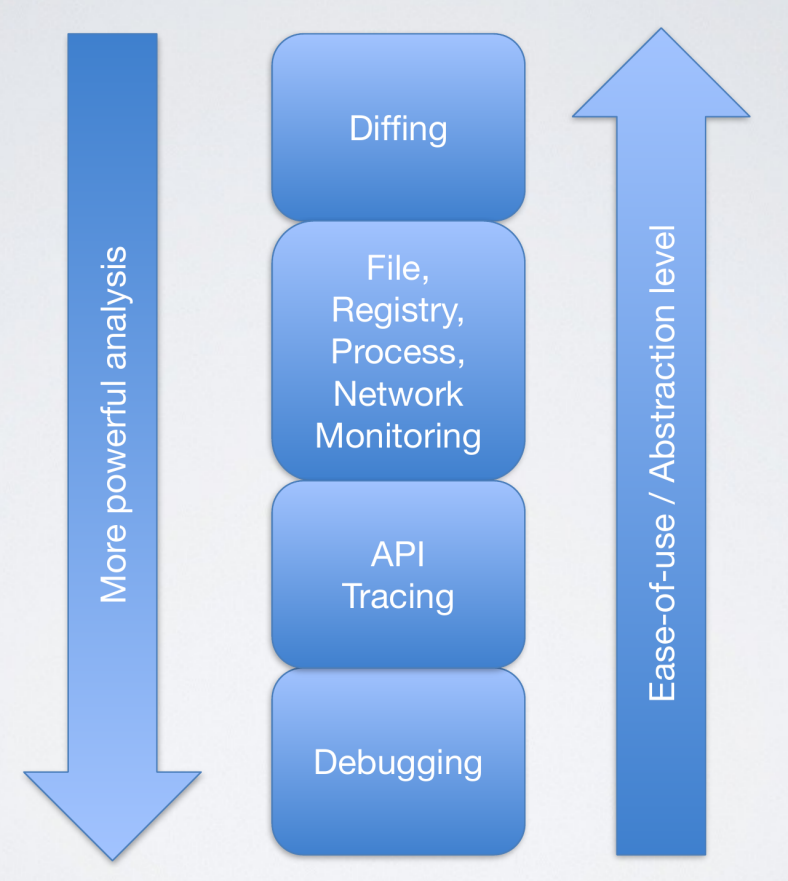


Figure 7.1: The four approaches as a trade-off between analysis power and ease of use (slides, Basic dynamic analysis).

The four approaches are a trade-off. Going **down** the stack (diffing, monitoring, API tracing, debugging), the analysis gets **more powerful**. Going **up**, **ease of use and abstraction level** increase. In practice you start from the top and go down only when the cheaper technique no longer answers your question.

7.3 Limits of dynamic analysis

- **In general, a single path (execution trace) is examined.** A branch that needs a specific date, command or file does not run, so its behaviour stays invisible.
- **The analysis environment is possibly not invisible.** The sample can detect it (chapter 12).
- **The analysis environment is possibly not comprehensive.** It may not record everything.
- **Scalability issues.** Each run takes real time and resources.

7.4 Instrumentation levels

How is the observation technically performed? By **instrumenting** the program, the operating system or the hardware.

Program instrumentation. The analysis operates **in the same address space** as the sample.

- Examples are manual analysis with a debugger, or **Detours**, a Windows API hooking mechanism.
- The binary under analysis is **modified**: breakpoints are inserted, functions are rewritten, debug registers are used.
- It is **not invisible**: malware can detect the analysis artifacts.
- It may require significant manual effort.

OS instrumentation. The analysis operates **in the OS** where the sample runs, for instance with Windows **system call hooks**.

- It is somewhat invisible to **user-mode** malware.
- It can cause problems when malware runs **in the OS kernel**, at the same privilege as the monitor.

- It has **limited visibility of activity inside the program**: for example, you cannot set function breakpoints.

Hardware instrumentation. The analysis provides **virtual hardware** (a processor) on which the sample executes, sometimes including the OS.

- An example is **software emulation** of the executed instructions.
- The analysis observes the activity "from the outside".
- It is **transparent** to the sample and to the guest OS.
- An **OS environment needs to be provided**: a limited environment could be detected but is faster, while a complete one is more comprehensive but slower.
- This is the approach behind **sandboxes** (section 7.7).

> **Do not confuse**
>
> **"Transparent" is an ideal.** Nothing inside the guest is modified, so this level has no in-guest artifacts. It is still not undetectable: section 12.3 shows how the virtual hardware itself can be revealed.

Level	Where the analysis lives	Visibility inside the program	Main weakness
Program	Same address space as the sample	Highest	Leaves artifacts the sample can detect
OS	OS where the sample runs	Limited: calls, not functions	Kernel-mode malware
Hardware	Outside the guest	Needs reconstruction (the semantic gap, section 12.11.2)	Environment and speed trade-off

7.5 What to observe: system calls and Windows APIs

A process interacts with the OS via **system calls**: it needs the OS for every interaction with the environment, such as the file system, network and registry. So monitoring system calls is a natural choice, with a Windows-specific catch:

- many Windows system calls (**Native APIs**) are **undocumented and can change without notice**;
- developers are expected to use the **Windows APIs**, a collection of stable user-mode shared libraries;
- of course, **Windows APIs can be bypassed**: malware can call the Native API directly (section 9.3), so hooking only the upper layer can miss activity.

7.6 What an analysis report contains

The typical output is organised by activity:

- **file** activity: read, write, create, open and so on;
- **registry** activity;
- **service** activity: start and stop of Windows services, via the Service Manager;
- **process** activity: start and terminate processes, inter-process communication;
- **network** activity: API calls and packet logs.

These are the same categories as the expected behaviours guessed during static triage (section 2.5): dynamic analysis confirms or refutes them.

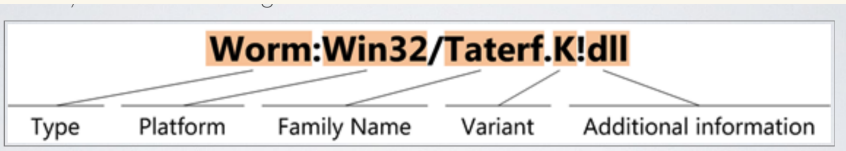
7.7 Sandboxes

A **sandbox** is all-in-one software for basic dynamic analysis: a virtualised environment that also simulates network services.

- Examples are Norman Sandbox, GFI Sandbox, Anubis, Joe Sandbox, ThreatExpert, BitBlaze and Comodo Instant Malware Analysis.
- They are **expensive but easy to use**, and some offer a free tier.
- They **automate** dynamic analysis, which is why they scale to the daily flood of samples.
- They produce a nice **PDF report** of results. An example is `https://www.joesecurity.org/joe-sandbox-reports`.

7.8 Malware classification

A nice by-product of sandboxes is a tentative **family classification**. Naming conventions derive from the **CARO** (Computer Antivirus Research Organization) Malware Naming Scheme.



Vendor	Name Convention	Example
Symantec	Prefix.Name.Suffix	Infostealer.Banker.C
Avira	Prefix:Name [Type]	Win32:Zbot-BS [Trj]
Kaspersky	[Prefix:]Behaviour.Platform.Name[.Variant]	Trojan.Win32.Genome.taql

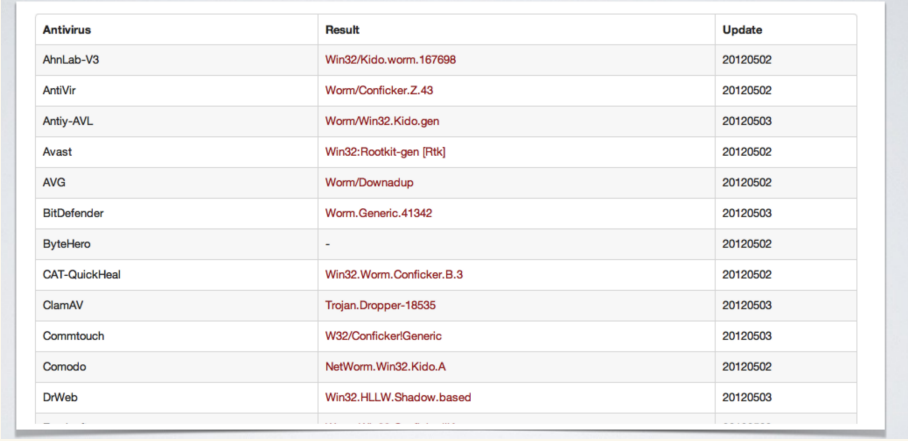
Figure 7.2: The CARO scheme applied to Worm:Win32/Taterf.K!dll, and vendor variations (slides, Basic dynamic analysis).

The name `Worm:Win32/Taterf.K!dll` splits into **type** (`Worm`), **platform** (`Win32`), **family name** (`Taterf`), **variant** (`K`) and **additional information** (`dll`). Each vendor uses its own variation:

Vendor	Name convention	Example
Symantec	`Prefix.Name.Suffix`	`Infostealer.Banker.C`
Avira	`Prefix:Name [Type]`	`Win32:Zbot-BS [Trj]`
Kaspersky	`[Prefix:]Behaviour. Platform.Name[.Variant]`	`Trojan.Win32.Genome.taql`

> **Correction**
>
> The slide attributes `Win32:Zbot-BS [Trj]` to **Avira**. That format looks like **Avast**'s: in the next slide's table, Avast reports `Win32:Rootkit-gen [Rtk]` (same shape), while AntiVir, the Avira engine, reports `Worm/Conficker.Z.43`.

Classification schemes are hardly coherent. Here is the same sample as seen by different antivirus engines:



Antivirus	Result	Update
AhnLab-V3	Win32/Kido.worm.167698	20120502
AntiVir	Worm/Conficker.Z.43	20120502
Antiy-AVL	Worm/Win32.Kido.gen	20120503
Avast	Win32:Rootkit-gen [Rtk]	20120502
AVG	Worm/Downadup	20120502
BitDefender	Worm.Generic.41342	20120503
ByteHero	-	20120502
CAT-QuickHeal	Win32.Worm.Conficker.B.3	20120502
ClamAV	Trojan.Dropper-18535	20120503
Commtouch	W32/ConfickerGeneric	20120503
Comodo	NetWorm.Win32.Kido.A	20120502
DrWeb	Win32.HLLW.Shadow.based	20120503

Figure 7.3: The same specimen as named by different antivirus engines (slides, Basic dynamic analysis).

The same specimen is called `Kido` (AhnLab, Antiy, Comodo), `Conficker` (AntiVir, CAT-QuickHeal, Commtouch), `Downadup` (AVG), `Rootkit-gen` (Avast) and even `Trojan.Dropper` (ClamAV), while ByteHero does not detect it at all. So a vendor label is a **hint** rather than an identity: engines disagree on both the family and the type.

7.9 Where to run: real and virtual machines

7.9.1 Real machines

Analysis can also run on physical hardware.

- Disadvantage: no Internet connection, since the machine is air-gapped, so parts of the malware may not work.
- Disadvantage: malware can be difficult to remove, so **re-imaging** the machine will be necessary.
- Advantage: some malware detects virtual machines and will not run properly in one.

7.9.2 Virtual machines

VMs are **the most common method**:

- they protect the host machine from the malware;
- **VM-escape** attacks remain possible, but are currently unlikely;
- **naive VM usage is way more dangerous**: shared folders and a bridged network, for example, connect the sample to the host and to the LAN;
- you can easily **snapshot** the VM and revert to a clean state at the end of each analysis job.

Oracle VirtualBox is the tool used in the course: it is free, can read and write VMware disks, has reasonable performance for the goals of the course, and lets you take several snapshots and revert to them.

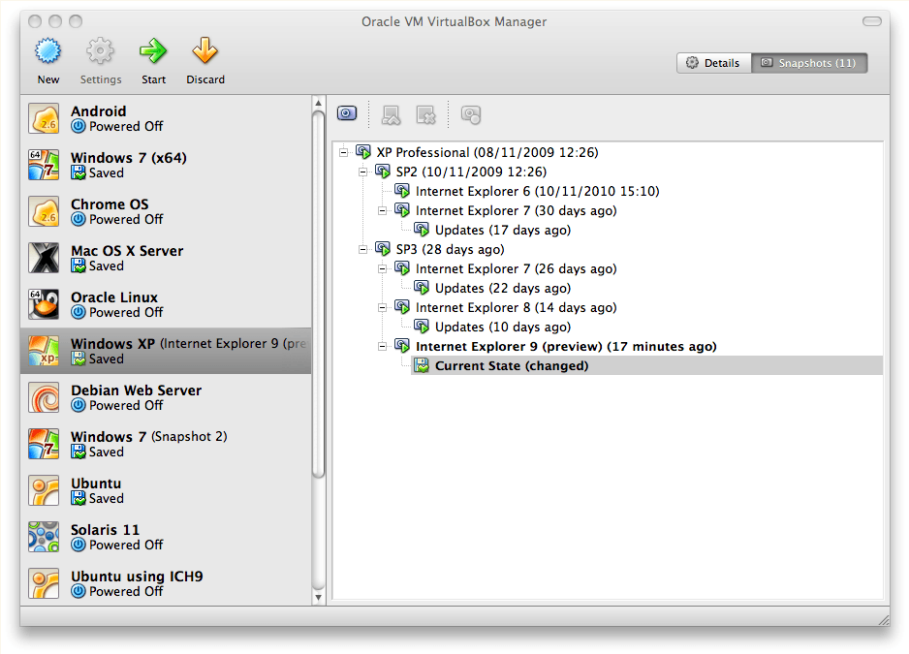


Figure 7.4: VirtualBox Manager: VMs on the left, snapshot tree of a Windows XP machine on the right (slides, Basic dynamic analysis).

The VirtualBox Manager shows the list of VMs on the left and, on the right, the **snapshot tree** of the selected machine. Each node is a saved state, for example with a specific Internet Explorer version, and `Current State` is where the VM is now. Snapshots therefore form a tree, not a line.

7.9.3 Configuring the network

You can **disable networking** by disconnecting the virtual network adapter while the VM is running. **Host-only networking** allows traffic to the host but not to the Internet.

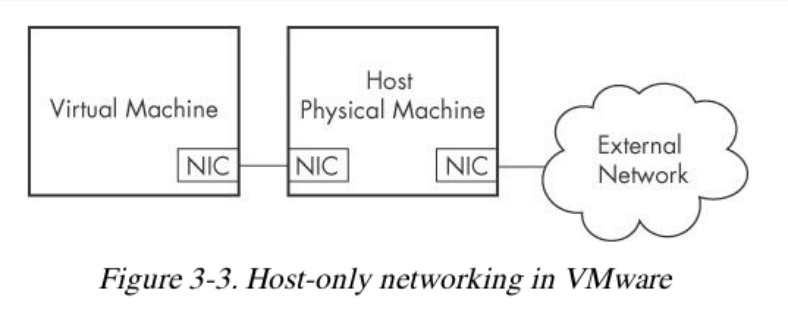


Figure 7.5: Host-only networking: the VM's network card talks to a virtual card on the host (slides, Basic dynamic analysis).

In host-only mode the VM's network card talks to a virtual card on the host. The host keeps its own separate card for the external network, and the VM cannot reach that one.

More complex setups are possible. **Ad-hoc networking** can fake external services, for example with host-only networking and a second VM acting as "the Internet".

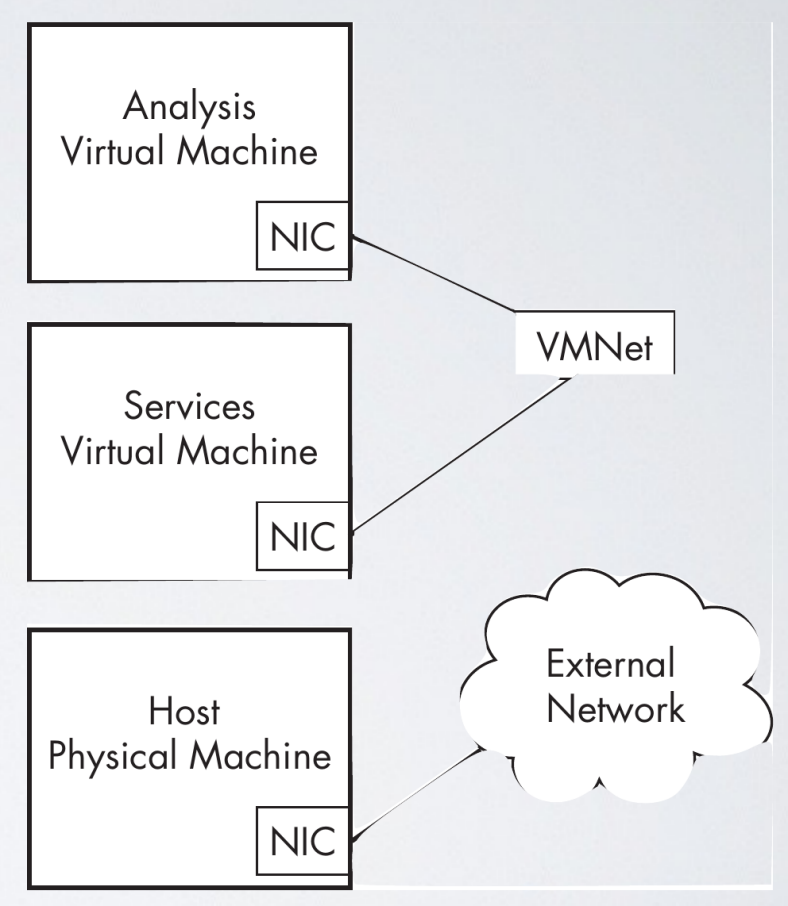


Figure 7.6: Analysis VM and services VM on the same virtual switch, separate from the host's external interface (slides, Basic dynamic analysis).

The **analysis VM** and a **services VM** share the same virtual switch (`VMNet`), while the host stays connected to the external network through a different interface. The malware believes it is online, but it can reach only the fake services (section 8.8.4).

> **From the book**
>
> PMA describes this as the configuration that "combines the best of all options": multiple VMs linked by a LAN but disconnected from the Internet

> and the host (PDF p. 66).

7.9.4 Snapshots

Use **snapshots** to protect a clean installation of the analysis environment, to keep track of ongoing progress during the analysis, and to hop instantly from job to job when analysing several samples at the same time.

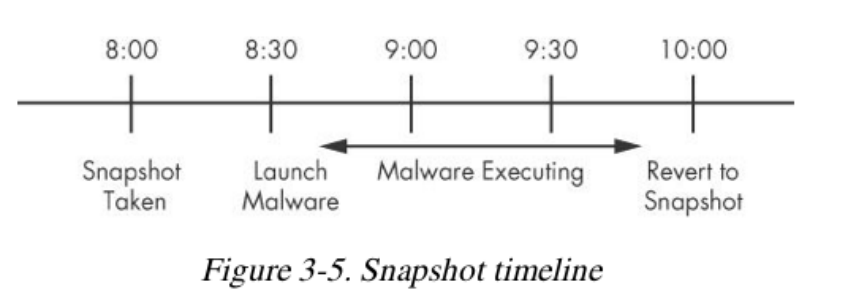


Figure 7.7: Timeline of a snapshot cycle (slides, Basic dynamic analysis).

The timeline is the standard analysis cycle. At 8:00 the snapshot is taken, at 8:30 the malware is launched, it runs until about 10:00, and then the VM reverts to the snapshot. Whatever the sample did is discarded along with that run.

7.9.5 Risks of using a VM

- Malware may **detect that it is in a VM** and run differently (chapter 12).
- Virtualisation environments may have **bugs**: in some cases malware may exploit them to spread or to affect the host.
- The slide states that all the samples analysed in this course are harmless.

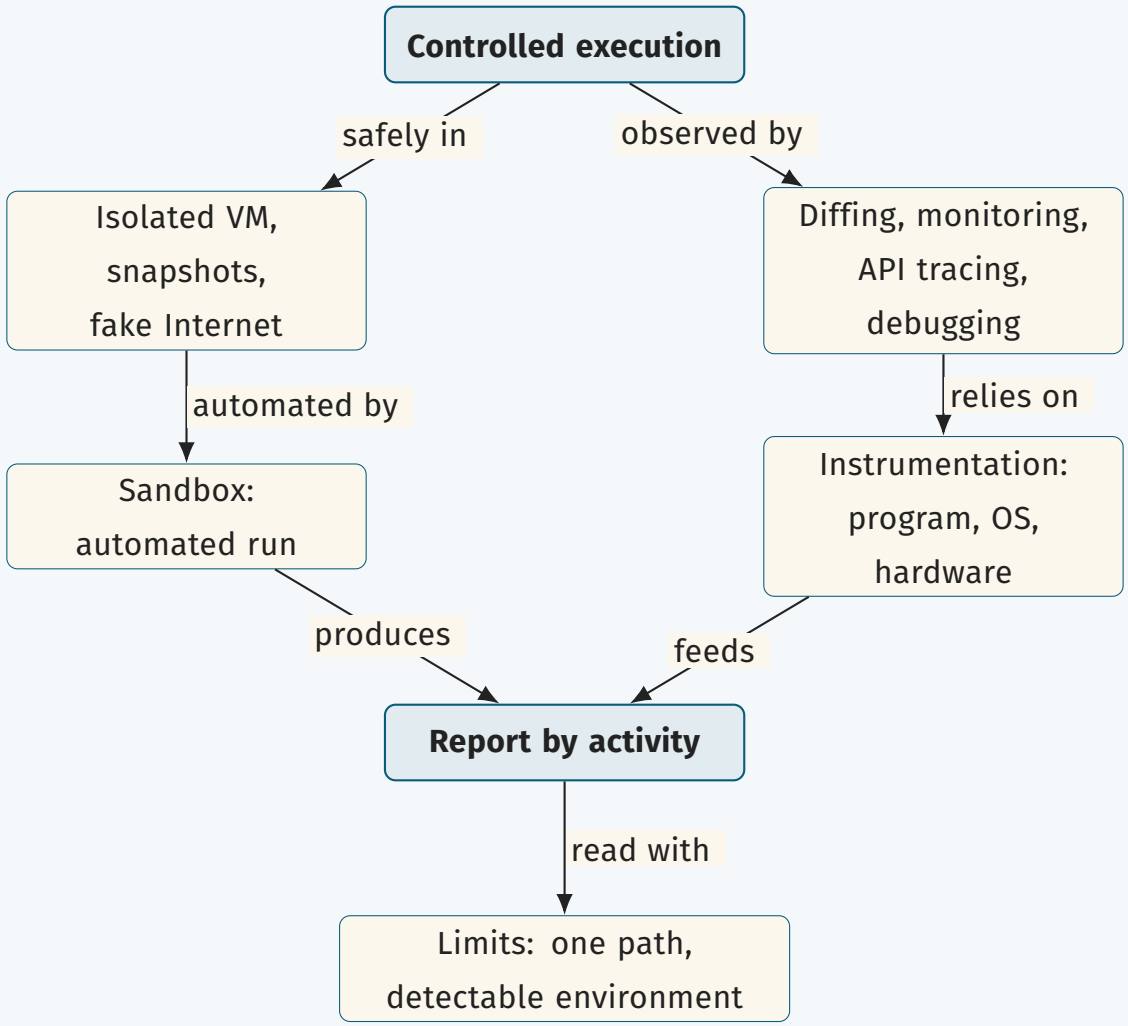
7.9.6 The Windows analysis VM

Most malware you will typically encounter is Windows based, so the course uses a **Windows 10 VM** for analysis.

- Some old malware may not run without **admin privileges**.

- Internal safety protections such as **Windows Defender** need to be disabled, using `gpedit.msc`, to prevent samples from being automatically quarantined or deleted.

Concept map: how the ideas connect



Controlled execution needs a safe environment and a way to observe. A sandbox automates the run, and the approaches rely on instrumentation at some level. Both lead to a report organised by activity. Read that report knowing its limits: it usually shows a single path, in an environment that may be detected.

In short

Dynamic analysis observes behaviour, but usually along a **single path** and in an environment that may be detected. Diffing, monitoring, API tracing and debugging offer increasing power and decreasing ease of use. Observation requires instrumentation at program, OS or hardware level, each with its

own weakness. Use isolated VMs with snapshots, and fake the Internet instead of connecting to it.

Review: try to answer without looking

1. Why can diffing miss a self-deleting dropper?
2. Why does hooking only the Windows APIs leave a blind spot?
3. Which instrumentation level has no in-guest artifacts, and why is it still detectable?
4. Why is a vendor's label not an identity?

Answers, with the section where to find them

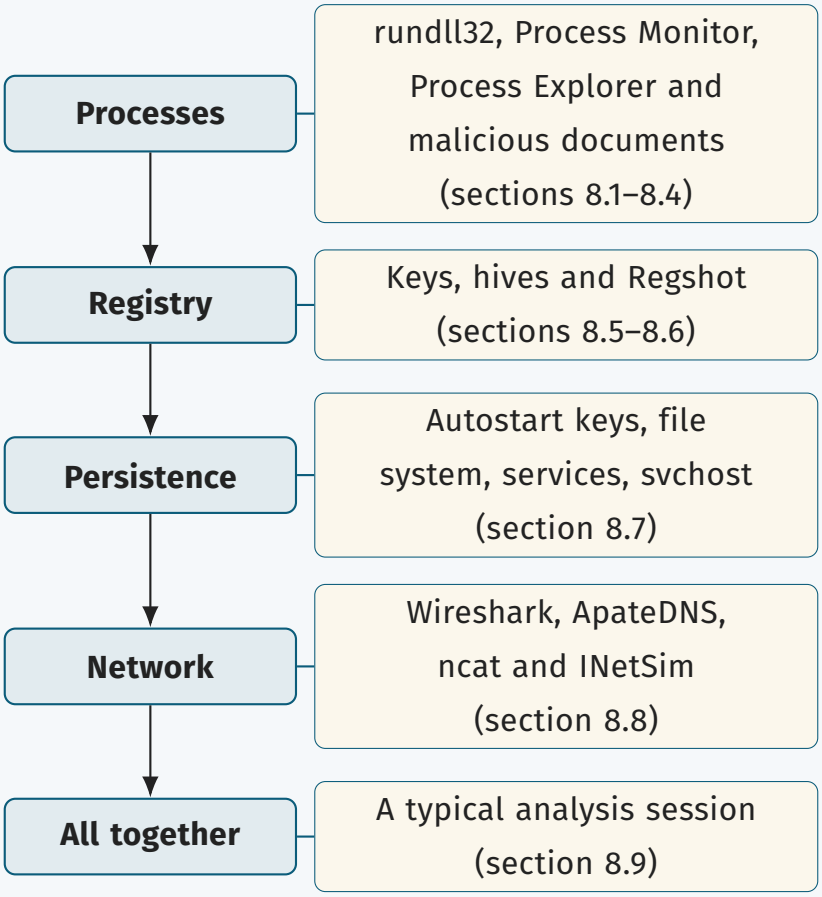
1. It compares only before and after, and the file is gone by then (section 7.2).
2. Malware can call the Native API directly (section 7.5).
3. Hardware; the virtual hardware itself can be revealed (section 7.4).
4. Engines disagree on family and type for the same specimen (section 7.8).

Chapter 8

Basic dynamic analysis: tools, registry and persistence

> **Before you start**
>
> **Central question:** which tool answers which question while the sample runs, and where does a sample leave its persistence?
>
> **Prerequisites:** the approaches and the lab of chapter 7; imports and exports (section 6.6).
>
> **Where it leads:** Lab03 in section 13.2 uses several of these tools. Chapter 9 shows the same registry and service activity as API calls in code.

Chapter structure: what comes, and in which order



The chapter goes through the tools by what they observe: processes, registry and network. Persistence sits between the registry and the network because it builds on the registry. The last section combines the tools in one session.

8.1 Launching DLLs

EXE files can be run directly, but DLLs cannot. Use the Windows tool **Rundll32.exe**:

```
rundll32.exe DLLname, Export arguments
```

`Export` is one of the exported functions found with Dependency Walker, PEview or PE Explorer (section 6.6.5). But many malicious DLLs **do not have an export**: the slide says launching them is covered in a hands-on session later in the

course. Section 9.7.2 explains why code can run on load through `DllMain` without any export call.

8.2 Process Monitor

Process Monitor (procmon) monitors registry, file system, network, process and thread activity.

- **All recorded events are kept**, but you can filter the display to make it easier to find items of interest.
- **Do not run it too long**, or it will fill up all RAM and crash the machine.

> **From the book**
>
> Procmon "doesn't capture everything": it can miss a user-mode component talking to a rootkit via device I/O controls, and certain GUI calls such as `SetWindowsHookEx`. It "usually should not be used for logging network activity" (PDF p. 81). Also, filters change only the **display**, so a filter does not stop procmon from consuming memory (PDF p. 82).

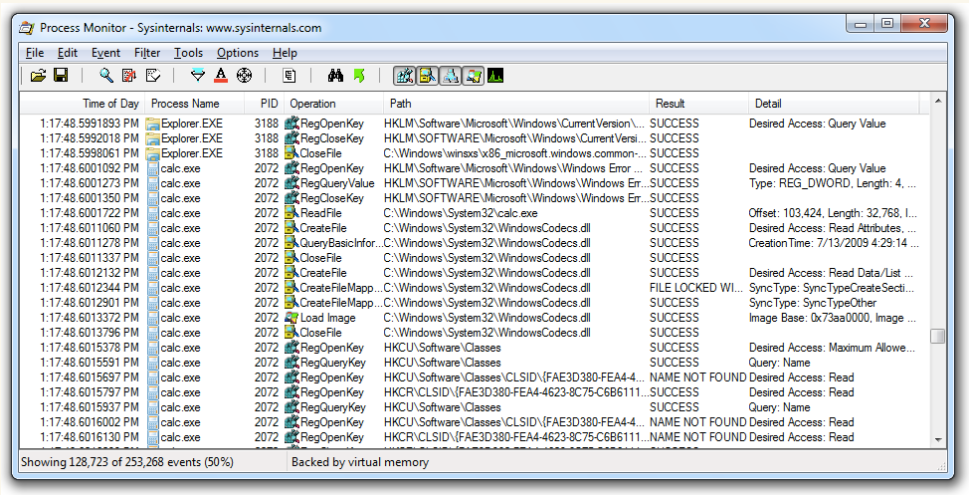


Figure 8.1: Procmon capture after launching calc.exe, filtered on that process (slides, Basic dynamic analysis).

This is the capture after launching `calc.exe`, filtered on that process. Even the calculator generates a long list of `RegOpenKey`, `RegCloseKey`, `CreateFile`

and `QueryBasicInformationFile` events, each with its time, PID, path and result. The status bar shows **128,723 of 250,268 events** displayed, which demonstrates both the cost of monitoring ("too much information") and why filters are essential.

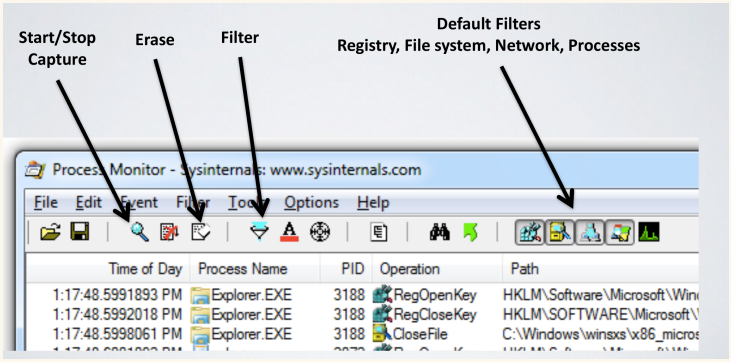


Figure 8.2: The procmon toolbar (slides, Basic dynamic analysis).

The toolbar icons used most often are **Start/Stop Capture** (magnifying glass), **Erase** the displayed events, **Filter**, and the **default filters** by category: Registry, File system, Network, Processes.

8.2.1 Filtering with Exclude

One technique is to **hide normal activity before launching malware**: right-click each Process Name and click **Exclude**. The most useful filters are **Process Name**, **Operation** and **Detail**.

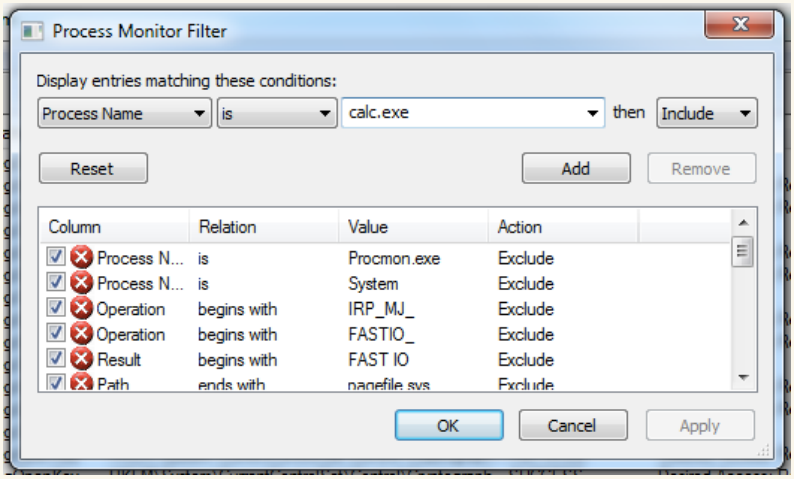


Figure 8.3: The procmon Filter dialog (slides, Basic dynamic analysis).

The Filter dialog builds rules in the form: column, relation, value, then Include or Exclude. Here `Process Name is calc.exe` is being added as an Include, while the default rules exclude noise such as `Procmon.exe`, `System`, operations beginning with `IRP_MJ_` or `FASTIO_`, and paths ending in `pagefile.sys`.

8.3 Process Explorer

Process Explorer is an advanced task manager: it shows the process **tree** with parent and child relationships, and details for each process.

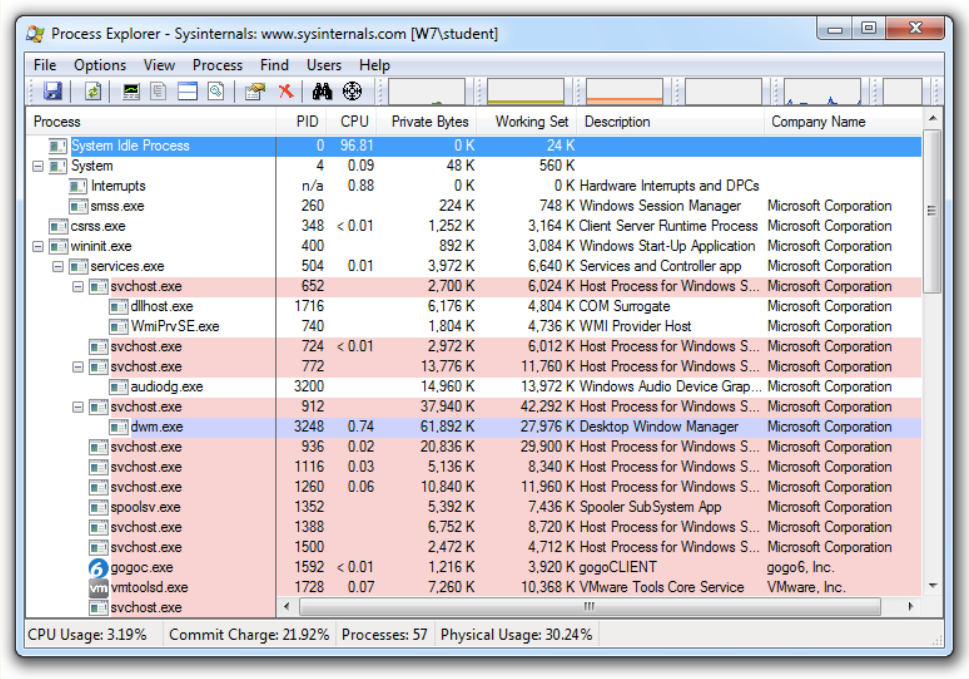


Figure 8.4: Process tree in Process Explorer (slides, Basic dynamic analysis).

The tree shows `services.exe` with its many `svchost.exe` children in pink, and the interactive desktop processes elsewhere. Each row reports PID, CPU, private bytes, working set, description and company name.

Colour	Meaning
Pink	Services
Blue	Processes
Green (briefly)	New processes
Red	Terminated processes

The green and red flashes are useful during analysis, because they show a sample spawning a child that exits quickly.

8.3.1 Loaded DLLs

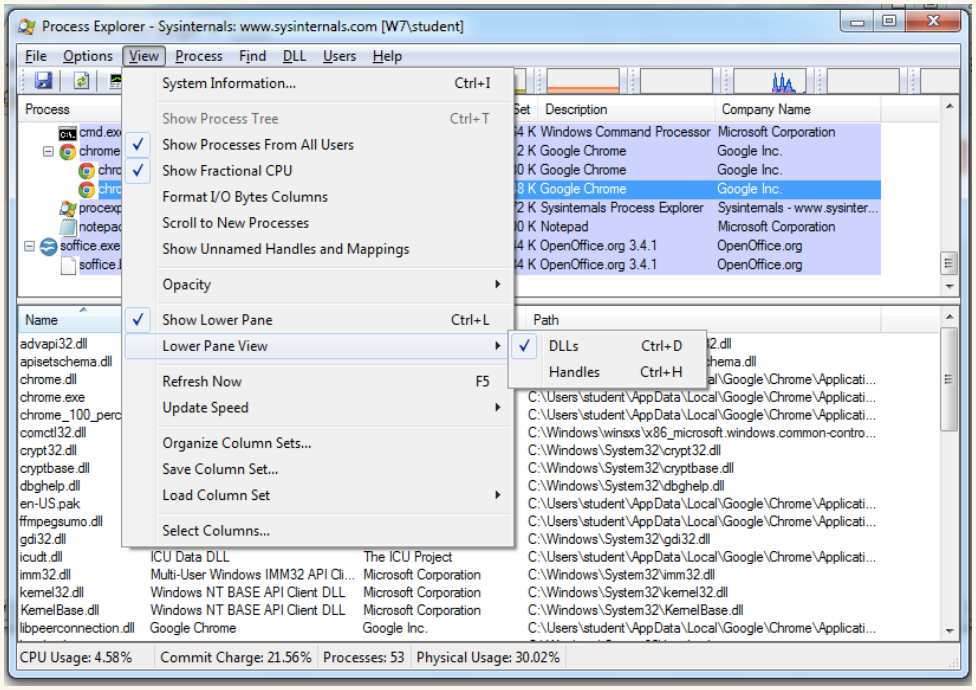


Figure 8.5: Lower pane of Process Explorer listing the DLLs of the selected process (slides, Basic dynamic analysis).

View, Lower Pane View, DLLs lists the libraries mapped into the selected process (here Chrome), with their path. A DLL with an unusual name or path, or one that was not among the static imports, is worth a closer look.

8.3.2 Properties and signature verification

The Properties window shows the **DEP** and **ASLR** status (section 3.7), and has a **Verify** button that checks the **disk file's** Windows signature, **but not the memory image**. So it **will not detect process replacement**.

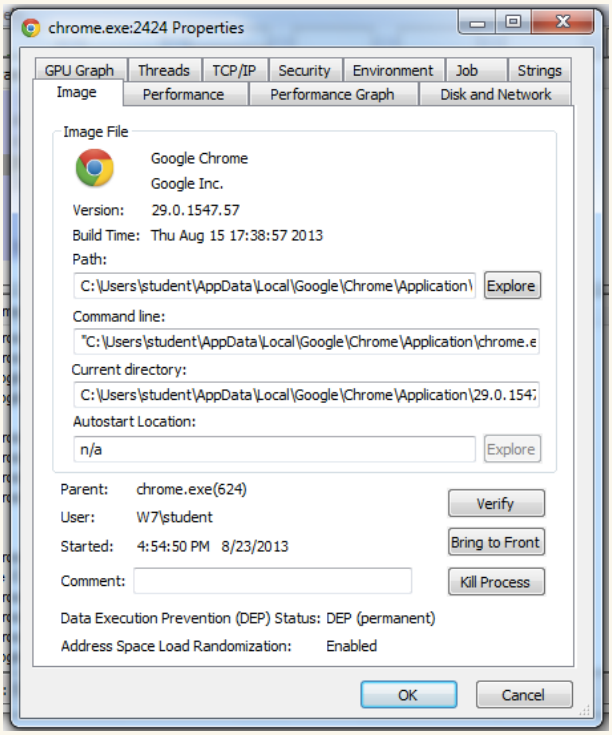


Figure 8.6: Image tab of chrome.exe in Process Explorer (slides, Basic dynamic analysis).

The Image tab of `chrome.exe` shows version, path, command line, current directory, parent and user. At the bottom appear `DEP (permanent)` and `Address Space Load Randomization: Enabled`. The **Verify** and **Kill Process** buttons are on the right.

> **From the book**
>
> **Process replacement** overwrites the memory space of a running process with a malicious executable, to disguise malware as a legitimate process. The user would see `svchost.exe` running from `C:\Windows\System32` and "probably think nothing of it". The key is creating the process in a suspended state (PDF p. 367).

8.3.3 Strings: image versus memory

Compare **Image** strings to **Memory** strings: if they are very different, it can indicate process replacement.

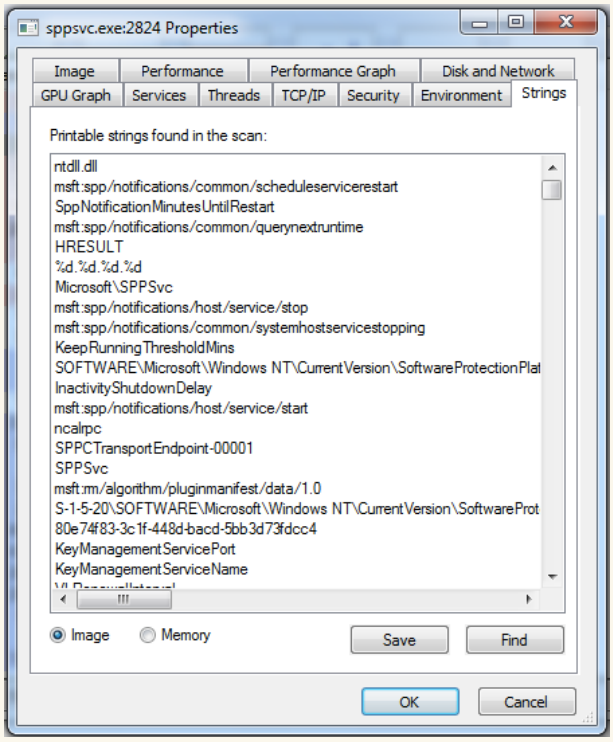


Figure 8.7: Strings tab of sppsvc.exe, with the Image and Memory radio buttons (slides, Basic dynamic analysis).

The Strings tab of `sppsvc.exe` has two radio buttons at the bottom. **Image** shows the strings of the file on disk, **Memory** those of the running process. This closes the gap left by Verify: the signature concerns the disk, while this comparison also looks at memory. It works for packed samples too, since the unpacked strings appear only in memory (section 6.3).

8.4 Detecting malicious documents

Documents can be tested with the same tools.

1. Open the document, for example a PDF, on a system with a **vulnerable application**, such as an old version of Adobe Reader.

2. Watch Process Explorer to see if it **launches a process**.
3. The **Image** tab of that process's Properties shows **where the malware is**.

8.5 The Windows Registry

The **registry** is the repository for the configuration and control of Windows systems.

- **System-wide** settings: which device drivers to load, how to configure the memory manager and process manager, and so on. Applications also read them.
- **Per-user** settings: user preferences, most-recently accessed documents.

It matters for malware analysis because it holds configuration **and** many persistence locations.

8.5.1 Keys, values and root keys

A **registry key** is a container of other keys (**subkeys**) or values: think of a folder. A **registry value** stores data of a given type, such as `REG_SZ`, `REG_DWORD` or `REG_BINARY`: think of a file.

Root key	Stored information	Link
HKEY_CLASSES_ROOT (HKCR)	File association and COM object registration, such as ProgID and CLSID	Merged
HKEY_CURRENT_USER (HKCU)	Data of the currently logged-on user	Yes
HKEY_LOCAL_MACHINE (HKLM)	Global settings for the machine	No
HKEY_USERS (HKU)	All the accounts on the machine	No
HKEY_CURRENT_CONFIG (HKCC)	Current hardware profile	Yes

The HKCR entry returns with COM (section 9.7.8).

8.5.2 Links and hives

REG_LINK: some keys are only links to others.

```
HKEY_CURRENT_USER        -> HKEY_USERS\<SID of the current
  user>
HKEY_CURRENT_CONFIG      ->
  HKLM\SYSTEM\CurrentControlSet\Hardware Profiles\Current
HKLM\SYSTEM\CurrentControlSet -> HKLM\SYSTEM\ControlSet00X
  (X is a number)
```

A **registry hive** is a "logical group of keys, subkeys and values in the registry that has a set of supporting files containing backups of its data". For example, HKLM\SAM is stored in `c:\windows\system32\config\SAM`. Other hives are constructed dynamically in memory.

8.6 Regshot

Regshot checks how a process modified the registry. It is the classic **diffing** tool.

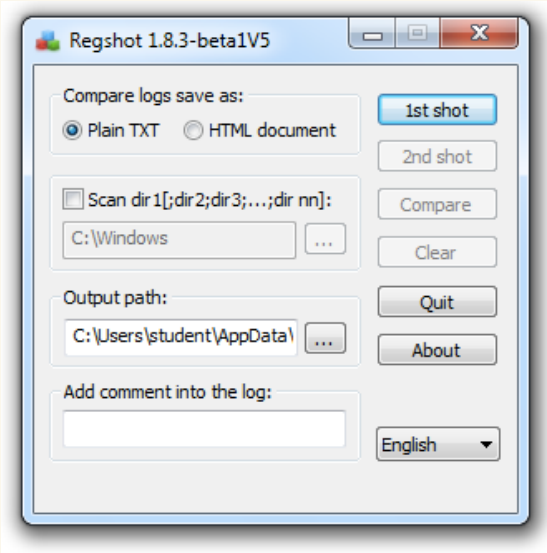


Figure 8.8: The Regshot window (slides, Basic dynamic analysis).

The usage is simple: press **1st shot** before running the sample and **2nd shot** after, then **Compare**. The output shows, for each category, what changed:

```
Regshot
Comments:
Datetime: <date>
Computer: MALWAREANALYSIS
Username: username
----------------------------------
Keys added: 0
----------------------------------


----------------------------------
Values added:3
----------------------------------
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\ckr:
C:\WINDOWS\system32\ckr.exe
...
----------------------------------
Values modified:2
----------------------------------
HKLM\SOFTWARE\Microsoft\Cryptography\RNG\Seed: 00 43 7C 25
```

```
9C 68 DE 59 C6 C8 9D C3 1D E6 DC 87 1C 3A C4 E4 D9 0A B1 BA
...
---------------------------------
Total changes:5
---------------------------------
```

Reading it critically:

- the new value under `...\CurrentVersion\Run` is the signal: `ckr.exe` will restart at every logon, so this is **persistence**;
- the modified `RNG\Seed` is noise: Windows updates it regularly, so not every change in the diff is the malware's doing. This is why you weed out results with care.

8.7 Persistence on Windows

Persistence means techniques to **survive a reboot** (section 2.5.1). The main locations are:

- **registry keys**;
- the **file system**: startup locations, DLL search order hijacking, trojanising system files;
- the **Master Boot Record (MBR)**;
- the **Basic Input/Output System (BIOS)**.

8.7.1 Frequently used registry keys

Neither list below is comprehensive, and the listed keys are not more important than others.

Administrator privilege is required to update HKLM:

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKLM\SOFTWARE\Microsoft\Windows
  NT\CurrentVersion\Winlogon\"Shell" and "UserInit"
HKLM\SOFTWARE\Microsoft\Windows
  NT\CurrentVersion\Windows\"Appinit_Dlls"
HKLM\System\CurrentControlSet\Control\Session
  Manager\KnownDlls
HKLM\System\CurrentControlSet\Services
HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File
  Execution Options
HKLM\Software\Microsoft\Windows\CurrentVersion\Explorer\Brows
  er Helper Objects
```

Without administrator privileges, malware can persist with:

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
HKCU\Software\Policies\Microsoft\Windows\System\Scripts\Logon
HKCU\SOFTWARE\Microsoft\Windows
  NT\CurrentVersion\Winlogon\Shell
```

8.7.2 Persistence through the file system

The slide gives these startup locations:

```
logged-in user:  %USERPROFILE%\Start Menu\Programs\Startup
all users:       %ALLUSERSPROFILE%\Start Menu\Programs\Startup
```

> **Correction**
>
> These are the **Windows XP** paths. On the Windows 10 VM used in the course, the folders are `%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup` (user) and `%ProgramData%\Microsoft\Windows\Start Menu\Programs\Startup` (all users).

8.7.3 Windows services

Services are long-running executables without user interaction, like a Unix daemon. They can be started automatically when the computer boots. `CreateService()` is the Windows API that registers a service, and registered services are found under `HKLM\System\CurrentControlSet\Services`. Section 9.7.7 gives more detail.

8.7.4 Svchost

`C:\Windows\System32\svchost.exe` is a generic **host process** for services that run from DLLs.

- Multiple instances are often running, and one instance contains a group of services.
- Groups are listed in `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Svchost`.
- It is common for malware to **name itself svchost.exe** but run from somewhere other than `C:\Windows\System32`, for example `C:\Windows`.
- Alternatively, it just **adds a new DLL for the real svchost** to run as a service.

So the two questions to ask about any `svchost.exe` are: **where does it run from?** and **which DLLs does it host?**

8.8 Network analysis

Most malware communicates, but the analysis network is isolated. These tools observe the traffic and give the sample something to talk to.

8.8.1 Packet sniffing with Wireshark

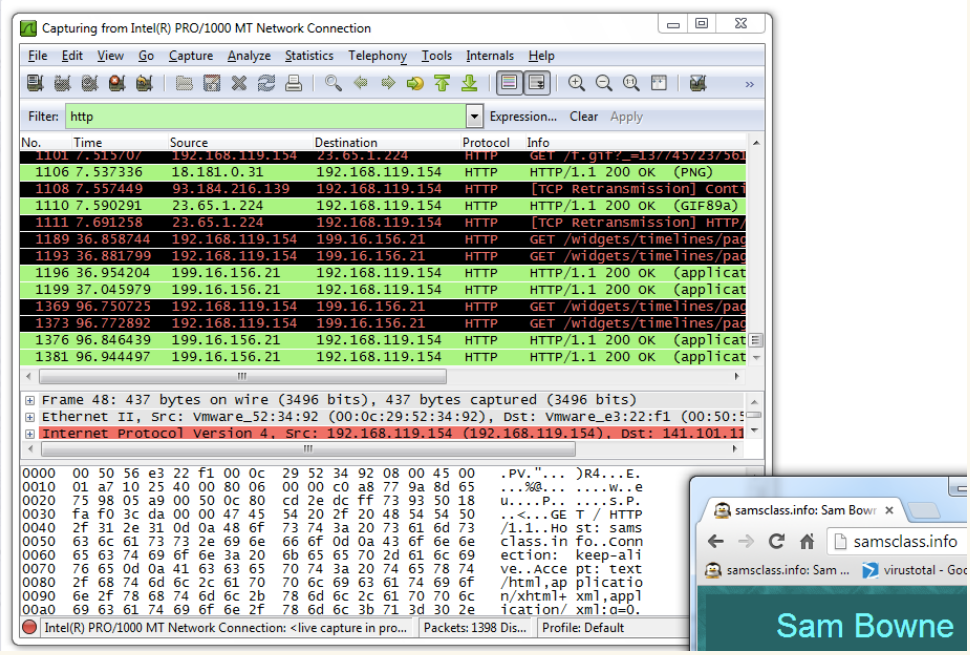


Figure 8.9: Wireshark with the display filter http (slides, Basic dynamic analysis).

Wireshark captures the traffic on an interface, here with the display filter `http`. The window has three panes: packet list, the protocol layers of the selected packet, and raw bytes. Network indicators of compromise (domains, URLs, User-Agent) come from here.

Follow TCP Stream rebuilds a whole conversation from the individual packets. You can also **save files** from streams here.

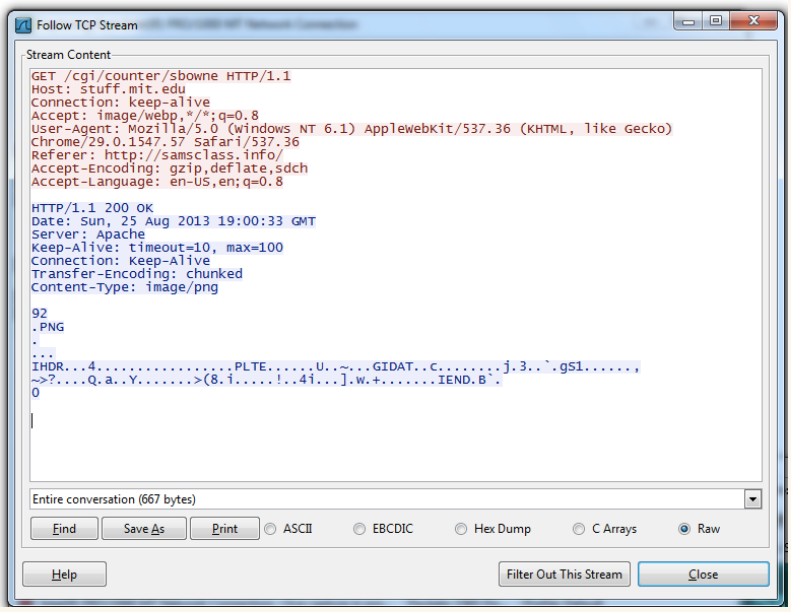


Figure 8.10: Follow TCP Stream in Wireshark (slides, Basic dynamic analysis).

The two directions appear in different colours: the client's `GET /cgi/counter/sbowne HTTP/1.1` with its headers, in red, and the server's `HTTP/1.1 200 OK` with a PNG body, in blue. The buttons at the bottom save the data or filter out this stream.

8.8.2 Redirecting DNS with ApateDNS

ApateDNS answers DNS requests with an IP of your choice, so the sample's connections are redirected to a machine you control.

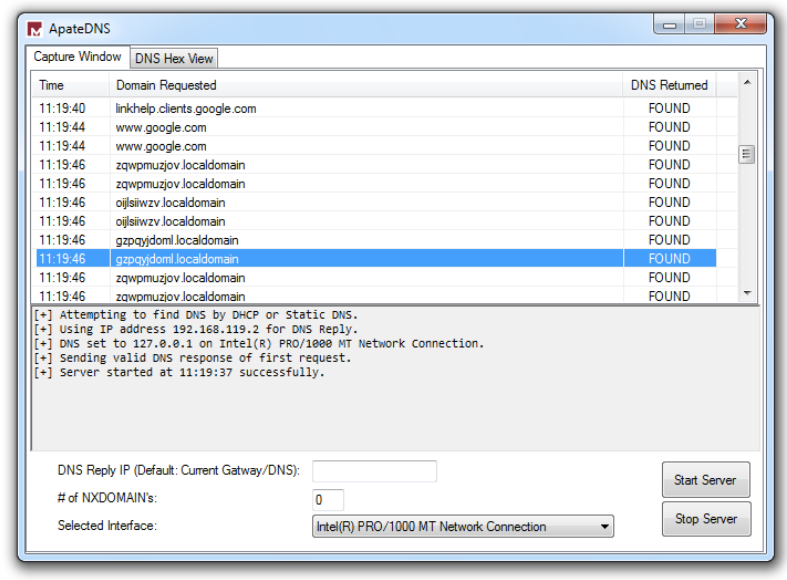


Figure 8.11: The ApateDNS window (slides, Basic dynamic analysis).

The capture window lists the time and the domain requested, including several random-looking names under `.localdomain`. The log at the bottom shows the setup: the reply IP is set and the local DNS is pointed to `127.0.0.1`. There are also a **DNS Reply IP** field and a **# of NXDOMAINs** field. The domain list is itself an indicator, and a burst of random names is typical of generated domains.

8.8.3 Monitoring with ncat

ncat (included with Nmap) listens on a port and prints whatever arrives.

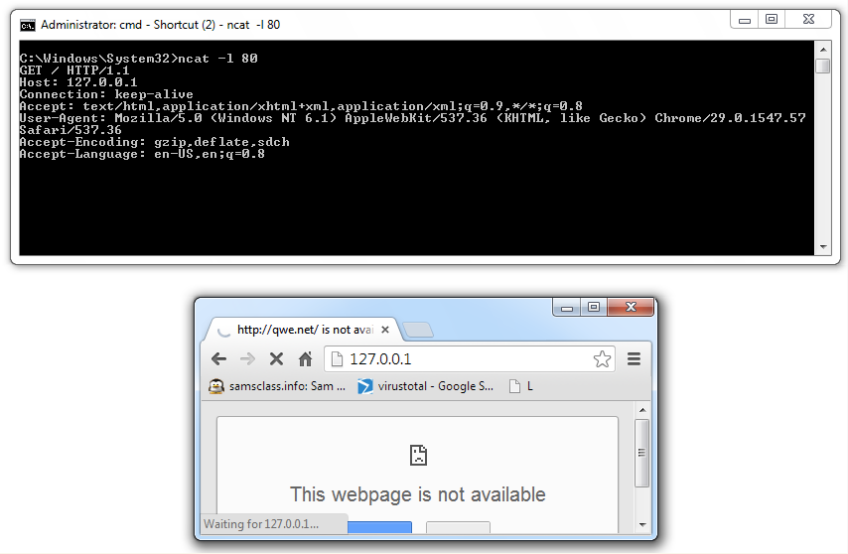


Figure 8.12: ncat listening on port 80 while a browser visits 127.0.0.1 (slides, Basic dynamic analysis).

`ncat -l 80` opens port 80. When the browser visits `127.0.0.1`, its full HTTP request appears in the console. The browser keeps waiting and finally reports an error, because ncat does not answer. This reveals what a sample sends, but only until it expects a real reply. INetSim solves that limit.

8.8.4 INetSim

INetSim simulates common Internet services, so that the sample gets plausible replies.

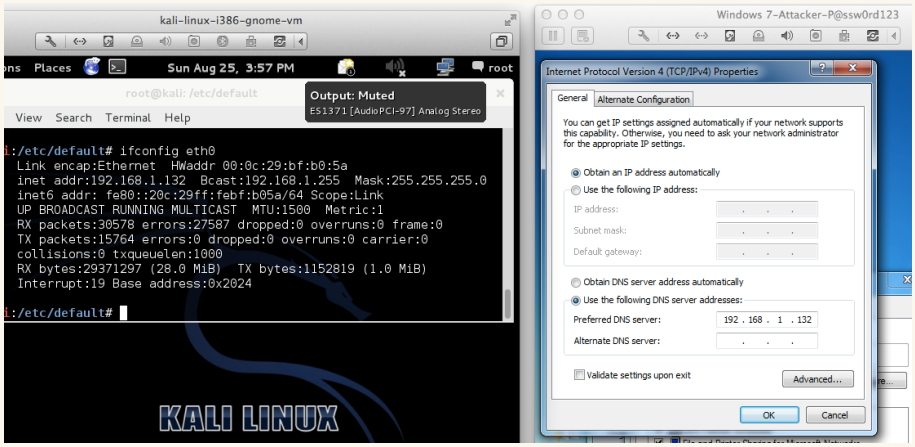


Figure 8.13: INetSim on a Kali Linux VM, with its address set as the preferred DNS server of the Windows VM (slides, Basic dynamic analysis).

The screenshot shows the setup. INetSim runs on a **Kali Linux** VM (`ifconfig` shows `192.168.1.132`), and the Windows VM has that same address as its **preferred DNS server**. All name resolutions therefore lead to the fake services.

> **From the book**
>
> INetSim is "a free, Linux-based software suite for simulating common Internet services"; if the base OS is Windows, the easiest way to run it is on a Linux VM on the same virtual network (PDF p. 99).

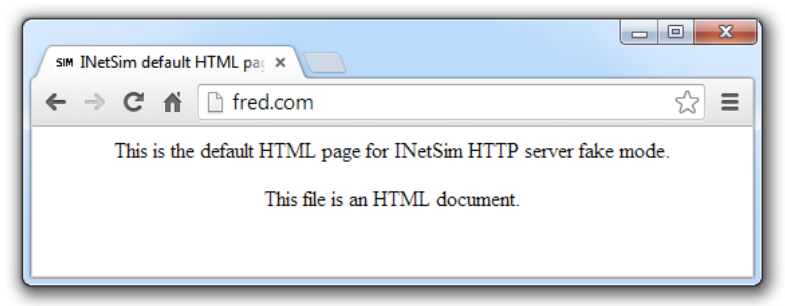


Figure 8.14: INetSim fools a browser: fred.com returns the default HTML page (slides, Basic dynamic analysis).

INetSim fools a browser. Visiting `fred.com` returns INetSim's default HTML page. There is no such page on the real site: any name resolves to the fake server, which answers with a default file.

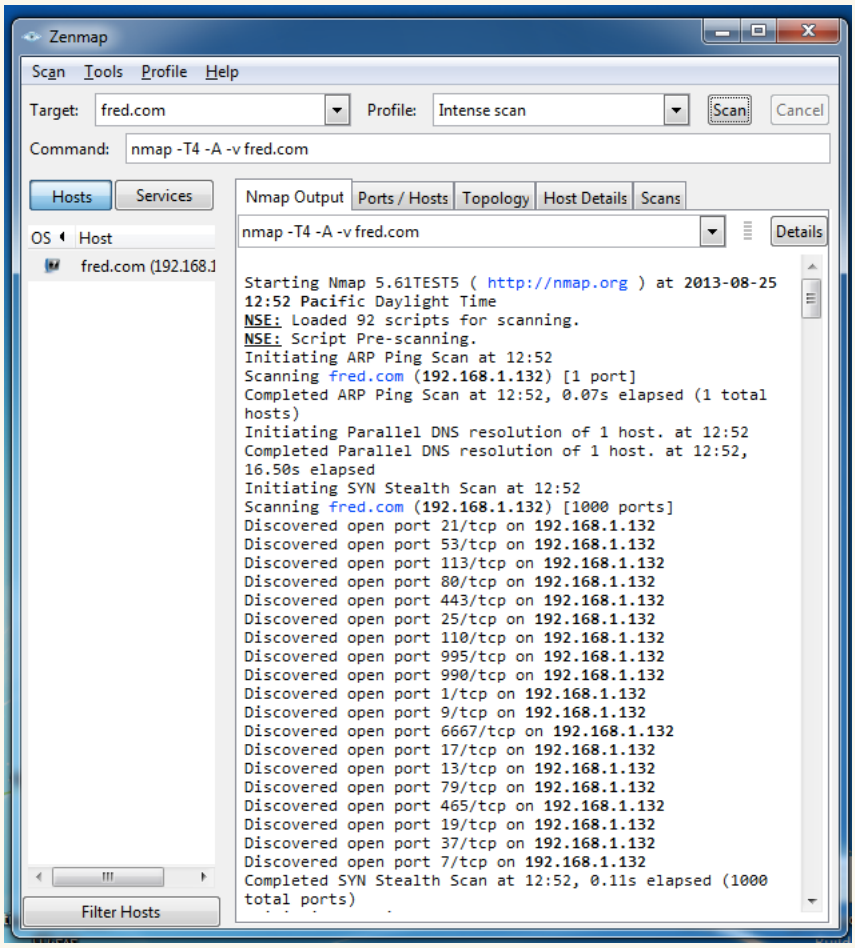


Figure 8.15: INetSim fools nmap: Zenmap finds many open ports (slides, Basic dynamic analysis).

INetSim fools nmap. Zenmap scanning `fred.com` (`nmap -T4 -A -v`) finds many open ports on `192.168.1.132`: from the outside, that host looks like a full server. Likewise, a sample looking for HTTP, FTP or SMTP will find what it expects and keep running, revealing more behaviour.

8.9 Using the tools together

The slide names five items for a typical session: **procmon** (filter on the malware executable name and clear all events just before running it), **Process Explorer**, **Regshot**, a **virtual network** with ApateDNS or INetSim, and **Wireshark**. A workable order follows. The sequence is the one proposed in the course notes.

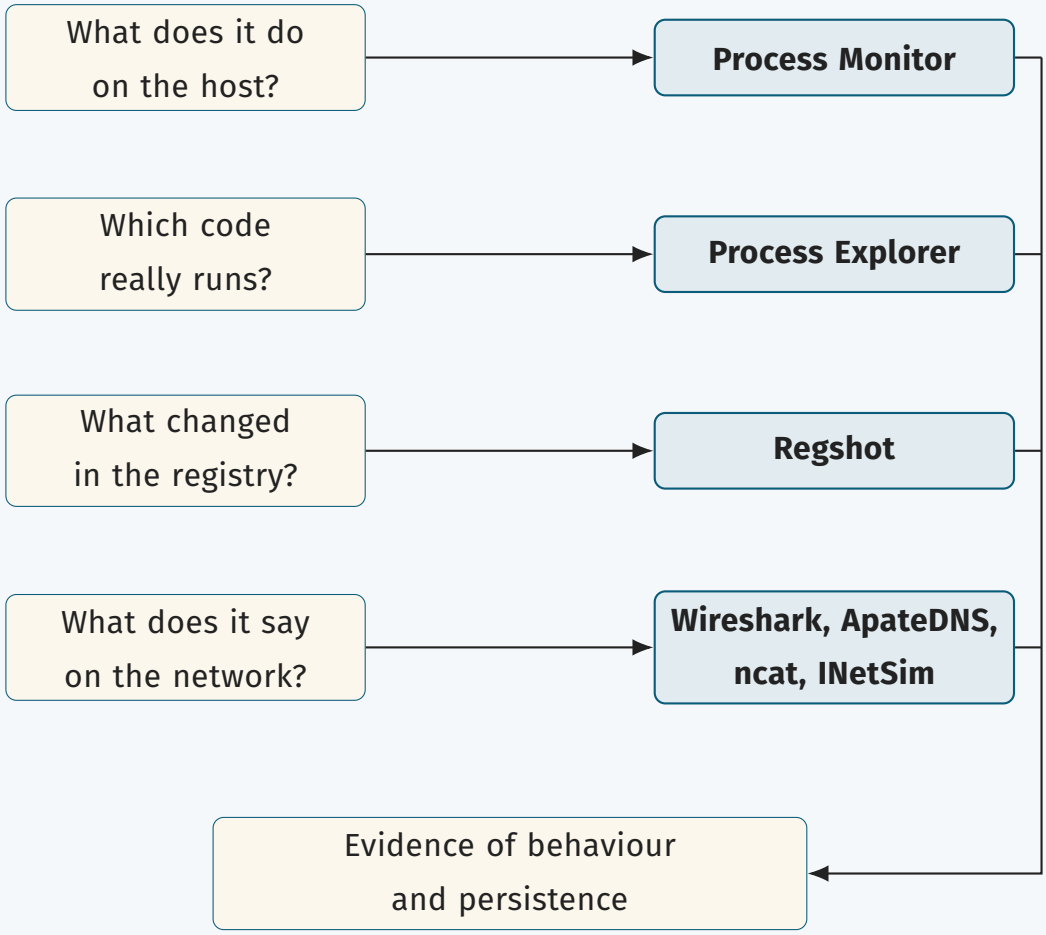
Step by step: A typical analysis session

1. Revert the VM to a clean snapshot.
2. Start the virtual network with ApateDNS or INetSim.
3. Start Wireshark.
4. Regshot: take the first shot.
5. Procmon: filter on the sample name and clear the events.
6. Open Process Explorer.
7. Run the malware.
8. Stop the procmon capture.
9. Regshot: take the second shot and compare.
10. Review all the outputs and write the report, then return to step 1 for the next run.

Linux has analogous monitoring tools, such as `strace` for system calls.

Question	Tool
What did it touch, event by event?	Process Monitor
What processes did it start? What does the process hold in memory?	Process Explorer
What changed in the registry?	Regshot
What does it send on the network?	Wireshark, ncat
Which domains does it want?	ApateDNS
How does it behave if the services reply?	INetSim

Concept map: how the ideas connect



Each question on the left has its tool on the right. Process Monitor records what the sample does on the host. Process Explorer shows which code really runs, by comparing image and memory strings. Regshot gives a before-and-after view of the registry. The network tools show or simulate communication. Together they give the evidence of behaviour and persistence.

Remember

- Procmon filters change only the display, and long captures fill the RAM.
- Process Explorer's **Verify** checks only the disk: compare **Image and Memory strings** to catch process replacement.
- Not every change in a Regshot diff comes from the malware.
- Check the registry autostart keys, startup folders, services and

`svchost.exe` paths for persistence.

Review: try to answer without looking

1. A signed `svchost.exe` passes Verify. Can it still be a replaced process?
2. Which Run key can a non-administrator sample write?
3. Why is ncat not sufficient for a sample that expects an HTTP reply?

Answers, with the section where to find them

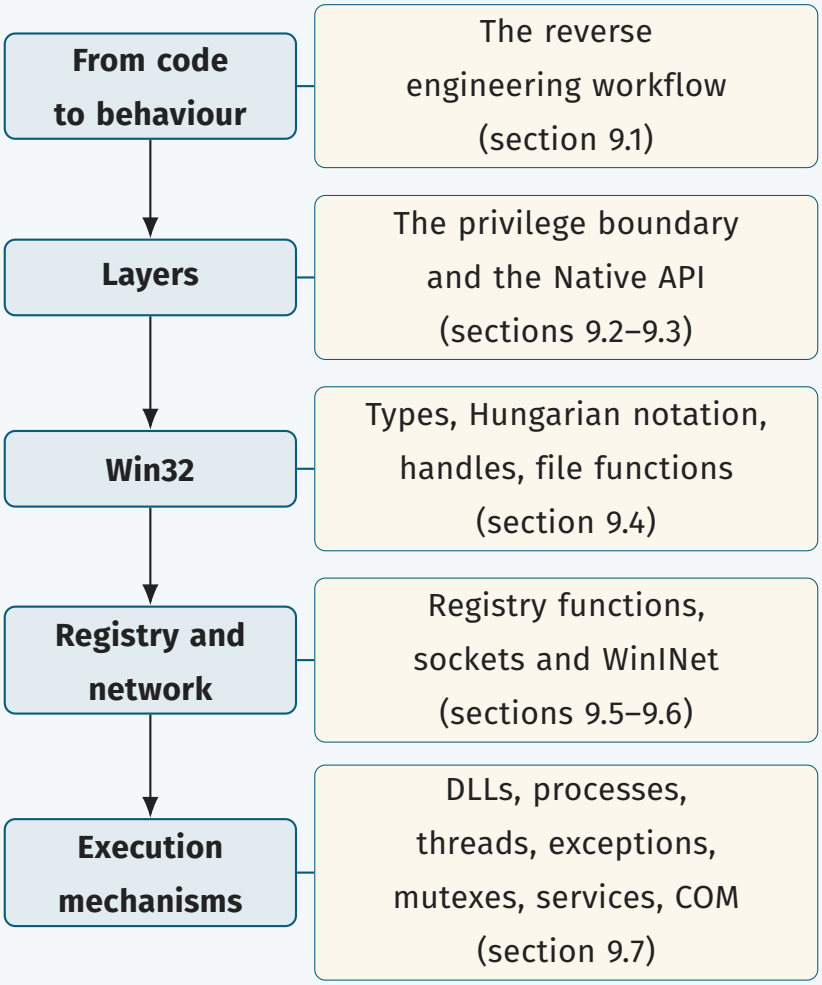
1. Yes: Verify checks the disk file, not the memory image (section 8.3.2).
2. The one under HKCU (section 8.7.1).
3. It prints what arrives but does not answer (section 8.8.3).

Chapter 9

Windows from the analyst's point of view

> **Before you start**
>
> **Central question:** when we read code, how do we recognise what it asks of the operating system?
>
> **Prerequisites:** calling conventions (section 4.5), imports (section 6.6), registry and services at tool level (chapter 8).
>
> **Where it leads:** chapter 10 adds the C patterns, chapter 11 steps through these calls, and chapter 12 shows how the same APIs serve anti-analysis.

Chapter structure: what comes, and in which order

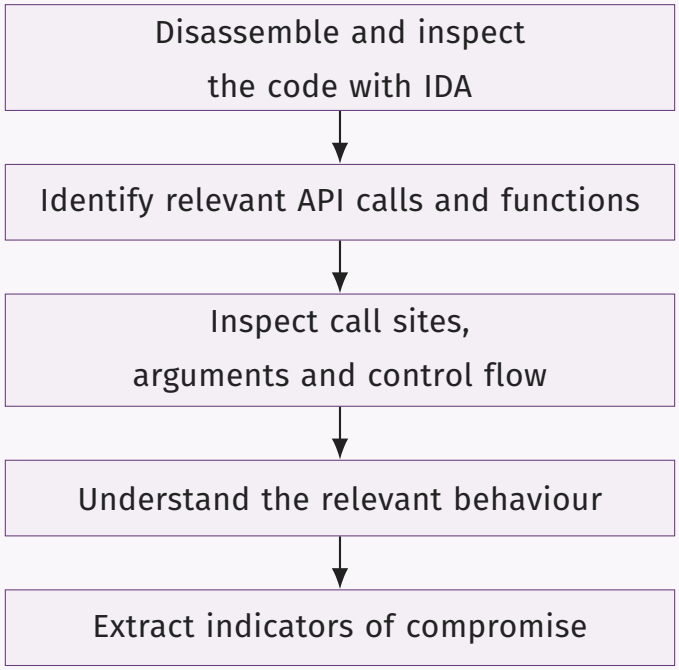


The chapter first sets the goal of turning code into behaviour. It then goes through the layers of Windows and the main API families. It ends with the mechanisms through which code runs.

9.1 From disassembly to malware behaviour

Advanced static analysis reads the code itself, going from disassembly to malware behaviour.

Flowchart: Reverse engineering workflow



Reverse engineer the sample by following these five steps from top to bottom. Tools like IDA provide features to skim through code, search for specific snippets and interpret the code flow. To simplify this part, we **assume that the malware code is not obfuscated or crippled in strange ways**. Dealing with that is another topic (section 4.9 and chapter 12).

From the lecture (27 October 2026, 01:30–05:50)

This step comes when basic analysis was inconclusive or the sample is a new threat. The aim is not to dissect every subroutine: skim the code, separating bulky subroutines full of logic from thin ones with one or two API calls, and add detail only where it matters. API calls characterize how a sample interacts with the system, so a sample making none leaves little mark worth analyzing. With practice you start recognizing telling arguments, such as sensitive file-system paths or memory allocated as executable. This is one common approach, not the only way to work. API call obfuscation is set aside for the next lectures.

9.2 The privilege boundary

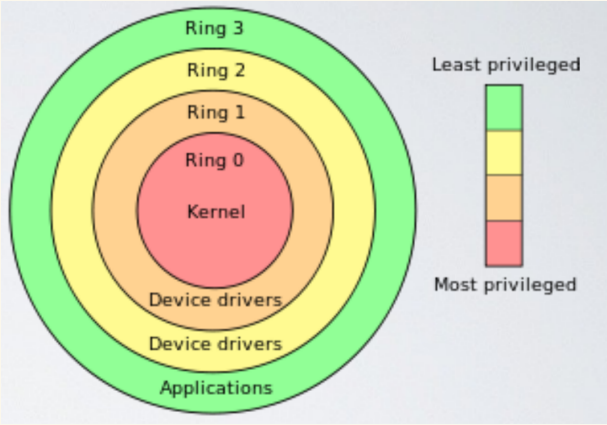


Figure 9.1: x86 protection rings, from the most privileged at the centre to the least privileged outside (slides, Advanced static analysis).

x86 CPUs define four privilege levels, drawn as concentric rings, from the most privileged at the centre to the least privileged outside:

- **Ring 0: kernel mode**;
- **Ring 3: user mode**;
- **rings 1 and 2 are not used by Windows**.

Nearly all code runs in user mode, except the OS and hardware drivers. User-mode code can manipulate hardware **only through the Windows API**, with no direct access, and access to some CPU instructions is restricted.

> **Do not confuse**
>
> The picture is the generic x86 model, which places device drivers in rings 1 and 2. Since Windows does not use those rings, its drivers run in ring 0.

User-mode processes. Each process has its own memory (**isolated address space**), security permissions and resources. If a user-mode program executes an invalid instruction and crashes, Windows can **reclaim the resources and terminate the program**. The damage stays contained.

Calling the kernel. It is **not possible to jump directly** from user mode to the kernel. **Special instructions** (`sysenter`, `syscall`, `int 0x2e`) act as **gates** that

invoke predefined functions through lookup tables, so user code can enter the kernel only at approved points. Their presence is an indicator that the code runs functions at kernel level. A normal program lets `ntdll.dll` execute them, so finding one in a sample's own code is suspicious.

> **From the lecture (27 October 2026, 09:13–11:50)**
>
> On Linux, system call numbers are public and stable. On Windows they can change between releases and are not officially documented, although reverse engineers publish tables per version. So direct system calls appear in highly sophisticated malware, which also gains the advantage that they are harder for security solutions to monitor. Most samples just rely on the APIs.

Kernel processes.

- All kernel processes **share resources and memory addresses**: there is no isolation between them.
- There are fewer security checks, but **PatchGuard** acts as an integrity mechanism.
- If kernel code executes an invalid instruction, the OS crashes with the **Blue Screen of Death (BSoD)**.
- Some security solutions are supported by kernel-mode components.
- **Kernel-mode malware** is rarer, more specialised and more sophisticated than user-mode malware. **Rootkits** are a common example.

> **From the lecture (27 October 2026, 13:40–15:46)**
>
> Security products have fewer kernel drivers than in the past because of PatchGuard. Windows instead offers components they can register with to be notified of events, which a malware author then tries to disable or circumvent.

> **To verify**
>
> The audio of the recording is unclear at 14:36, where the name of such a

> component sounds like “AMSI”. Check it against the recording before relying on it.

9.3 The Native API

The **Native API** is a lower-level interface for interacting with Windows.

- `ntdll.dll` manages interactions between user space and the kernel, and **ntdll functions make up the Native API**.
- It is rarely used by goodware.
- It is **popular among malware writers**, as it can be more powerful and stealthier than Windows API calls: monitors hooking only the upper layer do not see it.
- It has limited official documentation.

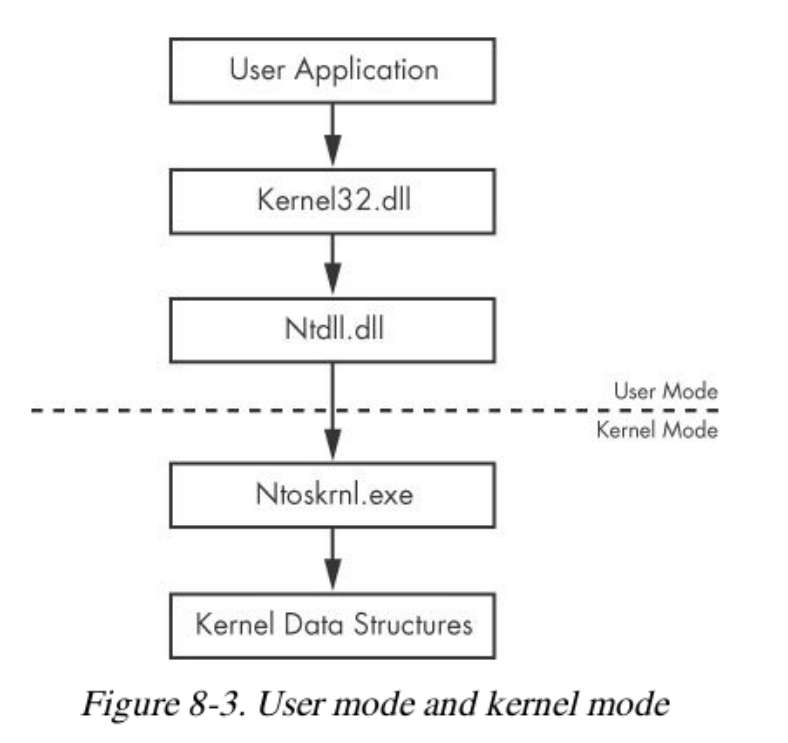


Figure 9.2: Path of a normal call across the user/kernel boundary (slides, Advanced static analysis).

The figure shows the path of a normal call: user application, `Kernel32.dll`,

`Ntdll.dll` in user mode, then across the boundary to `Ntoskrnl.exe` and kernel data structures. A sample that calls `ntdll.dll` directly simply skips the Kernel32 step. This is the bypass mentioned in section 7.5.

> **From the lecture (27 October 2026, 16:21–20:13)**
>
> `ntdll.dll` knows the running kernel version and the right system call numbers, and libraries like `kernel32.dll` rely on it. Some malware is written almost entirely with these APIs. Reverse-engineering sites document many of them, and the professor also suggests reading how **ReactOS**, an open-source implementation of the Windows kernel, calls them. In exams there are normally one or two `Nt` APIs, which are just a bit more complex than standard ones once you look them up.

Popular Native API calls in malware get information about the system, processes, threads, handles and other items. They provide more information than any available Win32 API from DLLs. The slide lists them with a misspelled prefix (see the correction): `NtQuerySystemInformation`, `NtQueryInformationProcess`, `NtQueryInformationThread`, `NtQueryInformationFile` and the key query. `NtQueryInformationProcess` is also a classic way to detect a debugger (section 12.12).

> **Correction**
>
> `NTtQuery...` is a typo in the slide: the real names are those given above. In particular, the last slide entry, `NTtQueryInformationKey`, is exported as `NtQueryKey`, not `NtQueryInformationKey`. Use the correct names when searching in IDA or in the documentation.

> **From the lecture (27 October 2026, 20:28–21:10)**
>
> Why use them: they expose more than normal APIs. For fingerprinting a sandbox, `NtQuerySystemInformation` alone can reveal more than 50 configuration options, according to the professor.

9.4 Windows API (Win32)

The Win32 API provides applications with functions to interact with Windows and access OS services and resources. Programs normally interact with OS resources through **API functions exported by system DLLs**. These are what you see in the imports and at the call sites.

> **From the lecture (27 October 2026, 21:52–23:40)**
>
> From a study by the group of the professor, as recalled in class: system DLLs offer close to 100,000 different APIs, with over 60,000 widely available across Windows versions. These stable libraries are part of why a program written in 2005 can still run in 2026.

9.4.1 Types and Hungarian notation

The Windows API has its own names for C data types, for example `DWORD` for 32-bit unsigned integers and `WORD` for 16-bit unsigned integers. With **Hungarian notation**, variable names are prefixed to indicate their type or intended use.

Type (prefix)	Meaning
`WORD` (w)	16-bit unsigned value
`DWORD` (dw)	32-bit unsigned value
Handle (H)	A reference to an object
Long Pointer (LP)	Points to another type

So `dwDesiredAccess` is a 32-bit value and `lpStartupInfo` is a pointer. IDA labels the arguments with these names, so the prefixes tell you what each `push` is.

> **From the lecture (27 October 2026, 25:35–26:27 and 50:53)**
>
> Prefixes combine: `LP` is a pointer to something mutable, as in `lpBuffer`, and the `C` in `LPCSTR` means constant, so it is a string the function will not

> modify. Often the parameter name alone already suggests the type, without reading its declaration.

9.4.2 Handles

Handles refer to items opened or created in the OS, such as a process, menu, file or window. They are like **immutable pointers** to those objects:

- you **cannot operate on them with arithmetic**;
- you can **store them** and use them later to refer to the same object;
- sometimes you can check validity against `INVALID_HANDLE_VALUE`.

For example, `CreateWindowEx` returns an `HWND`, a handle to the window. To do anything to that window, such as `DestroyWindow`, you use that handle. For the analyst, this means following a handle from the call that creates it to the calls that use it, which links apparently unrelated API calls together.

> **From the lecture (27 October 2026, 26:46–28:40)**
>
> A handle is similar to a Linux file descriptor, but applies to many more things. You can duplicate it, but that is the only form of sharing. Malware authors also compare the result of opening a resource with the invalid handle value, so look for that check.

9.4.3 File-system functions

- `CreateFile`, `ReadFile` and `WriteFile` perform normal file input and output.
- `CreateFileMapping` and `MapViewOfFile` are **frequently used by malware**, and load file contents into RAM. The first creates a file-mapping object. The second maps the view into the process address space, making the contents visible, and returns a pointer to the base address of the file in memory.

- Together they can be used to **execute a file without using the Windows loader**.

> **From the book**
>
> With the pointer returned by `MapViewOfFile`, a program can “read and write anywhere in the file”, which is “extremely handy when parsing a file format”. File mappings “are commonly used to replicate the functionality of the Windows loader” (PDF p. 205).

> **From the lecture (27 October 2026, 29:00–30:40)**
>
> Windows itself uses memory mapping to load libraries, which is why these two functions can implement custom code loading. Variants also exist through `Nt` counterparts, and a security solution may miss this kind of loading. The professor returns to it with code injection.

9.4.4 Special files

Special paths. `\\server\share\file.txt` lets programs access a shared resource on another computer: these are **UNC paths** (Universal Naming Convention). The `\\?\` prefix for extended paths, both local and UNC, bypasses normal Win32 path processing, allowing extended-length paths beyond 260 characters.

Namespaces are special folders in the Windows file system. `\` is the lowest namespace and contains everything. `\\.\` is the **device namespace**, used for direct disk I/O. The **Witty worm** (2004) wrote to `\\.\PhysicalDisk1` to corrupt the disk. A string with one of these prefixes in a sample is therefore a strong signal: direct disk access does not go through the file system.

Alternate Data Streams (ADS) are a second stream of data attached to a filename: `File.txt:otherfile.txt`. The stream is kept in addition to the primary, visible content of the file. It is a feature of the **NTFS** file system.

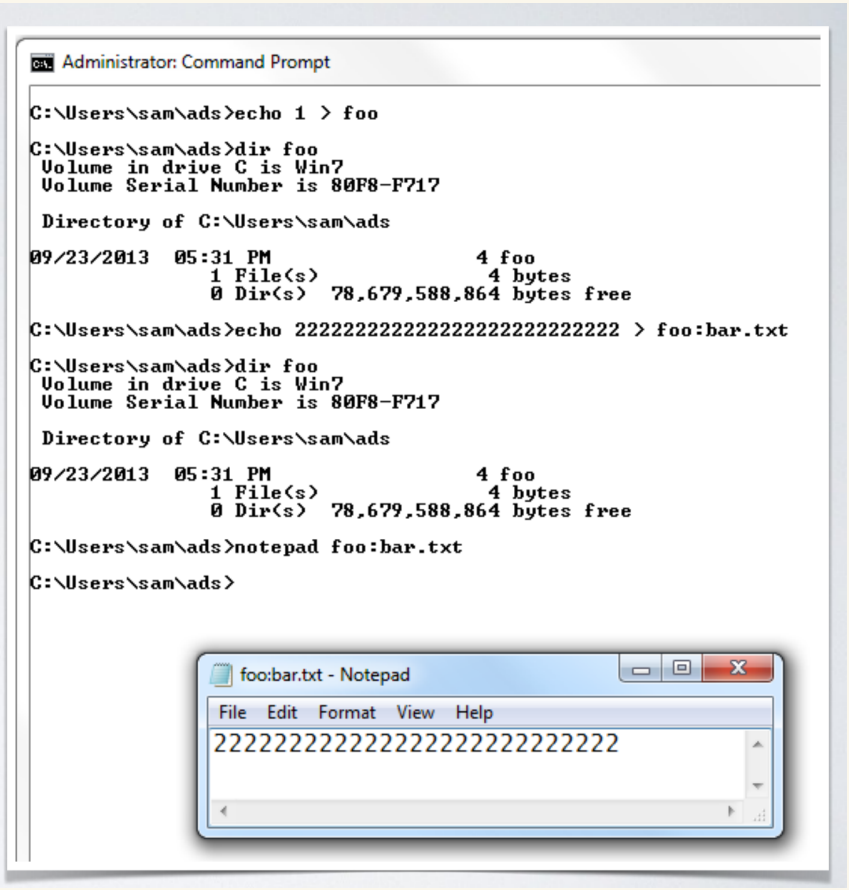


Figure 9.3: Console session showing an alternate data stream (slides, Advanced static analysis).

The console session shows the trick:

1. `echo 1 > foo` creates a 4-byte file;
2. `echo 2222... > foo:bar.txt` writes a long string into a stream of the same file;
3. `dir` still reports **4 bytes** with the same date, and the stream is not listed;
4. `notepad foo:bar.txt` opens the hidden content.

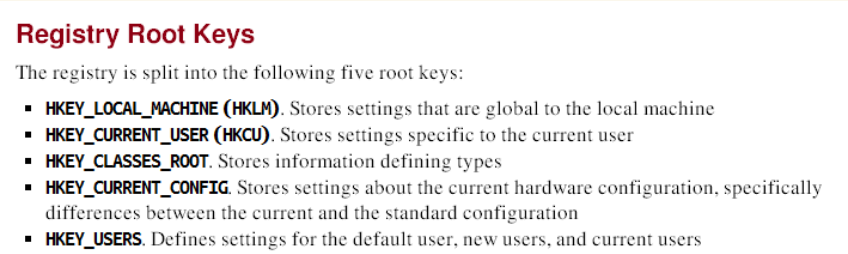
So plain `dir` does not reveal data hidden in a stream.

> **From the lecture (27 October 2026, 30:52–36:12)**
>
> - **UNC paths:** in malware they may appear when it tries to spread by writing to shared resources on the network.

- **`\\.\` on older Windows:** with admin rights, two API calls were enough to overwrite the boot sector and make the machine unusable.
- **Alternate Data Streams:** the professor has not personally seen them used by malware yet, but it is possible. The feature exists only on NTFS.

9.5 Windows registry functions

The registry stores OS and program configuration settings, such as desktop background and mouse preferences. Malware may use it for **persistence**, to restart when the system reboots. Section 8.5 describes its structure.



Registry Root Keys

The registry is split into the following five root keys:

- **HKEY_LOCAL_MACHINE (HKLM)**. Stores settings that are global to the local machine
- **HKEY_CURRENT_USER (HKCU)**. Stores settings specific to the current user
- **HKEY_CLASSES_ROOT**. Stores information defining types
- **HKEY_CURRENT_CONFIG**. Stores settings about the current hardware configuration, specifically differences between the current and the standard configuration
- **HKEY_USERS**. Defines settings for the default user, new users, and current users

Figure 9.4: Book excerpt on the five root keys (slides, Advanced static analysis).

The excerpt lists `HKEY_LOCAL_MACHINE` (global settings), `HKEY_CURRENT_USER` (current user), `HKEY_CLASSES_ROOT` (information defining types), `HKEY_CURRENT_CONFIG` (differences between the current and the standard hardware configuration) and `HKEY_USERS` (default, new and current users).

Example: the Run key. `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run` lists the executables that start when a user logs on.

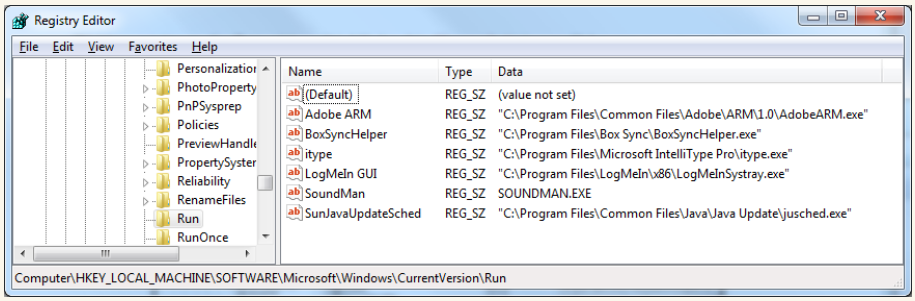


Figure 9.5: The Run key in Registry Editor (slides, Advanced static analysis).

In Registry Editor, each value in the Run key is a program, of type REG_SZ, with its path as data (AdobeARM.exe, jusched.exe and so on). A malicious entry would look just like these, so the **path** and the file behind it are what you check.

> **From the lecture (27 October 2026, 37:03–43:47)**
>
> **Reading the hierarchy.** Think of the registry as a file system: keys are folders, and values hold simple content such as numbers, booleans or strings. The H in HKEY is the Hungarian prefix for a handle. A normal user can modify only HKCU, while the others require an administrator. As an analyst you will mostly see HKLM and HKCU.
>
> **RunOnce.** Next to Run there is RunOnce, whose program runs at the next reboot and is then deleted. It was common for installers that needed a restart.
>
> **Hiding in Run.** An author can pick a name resembling an installed application, with just one letter changed. Security solutions would notice, but a person may not.

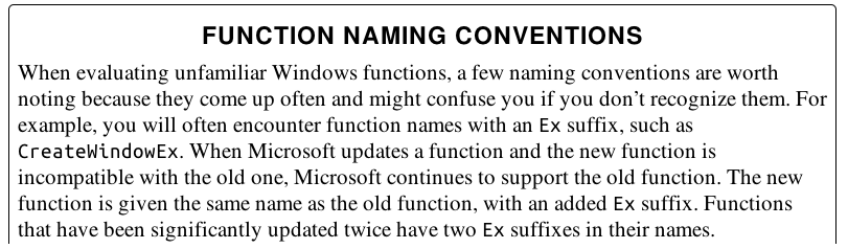
Common registry functions.

- RegOpenKeyEx opens a registry key for editing and querying.
- RegSetValueEx adds a new value to the registry and sets its data.
- RegGetValue returns the data for a value entry in the registry.

Documentation often omits the trailing W (wide-char) or A (ASCII) character in a call like RegOpenKeyExW.

9.5.1 Ex, A and W suffixes

These conventions come from Chapter 2 of the book.

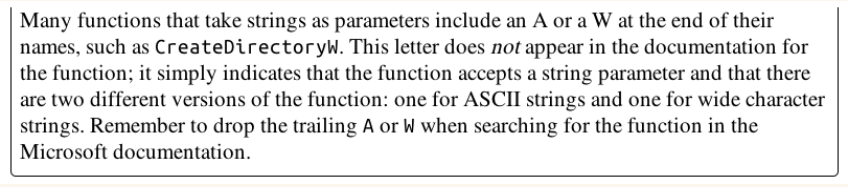


FUNCTION NAMING CONVENTIONS

When evaluating unfamiliar Windows functions, a few naming conventions are worth noting because they come up often and might confuse you if you don't recognize them. For example, you will often encounter function names with an `Ex` suffix, such as `CreateWindowEx`. When Microsoft updates a function and the new function is incompatible with the old one, Microsoft continues to support the old function. The new function is given the same name as the old function, with an added `Ex` suffix. Functions that have been significantly updated twice have two `Ex` suffixes in their names.

Figure 9.6: Book excerpt on the Ex suffix (slides, Advanced static analysis).

Ex. When Microsoft updates a function and the new one is incompatible with the old, it keeps supporting the old function and gives the new one the same name plus an Ex suffix. Functions updated significantly twice have two Ex suffixes.



Many functions that take strings as parameters include an A or a W at the end of their names, such as `CreateDirectoryW`. This letter does *not* appear in the documentation for the function; it simply indicates that the function accepts a string parameter and that there are two different versions of the function: one for ASCII strings and one for wide character strings. Remember to drop the trailing `A` or `W` when searching for the function in the Microsoft documentation.

Figure 9.7: Book excerpt on the A and W suffixes (slides, Advanced static analysis).

A and W. Many functions that take strings have an A or W at the end, such as `CreateDirectoryW`. This letter does **not** appear in the documentation: it only indicates that the function takes an ASCII or a wide-character string. Drop it when searching the Microsoft documentation.

From the lecture (27 October 2026, 46:03–48:09)

In memory, a wide-char string looks like familiar ASCII letters interleaved with zeros, each character taking 2 bytes, and it ends with a double zero. Recognizing this pattern helps when inspecting memory in the labs (section 14.2.4).

Correction

In the lecture wide-char strings are described as "not Unicode". On Windows they are in fact UTF-16LE, an encoding of Unicode. The two-byte description holds for common characters, but some characters take four bytes.

9.5.2 Reading registry code

Listing 7-1 of the book shows code that modifies registry settings. The first lines open the Run key, the rest write a value.

```
0040286F   push    2                 ; samDesired
00402871   push    eax               ; ulOptions
00402872   push    offset SubKey     ;
  "Software\\Microsoft\\Windows\\CurrentVersion\\Run"
00402877   push    HKEY_LOCAL_MACHINE ; hKey
0040287C   call    esi ; RegOpenKeyExW        ; (1)
0040287E   test    eax, eax
00402880   jnz     short loc_4028C5
00402882
00402882 loc_402882:
00402882   lea     ecx, [esp+424h+Data]
00402886   push    ecx               ; lpString
00402887   mov     bl, 1
00402889   call    ds:lstrlenW               ; (2)
0040288F   lea     edx, [eax+eax+2]
00402893   push    edx               ; cbData  ; (3)
00402894   mov     edx, [esp+428h+hKey]
00402898   lea     eax, [esp+428h+Data]       ; (4)
0040289C   push    eax               ; lpData
0040289D   push    1                 ; dwType
0040289F   push    0                 ; Reserved
004028A1   lea     ecx, [esp+434h+ValueName]  ; (5)
004028A8   push    ecx               ; lpValueName
004028A9   push    edx               ; hKey
004028AA   call    ds:RegSetValueExW
```

The markers in the slide, here numbered, follow the logic.

1. `call esi` is an **indirect call**: `esi` was loaded earlier with the address of `RegOpenKeyExW`, and IDA adds it as a comment. Arguments are pushed right to left (section 4.5). `samDesired` = 2 is `KEY_SET_VALUE`, so the code

intends to write. Then `test eax, eax; jnz` leaves on failure, because registry functions return `ERROR_SUCCESS` (0) when they succeed.

2. `lstrlenW` returns the length of the data string, in **characters**.
3. `lea edx, [eax+eax+2]` computes $2n + 2$: the size in **bytes** of a wide string, including the terminating NUL. This is an arithmetic use of `lea`.
4. `lpData` points to the data to write, and `push 1` is `dwType = REG_SZ`.
5. `lpValueName` is the value name, followed by the key handle.

Putting it together: the code **adds a value under the HKLM Run key**, so its data will run at logon. The `Data` string is thus an indicator of compromise, and writing to HKLM needs admin privileges.

The next two slides annotate the same pattern on another sample.

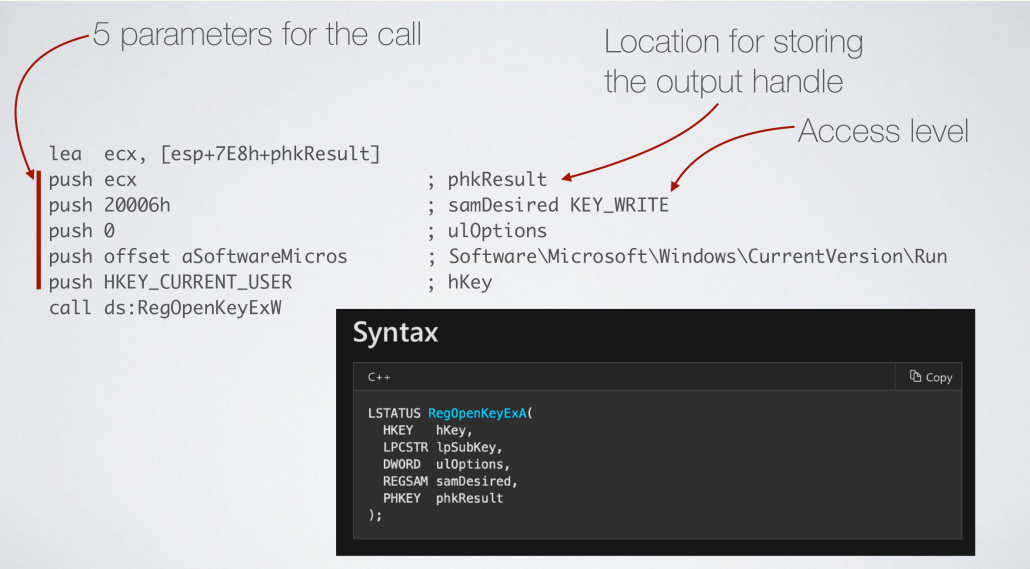


Figure 9.8: Annotated RegOpenKeyExW call (slides, Advanced static analysis).

Here the `RegOpenKeyExW` call has **5 parameters**. The two most interesting are the **location for storing the output handle** (`phkResult`, the address of a local variable, obtained with `lea`) and the **access level** (`20006h` = `KEY_WRITE`). The key is `HKEY_CURRENT_USER\...\Run`, so this one needs no admin rights. The Syntax box shows the prototype in the documentation order, which is the reverse of the `push` order.

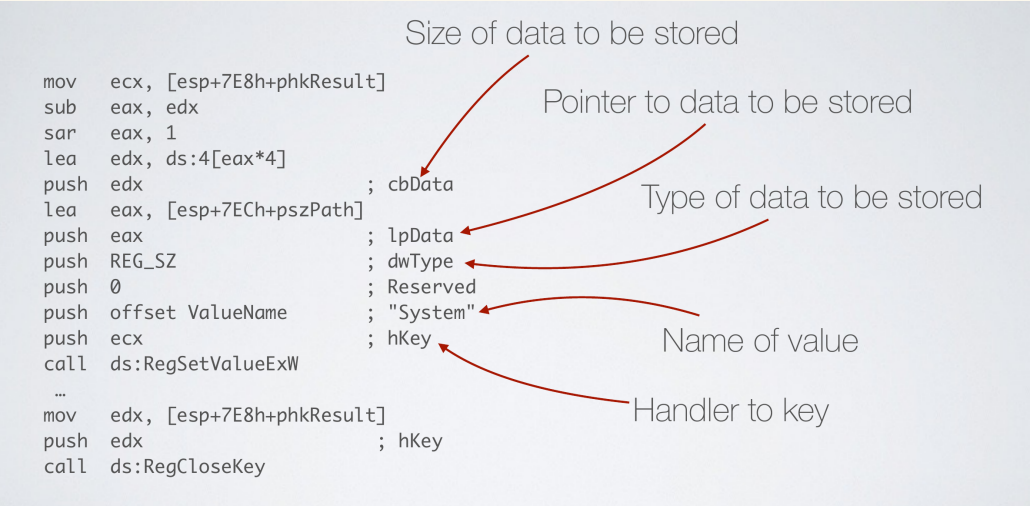


Figure 9.9: Annotated RegSetValueExW call (slides, Advanced static analysis).

`RegSetValueExW` then receives the size of the data (`cbData`), the pointer to the data (`pszPath`, a path, hence `lpData`), the type (`REG_SZ`), the value name (`"System"`, chosen to look harmless) and the handle obtained before, which is finally released with `RegCloseKey`.

> **To verify**
>
> The sub, `sar eax, 1` sequence turns a byte difference into a number of characters, from which `lea edx, ds:4[eax*4]` derives `cbData` ($4n + 4$ bytes). If `eax` is really the character count, this is more than the $2n + 2$ bytes a wide string needs. The code computing `eax` is not shown on the slide, so this cannot be confirmed.

> **Correction**
>
> The slide's annotation says "Handler to key". The correct term is **handle**. A handler is a function that handles an event, such as an exception handler.

> **From the lecture (27 October 2026, 48:13–53:36)**
>
> **Which arguments to read.** You do not need to dissect everything. In `RegOpenKeyEx`, look at `hKey`, which IDA usually recognizes as a standard constant, then at the string pushed for `lpSubKey`, and finally at the fifth argument. The root key goes in `hKey` and the rest of the path in the string. In `RegSetValueEx`, what matters is the key and the data, for example the

path of the malware.

Why `test` looks at a status. The professor first takes the `test eax, eax` for a handle check, then corrects the reading: the value returned is a status code. The handle is written through `phkResult`.

9.6 Network API

The **Winsock** libraries, primarily in `ws2_32.dll`, offer Berkeley-compatible sockets, almost identical in Windows and Unix.

Function	Description
`socket`	Creates a socket
`bind`	Attaches a socket to a particular port, prior to the `accept` call
`listen`	Indicates that a socket will be listening for incoming connections
`accept`	Opens a connection to a remote socket and accepts the connection
`connect`	Opens a connection to a remote socket; the remote socket must be waiting for the connection
`recv`	Receives data from the remote socket
`send`	Sends data to the remote socket

NOTE

The `WSAStartup` function must be called before any other networking functions in order to allocate resources for the networking libraries. When looking for the start of network connections while debugging code, it is useful to set a breakpoint on `WSAStartup`, because the start of networking should follow shortly.

Figure 9.10: Table of the core Winsock functions (slides, Advanced static analysis).

The table lists the core functions: `socket`, `bind`, `listen`, `accept`, `connect`, `recv` and `send`. The note adds an analysis tip: `WSAStartup` must be called

before any other networking function to allocate resources, so **a breakpoint on `WSAStartup`** finds the start of network activity while debugging.

Side	Role	Calls, in order
Server	Maintains an open socket waiting for connections	`socket`, `bind`, `listen`, `accept`, then `send` and `recv` as necessary
Client	Connects to a waiting socket	`socket`, `connect`, then `send` and `recv` as necessary

The call order tells you the role at a glance: a sample with `bind`, `listen` and `accept` opens a port (a backdoor, for instance), while one with `connect` reaches out to a server.

9.6.1 The WinINet API

WinINet is a higher-level API than Winsock, with functions in `Wininet.dll`. It implements **application-layer protocols** like HTTP and FTP.

- `InternetOpen` initialises use of the API.
- `InternetOpenURL` connects to a URL.
- `InternetReadFile` reads data from a URL.

With this API the malware does not implement HTTP by itself. The typical downloader is just these three calls, and the URL argument is an indicator of compromise.

> **From the lecture (27 October 2026, 55:51–56:51)**
>
> The functions used suggest the kind of sample. `accept`, `connect`, `recv` and `send` may indicate communication with a command-and-control server, perhaps a remote access tool that receives and parses commands. A simple sample that downloads and runs a second stage does not need that complexity, and `URLDownloadToFile` or similar APIs are enough.

9.7 Execution mechanisms

While `jmp` and `call` transfer control within the current thread, Windows provides mechanisms to **execute code in other threads or processes** and to coordinate that execution: DLLs, processes, threads, mutexes, services, the Component Object Model (COM) and exceptions.

9.7.1 Dynamic link libraries

DLLs enable sharing code among applications. Using the DLLs included in Windows makes code smaller, and software companies can also make custom ones. Malware authors use DLLs in multiple ways:

- use **Windows DLLs** to interact with the OS;
- **store malicious code in a DLL**, for example to load the DLL into another process;
- use **third-party DLLs**, for example a Firefox DLL to connect to a server instead of the Windows API.

> **From the lecture (27 October 2026, 58:51–63:04)**
>
> - **Diluting the behavior:** a solution focused on the delivered executable may see only that it creates a couple of DLLs on disk, which is why a good one analyzes those DLLs too.
> - **Search path tricks:** if a DLL is requested without a path, Windows resolves it, and a malicious DLL with the same name can be loaded instead of the legitimate one.
> - **Loading into another process:** a single API call can ask another process to load a DLL. The feature exists for flexibility and interoperability, and the professor can count at least 10 covert ways of forcing it.
> - **Third-party DLLs:** a security solution may watch less closely an

application it already trusts, for instance because it is digitally signed.

9.7.2 Basic DLL structure

- DLLs are very similar to EXEs and **share the PE file format**.
- A **single flag in the PE header** discriminates between a DLL and an EXE.
- DLLs have **more exports and fewer imports**.
- `DllMain` is a highly relevant function for initialisation and cleanup. It is not exported, but specified as the **entry point in the PE header**. It is called when a process loads or unloads the library, and on thread creation and termination.

Correction

The slide says "called when a **function** loads or unloads the library". It should read "when a **process** loads or unloads the library". PMA: the function "is called to notify the DLL whenever a process loads or unloads the library" (PDF p. 217).

This matters because code in `DllMain` runs as soon as the DLL is loaded, without any explicit export call. It explains why many malicious DLLs do not need an export (section 8.1).

From the lecture (27 October 2026, 63:08–66:16)

Because one bit separates the two, an editor can turn a DLL into an executable. `DllMain` is short and works like a callback, receiving the event in its `dwReason` parameter (values 0–3). Malware can use the thread events to tamper with every thread created. When analyzing a DLL, read `DllMain` first: it shows whether the sample starts on its own at these events or needs a specific export to be called. The professor returns to this at the end of the course.

9.7.3 Processes

A program being executed by Windows is a **process**, and each process has its own resources (handles, memory and so on). Each process has one or more execution units called threads. Malware can run in a standalone process, or also have some code execute **in another process**. Section 3.6.1 compares processes and threads with Linux.

`CreateProcess` creates a new process:

- `CommandLine` contains the command line to execute for the desired program;
- the `STARTUPINFO` parameter contains handles for the **standard input, output and error** streams. They could be set to a **socket for a remote shell**!

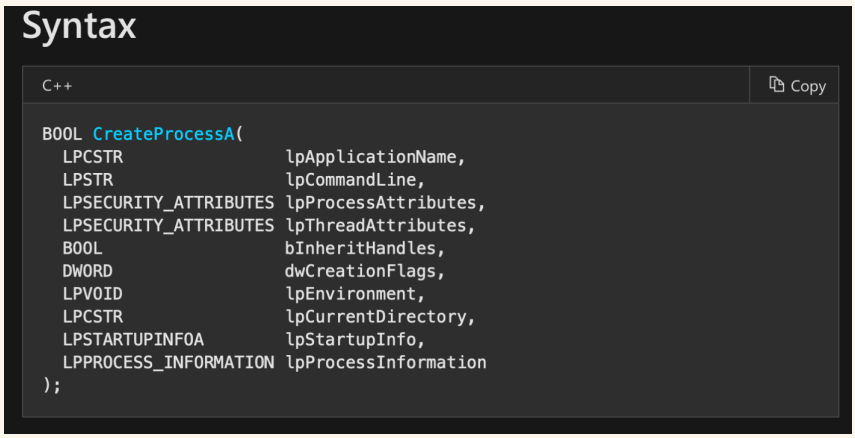


Figure 9.11: Prototype of CreateProcessA (slides, Advanced static analysis).

The prototype of `CreateProcessA` has ten parameters. The two to look at first are `lpCommandLine` (what runs) and `lpStartupInfo` (where its I/O goes). The remote-shell pattern is thus recognisable statically. The sample connects a socket, copies it to the three handles of `STARTUPINFO`, and launches `cmd.exe`: everything typed on the socket reaches the shell, and its output goes back over the network.

> **From the lecture (27 October 2026, 68:06–68:57)**
>
> Most `CreateProcess` parameters can be zero. The creation flags matter too, as seen later. About the remote-shell trick, the professor admits not

> having done it personally and shows it because the book and many resources consider it important.

9.7.4 Threads

Processes are containers, and each contains one or more threads. **Threads are what Windows actually executes**:

- they are independent sequences of instructions, independently scheduled by the OS, which may execute concurrently or in parallel;
- threads within a process **share the same memory space**;
- each thread has **its own stack and CPU execution context**;
- the context is the register state, including instruction and stack pointers, and the OS saves and restores it in a dedicated structure when switching threads on a CPU core;
- they are created with `CreateThread`, to run from a designated start function.

For the analyst, the start function passed to `CreateThread` is a new entry point to analyse: following the `call` alone would not reach it.

> **From the lecture (27 October 2026, 66:28–67:26 and 70:35–71:19)**
>
> **Thread hijacking.** Registers are saved in the context of the thread, which can also be read and changed from software: suspend a thread, change its instruction pointer, resume it. So look out for `CreateThread`, `ResumeThread`, `GetThreadContext` and `SetThreadContext`. A thread can also be started in another process, with an API for remote threads.

9.7.5 Exceptions

Exceptions are caused by **hardware faults**, such as division by zero or invalid memory access, or are **raised explicitly by software**. Windows provides

Structured Exception Handling (SEH) to dispatch them: when an exception occurs, execution may transfer to an appropriate handler.

FS is a segment register used to access **thread-specific data**.

Example 8-13. Storing exception-handling information in fs:0

```
01006170  push  1offset loc_10061C0
01006175  mov     eax, large fs:0
0100617B  push  2eax
0100617C  mov     large fs:0, esp
```

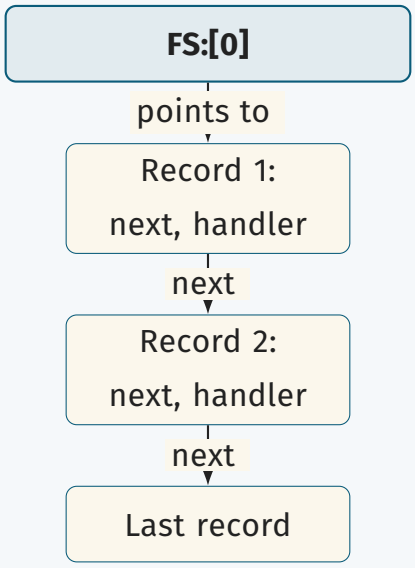
Figure 9.12: Registering an exception handler through FS:[0] (slides, Advanced static analysis).

This pattern **registers a handler** by building a new record on the stack:

```
push offset loc_10061C0     ; (1) handler address
mov  eax, large fs:0        ; current head of the chain
push eax                    ; (2) previous record: "next" field
mov  large fs:0, esp        ; the new record becomes the head
```

After these lines, the stack contains the record {next, handler} and `FS:[0]` points to it. When an exception occurs, Windows uses `FS:[0]` to find the **first SEH registration record on the stack**, then traverses the chain to locate a suitable handler.

Diagram: The SEH chain



`FS:[0]` points to the newest record, and each record links to the previous one through its `next` field. So code that writes to `fs:0` deserves attention: because control flow reaches the handler through the exception, a disassembler does not show it as a normal branch. Malware uses this to hide its flow (section 11.6).

> **Do not confuse**
>
> This stack-based mechanism belongs to **32-bit** code. On x64, handlers are described by tables in the PE file, and the thread block is reached through GS.

> **From the lecture (27 October 2026, 72:12–75:51)**
>
> A registration record is like a catch clause: Windows walks the chain until it finds a handler for that error, otherwise the program is terminated. Besides SEH, Windows has **Vectored Exception Handling**, not covered here.
>
> **Anti-debugging.** Malware can raise exceptions on purpose, since it expects to run without a debugger. If the analyst catches the exception in the debugger, the sample can detect it. So when the debugger reports an exception, you may need to let the program handle it, because the sample may have registered a handler (section 12.12).

9.7.6 Mutexes

Mutexes are synchronisation objects to coordinate access to shared resources, the same concept studied with semaphores in Sistemi Operativi.

- They are **inter-process communication (IPC) objects**, so they can be shared across threads and processes.
- They can be **named**, and such names are often **hard-coded**. This makes them a good source of indicators to identify malware, and malware may use them to check whether another instance is already running.

Functions for mutexes.

- `CreateMutex` creates a mutex, or opens an existing named mutex.

- `OpenMutex` opens an existing named mutex.
- `WaitForSingleObject` waits to acquire ownership of a mutex (blocking call). Any subsequent threads attempting to gain access must wait.
- `ReleaseMutex` is called when a thread is done using the mutex.

Malware single-instance check. `OpenMutex` checks whether the mutex `HGL345` exists. If not, it is created with `CreateMutex`. `test eax, eax` sets the Z flag if `eax` is zero.

```
00401007   push  1F0001h          ; dwDesiredAccess
0040100C  1call  ds:__imp__OpenMutexW@12 ;
OpenMutexW(x,x,x)
00401012  2test  eax, eax
00401014  3jz    short loc_40101E
00401016   push  0                ; int
00401018  4call  ds:__imp__exit
0040101E   push  offset Name      ; "HGL345"
00401023   push  0                ; bInitialOwner
00401025   push  0                ; lpMutexAttributes
00401027  5call  ds:__imp__CreateMutexW@12 ;
CreateMutexW(x,x,x)
```

Figure 9.13: Single-instance check with a mutex (slides, Advanced static analysis).

> **Step by step: Following the numbered markers**
>
> 1. `OpenMutexW` returns a handle, or `NULL` if the mutex does not exist.
> 2. `test eax, eax` checks the result.
> 3. `jz` jumps when `eax` is 0, that is, when the mutex **does not exist**, and goes to create it.
> 4. Otherwise `exit` is called: another instance is already running.
> 5. `CreateMutexW` creates `"HGL345"`, marking this instance as the running one.

The name is a hard-coded indicator, and it can even be used to **vaccinate** a machine, since a pre-existing mutex with that name makes this sample exit on its own.

> **From the lecture (27 October 2026, 76:41–78:57)**
>
> Why a single-instance check matters to the author: a user may click 20 times on the executable while waiting for something to appear, and the instances would interfere with each other. Wait and release functions are normally not what malware uses mutexes for. The clue is `CreateMutex` or `OpenMutex` with an `ExitProcess` nearby, or an infinite sleep.

9.7.7 Services

Services are background processes running without user input.

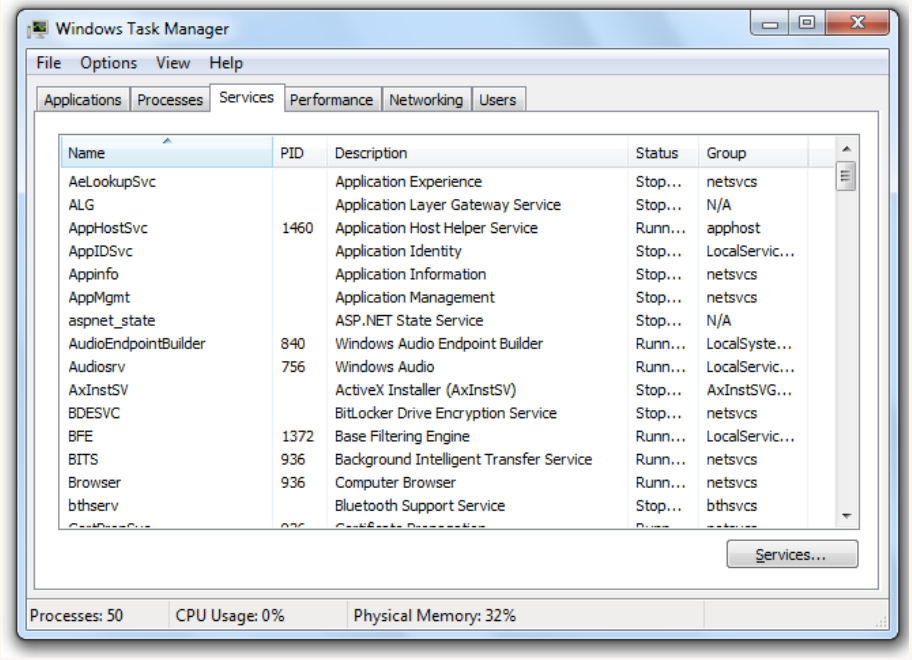


Figure 9.14: Services tab of Task Manager (slides, Advanced static analysis).

The Services tab of Task Manager lists name, PID, description, status and group. Note the many services in the `netsvcs` group: they relate to `svchost`, discussed below.

These processes are managed by the Windows **Service Control Manager (SCM)**. They can run independently of user logon and start automatically when Windows boots. Each service runs under a **configured security account**:

- some run as **LocalSystem (SYSTEM)**, with extensive local privileges,

including access to many protected system resources;

- others run under restricted service accounts or specific user accounts;
- service creation and configuration **require appropriate privileges**.

Malware can install services to establish **persistence**, potentially with elevated privileges.

Service API functions.

- `OpenSCManager` returns a handle to the Service Control Manager.
- `CreateService` adds a new service to the SCM, and can specify whether it will start automatically at boot time.
- `StartService` starts a stopped service, for example one set to start manually.

The sequence of these three calls in a sample is a clear sign of installation as a service, and the `CreateService` arguments give its name and binary path (indicators).

> **From the lecture (27 October 2026, 78:57–83:28)**
>
> - **Before login:** a service can run while the machine waits at the login prompt.
> - **SYSTEM:** think of it as a special root-like user. According to the professor, it can sometimes do what even an administrator cannot.
> - **UAC:** besides persistence, abusing a service may be an alternative to bypassing the User Account Control prompt, since bypasses are detectable and do not work on every version.
> - **Second stage:** a payload may just start a service created by another component, so `StartService` can appear alone.

svchost.exe. The `dwServiceType` of `CreateService` determines the `SERVICE_` type:

- `WIN32_OWN_PROCESS` (`0x10`): dedicated process;

- `WIN32_SHARE_PROCESS` (`0x20`): can share a process with other services.

`svchost.exe` is a Windows service-host executable that loads and runs services implemented as DLLs. Different services may run within the same instance or in separate ones. The actual service code resides in a **DLL**, not in `svchost.exe` itself. Analysts shall inspect the service configuration and its **`ServiceDll`** entry to identify the DLL being executed. While `svchost.exe` is intended for Windows services, that does not prevent malware from disguising malicious activity through it.

Service information in the registry. Under `HKLM\System\CurrentControlSet\Services\<ServiceName>`:

- `Start` value = `0x03` for "Load on Demand";
- `Type` = `0x20` for `WIN32_SHARE_PROCESS`;
- `ImagePath` = executable or driver path.

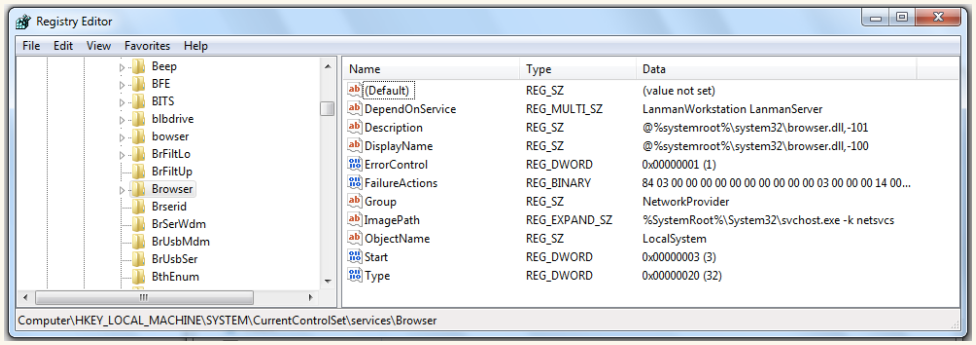


Figure 9.15: The Browser service in the registry (slides, Advanced static analysis).

The `Browser` service matches the description: `Start` = 3, `Type` = `0x20`, `ObjectName` = `LocalSystem`, and `ImagePath` = `%SystemRoot%\System32\svchost.exe -k netsvcs`. So the `ImagePath` only says "run inside svchost, group netsvcs". The DLL is found through `ServiceDll`, usually in the `Parameters` subkey of the service.

> **From the lecture (27 October 2026, 83:28–85:27)**
>
> Originally almost all services ran in a single `svchost.exe`. Malware authors noticed that they could load their own DLL there as a service, and

> nobody looking at Task Manager would notice, because `svchost.exe` is always running. It was not meant for applications, only for Windows maintenance work. Today there are several instances, and the check remains opening the service configuration to read `ServiceDll`.

9.7.8 Component Object Model

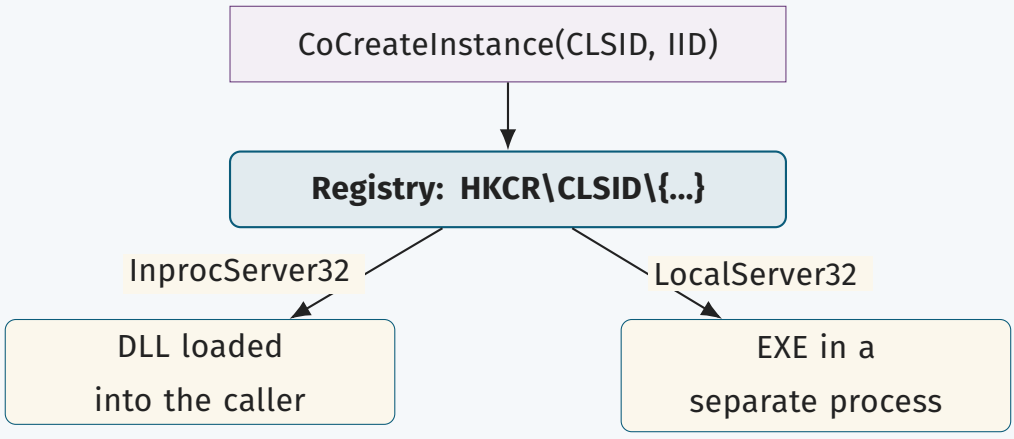
COM allows software components to expose functionality through **standardised interfaces**, including across process boundaries. Every thread using COM generally initialises the COM library by calling `OleInitialize` or `CoInitializeEx`.

COM classes and interfaces are identified via **Globally Unique Identifiers (GUIDs)**. The main types are:

- **Class Identifiers (CLSIDs)** identify classes of objects, in the registry under `HKEY_CLASSES_ROOT\CLSID`;
- **Interface Identifiers (IIDs)** identify interfaces exposed by objects, in the registry under `HKEY_CLASSES_ROOT\Interface`.

COM object activation. Applications instantiate COM objects using `CoCreateInstance`, specifying a CLSID and a requested IID. Windows typically uses the CLSID registration to locate the corresponding implementation.

Diagram: COM activation



The call names a CLSID, and Windows looks it up in the registry. An

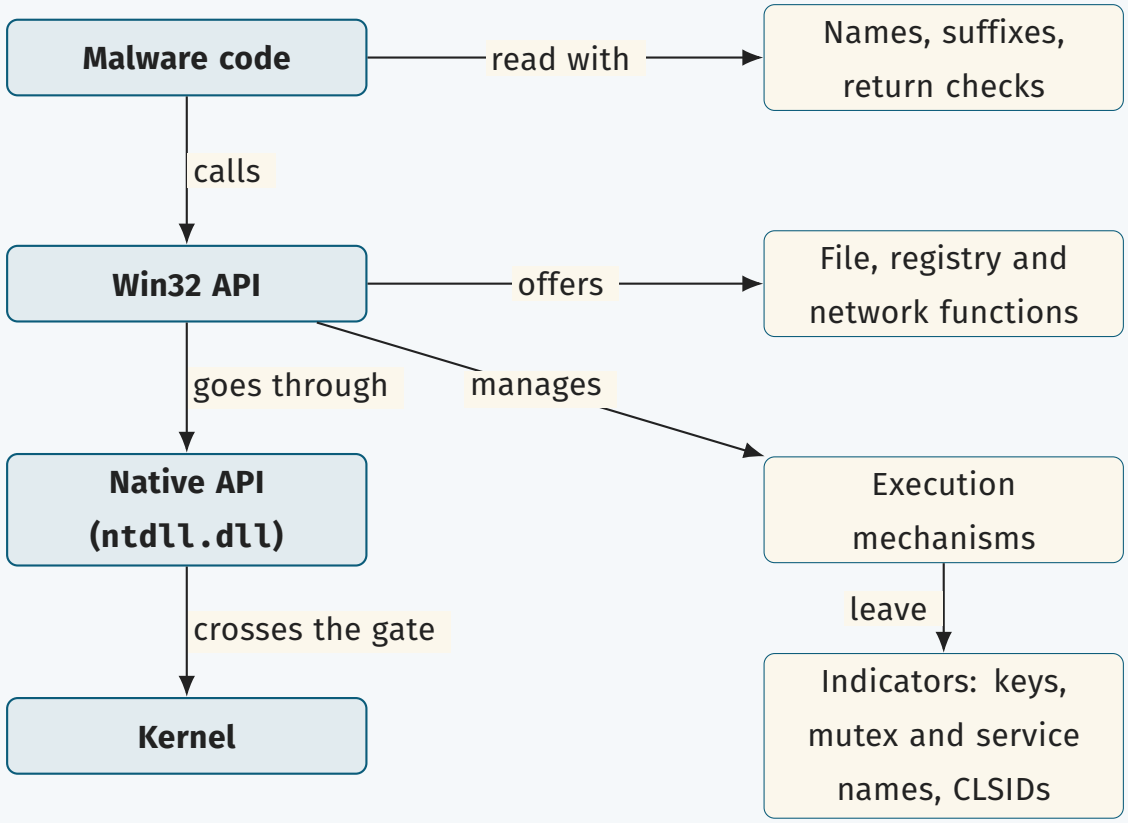
`InprocServer32` entry leads to a DLL loaded into the calling process. A `LocalServer32` entry leads to an EXE running in a separate process.

Malware relevance.

- Malware can use COM objects to access Windows functionality. In the disassembly this appears as calls through an interface pointer rather than as named imports, so the CLSID and IID constants are the clues to look up.
- **COM hijacking**: malicious modifications to COM registrations can cause legitimate applications to load attacker-controlled code, which also gives persistence.

> **From the lecture (27 October 2026, 85:28–89:43)**
>
> COM is more structured than DLLs: it shares interfaces, not just raw code. You may know it by another name, **OLE**, typical of Office interoperability, as when a table moves between Excel and PowerPoint. It is widely abused yet understudied. Checks usually cover only the expected usage patterns, so a hijacked COM object entitled to elevated operations can offer escalation primitives, without bypassing UAC or registering a privileged service. The group of the professor found more such ways in a thesis project, and a seminar may follow at the end of the course.

Concept map: how the ideas connect



On the left is the path of a call. Malware code calls Win32, Win32 goes through the Native API, and the Native API crosses the gate into the kernel. Code that uses the Native API directly is suspicious. On the right is what the analyst reads: names and return checks at the call site, the API families, and the execution mechanisms with the indicators they leave.

Remember

- User-mode code reaches the kernel only through gates, via `ntdll.dll`; direct Native API use is suspicious.
- Read call sites: Hungarian names, `Ex`, `A` and `W` suffixes, `test eax, eax` on the return value, and handles linking the calls together.
- Each mechanism has a signature and indicators: Run keys, mutex names, service names, `ServiceDll`, CLSIDs.
- `socket`, `bind`, `listen`, `accept` indicates a server; `socket`, `connect` a client; WinINet an HTTP or FTP client.

Review: try to answer without looking

1. What does `lea edx, [eax+eax+2]` compute before `RegSetValueExW`?
2. A DLL has no exports. Can it still run code when loaded?
3. Why does a disassembler not show the path to an SEH handler as a normal branch?
4. Where do you find the DLL of a service hosted by `svchost.exe`?

Answers, with the section where to find them

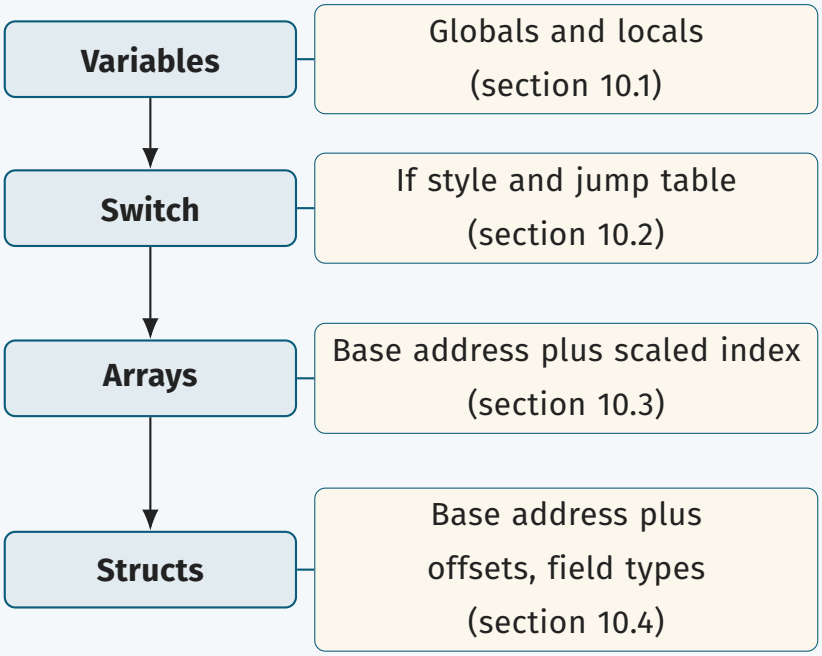
1. The size in bytes of a wide string with its NUL, $2n + 2$ (section 9.5.2).
2. Yes, in `DllMain` (section 9.7.2).
3. Control reaches it through the exception (section 9.7.5).
4. In the `ServiceDll` entry (section 9.7.7).

Chapter 10

Recognising C constructs in assembly

> **Before you start**
>
> **Central question:** when a quick code inspection shows only instructions, how do we recover the variables, branches, arrays and structures that the programmer wrote?
>
> **Prerequisites:** addressing modes (section 3.8.1), flags and conditional jumps (section 4.2), stack frames (section 4.4).
>
> **Where it leads:** these patterns make the listings of the debugging and evasion chapters readable (chapters 11 and 12).

Chapter structure: what comes, and in which order



The chapter goes through four C constructs, from the simplest to the richest. Each section shows the source and then the assembly pattern it produces.

The Windows API tells us what a program asks of the system. The patterns in this chapter tell us how the program organises its own data and decisions. They complement the hands-on sessions on x86 assembly (chapter 5 and section 13.1).

Each section puts the C source and the compiled code side by side. Read the source first, then look for where each C element went.

From the lecture (27 October 2026, 89:43)

The patterns of this chapter were not covered in this lecture: they were left for the labs, since the professor considers them better treated there. Section 14.1 applies the workflow to a real binary in IDA, with strings and references, calling conventions, loops and arrays.

10.1 Global and local variables

A **global** variable can be accessed and used by any function in a program. A **local** variable can be accessed only by the function that defines it. Both are declared similarly in C, but they **look completely different in assembly**.

Globals are referenced by memory address.

```
int x = 1;
int y = 2;

void main()
{
    x = x+y;
    printf("Total = %d\n", x);
}
```

```
00401003   mov   eax, dword_40CF60
00401008   add   eax, dword_40C000
0040100E   mov   dword_40CF60, eax    ; x is changed in memory
00401013   mov   ecx, dword_40CF60
00401019   push  ecx
0040101A   push  offset aTotalD       ; "total = %d\n"
0040101F   call  printf
```

Locals are referenced by stack address, at a constant offset from ebp (section 4.4.3).

```
void main()
{
    int x = 1;
    int y = 2;

    x = x+y;
    printf("Total = %d\n", x);
}
```

```
00401006   mov   dword ptr [ebp-4], 1
0040100D   mov   dword ptr [ebp-8], 2
00401014   mov   eax, [ebp-4]
00401017   add   eax, [ebp-8]
```

```
0040101A   mov   [ebp-4], eax
0040101D   mov   ecx, [ebp-4]
00401020   push  ecx
00401021   push  offset aTotalD       ; "total = %d\n"
00401026   call  printf
```

> **Correction**
>
> The slide's constants are 1 and 2, patched over the original listing to match the C code. The listing in the book assigns 0 to `[ebp-4]` and 1 to `[ebp-8]` (PMA, PDF pp. 170–171), which does not match `x = 1; y = 2`. This book follows the slide.

So a fixed address means that the value outlives the function and that every function using it is affected. An `ebp-N` location lives only in the current frame.

10.2 Switch

A **switch** chooses among several code blocks according to a value. Compilers translate it in two ways, and the analyst should recognise both.

10.2.1 If style

```
switch(i)
{
    case 1: printf("i = %d", i+1); break;
    case 2: printf("i = %d", i+2); break;
    case 3: printf("i = %d", i+3); break;
    default: break;
}
```

```
00401013   cmp   [ebp+var_8], 1
00401017   jz    short loc_401027
00401019   cmp   [ebp+var_8], 2
```

```
0040101D   jz    short loc_40103D
0040101F   cmp   [ebp+var_8], 3
00401023   jz    short loc_401053
00401025   jmp   short loc_401067      ; default
00401027 loc_401027:
00401027   mov   ecx, [ebp+var_4]
0040102A   add   ecx, 1
0040102D   push  ecx
0040102E   push  offset unk_40C000     ; i = %d
00401033   call  printf
00401038   add   esp, 8
0040103B   jmp   short loc_401067
0040103D loc_40103D:
0040103D   mov   edx, [ebp+var_4]
00401040   add   edx, 2
00401043   push  edx
00401044   push  offset unk_40C004     ; i = %d
00401049   call  printf
0040104E   add   esp, 8
00401051   jmp   short loc_401067
00401053 loc_401053:
00401053   mov   eax, [ebp+var_4]
00401056   add   eax, 3
00401059   push  eax
0040105A   push  offset unk_40C008     ; i = %d
0040105F   call  printf
00401064   add   esp, 8
```

A chain of `cmp` and `jz` selects the case. Each case ends with a `jmp` to the common exit, which is the compiled `break`.

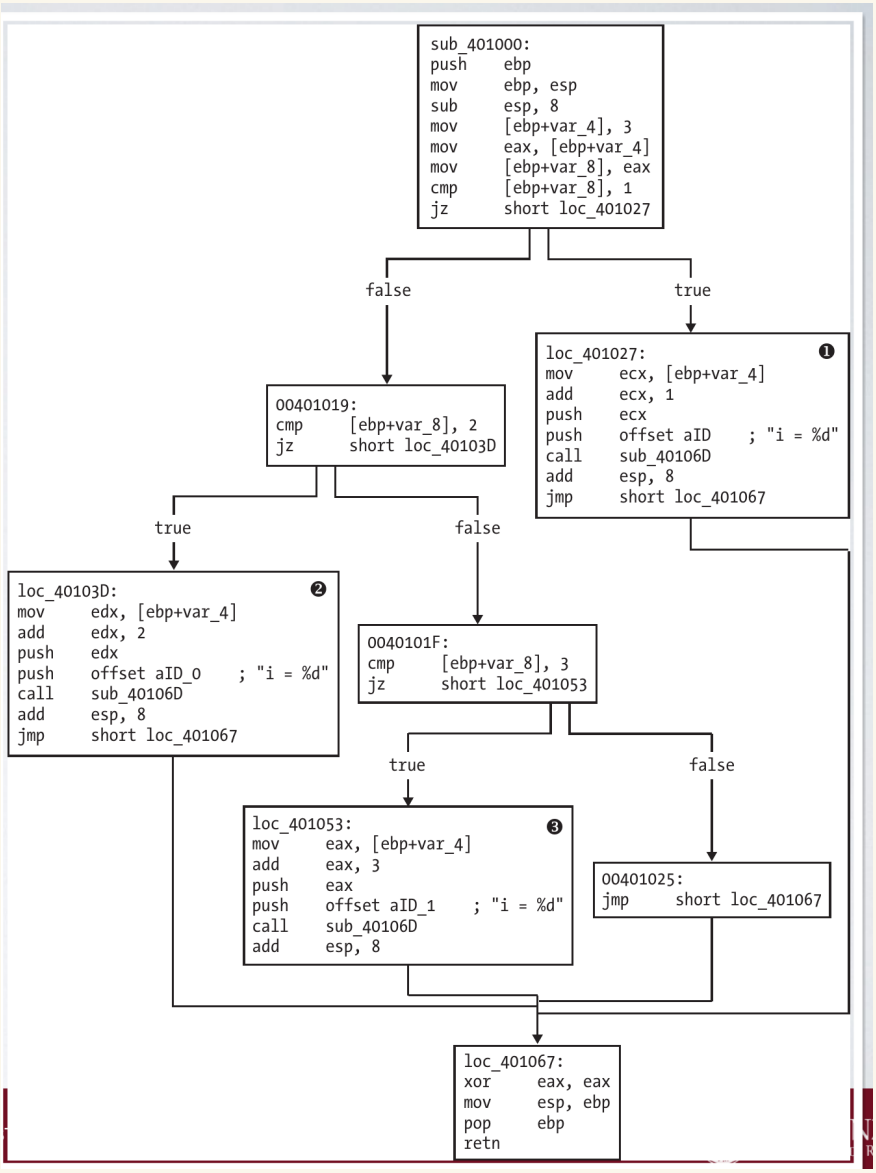


Figure 10.1: The if-style switch as an IDA graph: a cascade of comparisons that reconverges at `loc_401067` (slides, Advanced static analysis).

In the graph the same code looks like a **cascade**. Each comparison has a true branch to its case and a false branch to the next comparison, and all paths reconverge at `loc_401067`. It is hard to distinguish from a real sequence of `if` and `else if`.

The cost is **linear**: with `i = 3`, all three comparisons are executed.

10.2.2 Jump table

With one more case, the compiler changes strategy and the assembly is drastically different.

```
switch(i)
{
    case 1: printf("i = %d", i+1); break;
    case 2: printf("i = %d", i+2); break;
    case 3: printf("i = %d", i+3); break;
    case 4: printf("i = %d", i+3); break;
    default: break;
}
```

```
00401016   sub   ecx, 1
00401019   mov   [ebp+var_8], ecx
0040101C   cmp   [ebp+var_8], 3
00401020   ja    short loc_401082      ; default
00401022   mov   edx, [ebp+var_8]
00401025   jmp   ds:off_401088[edx*4]
0040102C loc_40102C:
           ...
00401040   jmp   short loc_401082
00401042 loc_401042:
           ...
00401056   jmp   short loc_401082
00401058 loc_401058:
           ...
0040106C   jmp   short loc_401082
0040106E loc_40106E:
           ...
00401082 loc_401082:
00401082   xor   eax, eax
00401084   mov   esp, ebp
00401086   pop   ebp
```

```
00401087   retn
00401087 _main endp
00401088 off_401088 dd offset loc_40102C
0040108C            dd offset loc_401042
00401090            dd offset loc_401058
00401094            dd offset loc_40106E
```

Step by step: Reading the jump table

1. `sub ecx, 1` **normalises** `i` to the range 0–3, since the table starts from index 0.
2. `cmp [ebp+var_8], 3` followed by `ja` sends out-of-range values to the default. `ja` is an **unsigned** comparison, so `i = 0` ends there as well, because 0 - 1 becomes `0xFFFFFFFF`.
3. `jmp ds:off_401088[edx*4]` is the jump itself. It reads the address from the table at offset index × 4, since **each entry is a 4-byte address**.
4. `off_401088` is the table: four addresses stored right after the function.

With `i = 3`, edx is 2, and execution jumps to the third entry, `loc_401058`, **in constant time**.

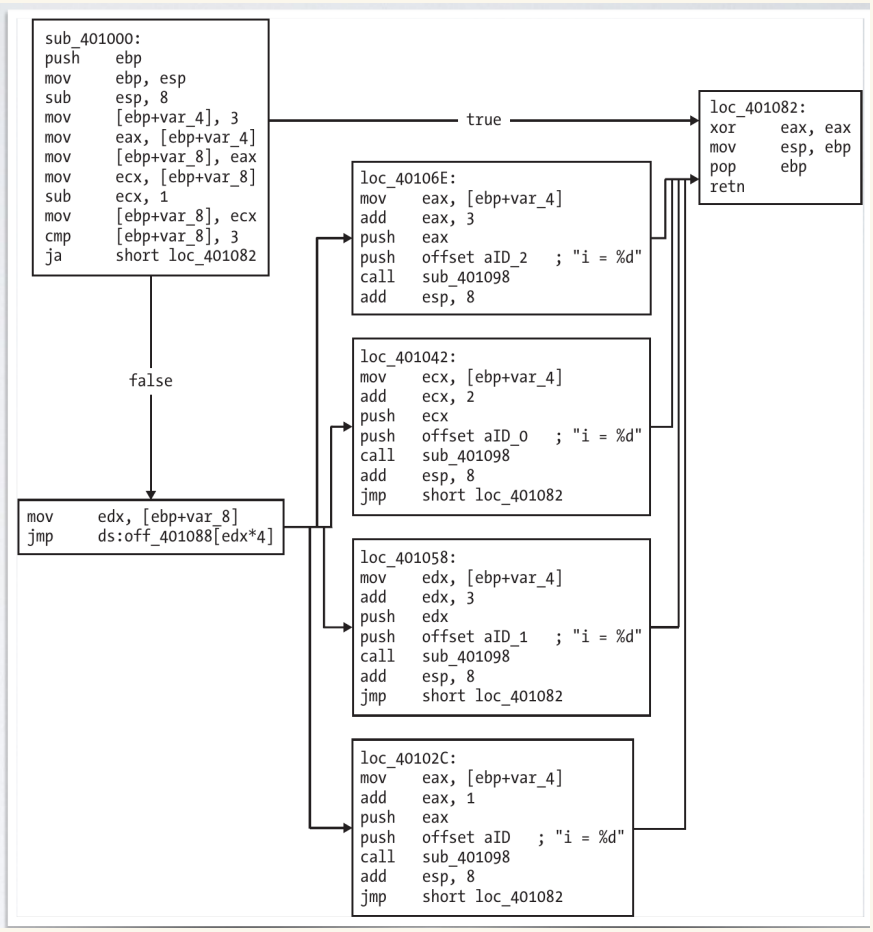


Figure 10.2: The jump-table switch as an IDA graph. The block with the indirect jump shows no ordinary edges towards the cases (slides, Advanced static analysis).

The graph shows the difference from the if style. A range check leads to the default (`loc_401082`). The block ending in `jmp ds:off_401088[edx*4]` has no visible edges towards the four cases, which are simply listed as separate blocks. IDA reconstructs these targets from the table.

> **Remember**
>
> The **indirect `jmp` through a table** is the signature to look for.

> **To verify**
>
> Case 4 prints `i+3`, the same as case 3. This looks like a copy-paste in the example, but the assembly matches according to the course notes: `loc_40106E` also adds 3. The body of that block is abbreviated in the listing above.

10.3 Arrays

In assembly, **arrays** are accessed using a **base address** as a starting point. The size of each element is not always obvious, but it can be determined by seeing **how the array is indexed**.

```
int b[5] = {123,87,487,7,978};
void main()
{
    int i;
    int a[5];

    for(i = 0; i<5; i++)
    {
        a[i] = i;
        b[i] = i;
    }
}
```

```
00401006   mov   [ebp+var_18], 0
0040100D   jmp   short loc_401018
0040100F loc_40100F:
0040100F   mov   eax, [ebp+var_18]
00401012   add   eax, 1
00401015   mov   [ebp+var_18], eax
00401018 loc_401018:
00401018   cmp   [ebp+var_18], 5
0040101C   jge   short loc_401037
0040101E   mov   ecx, [ebp+var_18]
00401021   mov   edx, [ebp+var_18]
00401024   mov   [ebp+ecx*4+var_14], edx    ; local array a
00401028   mov   eax, [ebp+var_18]
0040102B   mov   ecx, [ebp+var_18]
0040102E   mov   dword_40A000[ecx*4], eax   ; global array b
00401035   jmp   short loc_40100F
```

- `var_18` is `i`: initialisation, increment and the check against 5 form the `for` loop.
- `a` is local, so its base is on the stack (`ebp + var_14`). `b` is global, so its base is a fixed address (`dword_40A000`).
- The multiplier 4 reveals the **element size**: 4 bytes, that is, `int`. The two addressing modes have the shape base + index × size (section 3.8.1).

10.4 Structs

Structs are similar to arrays, but elements can have **different types**. They are accessed from a **base address** plus a constant **offset**.

```
struct my_structure {
    int x[5];
    char y;
    double z;
};

struct my_structure *gms;

void test(struct my_structure *q)
{
    int i;
    q->y = 'a';
    q->z = 15.6;
    for(i = 0; i<5; i++){
        q->x[i] = i;
    }
}

void main()
{
    gms = (struct my_structure *) malloc(
```

```
    sizeof(struct my_structure));
    test(gms);
}
```

In `main`, `gms` is a global, so its base pointer is stored at the fixed location `dword_40EA30`. It is passed to the `test` function via the stack.

```
00401050   push  ebp
00401051   mov   ebp, esp
00401053   push  20h               ; sizeof(struct
  my_structure)
00401055   call  malloc
0040105A   add   esp, 4
0040105D   mov   dword_40EA30, eax
00401062   mov   eax, dword_40EA30
00401067   push  eax
00401068   call  sub_401000
0040106D   add   esp, 4
00401070   xor   eax, eax
00401072   pop   ebp
00401073   retn
```

In `test`, `arg_0` is the base address of the structure.

```
00401000   push  ebp
00401001   mov   ebp, esp
00401003   push  ecx
00401004   mov   eax,[ebp+arg_0]
00401007   mov   byte ptr [eax+14h], 61h   ; q->y = 'a'
0040100B   mov   ecx, [ebp+arg_0]
0040100E   fld   ds:dbl_40B120             ; load 15.6
00401014   fstp  qword ptr [ecx+18h]       ; q->z
00401017   mov   [ebp+var_4], 0
0040101E   jmp   short loc_401029
00401020 loc_401020:
```

```
00401020   mov   edx,[ebp+var_4]
00401023   add   edx, 1
00401026   mov   [ebp+var_4], edx
00401029 loc_401029:
00401029   cmp   [ebp+var_4], 5
0040102D   jge   short loc_40103D
0040102F   mov   eax,[ebp+var_4]
00401032   mov   ecx,[ebp+arg_0]
00401035   mov   edx,[ebp+var_4]
00401038   mov   [ecx+eax*4],edx           ; q->x[i] = i
0040103B   jmp   short loc_401020
0040103D loc_40103D:
0040103D   mov   esp, ebp
0040103F   pop   ebp
00401040   retn
```

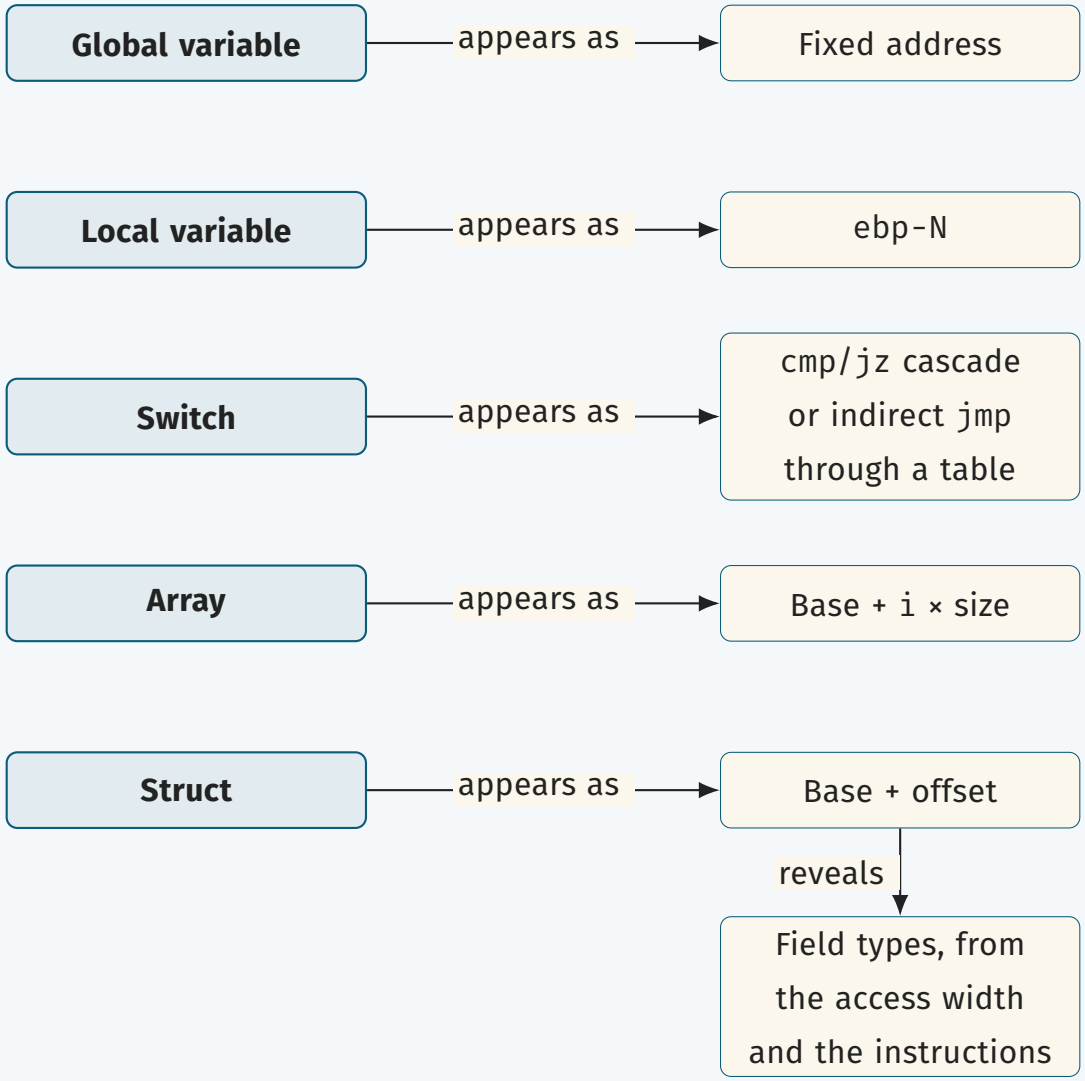
The field types are recovered from how they are accessed.

Offset	Access	Deduction	C field
`0x00`	`[ecx+eax*4]`, loop to `5`	Array of `5` dwords (`0x14` bytes)	`int x[5]`
`0x14`	`mov byte ptr`, value `0x61` = `'a'`	One byte, a character	`char y`
`0x18`	`fld/fstp qword ptr`	8-byte floating point	`double z`

The size of the struct is `0x18` + `8` = `0x20`, which is the `push 20h` before `malloc`. Offsets `0x15`–`0x17` are **padding**, which aligns the double to an 8-byte boundary. This is the reason why `y` is at `0x14` and `z` at `0x18`, rather than `0x15`.

> **Do not confuse**
>
> **Array** and **struct** both start from a base address. An array uses a variable index scaled by the element size. A struct uses constant offsets, and each field can have a different width.

Concept map: how the ideas connect



Read each row from left to right: the source construct, then the pattern it leaves in assembly. Globals use fixed addresses and locals use offsets from EBP. A switch is a cascade or a jump table. Arrays use a scaled index and structs a fixed offset. The way the code accesses a struct field then reveals the type of that field.

In short

- Globals use fixed addresses, and locals use `ebp-N`.
- A `switch` is either a `cmp/jz` cascade (linear cost) or an indirect `jmp` through a table (constant cost).
- Arrays are base + `i` × size. Structs are base + offset.

- Field types are recovered from the way the code accesses them, including the width of the access and the instructions used.

Review: try to answer without looking

1. Why does `i = 0` reach the default branch of the jump-table switch?
2. In the array example, which detail tells you that the elements are 4 bytes wide?
3. Why is `z` at offset `0x18` rather than `0x15`?

Answers, with the section where to find them

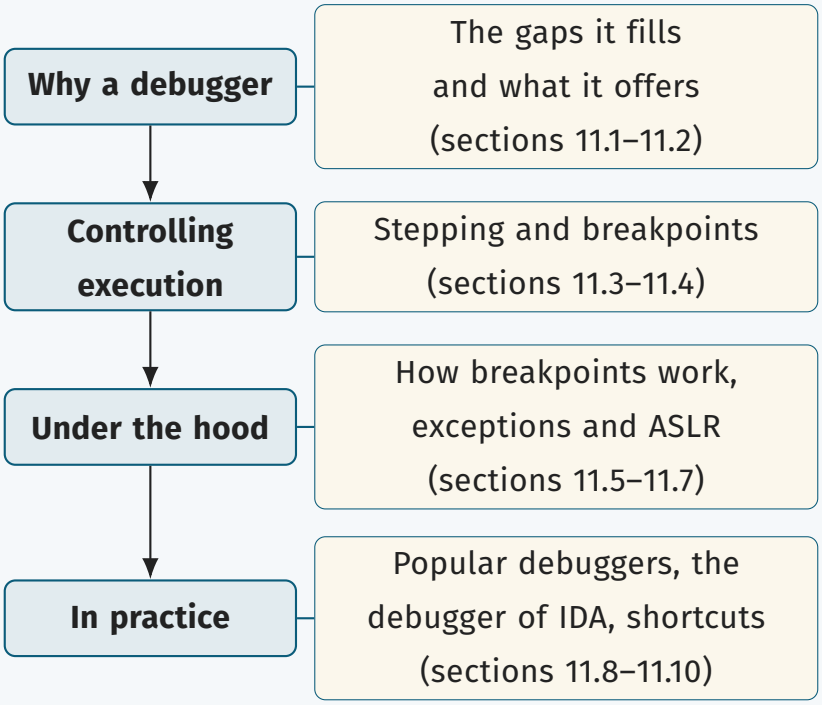
1. `sub ecx, 1` turns 0 into `0xFFFFFFFF`, and the unsigned `ja` treats it as greater than 3 (section 10.2.2).
2. The scale factor in `[ebp+ecx*4+var_14]` and `dword_40A000[ecx*4]` (section 10.3).
3. Padding at `0x15`–`0x17` aligns the double to an 8-byte boundary (section 10.4).

Chapter 11

Advanced dynamic analysis: the debugger

> **Before you start**
>
> **Central question:** how can we follow a running sample instruction by instruction, and change what it does?
>
> **Prerequisites:** basic dynamic analysis (chapters 7 and 8), code reading (chapters 9 and 10), stack frames (section 4.4).
>
> **Where it leads:** many tricks in chapter 12 target exactly the mechanisms explained here.

Chapter structure: what comes, and in which order



The chapter starts from what a debugger adds to the earlier techniques. It then explains how to control execution and how those controls work inside. It ends with practical use in IDA.

Basic dynamic analysis watches a sample from the outside, and advanced static analysis reads its code without running it. **Advanced dynamic analysis combines the two with a debugger**: it runs the code instruction by instruction, shows registers and memory, and lets the analyst change the execution.

The chapter first explains how debuggers work: stepping, breakpoints, exceptions and ASLR. It then shows how to use the debugger of IDA in practice.

11.1 Why we need a debugger

Basic dynamic analysis focuses on externally observable behaviour, while advanced static analysis focuses on malware internals. Both can fall short, for different reasons:

- **malware evasion**, for example checks for analysis tools and virtual machines (chapter 12);

- **trigger-based behaviour**, for example a packet carrying a command;
- **code obfuscation and packing** (section 6.4);
- **anti-disassembly and self-modifying code** (section 4.9).

A debugger addresses these cases directly. A packed sample eventually unpacks itself in memory, where it can be read. A trigger can be reached by changing a register or forcing a branch, without having to reproduce the real condition.

> **From the lecture (3 November 2026, 01:30–06:00)**
>
> The professor considers trigger-based behavior one of the hardest cases to model. A dormant feature may need a network packet carrying a command, such as downloading a DLL or stealing documents. Even when the sample does connect and the attacker responds, you only see the commands sent during that observation, so other features stay hidden.

11.2 What a debugger offers

A **debugger** supports the analysis of execution at different levels:

- instruction level;
- register and memory contents;
- function calls;
- threads and exceptions.

It gives the ability to **inspect and control execution** to gain insights. Analysts can also use it to **alter execution**.

There are two ways of debugging in user space:

- **load and run** a program from the debugger;
- **attach** the debugger to a running process.

Loading lets you observe the program from its very first instruction. Attaching is useful when the process is already running, or when you only care about what it does from a certain moment on.

Desirable features are the following:

- inspect and modify memory and registers, including `EIP`;
- capture control-flow transfers and specific instructions;
- control threads (pause and resume);
- modify code.

> **From the lecture (3 November 2026, 07:30–14:40)**
>
> - **Attaching in practice:** you normally attach to a program that malware is abusing, not to the running malware. A sample may force a victim process to load a malicious library, so you attach to the victim and intercept that moment.
> - **Why control threads:** a sample may have many threads handling background activities. Pause the ones you do not care about, but resume them when the sample needs them to make progress.
> - **Forcing a branch:** this can be unsound, because the data of the program are set for the other direction. It suits checks such as “am I in a VM?”, which do not affect the rest of the execution (section 14.2.3 shows the cost).
> - **Kernel debugging:** not covered in this course. The principles are similar, the tools differ, and it can even be done through a hypervisor.

11.3 Following instructions

A debugger can follow **one thread at a time**. **Stepping** lets you execute instructions in a controlled way.

Single-stepping goes through the instructions one at a time, exactly as the CPU

sees them arriving. It should be used **selectively, on limited code portions**. Stepping through a whole program would take forever, so you first locate the interesting region, for example with the API call sites from static analysis (section 9.1).

Function calls and returns are **milestones** in the execution.

Action	What happens	When to use it
Step into a `call`	Enters the called function, whose instructions can then be examined	The function is unknown or interesting
Step over a `call`	Single-stepping resumes only when the called function returns (if it ever does so)	The function is already understood or irrelevant, for example a library call
Step out of a function	Single-step execution is suspended, and resumes only once execution leaves the function	You entered a function and have seen enough

Remember

"If it ever does so." A step over assumes that the function returns. If it never does, for example because it calls `exit` or jumps elsewhere, the debugger does not regain control there and the code runs freely. So set a breakpoint further on when you are not sure.

From the lecture (3 November 2026, 15:40–19:50)

`Step over` works by waiting for execution to return to the return address written by `call`. `Step out`, also called *run until return*, works well right after stepping in by mistake, because the debugger can look up the return address on the stack, but it is not guaranteed to succeed. Never step into a library call like `CreateFile`: you do not need to debug how Windows is implemented.

11.4 Breakpoints

Breakpoints let the program **run until a point of interest is reached**. This complements stepping: you skip what you do not need and stop only where it matters.

- **Instruction breakpoints** pause execution once `EIP` reaches the address of interest.
- **Memory breakpoints** are more general and trap when the CPU accesses specific memory addresses. They can act on read and write access, on write access only, or on execute (like an instruction breakpoint).
- **Conditional breakpoints** suspend execution only when an additional condition is matched, for example stopping at address `0x401130` only when `ECX = 8`.

> **From the lecture (3 November 2026, 21:55–26:30)**
>
> Instruction breakpoints assume you can pinpoint an instruction, which can be limiting. A memory breakpoint fits when you know a behavior instead, such as “stop when this variable changes”. The example is the global compared with 6 in the secret stage of Lab 04 (section 14.1.5). For a conditional breakpoint, the typical case is a loop running 1000 times when only the last iteration interests you: you would otherwise press *continue* 999 times.

11.5 How breakpoints are implemented

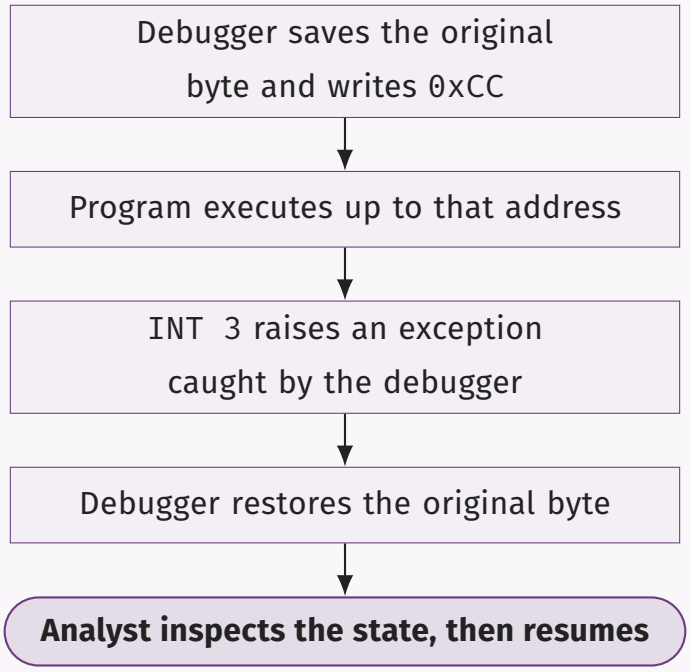
Knowing how each breakpoint is implemented explains its limits and how malware can detect it. This is also the basis of anti-debugging (section 12.12). Instruction breakpoints can be implemented via software or with hardware assistance.

11.5.1 Software breakpoints

A **software breakpoint** replaces the first byte of the instruction at the given address with `INT 3` (opcode `0xCC`).

- The CPU generates a software interrupt, and the debugger captures the event.
- Analysts can insert **as many software breakpoints as they want**.
- They are **conspicuous**, and malware may look for them, by performing a checksum of `.text` or by looking for `0xCC` in its code.

Flowchart: The life of a software breakpoint



The debugger replaces a byte before execution reaches it. When the CPU executes `0xCC`, control passes to the debugger, which puts the original byte back so that the analyst can inspect the state and then resume.

From the book

The debugger shows you the original byte, so you do not see the `0xCC` in its view, but other programs reading that memory do. Self-modifying code may also overwrite the breakpoint and remove it (PMA, PDF pp. 250–251).

From the lecture (3 November 2026, 29:26–32:40)

`INT 3` is one byte long for a precise reason: the smallest x86 instruction is one byte, so it can replace the start of any instruction. The debugger must remember the overwritten byte, because it has to put it back to run the code and then place the breakpoint again. Some packers and executable protectors also detect software breakpoints. A scan for `0xCC` could give false positives, but not for the author of the code, who knows which ones are legitimate.

11.5.2 Hardware breakpoints

A **hardware breakpoint** does not alter code. It leverages the **CPU debug registers** to trap execution automatically.

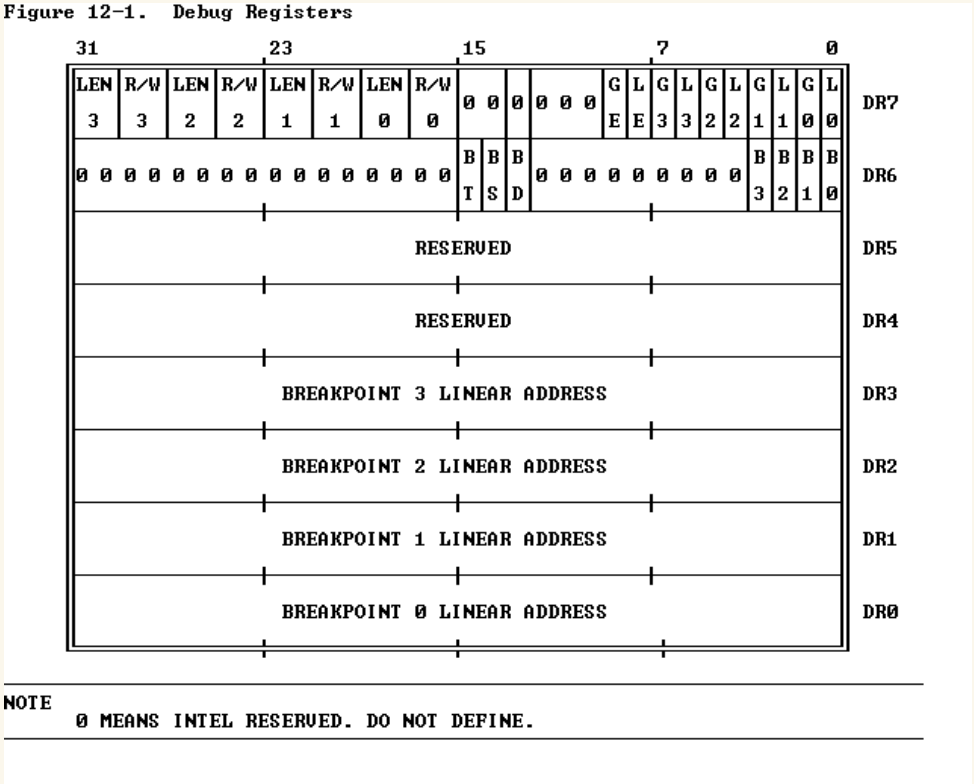


Figure 11.1: The debug registers, from the Intel manual (slides, Advanced dynamic analysis).

Register	Role
DR0–DR3	The linear addresses of the breakpoints
DR4–DR5	Reserved
DR6	Status: which condition was detected
DR7	Control: enable bits, type and length of each breakpoint

Two consequences follow:

- their number is **limited** (4 on x86), one per address register;
- they are **harder to detect**, because no byte changes in memory, so a checksum of `.text` finds nothing.

> **From the book**
>
> Malware can still modify the debug registers to interfere with hardware breakpoints. As a countermeasure, the General Detect flag in DR7 triggers a breakpoint before any `mov` that accesses a debug register, so it notices such changes, though only those made with `mov` (PMA, PDF p. 252).

> **From the lecture (3 November 2026, 32:47–40:03)**
>
> **Detecting hardware breakpoints.** Debug registers cannot normally be read from user space, but when a thread is suspended and its context read, Windows also copies DR0–DR3 into the context structure. A sample can check whether those values are 0. Analysts recognize these accesses to a context structure, or use plugins or their own code to hide them.
>
> **Do they slow execution?** A student asks. The professor says the impact is barely noticeable and comparable to background noise, though in principle a very competent attacker might measure a micro-architectural side effect. Mainstream malware is not that sophisticated.

11.5.3 Conditional breakpoints

Conditional breakpoints evaluate a user-provided condition and suspend execution only when it is satisfied.

- They are very useful on **loops**, for example to break only after N iterations.
- They are useful also for **function calls**, for example an argument-related condition such as a `CreateFile` on a specific path.
- **Careful:** they slow down execution even when they do not suspend it.

The slowdown comes from their implementation. PMA describes them as software breakpoints that break only if a certain condition holds (PDF p. 252). The program stops at every hit, the debugger evaluates the condition, and it resumes if the condition is false. A conditional breakpoint inside a hot loop can therefore make the program much slower, and this delay is measurable by timing checks (section 12.8).

11.5.4 Memory breakpoints

Memory breakpoints can be software or hardware-based:

- **Hardware:** up to 4 bytes (dword size) per registered breakpoint.
- **Software:** they change memory protection attributes (not 100% reliable).
- They are very useful when dealing, for example, with **packed malware**.

The course notes add two details. “Up to 4 bytes” is the limit in 32-bit code; in x64 long mode, a debug register can also watch 8 bytes. Software memory breakpoints work on whole pages, because that is the granularity of memory protection, so the debugger must filter out the accesses to the rest of the page.

Packing is the typical use. Unpacking stubs first write the code into memory and then jump to it. A breakpoint **on write** finds the code that unpacks, and one **on execute** in the same region stops just as the unpacked code starts. A write

breakpoint on a variable also answers the question “who changes this value?”, which is hard to answer by reading the code.

> **Do not confuse**
>
> A **software** breakpoint changes a byte of code and has no fixed limit. A **hardware** breakpoint changes only debug registers and is limited to four. A **conditional** breakpoint describes when to stop, and is slow because the debugger takes control at every hit.

> **From the lecture (3 November 2026, 40:50–43:05)**
>
> Why software memory breakpoints are slow and unreliable: you cannot mark a single address, so to watch address `1008` the debugger makes the whole page starting at `1000` inaccessible. Any access to that page, such as `1004`, also raises an exception. So when you can, use a hardware breakpoint for memory.

11.6 Exceptions

Exceptions are a mechanism for supporting alternative control flow, for example to recover from errors. Section 9.7.5 describes how handlers are registered on Windows.

- From the perspective of the CPU, they are an **interrupt**, just like the `int` instruction.
- **Debuggers use them to regain control** of the execution: for instruction and memory breakpoints, and for single-stepping when the **trap flag** `TF` in `EFLAGS` is set.

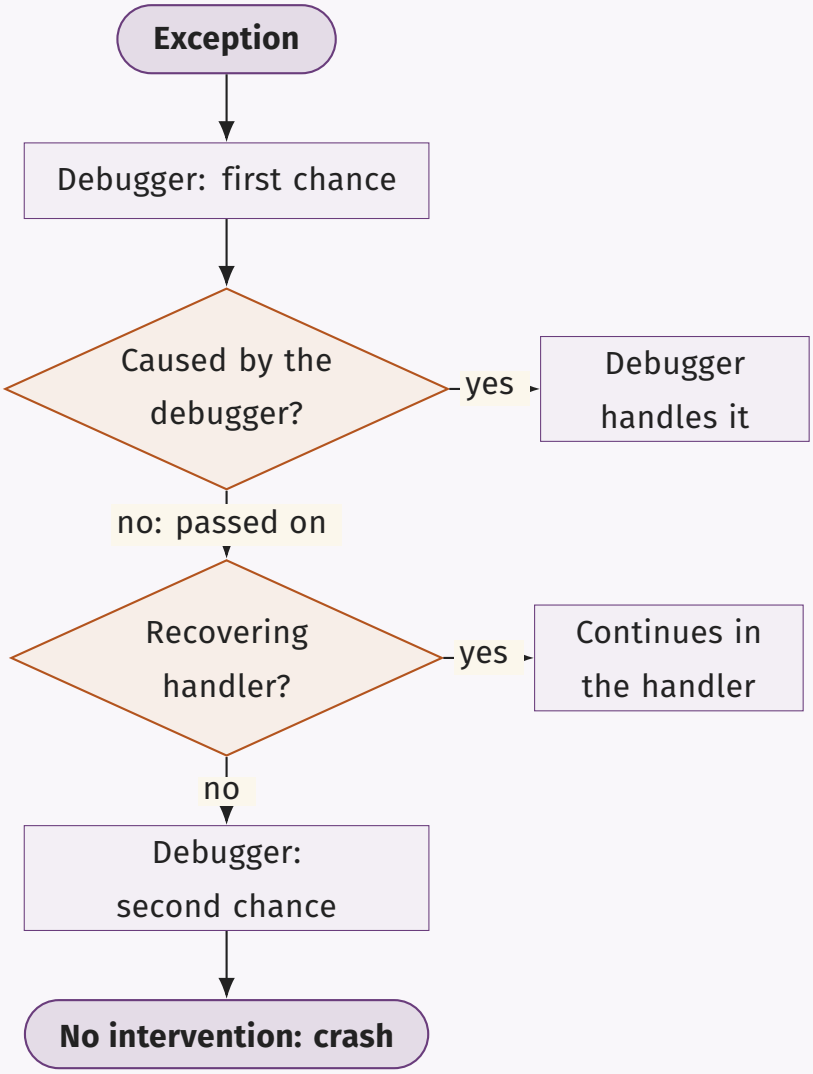
To handle an exception, a **handler must have been registered**. Benign programs use handlers for error situations, such as division by zero. Malware authors use them to hinder analysis. For instance, if the debugger catches an exception meant for the malware, it can give away its presence.

Exceptions are passed to the debugger as they occur, and it can catch each one

up to two times:

- Some APIs behave differently when executing under a debugger, raising exceptions to inform the user.
- **First-chance** exceptions can be passed to the program when they are unrelated to debugging, for example a floating-point or division-by-zero error.
- When the program cannot recover from the exception, for example because there is no corresponding registered handler, the debugger receives it again.
- **Second-chance** exceptions require intervention, or the program will crash.
- Malware often uses exceptions as an **anti-debug technique**.

Flowchart: Who receives an exception



The debugger sees every exception first. If it caused the exception, as with a breakpoint or a single step, it handles it. Otherwise the exception goes to the program. A registered handler that recovers lets execution continue there. If there is none, the debugger receives the exception a second time, and without intervention the program crashes.

From the book

First-chance exceptions can often be ignored, since the program may be designed to handle them. Second-chance ones cannot: in malware analysis they may reveal bugs, but it is “more likely that the malware doesn’t like the environment” (PMA, PDF pp. 254–255).

So a second-chance exception does not necessarily mean a broken sample: it

can be a sign of an environment check. To debug a sample that deliberately raises exceptions, pass the first-chance ones to the program, otherwise the logic in its handler never runs.

> **From the lecture (3 November 2026, 43:17–49:32)**
>
> From the perspective of the CPU, even `INT 3` is an exception. A malware author registers a handler and raises an exception: without a debugger, the handler runs and everything looks normal. With a debugger, the exception is first shown to you, and as a human you tend to accept inspecting it.
>
> The suggestion of the professor is to pass it to the program on the first attempt. If there is no handler, the program does not crash fatally, because you get a second chance. At that point, either fix the cause from the debugger or the program terminates. This does not apply to breakpoints and single-stepping. Exceptions may also be a technique to detect the debugger, as with `CloseHandle` (section 12.12.2), which is a case where you should not pass the exception.

11.7 Debugging and ASLR

With **ASLR** (Address Space Layout Randomization), changing addresses can slow down debugging. For programs compiled with `/DYNAMICBASE`, Windows randomises the load base address of the module. The addresses in the disassembly will then likely **not match** what you see in a debugger, and comments and notes taken statically no longer line up with the running program.

There are two ways to deal with this.

Rebasing in IDA. IDA offers manual loading for a PE and, better, a **rebase option for the current session** (Edit → Segments → Rebase Program).

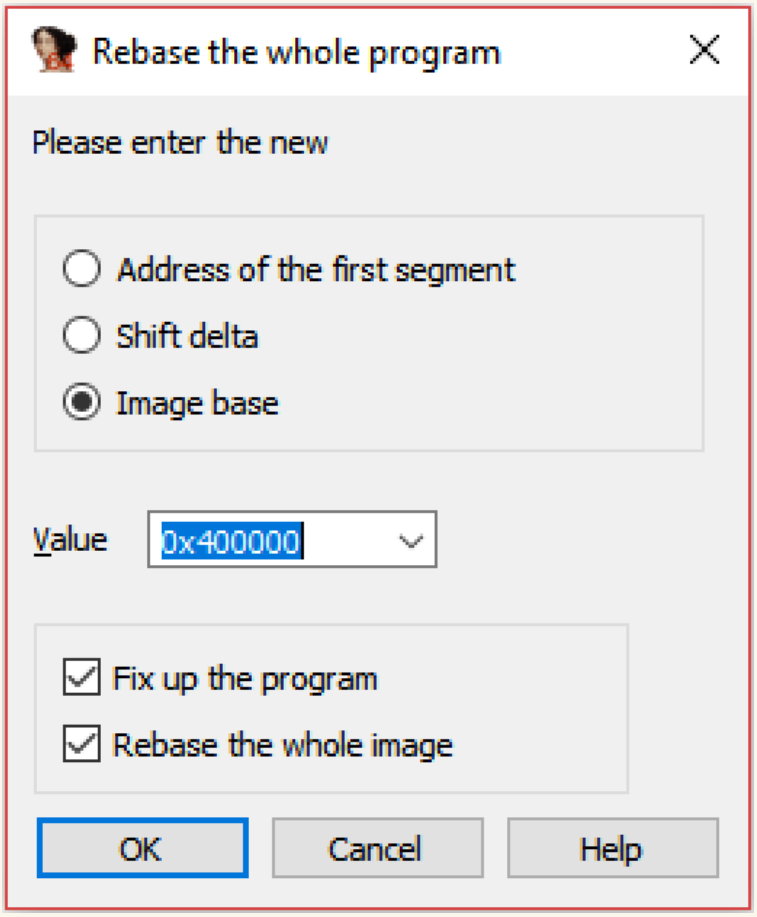


Figure 11.2: The Rebase Program dialog of IDA (slides, Advanced dynamic analysis).

The dialog lets you set the new value as the address of the first segment, as a shift delta or as the image base (here `0x400000`), with the options Fix up the program and Rebase the whole image. The address that matters is obtained as follows.

$$\text{runtime address} = \text{disassembly address} - \text{image base} + \text{runtime base}$$

Step by step: Example with image base `0x400000` and runtime base `0xD40000`

1. Start from the static address: `0x401390`.
2. Subtract the image base: `0x401390` − `0x400000` = `0x1390`.
3. Add the runtime base: `0xD40000` + `0x1390` = `0xD41390`.

The offset from the base never changes.

Editing the PE header. Alternatively, modify the PE header using CFF Explorer (OptionalHeader → DllCharacteristics → DLL can move), which clears the request for randomisation. Note that **some Windows 10+ settings may override it**.

> **From the lecture (3 November 2026, 52:13–54:17)**
>
> When you start the program from the debugger, IDA rebases for you and you get the addresses you expect. Rebasing by hand is needed in other cases, such as attaching to a running process. About the flag, its name is a leftover: ASLR was initially meant for libraries. Clearing it can be overridden on recent Windows versions, especially for a program that was already compiled for ASLR and so tolerates being moved.

11.8 Popular debuggers

Many debuggers exist. Some are particularly well suited for malware analysis and reverse engineering:

- WinDbg;
- IDA Pro and IDA Freeware;
- OllyDbg, x64dbg;
- Immunity Debugger;
- radare2 with plugins.

> **From the lecture (3 November 2026, 54:21–56:46)**
>
> The professor suggests IDA because one tool covers both static and dynamic analysis, but has no strong preference and welcomes other debuggers. For packers the professor personally prefers x64dbg, which integrates naturally with Scylla. For the Windows kernel WinDbg is great, though textual. When the course started, a debugger required IDA Pro. The Pro version also has IDAPython, a strong scripting engine, while the free

> one has a more limited scripting option.

11.9 The debugger of IDA

Recent versions of IDA Freeware include a debugger. It lacks integration with third-party tools like Scylla and some advanced scripting features, so you may still consider using x64dbg sometimes.

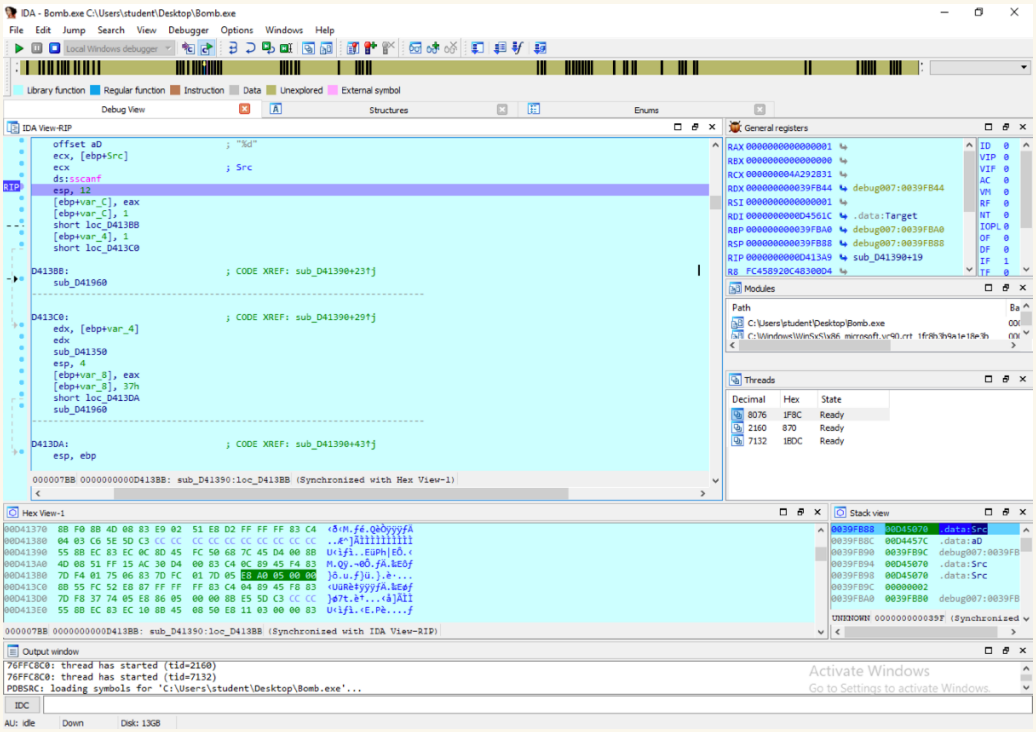


Figure 11.3: The IDA debugger running on `Bomb.exe` (slides, Advanced dynamic analysis).

Its advantage over a standalone debugger is that it keeps the analysis of IDA, so the names and comments from static work remain available at runtime.

11.9.1 Main interface

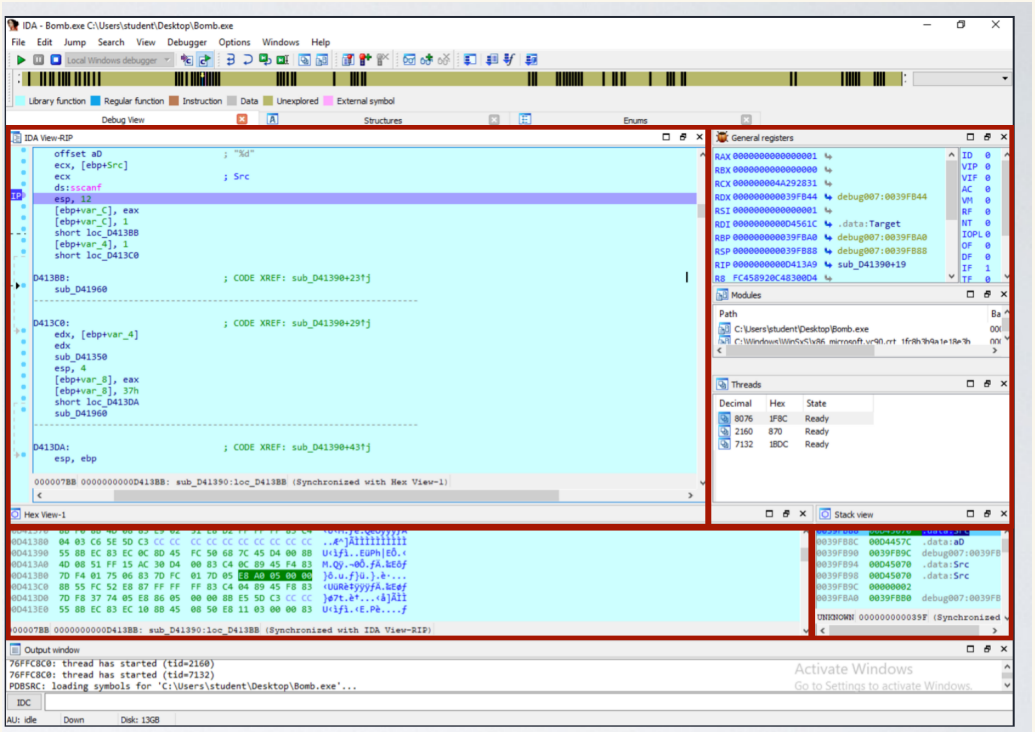


Figure 11.4: The main interface, with the four quadrants highlighted in red (slides, Advanced dynamic analysis).

There are four quadrants: the **disassembler** window, the **registers** window, the **stack** window and the **memory dumps** window.

> **From the lecture (3 November 2026, 56:46–58:30)**
>
> The background turns light blue to show you are in the debugger. The current instruction is highlighted, and the registers box also shows flags such as TF, OF and ZF. The views for threads and modules show what is running and which libraries are loaded.

11.9.2 Memory map

Layout information can be very valuable in the analysis.

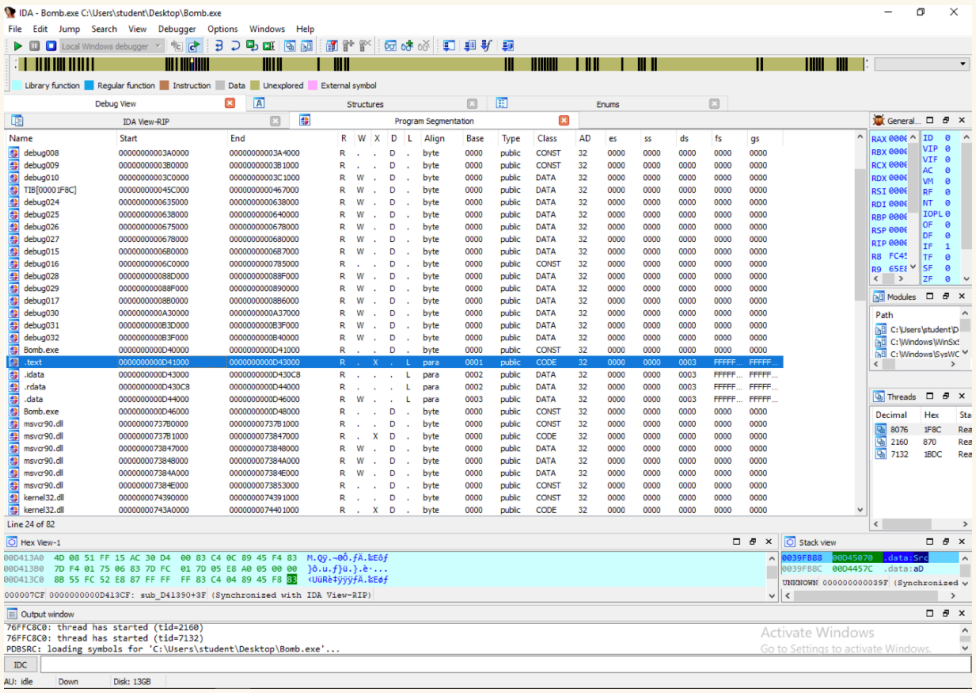


Figure 11.5: The Program Segmentation pane, with the R, W and X flags of each region (slides, Advanced dynamic analysis).

- **Writable and executable** regions may hint at packing or self-modifying code, since normal code is not writable and normal data is not executable.
- A **changed protection** for a region may suggest the same.
- The pane lists the **load address for DLLs**.
- Clicking on a base address updates the disassembler view (for code) or the memory dump view (for data).

Compare the map at different moments: a new RWX region appearing during execution is a good place for a memory breakpoint.

> **From the lecture (3 November 2026, 58:40–60:30)**
>
> Why RWX stands out: modern systems apply write-or-execute policies, so sections both writable and executable are not normal. A region that was read-write and later becomes read-execute formally complies with the policy, but the change of protection suggests packing, self-modifying code or custom loading of a module. If you suspect a buffer contains code, go to it: an executable shows the letters MZ.

11.9.3 Panes

IDA offers several panes besides CPU and Memory Map: **Graph**, **Breakpoints**, **Call stack**, **SEH** (exception handling), **Symbols**, **Threads**, and more.

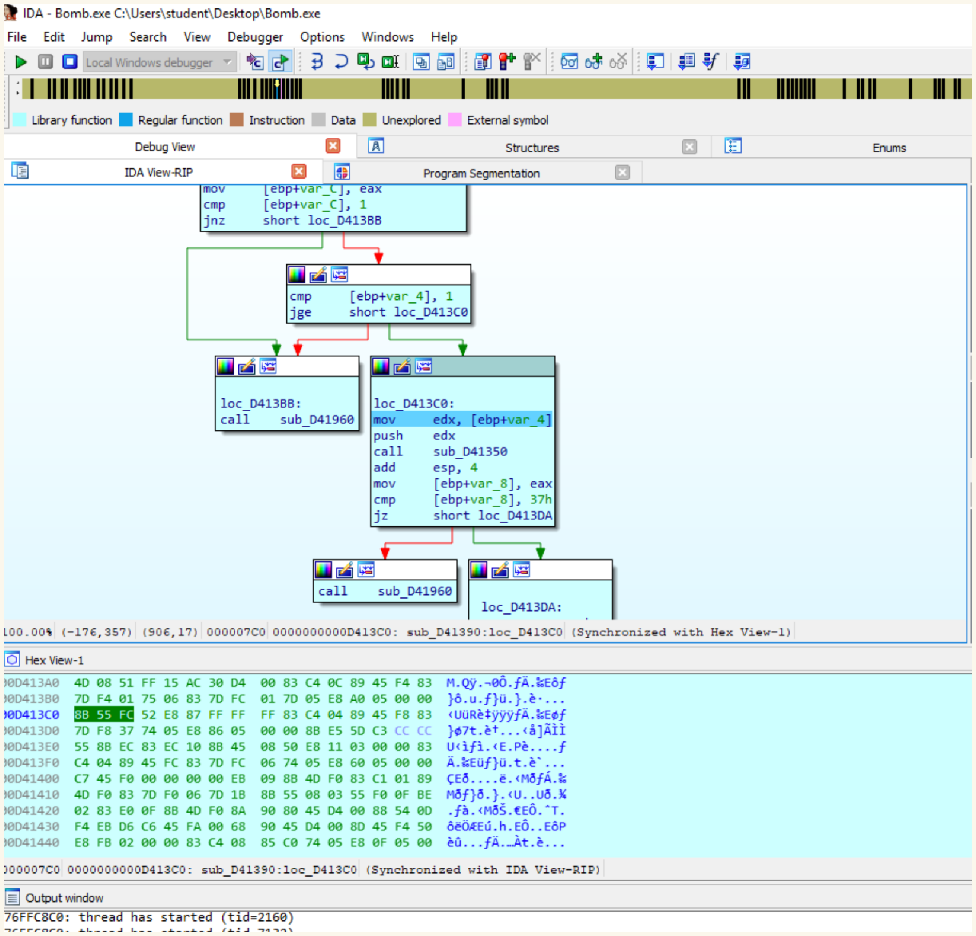


Figure 11.6: The graph pane during debugging (slides, Advanced dynamic analysis).

By right-clicking or pressing SPACE on an instruction address in the Disassembler window, you toggle between the classic graph view of IDA and the flat disassembly one. The graph makes the branches of the current function easy to follow, while the flat view shows the code in address order.

> **From the lecture (3 November 2026, 61:04–61:58)**
>
> In the graph during debugging, IDA makes the arrow of the branch that will be taken blink, based on the current state of the program.

The **Call Stack** pane shows the sequence of routine calls currently active on the

stack. The **Threads** pane lets the analyst control the state of each thread.

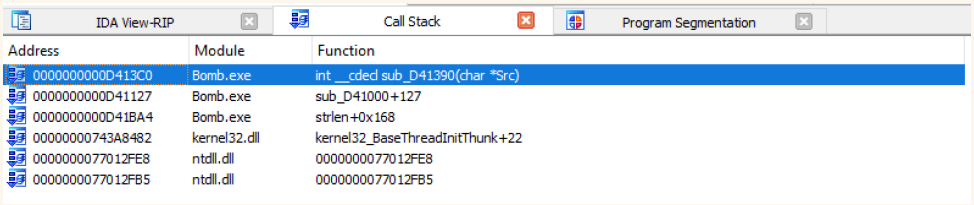


Figure 11.7: The Call Stack pane (slides, Advanced dynamic analysis).

The stack is read from the current function (`sub_D41390`, at the top) back through its callers (`sub_D41000+127`, and so on) down to `BaseThreadInitThunk` in `kernel32` and to `ntdll`. It answers the question "how did execution get here?", and it relies on the stack frames of section 4.4. The addresses are in the `0xD4xxxx` range, an effect of ASLR.

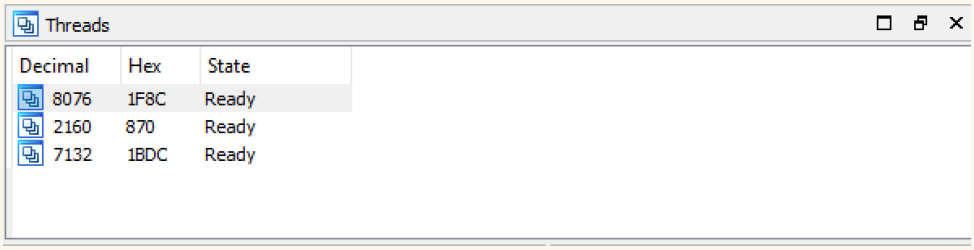


Figure 11.8: The Threads pane, with three threads in the Ready state (slides, Advanced dynamic analysis).

Suspending the other threads is useful to focus on one, given that the debugger follows one thread at a time (section 3.6.1).

11.9.4 Managing stepping

IDA gives fine-grained control of the execution. You can **run** until a breakpoint is hit, **step into** and **step over** for single-step, **execute till return**, and **restart** the execution.

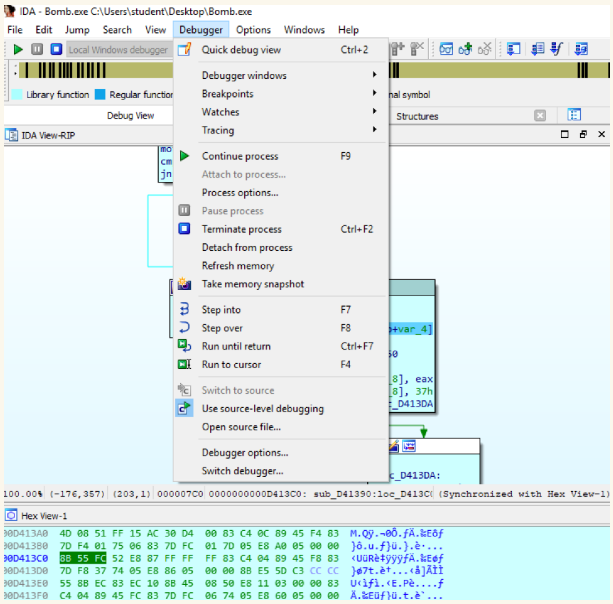


Figure 11.9: The Debugger menu of IDA (slides, Advanced dynamic analysis).

Command	Shortcut
Continue process	F9
Terminate process	Ctrl+F2
Step into	F7
Step over	F8
Run until return	Ctrl+F7
Run to cursor	F4

> **From the lecture (3 November 2026, 62:52–64:30)**
>
> *Detach* releases the debugger without ending the process. The debugger options let you set the command-line arguments of the program and stop automatically at events such as thread creation or library loading.

11.9.5 Controlling breakpoints

Instruction breakpoints are intuitive:

- toggle software breakpoints on and off with F2;

- hardware assistance can be enabled later;
- breakpoints can be controlled from the pane, for example to enable, disable or remove all;
- there is a **hit count** for each breakpoint;
- memory breakpoints are similar.

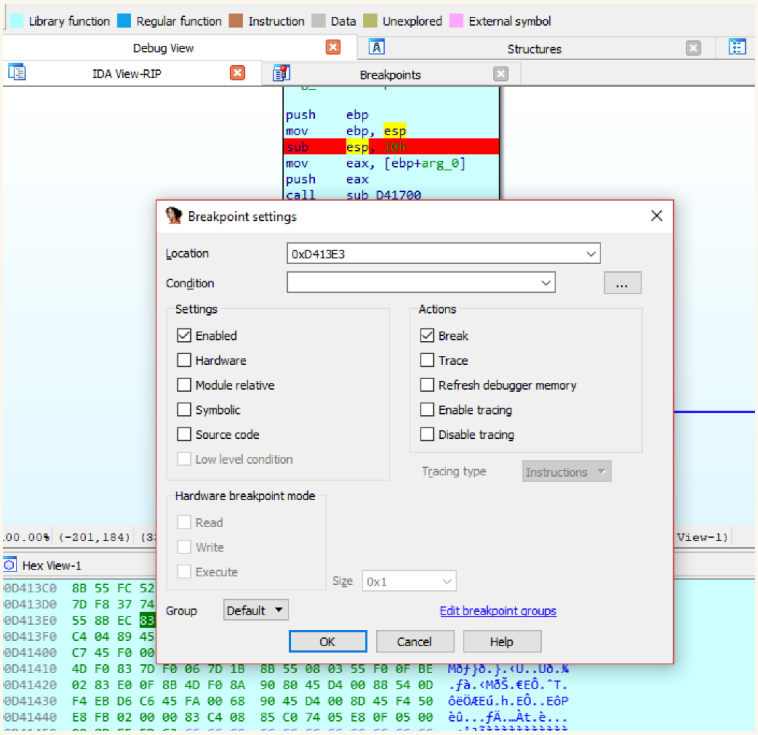


Figure 11.10: Breakpoint settings for location `0xD413E3` (slides, Advanced dynamic analysis).

The settings dialog contains everything seen in theory: a **Condition** field for conditional breakpoints, the **Hardware** checkbox, and the **Read/Write/Execute** mode with a **Size** for hardware breakpoints, that is, a memory breakpoint. The Actions section also lets you trace without breaking, which is lighter than stopping by hand each time.

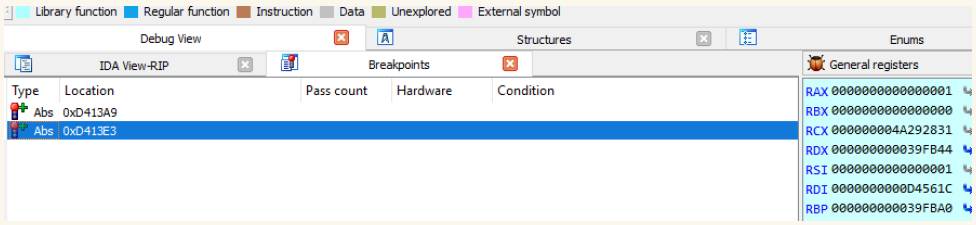


Figure 11.11: The Breakpoints pane, with columns for type, location, pass count, hardware and condition (slides, Advanced dynamic analysis).

> **To verify**
>
> The slide says “hit count”, while the column in the screenshot is labelled **Pass count**.

11.9.6 Modifying registers

You can edit general-purpose registers, such as ECX, or change the instruction pointer.

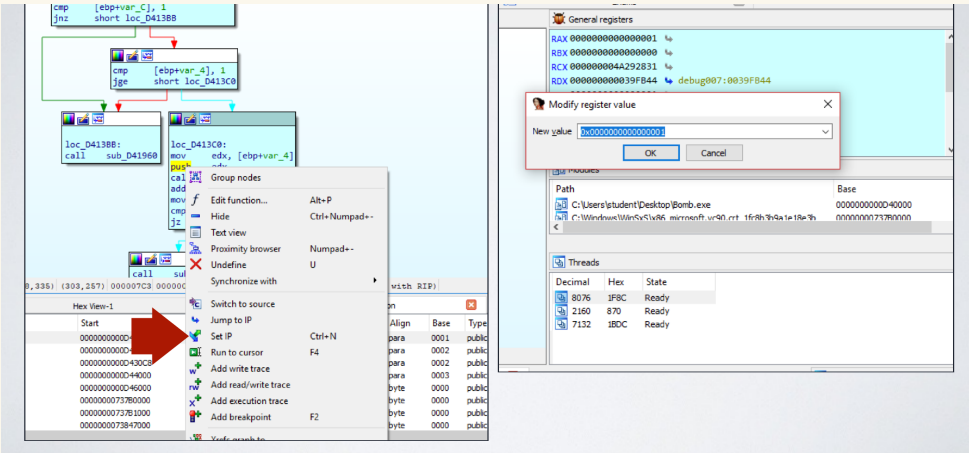


Figure 11.12: Left: the context menu in the graph view, with the arrow on Set IP (`Ctrl+N`). Right: the Modify register value dialog (slides, Advanced dynamic analysis).

Set IP moves execution to the selected instruction. These are powerful tools. For example, flipping EAX after a check, or setting `EIP` past it, skips an environment test without patching anything. But Set IP does not run the skipped instructions, so their effects on the stack and on registers are lost.

> **From the lecture (3 November 2026, 65:47–66:46)**
>
> After moving the instruction pointer, you may need to fix the state, by setting some values or moving the stack pointer, because the program does not expect to be there.

11.9.7 Modifying memory

You can alter memory easily. You can follow an address on the stack first, for example to dereference a pointer, or go directly to the dump, for example for ASCII data. Then you modify the corresponding data. Experience will tell you which to use every time.

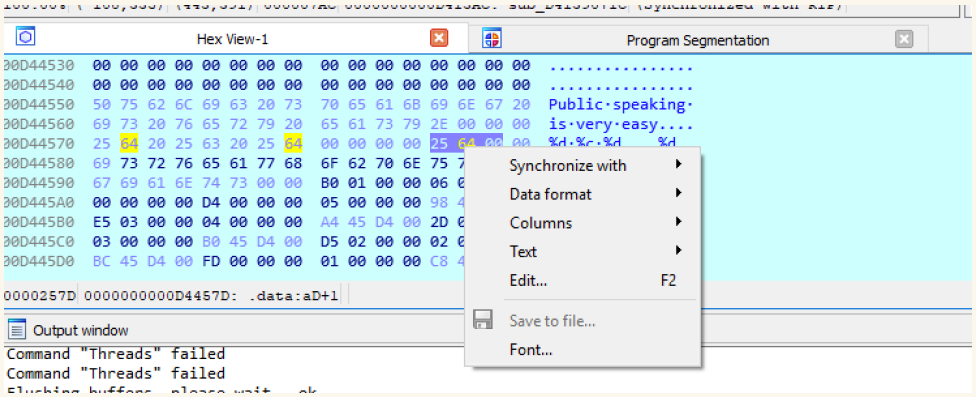


Figure 11.13: The Hex View context menu with Edit (F2), on the bytes of the string "Public speaking is very easy." (slides, Advanced dynamic analysis).

Editing a string like this one changes what the program compares or sends without touching its code.

> **From the lecture (3 November 2026, 66:46–67:40)**
>
> Using `Public speaking is very easy.` from Lab 04 as an example (section 14.1.4), the string the program expects can be changed in global memory, so that a different input is accepted.

11.9.8 Modifying code

You can also modify code, provided that there is **enough space to host the desired new instruction(s)**.

- Changes by default **do not survive restarts**. But they can be imported, exported or made permanent using patches.
- Beware of possible **integrity checks**: the same checksums that find software breakpoints.

- Ensure **stack pointer validity** with respect to the frame layout when deleting a function call.

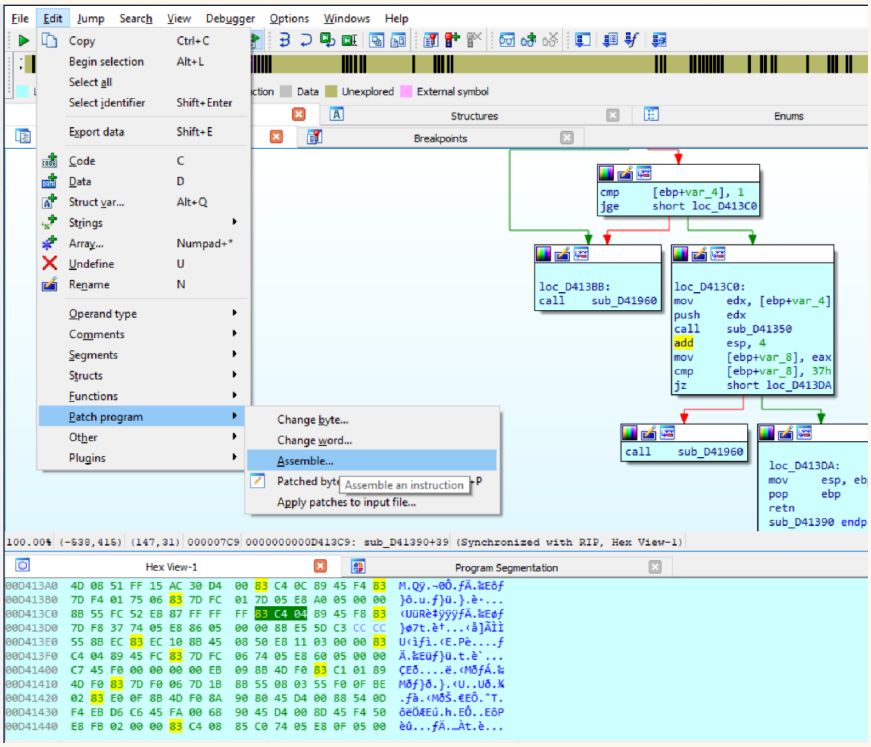


Figure 11.14: The patching commands of IDA (slides, Advanced dynamic analysis).

> **Correction**
>
> The slide says “File->Patches”, but the screenshot shows the commands under **Edit** → **Patch program**: Change byte, Change word, Assemble, Patched bytes and Apply patches to input file. Look for them there.

About the stack. Suppose you delete a `call` to a function taking arguments. The `push` instructions before it remain. In `cdecl` the `add esp, N` of the caller remains too, while in `stdcall` the callee cleaned up, so removing the call leaves the arguments on the stack (section 4.5.1).

You also have to remove the `push` instructions and fix the balance accordingly, otherwise `esp` no longer matches what the rest of the function expects. Any leftover space can be filled with `nop` (`0x90`).

> **From the lecture (3 November 2026, 67:40–69:34)**
>
> **A concrete patch.** A sequence verifies through a checksum that there are no

> software breakpoints, but you need more than 4 breakpoints. So flip the conditional instruction that compares the checksum. In other cases you can replace a whole block, if the bytes are enough. Be careful with any instruction doing add or sub on ESP. There can always be integrity checks on the code. IDA keeps a list of patches that you can export, revert or write back to disk.

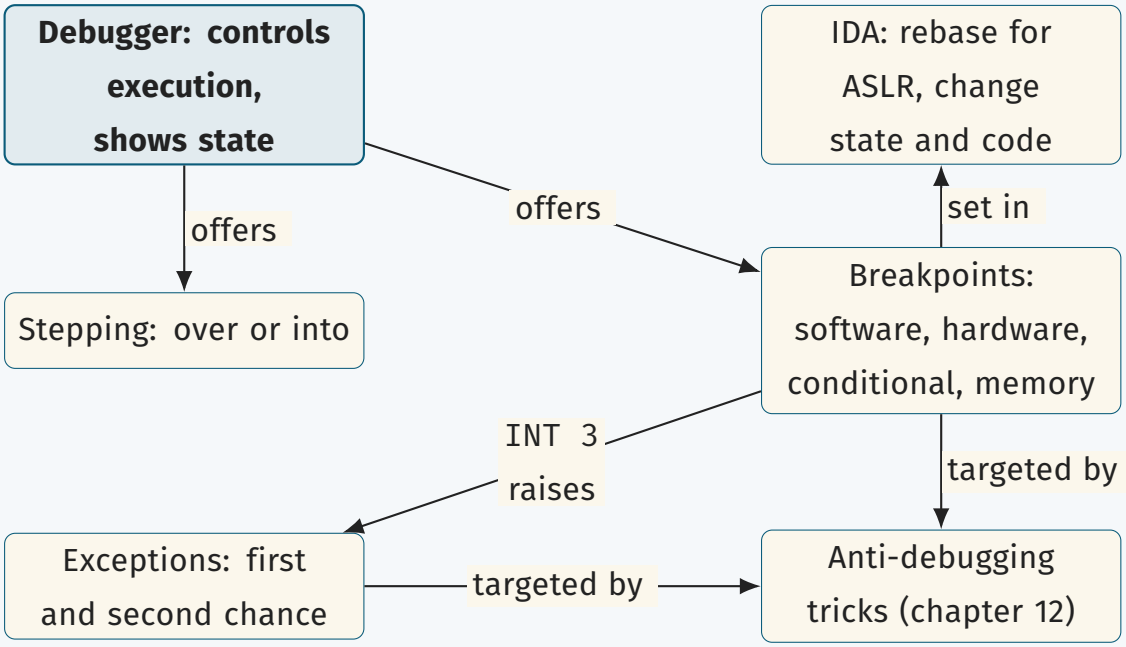
Section 14.2 uses this debugger on real binaries: stepping, watching flags, patching memory and forcing branches.

11.10 Common IDA shortcuts

Learn shortcuts to use IDA more efficiently.

Shortcut	Action
`Escape`	Go back to the previous position in the current view
`Ctrl+Enter`	Go forward to the next position in the current view
`Space`	Switch between text and graph mode
`G`	Jump to an address in the current view
`F2`	In disassembly view (static analysis): toggle a breakpoint at the current instruction. During debugging: enter edit mode to change the selected value, and press F2 again to apply the changes
`X`	Show cross-references to the selected item, such as a function
`N`	Rename the selected reference

Concept map: how the ideas connect



A debugger offers two controls: stepping and breakpoints. A software breakpoint works by raising an exception with INT 3, which reaches the debugger as a first-chance or second-chance exception. You set breakpoints in IDA, which also rebases the addresses under ASLR and can change registers, memory and code. Anti-debugging tricks target these same mechanisms.

In short

- A debugger fills the gaps of the basic and static techniques: it sees the unpacked code and lets you reach triggers.
- Use single-stepping selectively. Step over what you understand, step into what you do not, and stop where it matters with breakpoints.
- Software breakpoints write `0xCC` and are unlimited but detectable. Hardware ones use `DR0`–`DR3` and `DR7`, are limited to 4 and alter nothing.
- Conditional breakpoints are slow, and memory breakpoints are the tool for packed samples.
- Exceptions reach the debugger up to twice. Pass the first-chance ones to the program, and read second-chance ones as a possible sign of an

environment check.

- With ASLR, rebase in IDA: the offset from the base stays the same.
- In IDA you can change registers, `EIP`, memory and code, but mind integrity checks and the stack balance.

Review: try to answer without looking

1. Why can a software breakpoint be found by a checksum of `.text`, while a hardware breakpoint cannot?
2. Which breakpoint would you set to find the code that writes the unpacked payload?
3. With image base `0x400000` and runtime base `0xD40000`, what is the runtime address of `0x401390`?
4. Why can removing a `call` to a `stdcall` function break the stack?

Answers, with the section where to find them

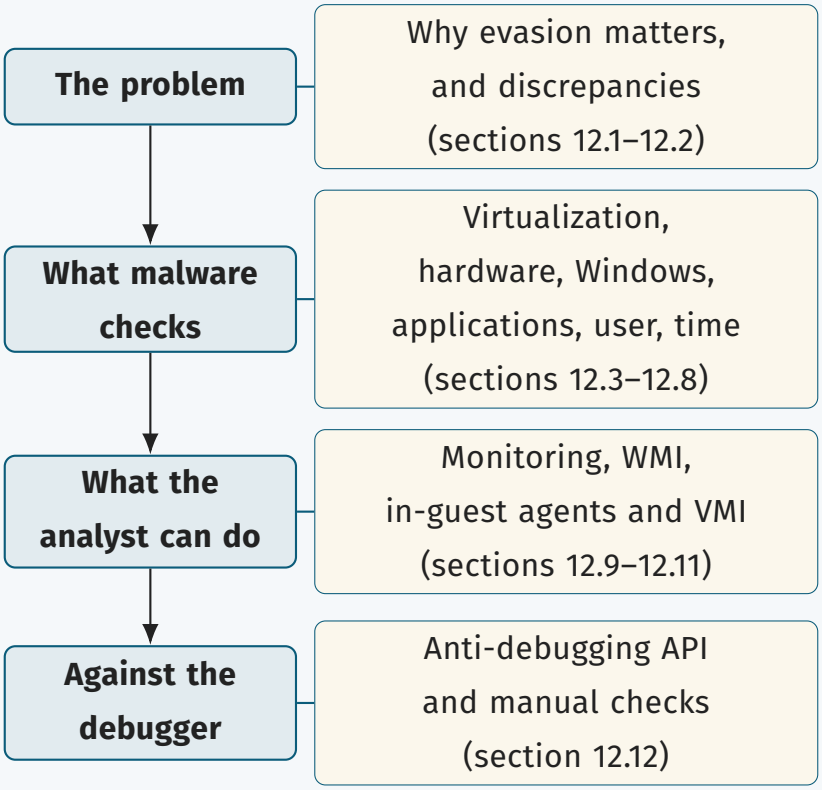
1. The software breakpoint overwrites a code byte with `0xCC`. The hardware one only writes debug registers (sections 11.5.1 and 11.5.2).
2. A memory breakpoint on write over the target region (section 11.5.4).
3. `0xD41390` (section 11.7).
4. The callee would have removed the arguments. Without the call, they stay on the stack (section 11.9.8).

Chapter 12

Malware evasion and anti-debugging

> **Before you start**
>
> **Central question:** how does malware notice that it is being analysed, and what can the analyst do about it?
>
> **Prerequisites:** sandboxes and virtual machines (chapter 7), x86 instructions and the PEB (chapter 3), breakpoints and exceptions (chapter 11).
>
> **Where it leads:** this chapter closes the theoretical path. Evasion is the reason why every result of the previous chapters needs its limits stated.

Chapter structure: what comes, and in which order



The chapter first states the problem. It then lists what malware checks, one family at a time. It continues with the analyst's options for monitoring, and ends with checks aimed at the debugger.

Dynamic analysis, running a sample inside a sandbox, answers the flood of new samples that static analysis cannot handle. This chapter shows the other side of the coin: **malware that notices it is being analysed and hides its behaviour**.

We go through the discrepancies that distinguish an analysis system from a real victim. We then look at the defensive answers: monitoring evasive code, in-guest agents and **virtual machine introspection**. Finally we study **anti-debugging**.

In plain words (explanation added by this book)

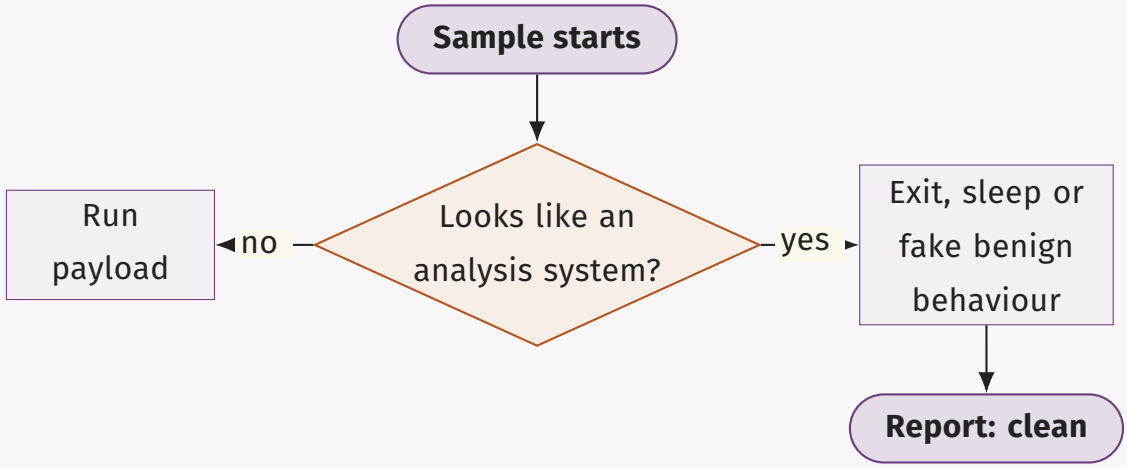
Evasion means that the sample behaves differently when it suspects that someone is watching. It is a test on the environment, followed by a change of behaviour.

12.1 Why evasion matters

- **Staying undetected is vital for malware.** Once a sample is recognised as malicious, antivirus signatures, blocklists and analysts shut it down.
- **Dynamic analysis is necessary** because new samples surface every day. Static analyses have limited capabilities (section 6.4). The externally observable behaviour (files written, registry keys, network traffic, processes created) is what allows threat identification and classification.
- **Only a few samples**, for instance new strains, undergo manual analysis. The vast majority is only seen by automated sandboxes.

The consequence is the point of the whole topic: **once the malware detects an execution or environmental artifact, it can deceive even the most sophisticated analysis system.** It can simply exit, sleep or behave as a harmless program, and the sandbox report will say “clean”.

Flowchart: The basic evasion decision



The sample tests the environment before doing anything. On a real victim it runs the payload. On something that looks like an analysis system, through a VM, a debugger, a fresh install or analysis tools, it exits, sleeps or fakes benign behaviour, and the report says clean.

Remember

A clean sandbox report is weak evidence. A sample that does nothing in a sandbox has **not** been shown to be benign: its trigger, or its victim, may

simply be absent (section 2.5.3).

From the lecture (20 October 2026, 02:00–06:30)

Static analysis can only go so far: flagging a sample on hints alone risks blocking legitimate software that uses obfuscation to protect intellectual property. *Externally observable* means looking at the interactions of the sample with the system (API calls, files, registry keys), not at how its functions call each other. An **artifact** is whatever gives the analysis system away, and the professor compares it to the Heisenberg effect: observing something changes it. Not all malware looks for artifacts, but most does.

12.2 Discrepancies

What makes an analysis system different from a victim machine? An analysis environment is built to be **controlled, reproducible and instrumented**, and each of these properties leaves a trace. The slides group the differences in six families, which are also the structure of the next sections.

Family	Typical tell
Virtualization artifacts	Special instructions, CPUID data, firmware tables, I/O ports
Hardware characteristics	Few cores, tiny disk, vendor MAC, missing audio device, no sensors
Windows installation	Timezone, language, uptime, registry keys of guest additions
Applications	Analysis tools, or missing “normal” software
User artifacts	A “fresh” machine with no history
Time overheads and memory footprint	Slower instructions, accelerated sleeps, injected agent

12.3 Virtualization artifacts

A **red pill** is an instruction sequence that detects whether the code is running in a virtual machine. The name comes from the film The Matrix: take it and you see reality. Where can the evidence come from?

- **Instructions that take longer to execute**, mainly `cpuid` (see below).
- **Instruction errata:** incomplete emulation of some instructions by emulators such as QEMU, which then behave differently from real silicon.
- **Information leaked by the machine:** CPU identification data from `cpuid`; system firmware tables (SMBIOS strings, ACPI tables); contents and **position in memory of specific structures**, for example the Interrupt Descriptor Table; the **micro-architectural state** of the machine (caches, branch predictors and so on).
- **I/O ports:** some hypervisors expose a backdoor channel through the `in` instruction.

> **From the book**
>
> PMA explains why this works. The x86 has instructions such as `sidt`, `sgdt`, `sldt`, `smsw` and `str` that read hardware state **without being privileged** and without trapping. A VMM would have to translate every instruction to virtualize them, with a huge performance cost, so it lets them run natively. The registers they read therefore contain values valid for the **host**, which diverge from what a guest on bare metal would see.
>
> The original Red Pill ran `sidt` and compared the fifth byte of the returned IDT base with `0xFF`, the VMware signature. It works reliably only on single-processor machines. No Pill does the same with `sldt`: the LDT is zero on Windows hosts but non-zero in VMware. The “Anti-VM x86 instructions” list in the book is `sidt`, `sgdt`, `sldt`, `smsw`, `str`, `in` (with VX) and `cpuid` (PMA, anti-VM chapter, PDF pp. 515–520).

From the lecture (20 October 2026, 11:00–16:10)

- **Instruction errata today:** running the course VM through an emulator on an ARM MacBook can expose missing features. When the professor last checked, hardware breakpoints on memory accesses were not emulated.
- **Corner cases:** *A Fistful of Red Pills* looked for instructions that emulators get slightly wrong. These work fine for most programs, but malware targets the corner cases, much like vulnerability discovery.
- **Micro-architectural state:** assembly has no notion of caches or TLBs, yet monitoring code in the VM monitor changes how they are used. A side-channel attack can reveal it. This is subtle and rarely demonstrated in malware, and the group of the professor has worked on it.

12.3.1 The cpuid instruction

`cpuid` retrieves **processor identification and feature information**.

- It reads from EAX the requested record type (the leaf) and writes an information record to `{EAX, EBX, ECX, EDX}`.
- For `EAX = 0x1` it writes feature information in ECX. The **31st bit**, the **hypervisor bit**, equal to 1 reveals the presence of a hypervisor.
- For `EAX = 0x40000000` it writes the **hypervisor brand** in `{EBX, ECX, EDX}` as ASCII bytes forming 12-byte strings.

Correction

The slide writes the output registers as `{EAX, EBC, ECX, EDX}`. EBC does not exist: the register is **EBX**.

Brand string	Hypervisor
`"KVMKVMKVM\0\0\0"`	KVM
`"Microsoft Hv"`	Microsoft Hyper-V
`"VMwareVMware"`	VMware
`"XenVMMXenVMM"`	Xen
`"prl hyperv  "`	Parallels
`"VboxVboxVbox"`	VirtualBox

This is a polite feature: it is designed so that guests can discover the hypervisor and use paravirtualized services. Malware just reads it for another purpose. Hyper-V is also what Windows itself runs on when VBS (Virtualization-Based Security) is enabled, so “a hypervisor is present” does **not** by itself mean “this is a sandbox”.

The hypervisor bit. The check on the slide sets `EAX = 1` and inspects bit 31 of ECX.

```
size_t eax, ebx, ecx, edx;
__asm__ volatile (
     "cpuid"
     : "=a"(eax), "=b"(ebx), "=c"(ecx), "=d"(edx)
     : "a"(1)
     : "memory" );
printf("%d\n", (int)(ecx >> 31));
```

The GCC extended-asm constraints bind the C variables to the registers. `"a"(1)` puts 1 in EAX before the instruction, and `"=a"`, `"=b"`, `"=c"`, `"=d"` read the results back. `ecx >> 31` isolates the top bit: 1 on a hypervisor, 0 on bare metal and on hypervisors configured to hide it.

> **From the lecture (20 October 2026, 19:34–20:10)**
>
> A sandbox must not leave this bit set: the VM monitor answering `cpuid` has to write 0 there. It requires modifying the hypervisor, but the professor

counts this among the checks that are still fixable.

The hypervisor brand.

```
#include "stdafx.h"
#include <iostream>
#include <string>

using namespace std;


int main()
{
 bool IsUnderVM = false;

 __asm {
 xor     eax, eax
 mov     eax,0x40000000
 cpuid
 cmp ecx,0x4D566572
 jne NopInstr
 cmp edx,0x65726177
 jne NopInstr
 mov IsUnderVM, 0x1
 NopInstr:
 nop
 }
 cout << IsUnderVM;
 return 0;
}
```

Figure 12.1: Detecting VMware through leaf `0x40000000`, from the rayanfam article cited on the slide (slides, Malware evasion).

The screenshot shows a small MASM-style C++ program.

Step by step: Reading the VMware check

1. `xor eax, eax; mov eax, 0x40000000; cpuid` asks for the hypervisor vendor leaf.
2. `cmp ecx, 0x4D566572` and `cmp edx, 0x65726177` compare the returned registers with constants.
3. Because x86 is **little-endian** (section 3.3), the string `"VMwareVMware"` is returned as three dwords: `EBX = "VMwa"` (`0x61774D56`), `ECX = "reVM"` (`0x4D566572`), `EDX = "ware"` (`0x65726177`). The program checks only the last two.
4. If both match, `IsUnderVM` is set to 1 and printed.

Anti-VM checks in real samples look exactly like this: a handful of `cpuid` calls

and `cmp` with magic constants. This makes them easy to spot, because `cpuid` is useless in normal user-mode code.

> **Correction**
>
> In the lecture (21:30) the professor reasons live about the constants and concludes they check the *first* 8 characters of the string. They correspond to `ECX = "reVM"` and `EDX = "ware"`, so they are the last 8 of `"VMwareVMware"`. The general point of the professor holds: the string repeats, so checking two registers out of three is enough.

12.3.2 Execution time of cpuid

Under a hypervisor, `cpuid` is a **sensitive instruction that always causes a VM exit**. The CPU stops the guest and gives control to the **virtual machine monitor** (VMM) component of the hypervisor, which emulates the answer and resumes the guest. That detour costs many hundreds or thousands of cycles, compared to some tens on bare metal.

Measuring the latency does not even need to read the brand, so it works if the hypervisor **lies** about the bit or the string. The slide measures the time with `rdtsc` (read time-stamp counter).

```
int elapsed; // gcc -masm=intel
__asm__ volatile (
         "rdtsc ;"
         "mov %0, %%eax ;"
         "mov %%eax, 1;"
         "cpuid ;"
         "rdtsc ; "
         "sub %0, %%eax ;"
         "neg %0 ;"
         : "=r"(elapsed) :
         : "rax", "rbx", "rcx", "rdx");
```

Correction

An earlier version of these notes called this snippet invalid, reading its instructions in AT&T order: `mov %%eax, 1` would write to an immediate, and the first timestamp would not be saved. That reading is wrong. The comment says `-masm=intel`, so the destination comes first, as the lecture specifies below. `mov %0, %%eax` does save the first reading, and `mov %%eax, 1` does write 1 into EAX.

Checked for this book by compiling the snippet with `gcc -masm=intel` on x86-64 Linux: it builds, runs and prints small positive cycle counts. The Al-Khaser version below remains the more careful implementation, because it averages several measurements.

From the lecture (20 October 2026, 22:59–29:40)

Orders of magnitude. On real hardware `cpuid` takes about 80–100 cycles (it can be 60 or 130), which refines the rough "some tens" above. Under a hypervisor it takes about 1000, or at least 500. Time this short is measured in cycles because no ordinary clock is precise enough.

`rdtsc`. It writes the timestamp counter to EAX and EDX. Most code reads only EAX, since the difference between two close readings fits in the lower 32 bits. On CPUs of about the last 10 years, the counter ticks at a fixed pace regardless of frequency scaling.

Reading the snippet. The professor reads it in Intel syntax, as its `-masm=intel` comment says. Both `rdtsc` and `cpuid` overwrite EAX, so `mov %0, %%eax` saves the first reading into a location chosen by the compiler. `mov %%eax, 1` then selects the leaf. After the second `rdtsc`, `sub %0, %%eax` computes start minus end, and `neg %0` flips the negative result.

```
/*
CPUID is an instruction which cauz a VM Exit to the VMM,
this little overhead can show the presence of a hypervisor
*/

BOOL rdtsc_diff_vmexit()
{
    ULONGLONG tsc1 = 0;
    ULONGLONG tsc2 = 0;
    ULONGLONG avg = 0;
    INT cpuInfo[4] = {};

    // Try this 10 times in case of small fluctuations
    for (INT i = 0; i < 10; i++)
    {
        tsc1 = __rdtsc();
        __cpuid(cpuInfo, 0);
        tsc2 = __rdtsc();

        // Get the delta of the two RDTSC
        avg += (tsc2 - tsc1);
    }

    // We repeated the process 10 times so we make sure our check is as much reliable as we can
    avg = avg / 10;
    return (avg < 1000 && avg > 0) ? FALSE : TRUE;
}
```

Figure 12.2: Al-Khaser, function `rdtsc_diff_vmexit` (slides, Malware evasion).

Al-Khaser implements the idea carefully. The loop runs 10 times "in case of small fluctuations". Each time it reads `rdtsc`, calls `__cpuid(cpuInfo, 0)`, reads `rdtsc` again and accumulates `tsc2 - tsc1`. Finally `avg /= 10`, and the verdict is `(avg < 1000 && avg > 0) ? FALSE : TRUE`.

So an average **below 1000 cycles means bare metal**, and above it the function reports a VM. The `avg > 0` test excludes a pathological zero, for example a virtualized TSC that does not advance. Averaging is needed because a single measurement is noisy, due to interrupts, scheduling and caches.

The course notes explain why this is a hard check to defeat. The hypervisor cannot make the VM exit disappear without giving up its own control. Cheating on the TSC, by offsetting it so that the time spent in the VMM is subtracted, is itself detectable by cross-checking other time sources (sections 12.8 and 12.11.1).

12.4 Hardware characteristics

Some hardware details reveal a VM or a sandbox:

- **CPU model and number of cores**, since a sandbox often gives 1–2 cores;
- **number of network adapters** and the **MAC address family**;
- **disk size and serial number**, since a 40 GB virtual disk is unusual on a modern laptop;
- **peripherals**, for example audio capabilities;
- **sensor measurements**: fan speed and temperature, which a VM does not have.

Remember

Not every VM is a sandbox. These features are clearly relevant for evasive malware, but the use of virtualization may **indicate a sandbox and also be legitimate**: cloud servers, developers, VDI (Virtual Desktop Infrastructure).

Moreover, **targeted malware** may inspect the same properties to check **whether it has reached the intended victim**, for instance a specific model of machine in an industrial network. The same check can be an anti-analysis check or a targeting check (section 2.5.3).

From the lecture (20 October 2026, 29:40–34:30)

- **Inconsistencies matter:** a VM reporting a CPU model sold with 12–16 cores but exposing only 2 gives itself away.
- **Sensors:** temperature and fan speed are either missing or fixed, staying the same whether the CPU is under load or idle.
- **A VM is no longer enough:** Windows 11 itself uses some virtualization by default, and many machines run in the cloud. The professor would not stop at detecting a VM: it now only raises suspicion, to be followed by other checks.
- **Targeted malware:** a sample aimed at specific machines can reject a CPU model newer than the ones its victim has. Infecting the wrong machine brings nothing and may lead to detection.

> **From the lecture (20 October 2026, 34:38–36:40, a student's question)**
>
> A student asks whether these values can simply be altered. Yes: that has been the defensive strategy for years. A hypervisor can change the MAC prefix, or the environment can change the result when malware queries it through an API. The professor calls this a *blue pill*, and presented a system named **BluePill** at Black Hat Europe 2019. It covers what is known, but it does not help against what is not.

12.4.1 Example: MAC address

The first three bytes of a MAC address are the vendor identifier, or OUI (Organizationally Unique Identifier). A hypervisor must invent MAC addresses for its virtual network cards and typically uses its own assigned prefix.

Prefix	Hypervisor
`08:00:27`	VirtualBox
`00:05:69`, `00:0C:29`, `00:1C:14`, `00:50:56`	VMware
`00:1C:42`	Parallels
`00:16:3E`	Xen

A malware author only has to compare the first three bytes with this table. The cheap defence is to **change the MAC** of the guest network card to one with the OUI of a real vendor (PMA, anti-VM chapter, PDF p. 512).

12.4.2 Example: speaker check

```
{
    wchar_t* filterName = L"random_name";

    IGraphBuilder *pGraph;
    CoCreateInstance(CLSID_FilterGraph, NULL, CLSCTX_INPROC_SERVER, IID_IGraphBuilder, (void**)&pGraph);
    if (E_POINTER != pGraph->AddFilter(NULL, filterName))
        ExitProcess(-1);

    IBaseFilter* pBaseFilter;
    CoCreateInstance(CLSID_AudioRender, NULL, CLSCTX_INPROC_SERVER, IID_IBaseFilter, (void**)&pBaseFilter);

    // in case of no audio device, CoCreateInstance returns VFW_E_NO_AUDIO_HARDWARE 0x80040256
    // but the result is not checked and it fails later

    pGraph->AddFilter(pBaseFilter, filterName);

    IBaseFilter* pBaseFilter2;
    pGraph->FindFilterByName(filterName, &pBaseFilter2);
    if (NULL == pBaseFilter2)
        ExitProcess(1);

    FILTER_INFO info = { 0 };
    pBaseFilter2->QueryFilterInfo(&info);
    if (0 != wcscmp(info.achName, filterName))
        return;

    IReferenceClock* pClock;
    if (0 != pBaseFilter2->GetSyncSource(&pClock))
        return;

    if (0 != pClock)
        return;

    CLSID clsID;
    pBaseFilter2->GetClassID(&clsID);
    if (clsID.Data1 == 0)
        exit(1);

    if (NULL == pBaseFilter2)
        exit(-1);

    IEnumPins *pEnum = NULL;
    if (0 != pBaseFilter2->EnumPins(&pEnum))
        exit(-1);

    if (0 == pBaseFilter2->AddRef())
        exit(-1);
}
```

Figure 12.3: The COM-based audio check of TeslaCrypt, from the Joe Security blog (slides, Malware evasion).

The **TeslaCrypt** ransomware performs a COM-based **DirectShow** audio check. It looks not only for the presence of an audio device: it also invokes APIs like `AddFilter` to expose **inaccurate emulation** of the device by a malware sandbox.

> **Step by step: Reading the code**
>
> 1. `CoCreateInstance(CLSID_FilterGraph ...)` creates a DirectShow graph builder, followed by `AddFilter(NULL, L"random_name")`.
> 2. `CoCreateInstance(CLSID_AudioRender ...)` creates the **audio**

renderer. The comment says that with no audio device `CoCreateInstance` returns `VFW_E_NO_AUDIO_HARDWARE` (`0x80040256`), but the result is not checked and "it fails later". This already catches machines without a sound card.

3. `pGraph->AddFilter(pBaseFilter, filterName)` adds the renderer. Then `FindFilterByName` retrieves it and `QueryFilterInfo` compares the **filter name** with the one that was given.

4. `GetSyncSource` must return a valid **reference clock** and `GetClassID` must return a non-zero CLSID. `EnumPins` must yield pins and `AddRef` must succeed.

Every one of these steps is trivial on a real driver stack. A sandbox that fakes an audio device by returning a canned success from the first call usually does not implement the whole object model coherently, and the malware calls `ExitProcess` as soon as something is off. The lesson is the fidelity problem: it is not enough to have a device, it must behave like one.

From the lecture (20 October 2026, 36:49–38:40)

Why the idea of TeslaCrypt works: sandboxes run many VMs at once on servers, where a sound card makes little sense, so they emulate one. Telling the sample that a device exists is not enough, because the sample can try to use it, through features more complex than simply playing audio.

Correction

In the lecture DirectShow is described as part of the Direct3D framework. DirectShow is a separate Windows multimedia API, and Direct3D is for 3D graphics.

12.5 Windows installation

Context information distinguishes a real user machine from a template image: timezone, language, **uptime** (a sandbox that was just started has a few minutes

of uptime), install date and product keys (sandboxes often share a key or use evaluation images).

Hypervisors and their guest additions also leave traces: registry entries, processes and drivers. For VMware, PMA lists the processes `VMwareService.exe`, `VMwareTray.exe` and `VMwareUser.exe`, the services of VMware Tools, a registry value `Identifier = "VMware Virtual IDE Hard Drive"`, and the `VMware` strings in mouse and network card driver entries. The cheap fix in the book is to uninstall VMware Tools or run `net stop "VMware Tools Service"` (PMA, PDF pp. 511–514).

From the lecture (20 October 2026, 38:40–42:17)

- **Install date:** a machine installed the day before the sample arrives is suspicious.
- **Uptime:** being infected in the first minute after boot is unlikely for a real user, who would have to log in, open the email and click. An uptime of 10 days is also odd, unless the target is a server.
- **Timezone and language:** banking Trojans have used these to check that they are in the right region.
- **Product key:** some samples check whether the copy of Windows is genuine, since many analysts use copies that are not.
- **Guest additions:** the shared clipboard, folders and smooth graphics that analysts want are exactly what leaves these traces.

12.5.1 Example: VirtualBox and the registry

These are key **values** under `HKEY_LOCAL_MACHINE`.

Path	Value	Content
HARDWARE\DEVICEMAP\Scsi\Scsi Port 0\Scsi Bus 0\Target Id 0\Logical Unit Id 0	Identifier	"VBOX"
HARDWARE\Description\System	SystemBiosVersion	"VBOX"
HARDWARE\Description\System	VideoBiosVersion	"VIRTUALBOX"
HARDWARE\Description\System	SystemBiosDate	"06/23/99"

These are keys whose mere **existence** under HKEY_LOCAL_MACHINE is the signal:

```
HARDWARE\ACPI\DSDT\VBOX__
HARDWARE\ACPI\FADT\VBOX__
HARDWARE\ACPI\RSDT\VBOX__
SOFTWARE\Oracle\VirtualBox Guest Additions
SYSTEM\ControlSet001\Services\VBoxGuest
SYSTEM\ControlSet001\Services\VBoxMouse
SYSTEM\ControlSet001\Services\VBoxService
SYSTEM\ControlSet001\Services\VBoxSF
SYSTEM\ControlSet001\Services\VBoxVideo
```

```
VOID vbox_reg_keys()
{
    /* Array of strings of blacklisted registry keys */
    const TCHAR* szKeys[] = {
        _T("HARDWARE\\ACPI\\DSDT\\VBOX__"),
        _T("HARDWARE\\ACPI\\FADT\\VBOX__"),
        _T("HARDWARE\\ACPI\\RSDT\\VBOX__"),
        _T("SOFTWARE\\Oracle\\VirtualBox Guest Additions"),
        _T("SYSTEM\\ControlSet001\\Services\\VBoxGuest"),
        _T("SYSTEM\\ControlSet001\\Services\\VBoxMouse"),
        _T("SYSTEM\\ControlSet001\\Services\\VBoxService"),
        _T("SYSTEM\\ControlSet001\\Services\\VBoxSF"),
        _T("SYSTEM\\ControlSet001\\Services\\VBoxVideo")
    };

    WORD dwlength = sizeof(szKeys) / sizeof(szKeys[0]);

    /* Check one by one */
    for (int i = 0; i < dwlength; i++)
    {
        TCHAR msg[256] = _T("");
        _stprintf_s(msg, sizeof(msg) / sizeof(TCHAR), _T("Checking reg key %s "), szKeys[i]);
        if (Is_RegKeyExists(HKEY_LOCAL_MACHINE, szKeys[i]))
            print_results(TRUE, msg);
        else
            print_results(FALSE, msg);
    }
}
```

Figure 12.4: Al-Khaser, function `vbox_reg_keys` (slides, Malware evasion).

The function declares an array `szKeys[]` of blacklisted registry keys, exactly the list above, and computes the number of entries with `sizeof(szKeys) / sizeof(szKeys[0])`. It then loops: for every key it calls `Is_RegKeyExists(HKEY_LOCAL_MACHINE, szKeys[i])` and prints the result. A real malware would simply exit on the first hit. The same pattern (array of known artifacts, loop, existence test) is used for files, processes and drivers.

> **Remember**
>
> From the point of view of the analyst, a binary that contains many strings such as `VBOX`, `VBoxGuest`, `VMware` or `SOFTWARE\Oracle\VirtualBox Guest Additions` is almost certainly performing anti-VM checks. This is a very cheap static indicator (section 6.3).

> **From the lecture (20 October 2026, 42:48–43:26)**
>
> Some tools rewrite these strings and remove the guest-addition entries. According to the professor, they work against simple malware.

12.6 Applications

The presence of a software program may be:

1. a **necessary condition** to trigger a payload, for example a banking trojan that activates only if a specific banking app or browser is installed;
2. an **adversary to disarm**, for example anti-virus products that the malware kills or evades;
3. a **sufficient condition for evasion**, for example finding an analysis tool like IDA: the malware assumes that an analyst is at work and stops.

Analysts dissect malware using **multiple VM images, each with different applications and versions**, precisely because the set of installed programs changes what the sample does.

```
const TCHAR *szProcesses[] = {
    _T("ollydbg.exe"),          // OllyDebug debugger
    _T("ProcessHacker.exe"),    // Process Hacker
    _T("tcpview.exe"),          // Part of Sysinternals Suite
    _T("autoruns.exe"),         // Part of Sysinternals Suite
    _T("autorunsc.exe"),        // Part of Sysinternals Suite
    _T("filemon.exe"),          // Part of Sysinternals Suite
    _T("procmon.exe"),          // Part of Sysinternals Suite
    _T("regmon.exe"),           // Part of Sysinternals Suite
    _T("procexp.exe"),          // Part of Sysinternals Suite
    _T("idaq.exe"),             // IDA Pro Interactive Disassembler
    _T("idaq64.exe"),           // IDA Pro Interactive Disassembler
    _T("ImmunityDebugger.exe"), // ImmunityDebugger
    _T("Wireshark.exe"),        // Wireshark packet sniffer
    _T("dumpcap.exe"),          // Network traffic dump tool
    _T("HookExplorer.exe"),     // Find various types of runtime hooks
    _T("ImportREC.exe"),        // Import Reconstructor
    _T("PETools.exe"),          // PE Tool
    _T("LordPE.exe"),           // LordPE
    _T("SysInspector.exe"),     // ESET SysInspector
    _T("proc_analyzer.exe"),    // Part of SysAnalyzer iDefense
    _T("sysAnalyzer.exe"),      // Part of SysAnalyzer iDefense
    _T("sniff_hit.exe"),        // Part of SysAnalyzer iDefense
    _T("windbg.exe"),           // Microsoft WinDbg
    _T("joeboxcontrol.exe"),    // Part of Joe Sandbox
    _T("joeboxserver.exe"),     // Part of Joe Sandbox
    _T("joeboxserver.exe"),     // Part of Joe Sandbox
    _T("ResourceHacker.exe"),   // Resource Hacker
    _T("x32dbg.exe"),           // x32dbg
    _T("x64dbg.exe"),           // x64dbg
    _T("Fiddler.exe"),          // Fiddler
    _T("httpdebugger.exe"),     // Http Debugger
};
```

Figure 12.5: Al-Khaser, the list of analysis processes (slides, Malware evasion).

The code is a long array `szProcesses[]` of process names with comments. The malware enumerates running processes and compares names against this list:

```
ollydbg.exe, ProcessHacker.exe,
tcpview.exe, autoruns.exe, filemon.exe, procmon.exe,
  regmon.exe,
procexp.exe (Sysinternals suite)
idaq.exe, idaq64.exe (IDA)
ImmunityDebugger.exe, Wireshark.exe, dumpcap.exe,
  HookExplorer.exe,
ImportREC.exe, PETools.exe, LordPE.exe, SysInspector.exe,
proc_analyzer.exe, sysAnalyzer.exe, sniff_hit.exe, windbg.exe,
```

```
joeboxcontrol.exe, joeboxserver.exe (Joe Sandbox)
ResourceHacker.exe, x32dbg.exe, x64dbg.exe, Fiddler.exe,
httpdebugger.exe
```

Example: frustrating the analyst. The **Furtim** malware first detects and bypasses a wide range of anti-virus products. Then it searches for **29 tools that analysts launch manually**. Instead of exiting immediately when one is present, it **delays the termination until a later stage of the fingerprinting process**, intentionally causing frustration for the analyst. The sample seems to progress and then silently dies, which makes it hard to understand which check killed it (reference on the slide: SentinelOne blog on SFG, the parent of Furtim).

From the lecture (20 October 2026, 44:05–53:40)

Necessary condition. This is not only about targeted malware. General-purpose samples may also need a vulnerable component (JVM, browser, old Office) and quit if it is absent.

Machine “vaccination”. Some people proposed installing IDA and hypervisor tools as decoys, so that malware would believe it is being analyzed and leave the machine alone. The professor says it works against some malware but is not robust: a good sample can tell that the tools are fake, just as with the audio card.

Professional setups. Analysts use many VMs with different Windows and software versions. Some vendors instantiate customized VMs on demand from base models.

The asymmetry. Defenders must cover every corner, while attackers need just one gap.

Furtim. Besides the 29 tools on the slide, the professor recalls 300–400 checks overall, which no publicly known sandbox survived at the time. Even Joe Sandbox, the best at the time according to the professor, needed about two weeks to run it. The authors knew products that most people in the field did not know existed, so they probably worked in the security industry. The name comes from Latin and refers to moving cautiously.

12.7 User artifacts

Fresh Windows installations are suspicious. A real user machine has accumulated wear:

- recently opened files;
- navigation history;
- applications and their install dates;
- recently connected USB devices;
- number and type of files in `Documents` and `Desktop`.

VM sandboxes shall exhibit a realistic wear-and-tear state. The key reference is Spotless Sandboxes: Evading Malware Analysis Systems Using Wear-and-Tear Artifacts (IEEE S&P 2017), which trains a classifier on such artifacts to separate real and sandbox machines. The countermeasure is therefore not to hide one artifact but to **age** the image.

> **From the lecture (20 October 2026, 53:40–55:54)**
>
> Windows keeps a list of the last connected USB devices in the registry. According to the wear-and-tear paper, as the professor recalls it, three checks of this kind were enough for a properly written sample to detect over 95% of the systems. The authors of sandboxes are not careless: realistically emulating a used machine is just very hard.

12.8 Timing attacks

> Playing with time is one of the oldest, yet still one of the most effective ways to detect malware analysis systems.
>
> (slides, Malware evasion)

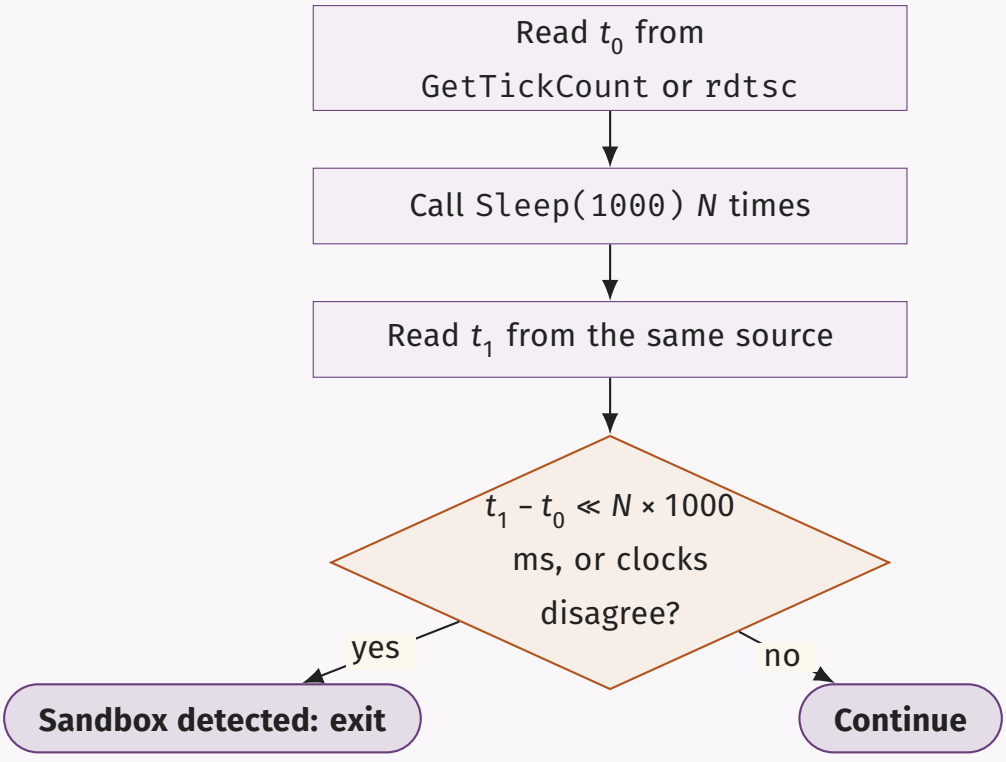
There are different options.

1. **Time an instruction sequence and compare it with a baseline.** This is what red pills do with `cpuid` under VMs. The method can **also detect binary instrumentation schemes** (dynamic binary translation, single-stepping, tracing), which slow code down.
2. **Make the analysis run out of budget with sleeps.** Sandboxes run each sample for a few minutes, so the malware simply sleeps through it.

The sleep option has several refinements:

- Use **multiple sleep actions** instead of just one with a large value. Sandboxes often accelerate a single big `Sleep()` by patching it to return immediately, so the malware issues many small ones and checks the elapsed time.
- Check **whether time has been accelerated** and whether the different **time sources are consistent** with each other, for example `GetTickCount`, `rdtsc` and network time. Windows offers plenty of time sources.
- Run **long computations in the initial stages**, which burn the sandbox budget without involving any sleep.

Flowchart: Detecting an accelerated sleep



The sample takes a time reading, sleeps in many small steps and reads the time again. If much less time has passed than the sleeps require, or if other clocks disagree, the sleeps were accelerated, so it concludes that it is in a sandbox and exits.

From the book

The same idea is used against **debuggers**, because single-stepping slows execution. The examples in PMA are two `rdtsc` readings compared with a threshold (the sample compares the difference with `0xFFF`; on failure it jumps to a random address by `push eax; ret`), `QueryPerformanceCounter`, and `GetTickCount` (with threshold `0x1A` ms in its example), plus timing the handling of a raised exception. The recommended workaround is to run past the check and set a breakpoint just after it (PMA, anti-debugging chapter, PDF pp. 495–497).

From the lecture (20 October 2026, 56:33–61:30)

- **Budget:** a sandbox runs a sample for 3–10 minutes. Studies show this is usually enough, unless the sample is evading.
- **Sleeps:** a sample loses little by activating after 20 minutes if nothing disturbs it.
- **Hiding the skipped time:** a sandbox can accumulate the skipped sleeps and subtract them whenever the sample asks for the time. But this must stay consistent across all the time sources Windows offers, so some samples still require manual help.
- **Long computations:** computing digits of pi, for example, burns the budget without suspicious actions. The professor sees room for analyses that recognize useless work and fast-forward past it.

12.9 What could one monitor?

> There is no such thing as a transparent sandbox.
>
> (slides, Malware evasion)

We may learn from **what imperfections the samples look for**, and either fix these issues or come up with designs unaffected by them. The analysis system itself can also **look for evasive sequences** and flag the sample: an evasion attempt is a strong indicator even if the payload is never seen.

Observable	Examples
Special instructions	`cpuid`, `int`, `rdtsc`
Library calls	Files, registry, GUI events, hardware features, drivers, processes, pipes, DLL handling, network, IPC objects, time sources
System calls	The sample talks to the kernel directly, bypassing the hooked user-mode APIs
WMI	Queries to the OS through the Windows Management Instrumentation system
Process environment	Data structures like the PEB (Process Environment Block) may expose relevant information, such as CPU cores or the presence of debuggers

The **PEB** is the per-process user-mode structure holding environment, loaded modules and debugger status (section 3.6). Because reading it is a plain memory access, **no API call is visible to the sandbox**. That is why malware prefers to inspect it manually (PMA, PDF p. 489).

> **From the lecture (20 October 2026, 61:38–64:10)**
>
> The professor compares this to any cat-and-mouse game in security, such as defenses moving from buffer overflows to use-after-free bugs. During manual analysis, the practical advice is to look closely at special instructions like `cpuid` and `rdtsc` and at the surrounding subroutines, because they may be an evasion to disarm. There are plugins that hide a debugger or alter values such as the number of cores in the PEB.

12.9.1 WMI queries

The **Windows Management Instrumentation** subsystem supports **SQL-style queries**, in the language called WQL, to the OS about hardware and software. These are some examples of what can be checked.

SELECT * FROM	Properties
`Win32_Bios`	`SerialNumber`
`Win32_PnPEntity`	`DeviceId`
`Win32_NetworkAdapterConfiguration`	`MACAddress`
`Win32_Processor`	`NumberOfCores`, `ProcessorId`
`Win32_LogicalDisk`	`Size`
`Win32_ComputerSystem`	`Model`, `Manufacturer`
`Win32_NTEventLogFile`	`FileName` (related to VBOX)
`MSACPI_ThermalZoneTemperature`	`CurrentTemperature`

Everything in the previous sections (hardware, MAC, disk size, BIOS) can therefore be read **through one uniform and high-level interface**, with no `cpuid` and no registry access in the code.

```
// Get the value of the Name property
hRes = pclsObj->Get(_T("SerialNumber"), 0, &vtProp, 0, 0);
if (SUCCEEDED(hRes)) {
    if (vtProp.vt == VT_BSTR) {

        // Do our comparison
        if (
            (StrStrI(vtProp.bstrVal, _T("VMWare")) != 0) ||
            (wcscmp(vtProp.bstrVal, _T("0")) == 0) || // VBox (serial is just "0")
            (StrStrI(vtProp.bstrVal, _T("Xen")) != 0) ||
            (StrStrI(vtProp.bstrVal, _T("Virtual")) != 0) ||
            (StrStrI(vtProp.bstrVal, _T("A M I")) != 0)
            )
        {
            VariantClear(&vtProp);
            pclsObj->Release();
            bFound = TRUE;
            break;
        }
    }
    VariantClear(&vtProp);
}
```

Figure 12.6: Al-Khaser, WMI check on `Win32_Bios` (slides, Malware evasion).

After a WQL query, the code calls `pclsObj->Get(L"SerialNumber", ...)`. If the variant is a string (`VT_BSTR`), it compares it with the **common BIOS serial numbers in hypervisors and emulators**:

- it contains `"VMware"`;
- it equals `"0"`, since the serial of VirtualBox is just `"0"`, as the comment says;
- it contains `"Xen"`, `"Virtual"` or `"A M I"`, the American Megatrends string used by some VMs.

On a match it sets `bFound = TRUE` and breaks. `StrStrI` is a case-insensitive substring search, and `wcscmp` is an exact comparison.

> **From the lecture (20 October 2026, 65:43–67:28)**
>
> WMI can also list running processes and installed antivirus products. So when a sample uses WMI, inspect its queries. According to the professor, `Virtual` in the Al-Khaser check most likely refers to Virtual PC, an old emulator, not VirtualBox.

12.10 In-guest agents

We discussed two main **attack surfaces**: artifacts of the execution technology (imperfections, overheads) and of the software setup (what is installed). However, for sandboxes and general dynamic analysis systems, **operating inside a virtualized guest is already a weakness**.

- Monitoring agents are often implemented in **user space**, frequently as an **injected DLL** that hooks APIs. The DLL, the hooks (modified function prologues) and the extra processes are all visible to the sample.
- Agents implemented in **kernel space** are more rare now, because **PatchGuard** (Windows Kernel Patch Protection) forbids modifying kernel structures.

We can always study fingerprinting techniques and fix or randomise the implementation accordingly. But in the end **these systems will always share the execution context with the sample under analysis**.

> **From the lecture (20 October 2026, 68:10–72:50)**
>
> Whenever attacker and defender coexist in the same space, the professor considers the defender destined to fail, unless the defender moves to a higher layer. A DLL injected into a process is visible, because the process can query its own loaded modules.
>
> **PatchGuard** detects modifications to the running kernel, such as patching a system call as a rootkit would, and answers with a blue screen. It has over 200 ways of checking, and bypassing it is very hard. Windows meanwhile offers standard ways to receive kernel events, such as **Event Tracing for Windows**, and some people experiment with **eBPF for Windows**.

12.11 Virtual machine introspection

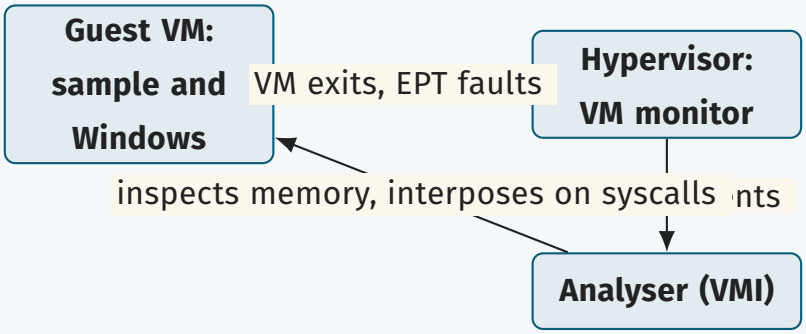
The idea is to perform the analysis from outside the guest VM. Virtual machine introspection (VMI) was pioneered in **intrusion detection systems** (Garfinkel and

Rosenblum, NDSS 2003) for greater attack resistance while retaining visibility. It **requires cooperation from the VM monitor of the hypervisor**.

Three capabilities are crucial to support good visibility while offering resistance to evasion and subversion.

Capability	Meaning
Isolation	The program cannot access the code or state of the monitoring system
Inspection	The monitoring system has full access to the state of the analysed machine (memory, registers, disk)
Interposition	The monitoring system can interpose on specific operations, such as the kernel transitions caused when issuing a system call

Diagram: The analyser outside the guest



The sample and Windows run in the untrusted guest. The VM monitor sees events such as VM exits and EPT (Extended Page Table) faults and passes them to the analyser. The analyser reads the memory of the guest and interposes on system calls, without being in the same execution context as the sample.

12.11.1 Ether

The **Ether** malware analyser (ACM CCS 2008) pioneered the use of the **VT-x** hardware virtualization extensions to build a **transparent** analysis system. The analyser **resides in the VM monitor**. Its authors identified **5 requirements** for a

system that wants to hide the memory and CPU changes inevitably caused by its very presence.

1. **Higher privilege:** the analyser runs at a privilege level that the analysed code cannot reach.
2. **Privileged access to side effects:** if side effects are introduced, they should be visible only with a privilege level that analysed code cannot reach.
3. **Same basic instruction semantics:** side effects are allowed only during exceptions, and normal instructions must behave as on real hardware.
4. **Transparent exception handling:** when an exception occurs, the analyser can reconstruct the expected context where needed.
5. **Identical timing information:** access to time sources is shepherded. The analyser forges the values so that elapsed time looks native, which defeats timing attacks.

The closing remark of the slide is that the design was **undermined by practical limitations: the imperfections of VT-x**. For example, VM exits cost time that cannot be fully hidden, so the timing checks of this chapter still work.

> **From the lecture (20 October 2026, 76:37–78:25)**
>
> Why the professor stays skeptical of Ether: VT-x was designed for near-native speed in the cloud, not for security. Its authors could not foresee in 2008 that a user-space program could reconstruct cache utilization through a side channel without violating any of the five requirements. Even so, the design is far better than a DLL injected in the sample, and top sandboxes have adopted VMI.

12.11.2 The semantic gap problem

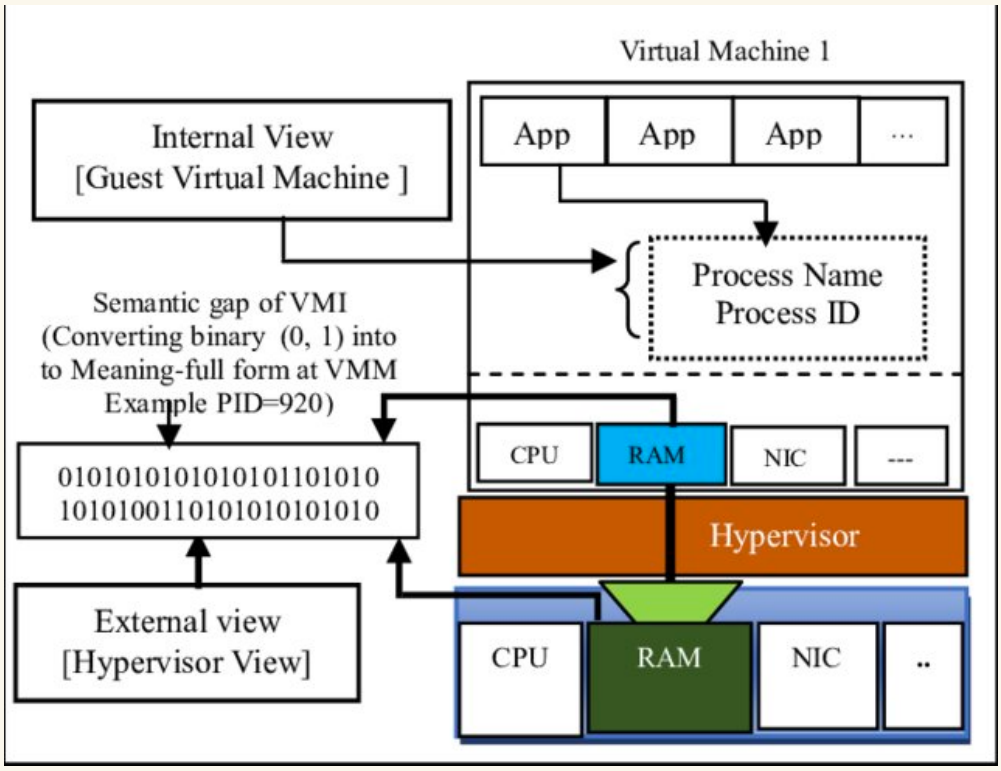


Figure 12.7: The internal and external views of a guest, from Ajay Kumar et al., ICPADS 2015 (slides, Malware evasion).

VMI methods incur a **semantic gap** when trying to inspect high-level concepts of the guest system, such as **API calls or threads**. The figure shows two views. The **internal view**, inside the guest, sees applications, a process name and a PID. The **external view**, from the hypervisor, sees only raw memory: CPU, RAM and network card as bits.

The VMM must **reconstruct meaning from raw bytes**: “converting binary (0, 1) into meaningful form at VMM; example PID = 920”. That requires knowing the kernel data structures of the guest OS (offsets and layouts of EPROCESS, thread lists and so on) and walking them in the memory of the guest. This is fragile and version-dependent.

> **From the lecture (20 October 2026, 78:25–80:40)**
>
> An injected DLL can simply call Windows APIs, while VMI only has raw bytes. The tooling has improved a lot. A few years ago, even listing processes was complex. Now a command on the host can list the processes of the guest

with PID and name, and VMI can even inject a thread to run code inside an application.

12.11.3 Automatic and manual analysis

- **VMI-based systems face higher implementation complexity** than in-guest agents but **offer better transparency**. Sandbox vendors have started adopting VMI-based or **hybrid** approaches. **DRAKVUF** is the reference system in academic research (ACSAC 2014).
- **Automatic analysis is only one part of the story.** When dissection is required, **VMI is no lifeboat for your conspicuous manual work**. Analysts still work on **their own laptops with classic virtualization products**, so they have to dismantle evasions again, this time **manually**, by patching the checks and hiding artifacts.
- The **latest trend** is **out-of-VM debugging** of a process running in the VM. The debugger runs outside the guest, so that anti-debugging checks for in-guest tools find nothing.

From the book

PMA covers the manual side for VMware. First look for strings (`VMwareTray.exe`, `VMMouse`, `net start | findstr VMware`) and follow the cross-reference to the check. Then **patch** the conditional jump so that it is never taken, or edit the string (`XXXareTray.exe`), or uninstall VMware Tools. You can also NOP out `sidt` and `in`, run on a multicore host, or add the undocumented `.vmx` options, which are listed in the book, but it advises against them as they disable VMware Tools.

It also gives an IDAPython script that highlights `sidt`, `sgdt`, `sldt`, `smsw`, `str`, `in` and `cpuid`, and mentions the tool **ScoopyNG**, with seven VMware checks (PMA, PDF pp. 512–523).

From the lecture (20 October 2026, 80:43–83:17)

Analysts are few and their time is precious, so research that saves it is

> valued. This is why the group of the professor built BluePill: the analyst opening a sample in a debugger meets again the same tricks that sandboxes worked hard to bypass. Debugging from a hypervisor is progressing but is not yet plug and play. The anti-debugging slides below were not covered in this lecture, because the professor wanted to work with debuggers first.

12.11.4 References on evasion

The slides contain two reference slides, kept together here.

- Al-Khaser: `https://github.com/LordNoteworthy/al-khaser/`
- A fistful of red-pills: How to automatically generate procedures to detect CPU emulators, WOOT 2009.
- A Virtual Machine Introspection Based Architecture for Intrusion Detection, NDSS 2003.
- Ether: Malware Analysis via Hardware Virtualization Extensions, ACM CCS 2008.
- Scalability, Fidelity and Stealth in the DRAKVUF Dynamic Malware Analysis System, ACM ACSAC 2014.
- Spotless Sandboxes: Evading Malware Analysis Systems Using Wear-and-Tear Artifacts, IEEE S&P 2017.
- On the Dissection of Evasive Malware, IEEE TIFS 2020.
- BluePill: Neutralizing Anti-Analysis Behavior in Malware Dissection, Black Hat Europe 2019.
- My Ticks Don’t Lie: New Timing Attacks for Hypervisor Detection, Black Hat Europe 2020.
- Check Point Research: evasion techniques at `https://evasions.checkpoint.com/` (proofs of concept in `CheckPointSW/Evasions`) and anti-debug tricks at `https://anti-debug.checkpoint.com/` (proofs of concept in `CheckPointSW/showstopper`).

- Defeating evasive malware, VMRay whitepaper, Winter 2020.
- PoW-How: An Enduring Timing Side-Channel to Evade Online Malware Sandboxes, ESORICS 2021.

12.12 Anti-debugging

The presence of a debugger alters the normal working of a process. The OS reflects such effects, both **directly and indirectly**, and the malware can read them. This is the basis of **anti-debugging**.

Mechanism	Detail
Library functions	`IsDebuggerPresent()`, `CheckRemoteDebuggerPresent()`
System calls	For example `NtQueryInformationProcess`
PEB flags	`BeingDebugged` (checked by `IsDebuggerPresent`), `NtGlobalFlag`
`int 2d` and other low-level tricks	For example `sti`
API semantics	For example `CloseHandle` on an invalid handle

The slide also notes that **Peter Ferrie** wrote a guide with about 80 red pills to expose debuggers.

> **From the book**
>
> `IsDebuggerPresent` reads the PEB field (`BeingDebugged`, at **offset 2**; the text of PMA calls it `IsDebugged`) and returns non-zero under a debugger. `CheckRemoteDebuggerPresent` is the same for another process on the local machine: despite the name, it is not about remote machines. `NtQueryInformationProcess` with class `ProcessDebugPort` (`0x7`) returns a port number if a debugger is attached.
>
> The **manual** version loads the PEB address from `fs:[30h]` and tests `byte`

`[eax+2]`. It avoids the API calls, which a rootkit-style hook could falsify.

Besides `BeingDebugged`, the **ProcessHeap** flags (`ForceFlags` and `Flags` of the first heap) and **NtGlobalFlag** are manual PEB-based checks. The bypass is to set the flag to zero, force the conditional jump, or use a "hide debugger" plug-in (PMA, anti-debugging chapter, PDF pp. 487–492).

Other families are scanning your own code for `0xCC` or computing **checksums** over the code, **TLS callbacks** (code run before the entry point), exceptions and inserted interrupts (`int 3`, `int 2d`, `icebp`), and bugs in the debugger itself (PE header and `OutputDebugString("%s%s...")` crashes in OllyDbg 1.1) (PMA, PDF pp. 494–506).

Remember

Software breakpoints work by **overwriting an instruction with `0xCC`**, which is exactly what the `0xCC` scans and the checksums detect. **Hardware breakpoints** do not modify the code and avoid both (section 11.5.2). The analyst should also configure the debugger to pass exceptions to the program (section 11.6).

From the lecture (3 November 2026, 70:43–73:40)

These slides were covered after introducing debuggers. A good debugger plugin starts by restoring the expected values: the debugger sets `BeingDebugged` to 1, and the plugin sets it back to 0, doing the same for `NtGlobalFlag`. Many proofs of concept, mostly from the guide of Peter Ferrie, are in the Check Point anti-debug page listed in the references.

12.12.1 The NtGlobalFlag check

`NtGlobalFlag` is a field of the PEB set from system and image settings at process creation. When a process is **started under a debugger**, Windows enables heap debugging flags, so the field holds `0x70`. Without a debugger it is **likely 0**.

The value `0x70` is the OR of three flags:

Flag	Value
FLG_HEAP_ENABLE_TAIL_CHECK	0x10
FLG_HEAP_ENABLE_FREE_CHECK	0x20
FLG_HEAP_VALIDATE_PARAMETERS	0x40

The check detects a debugger **attached at process creation**, not one attached later.

```
#elif defined(ENV32BIT)
    /* NtGlobalFlags for real 32-bits OS */
    BYTE* _teb32 = (BYTE*)__readfsdword(0x18);
    DWORD _peb32 = *(DWORD*)(_teb32 + 0x30);
    pNtGlobalFlag = (PDWORD)(_peb32 + 0x68);

    if (IsWoW64())
    {
        /* In Wow64, there is a separate PEB for the 32-bit portion and the 64-bit portion
        which we can double-check */

        BYTE* _teb64 = (BYTE*)__readfsdword(0x18) - 0x2000;
        DWORD64 _peb64 = *(DWORD64*)(_teb64 + 0x60);
        pNtGlobalFlagWoW64 = (PDWORD)(_peb64 + 0xBC);
    }
#endif

    BOOL normalDetected = pNtGlobalFlag && *pNtGlobalFlag & 0x00000070;
    BOOL wow64Detected = pNtGlobalFlagWoW64 && *pNtGlobalFlagWoW64 & 0x00000070;

    if(normalDetected || wow64Detected)
        return TRUE;
    else
        return FALSE;
```

Figure 12.8: Al-Khaser, `NtGlobalFlag` on the 32-bit and on the 64-bit PEB (slides, Malware evasion).

On a real 32-bit OS the code reads the TEB self pointer with `__readfsdword(0x18)`, adds `0x30` to get the PEB and `0x68` to get `NtGlobalFlag` (the `pNtGlobalFlag` pointer).

On **64-bit Windows a 32-bit process (WOW64)** has two PEBs: the 32-bit one and a 64-bit one. So the code derives the 64-bit TEB as `_teb32 - 0x2000`, the 64-bit PEB at `+0x60`, and the flag at `+0xBC`. The verdict is `normalDetected || wow64Detected`, where each is `*pNtGlobalFlag & 0x70`.

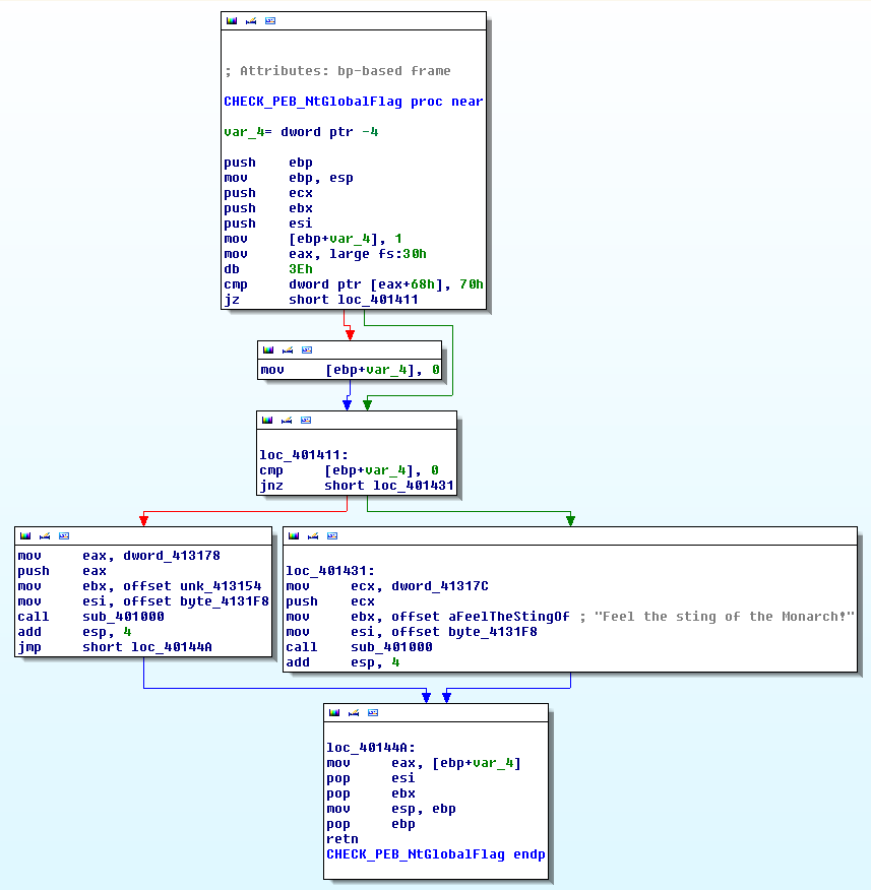


Figure 12.9: The same check in an IDA graph, from Challenge 7 of Flare-On 2014 (slides, Malware evasion).

The IDA graph comes from **Challenge 7 of the Flare-On series 2014**, a reverse-engineering CTF, and is the same check written directly in assembly.

Step by step: Reading the blocks from the top

1. The function `CHECK_PEB_NtGlobalFlag` first sets the local `var_4 = 1`, the "debugger present" default. It then loads the PEB with `mov eax, large fs:30h`. The stray `db 3Eh` is a `ds:` segment-override prefix that does nothing, which is why the book shows `ds:[eax+68h]`, and it may also confuse a naive disassembler.

2. `cmp dword ptr [eax+68h], 70h` compares `NtGlobalFlag` with `0x70`. If it is **equal** (`jz`) the jump skips the next block, so `var_4` stays 1. If it is **different**, the red arrow executes `mov [ebp+var_4], 0` (no debugger).

3. At `loc_401411` the code tests `var_4`. When it is non-zero (debugger), the green branch loads a **taunt string**, “Feel the sting of the Monarch!”, the message the challenge shows to an analyst who runs the program in a debugger, and calls `sub_401000`. Otherwise the other branch runs with different data, which is the real behaviour.
4. The function returns `var_4` in EAX.

The remark on the slide compares it with the C example: here the code **accesses the 32-bit PEB directly** (`fs:30h`) but **does not check the WOW64 PEB**. This is the pattern to look for during static analysis: `fs:[30h]` followed by a comparison with a constant (PMA, PDF p. 492, `mov eax, large fs:30h; cmp dword ptr ds:[eax+68h], 70h`).

The function is a pure predicate, `BOOL is_debugged()`. The caller uses the result to change its control flow, which is why the bypass of the book applies directly: force the `jz`, or set `NtGlobalFlag` to `0` before the `cmp`.

From the lecture (3 November 2026, 77:34–83:34)

- **Learn the offsets:** they look like boring details, but you will recognize them in code (TEB, PEB, `NtGlobalFlag`). Some authors build them at runtime, so they do not stand out in the disassembler.
- **Why two PEBs:** on WOW64 a 64-bit process can query a 32-bit one, but it would crash reading structures in the 32-bit format, where pointers have a different size. So the 32-bit process also has a 64-bit-friendly structure. A sloppy analyst fixes only one of them, and according to the professor this has worked for much malware.
- **The AND in the C check:** any bit of `0x70` that is set counts as detection, not only the exact value. The Flare-On code is simpler, comparing with `0x70` directly.

12.12.2 The CloseHandle check

```
/*
APIs making user of the ZwClose syscall (such as CloseHandle, indirectly)
can be used to detect a debugger. When a process is debugged, calling ZwClose
with an invalid handle will generate a STATUS_INVALID_HANDLE (0xC0000008) exception.
As with all anti-debugs that rely on information made directly available.
*/


BOOL NtClose_InvalideHandle()
{
    auto NtClose_ = static_cast<pNtClose>(API::GetAPI(API_IDENTIFIER::API_NtClose));

    __try {
        NtClose_(reinterpret_cast<HANDLE>(0x99999999ULL));
    }
    __except (EXCEPTION_EXECUTE_HANDLER) {
        return TRUE;
    }

    return FALSE;

}

BOOL CloseHandle_InvalideHandle()
{
    // Let's try first with user mode API: CloseHandle
    __try {
        CloseHandle(reinterpret_cast<HANDLE>(0x99999999ULL));
    }
    __except (EXCEPTION_EXECUTE_HANDLER) {
        return TRUE;
    }

    // Direct call to NtClose to bypass user mode hooks
    if (NtClose_InvalideHandle())
        return TRUE;
    else
        return FALSE;
}
```

Figure 12.10: Al-Khaser, the `CloseHandle` check (slides, Malware evasion).

The Al-Khaser snippet is introduced by a comment.

> APIs making use of the ZwClose syscall (such as CloseHandle, indirectly) can be used to detect a debugger. When a process is debugged, calling ZwClose with an invalid handle will generate a STATUS_INVALID_HANDLE (0xC0000008) exception. As with all anti-debugs that rely on information made directly available.
>
> (Al-Khaser source code, as transcribed from the slide in the course notes)

Two functions are shown:

1. `NtClose_InvalideHandle()` resolves `NtClose` dynamically (`API::GetAPI(API_IDENTIFIER::API_NtClose)`) and calls it with the

bogus handle `0x99999999` inside `__try`. If an exception is raised, the `__except(EXCEPTION_EXECUTE_HANDLER)` block returns `TRUE` (debugger present), otherwise `FALSE`.

2. `CloseHandle_InvalideHandle()` tries first with the **user-mode** API `CloseHandle`. If an exception is raised, it returns `TRUE`. Otherwise it calls `NtClose_InvalideHandle()` directly "to **bypass user mode hooks**".

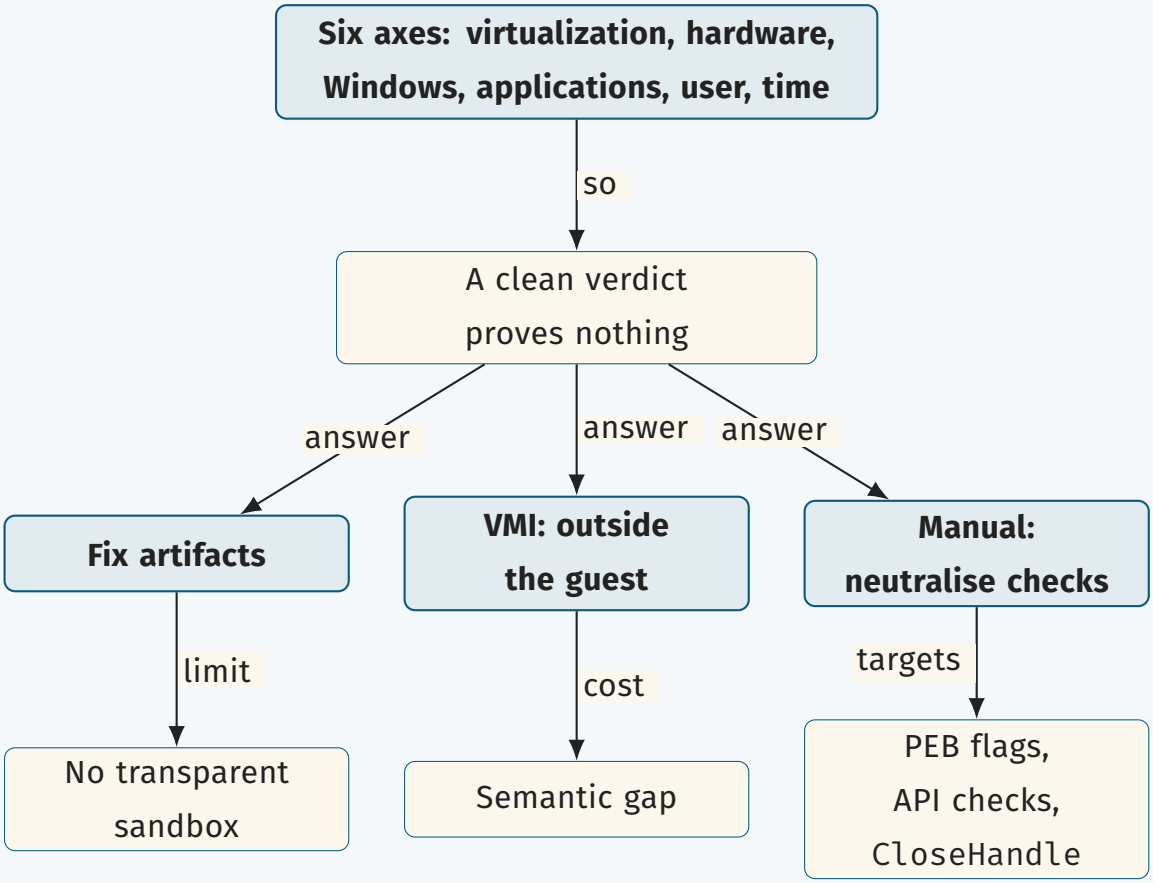
Why does it work? With no debugger, `CloseHandle` on an invalid handle just returns `FALSE` and sets the last error, silently. With a debugger attached, the kernel raises the exception, normally delivered to the debugger first and then handled by the `__except` of the process.

The second function is a good example of the cat-and-mouse game. If a sandbox or an anti-anti-debug tool hooked `CloseHandle` in user mode to hide the exception, calling the native function bypasses the hook.

> **From the lecture (3 November 2026, 74:04–77:30)**
>
> Closing an invalid handle is something the program wanted to release anyway, so normally it is not a reason to stop. This check is an exception to the usual advice of passing exceptions to the program: if you pass it, the `__except` block runs and returns `TRUE`. So here you should handle it in the debugger. The second function is more thorough because a modified `CloseHandle` could simply skip invalid handles, whereas `NtClose` is harder to modify.

> **To verify**
>
> The function names `NtClose_InvalideHandle` and `CloseHandle_InvalideHandle` are reported in the course notes as the original Al-Khaser spelling ("Invalide"). This is not a transcription mistake here.

Concept map: how the ideas connect



Malware compares the environment with a real victim along six axes, so a clean verdict does not prove the absence of malicious behaviour. The analyst has three answers. Fixing artifacts one by one never ends. VMI moves the analyser outside the guest, and pays with the semantic gap. In manual work, you recognise and neutralise the checks.

In short

Evasion is the reason why **a sandbox verdict never proves the absence of malicious behaviour**. Malware compares the environment with what a real victim looks like along six axes: virtualization, hardware, Windows installation, applications, user wear-and-tear and **time**.

The answers of the analyst are three:

1. fix the artifacts one by one, which is an endless game (“no transparent sandbox”);

2. move the analyser **out of the guest** with **VMI** (isolation, inspection, interposition; Ether, DRAKVUF), paying with the **semantic gap**;
3. for manual work, recognise and neutralise the checks: `fs:[30h]`, `NtGlobalFlag == 0x70`, `IsDebuggerPresent`, `CloseHandle` exceptions.

Review: try to answer without looking

1. Why does measuring `cpuid` latency still work when the hypervisor hides its brand string?
2. Why is a VM not proof of a sandbox, and a check on a machine model not proof of anti-analysis?
3. Why is reading the PEB more attractive to malware than calling an API?
4. What are the three VMI capabilities?
5. Which debugger does the `NtGlobalFlag` check miss?

Answers, with the section where to find them

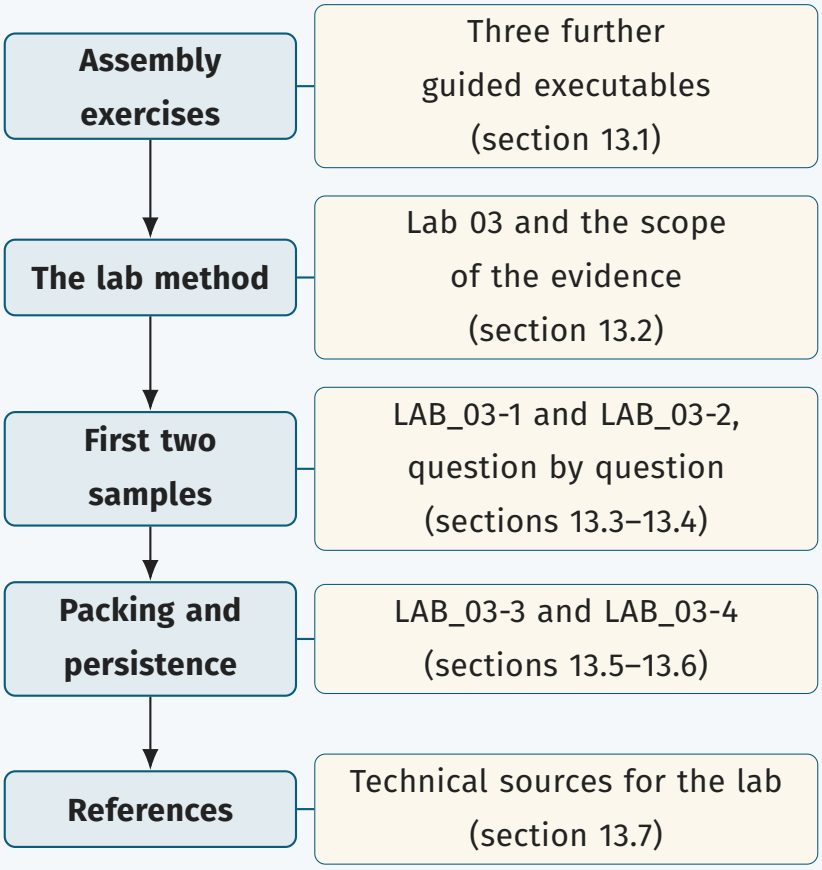
1. `cpuid` always causes a VM exit, and that cost remains (section 12.3.2).
2. Virtualization has legitimate uses, and targeted malware reads the same properties to recognise its victim (section 12.4).
3. It is a plain memory access, so no API call is visible to the sandbox (section 12.9).
4. Isolation, inspection and interposition (section 12.11).
5. One attached after the process was created (section 12.12.1).

Chapter 13

Hands-on exercises and Lab 03

> **Before you start**
>
> **Central question:** how do we turn the theory into a method for real binaries, and how do we report only what the evidence supports?
>
> **Prerequisites:** x86 code and IDA (chapters 4 and 5), basic static analysis (chapter 6), dynamic tools (chapter 8).
>
> **Where it leads:** it is the practical counterpart of the theoretical path, and you can use it at any time once the prerequisites are clear.

Chapter structure: what comes, and in which order



The chapter starts with the assembly exercises. It then sets the method for the lab and applies it to the four samples. The arrows show the order of the book: you can use the parts separately once their prerequisites are clear.

This chapter has two parts. The first collects the assembly exercises from the slides that chapter 5 does not cover. The second follows Lab 03, on basic analysis, with its four samples.

Remember

What the material contains. The exercise slides give no executables, and the Lab 03 PDF is an assignment with no solutions. This chapter therefore explains what to investigate, how to do it and what the evidence can establish. It does not report results for the samples, except where marked as an observation visible in the screenshots or in the notes of Simone, and in the "From the lab" boxes, which come from the recording of 9 October 2026.

13.1 Assembly exercises

Chapter 5 covers the setup, the use of IDA, `CONSTRUCTS.EXE` and `CDECL.EXE`. The three executables below use the same workflow (section 5.1), but the slides only give the questions, and no IDA demonstration of them is in the material.

13.1.1 CALLS\POINTERS.EXE

In `main()`, the address of a local variable is commonly calculated with `lea reg, [ebp-offset]` or an equivalent ESP-relative expression. The address of a global can be loaded or calculated from its image location, possibly with relocations.

If the program prints both, compare their regions: a local usually lies in the stack mapping of the thread, and a global in the data mapping of the image. Their **exact numerical addresses and order are not universal**, especially with ASLR (Address Space Layout Randomization) and differing layouts.

In `mysum()`, identify pointer dereferences by the brackets around a register or computed address, such as `mov eax, [ecx]`. Keep three things separate: the pointer value, the address calculation, and the value fetched from the memory pointed to. Follow the width (byte, word, dword) and any pointer arithmetic, to see whether the code accesses a scalar, a field or an array element.

13.1.2 CALLS\STDCALL.EXE

The exercise asks for the function addresses and the calling conventions of `test()` and `mysum()`. Infer the conventions from a combination of argument setup, register preservation, return instruction (`ret` versus `ret N`) and caller cleanup. One instruction in isolation is often insufficient.

- `sub esp, 28h` reserves `0x28` = 40 bytes. Inspect the uses of the reserved space before labelling all of it “locals”.
- A later `sub esp, 0Ch` reserves another 12 bytes at that point, perhaps

temporary space, outgoing argument space or alignment. Determine its role from the following stores, calls and matching cleanup.

- **Lowest ESP.** Start with a known ESP and account for every decrement: local reservations, saved registers, argument pushes, and the four-byte return address pushed by a nested `call`. Account for `ret N`, `pop` and `add esp, N` on the way back. The minimum occurs at the deepest point, not necessarily after the two visible `sub` instructions.
- **Optimization question.** Look for allocations whose lifetimes do not overlap, and for stack space that could be reused. Any proposed change must still respect the convention, the alignment requirements and the live values.

The numerical minimum and the best concrete rewrite depend on the actual disassembly, which is not included in the slides.

13.1.3 REP*.EXE

These binaries illustrate the compact buffer patterns of section 4.7. Identify the initialisation of the following registers and flag:

Item	Role
EDI	Destination or search buffer
ESI	Source
AL	Fill or search byte
ECX	Count
DF	Direction

Then inspect the exact repeat prefix, and what is tested after it:

- **Buffer comparison:** `cmpsb` with `repe` or `repz` is a common pattern, and it terminates on a mismatch or on an exhausted count.

- **Copy:** `rep movsb` copies ECX bytes from `ESI` to `EDI`.
- **Search:** `scasb` with `repne` or `repnz` commonly stops on a match or on an exhausted count.
- **Fill:** `rep stosb` writes `AL` repeatedly to the destination.

> **Correction**
>
> The glance line in the PDF appears to say `rep cmbp` for comparison. Interpret it as the **cmpsb family with an appropriate repeat condition**: plain `rep` does not supply the mismatch-based early stop of `repe cmpsb`.

Verify the actual instructions of each executable rather than assuming a library call from the mnemonic alone.

The other bonus topics of the same slides are treated with their theory: the other Windows conventions in section 4.5.5, `loop` in section 4.8, and anti-disassembly with the `antidisassembly.exe` exercise in section 4.9.1.

13.2 Lab 03: method

13.2.1 Scope of the evidence

The supplied PDF is an assignment, not a completed malware report. It does not provide the binaries, their hashes, actual timestamps, execution traces or final answers. For the first sample it explicitly highlights only `ShellExecuteExA` and the string `60.248.52.95:443`. Every other API, artifact and scenario below is an explanatory example unless marked otherwise.

Basic analysis combines **static analysis**, which examines a file without executing it, and **dynamic analysis**, which observes what happens during a controlled execution. Static analysis suggests capabilities and possible intent, and dynamic analysis tests those hypotheses in a particular environment. Neither alone reveals every possible behaviour.

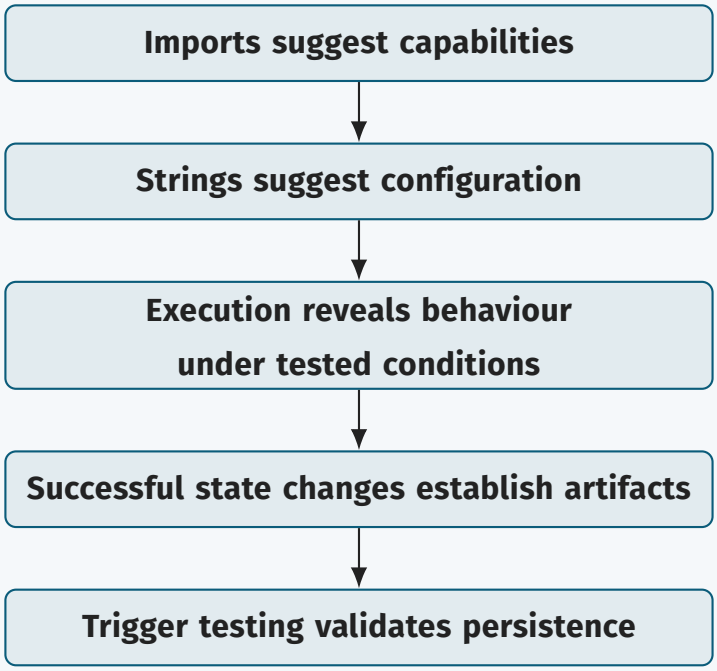
For each conclusion, keep three things separate.

Level	Meaning	Example
Observation	What a tool actually showed	A readable address
Interpretation	What that evidence suggests	A possible command-and-control endpoint
Limitation	What remains untested	Actual communication requires runtime evidence

Remember

Execution environment. All execution and unpacking work belongs in a disposable malware-analysis environment with a recoverable snapshot. Keep it separate from personal files, credentials, shared folders and production systems. If network behaviour must be exercised, use a controlled lab network or simulated services, rather than contacting the address printed in the PDF. Start monitoring before execution, and preserve evidence before restoring the snapshot.

Diagram: From weak to strong evidence



The diagram gives the evidence standard for every answer. A strong laboratory

answer preserves the exact observation, explains its significance and states its limit. Each level supports more than the one before it, and they must stay distinct throughout the analysis.

13.3 LAB_03-1.EXE

The assignment asks nine questions:

1. When was this file compiled?
2. List a few imports or sets of imports and describe how the malware might use them.
3. What are a few strings that stick out to you and why?
4. What happens when you run this malware? Is it what you expected and why?
5. Name a procmon filter and why you used it.
6. Are there any host-based signatures? (Files, registry keys, processes or services, etc). If so, what are they?
7. Are there any network-based signatures? (URLs, packet contents, etc) If so, what are they?
8. Is there anything that impeded your analysis? How so? How might you overcome this?
9. What do you think is the purpose of this malware?

The hints in the PDF are given below, each next to the question it serves.

> **Simone's note (original wording)**
>
> we have opened the lab3 zip file and pestudio
>
> the first thing to do in general is use VirusTotal and pass the MD5 into the search field

From the lab (9 October 2026, 00:00–05:00)

The samples come in an archive protected by the password `infected`. The good habit is to download it inside the VM, then disconnect the VM from the network before starting the tests. The samples are not executed until the professor asks for it, and a backup copy is made first. In the exam, choosing the right tool is part of the task; the preference of the professor here is PEStudio.

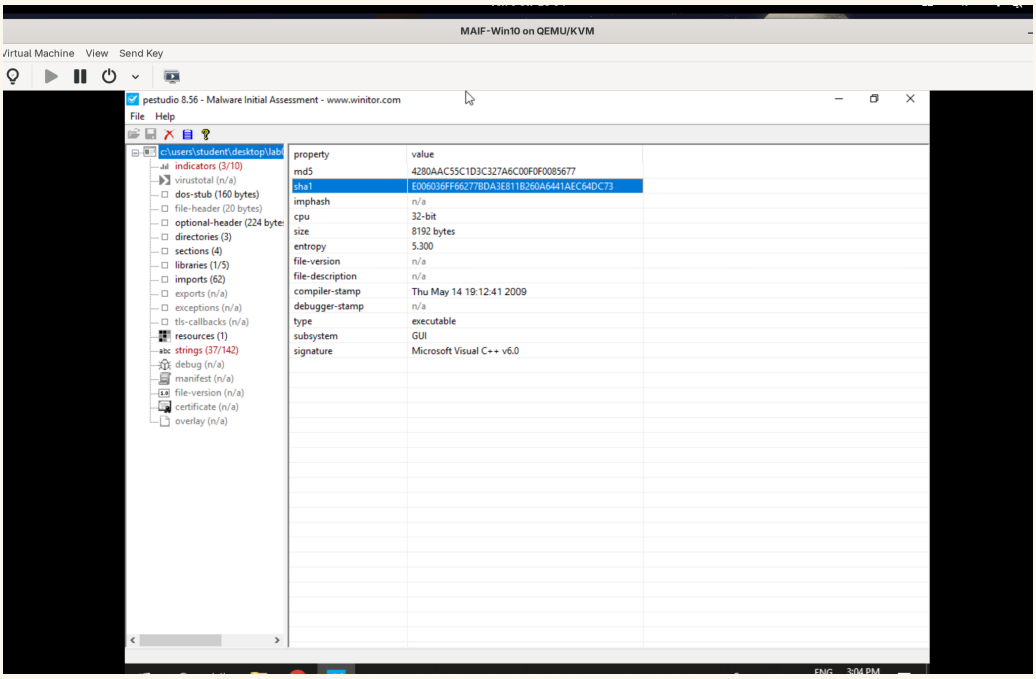


Figure 13.1: The first view of pestudio 8.56 on the sample, in the course VM (screenshot by Simone).

The screenshot reports the following values, which are observations from that session.

Property	Value
md5	`4280AAC55C1D3C327A6C00F0F0085677`
sha1	`E006036FF66277BDA3E811B260A6441AEC64DC73`
cpu, size	32-bit, 8192 bytes
entropy	5.300
compiler-stamp	`Thu May 14 19:12:41 2009`
debugger-stamp	n/a
type, subsystem	executable, GUI
signature	Microsoft Visual C++ v6.0
Counts in the tree	indicators 3/10, sections 4, libraries 1/5, imports 62, resources 1, strings 37/142

13.3.1 Question 1: the compilation date

Hints in the PDF: use software that allows you to inspect the PE header, open the EXE in PeStudio, and check the “compiler-stamp” value for the compilation date.

The intended starting point is the **PE (Portable Executable) header** (section 6.5). It lets you examine the structure of the file without intentionally running it: headers, sections, entry point, imports and resources. An unexpected entry-point location, an unusual section layout or sparse imports can guide the investigation, but no single anomaly establishes maliciousness.

For a conventional PE image, `compiler-stamp` normally represents the COFF header field `TimeDateStamp`, a number generally interpreted as seconds since 1 January 1970 in UTC. Despite the label, this is usually a **linker or build timestamp**, not proof of when every source file was compiled.

Record the raw value, the displayed date and time, and the time zone used by the viewer. Do not confuse this field with filesystem creation or modification times, which may change during copying or extraction. It also does not tell you

when the sample first infected a machine, or when an analyst received it.

The timestamp is only a clue. An author can modify it, a packer can alter metadata, and some reproducible-build workflows use a value that is not an ordinary creation time. An implausible date needs explanation, so compare it with other metadata and tools rather than silently replacing it (section 6.5.9).

> **Simone's note (original wording)**
>
> tuesday 14 May 2009, 19:12:42
>
> in pestudio

> **Correction**
>
> The note says "tuesday" and "19:12:42". The pestudio screenshot above shows `Thu May 14 19:12:41 2009`. The day is Thursday, and the seconds are 41 rather than 42. The value to report is the one displayed by the tool, together with the time zone of the viewer.

How to answer. State the observed PE timestamp and qualify its reliability, for example: "The header reports a build timestamp of `<observed time>` in `<time zone>`; this is metadata and has not been independently verified."

> **From the lab (9 October 2026, 35:00)**
>
> PEStudio shows the value on its first page. The field is not needed to run the file, so an attacker can easily change it and some compilers do not write a meaningful value. Here the professor considers it plausibly written by the compiler. The same page gives the MD5 and SHA-1 to look up, and the signature `Microsoft Visual C++ v6.0`.

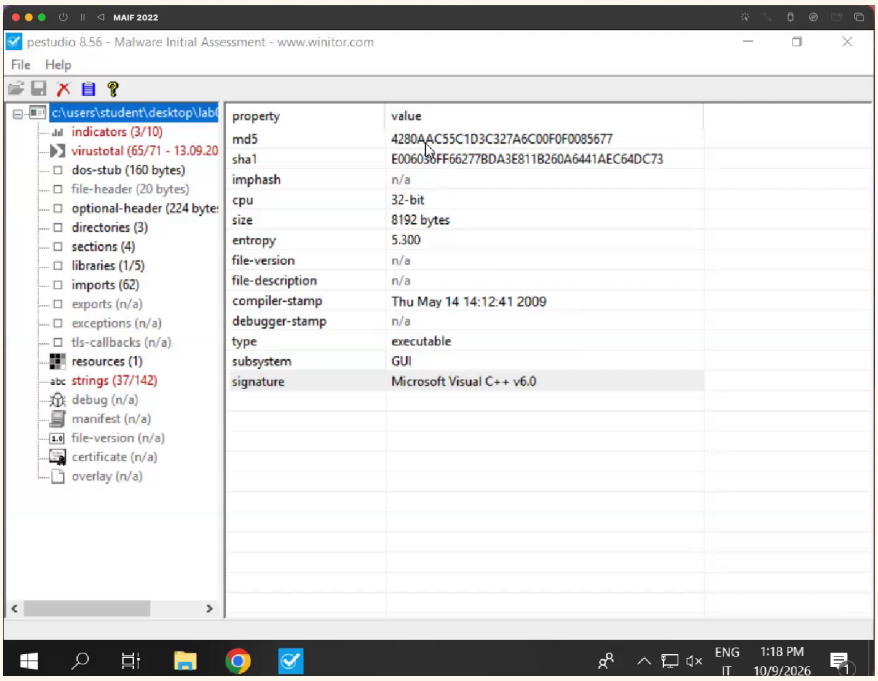


Figure 13.2: The first page of pestudio in the lab: same MD5 and SHA-1 as in the screenshot of Simone, but a compiler-stamp of 14:12:41 (lab recording, 9 October 2026).

Not from the lab: the same stamp, five hours apart

The screenshot of Simone at the start of this section shows 19:12:41 for the same file, while this one shows 14:12:41. The hash, date, minutes and seconds match. So the difference is most likely the time zone of the two virtual machines, which is why the answer must state the time zone of the viewer.

From the lab (9 October 2026, 35:00, sections)

The file has four standard sections, each with the characteristics you would expect.

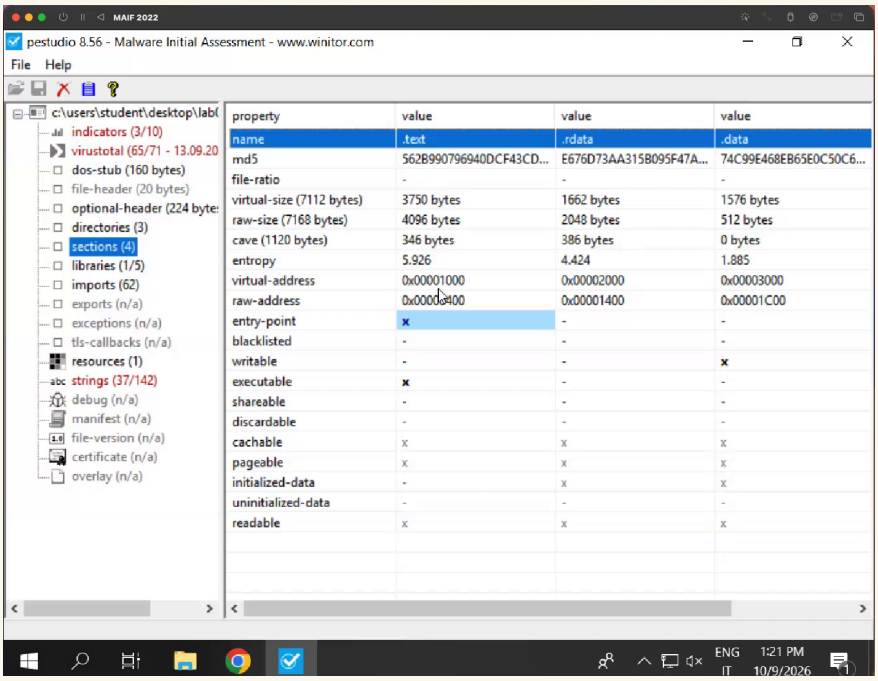


Figure 13.3: The sections of `LAB_03-1.EXE` in pestudio (lab recording).

> **From the lab (9 October 2026, 35:00, sections)**
>
> - **`.text`:** contains the entry point and is executable but not writable.
> - **`.data`:** is writable and not executable. Its virtual size (1576 bytes) is much larger than its raw size (512 bytes), because it holds data that exists only at runtime.
> - **`.rdata`:** is read-only and holds static data such as strings.
> - **`.rsrc`:** is almost empty, because the program contains no images or interface elements.

13.3.2 Question 2: imports

Hints in the PDF: look at the “Imports” tab, in particular `ShellExecuteExA`, which “launches other processes via the shell”.

> **Simone’s note (original wording)**
>
> libraries

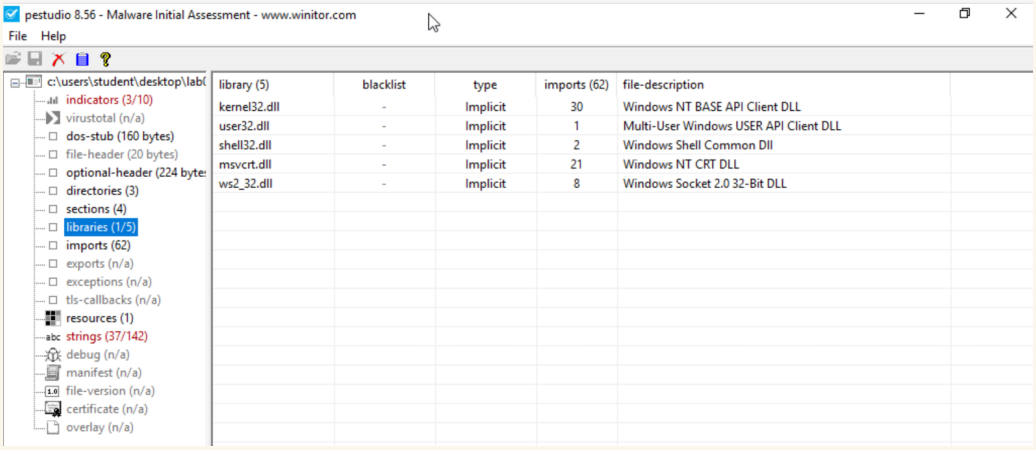


Figure 13.4: The libraries tab (screenshot by Simone).

The screenshot lists five implicitly linked libraries: `kernel32.dll` (30 imports), `user32.dll` (1), `shell32.dll` (2), `msvcrt.dll` (21) and `ws2_32.dll` (8).

> **Simone's note (original wording)**
>
> imports:

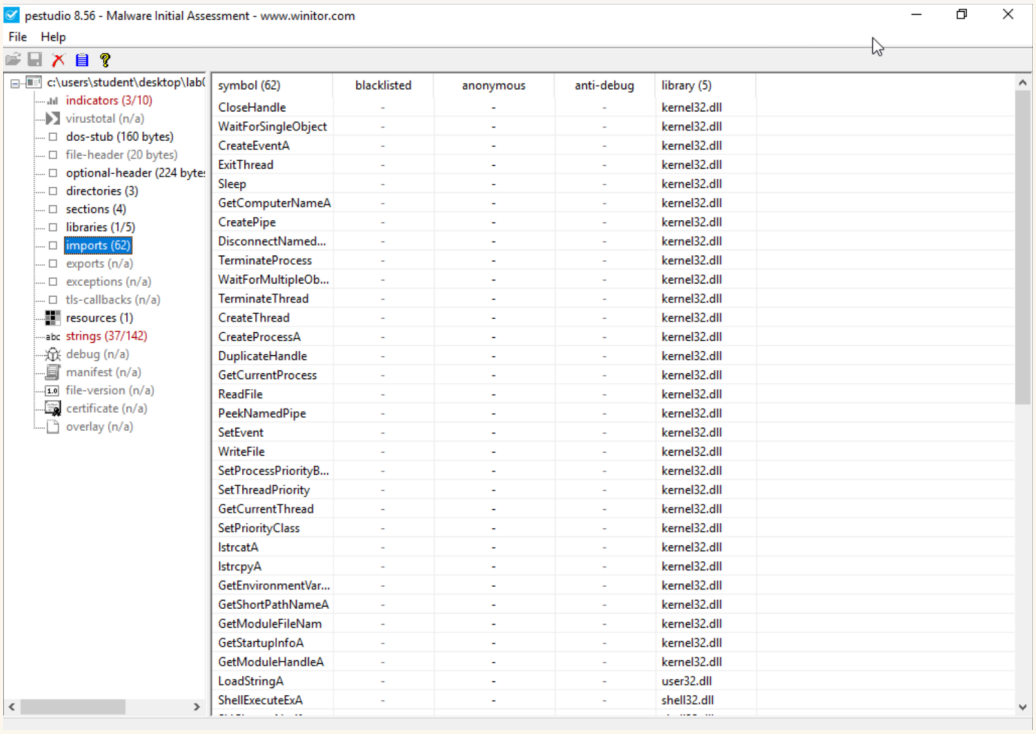


Figure 13.5: The imports tab, first part (screenshot by Simone).

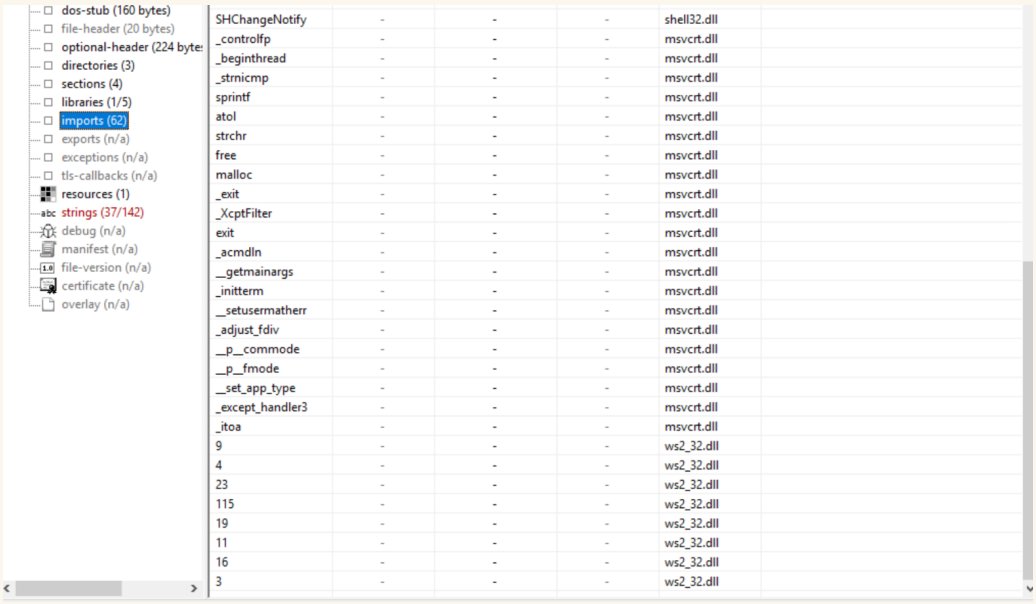


Figure 13.6: The imports tab, second part, ending with the numbered imports (screenshot by Simone).

The visible imports are the following. Names cut off in the screenshot end with an ellipsis.

Library	Imports visible in the screenshots
kernel32	CloseHandle, WaitForSingleObject, CreateEventA, ExitThread, Sleep, GetComputerNameA, CreatePipe, DisconnectNamed..., TerminateProcess, WaitForMultipleOb..., TerminateThread, CreateThread, CreateProcessA, DuplicateHandle, GetCurrentProcess, ReadFile, PeekNamedPipe, SetEvent, WriteFile, SetProcessPriorityB..., SetThreadPriority, GetCurrentThread, SetPriorityClass, lstrcatA, lstrcpyA, GetEnvironmentVar..., GetShortPathNameA, GetModuleFileNam..., GetStartupInfoA, GetModuleHandleA
user32	LoadStringA
shell32	ShellExecuteExA, SHChangeNotify
msvcrt	_controlfp, _beginthread, _strnicmp, sprintf, atol, strchr, free, malloc, _exit, _XcptFilter, exit, _acmdln, __getmainargs, _initterm, __setusermatherr, _adjust_fdiv, __p__commode, __p__fmode, __set_app_type, _except_handler3, _itoa
ws2_32	Numbers only: 9, 4, 23, 115, 19, 11, 16, 3

Simone's note (original wording)

we are going to see them and start to think about possible hipothesys and check later if we actually have found it. It is interesting also those last numbers in the end: someone in class gave an interesting point in a obfucscation point of view (i guess). possibly the attacker copuld remove also the dynamically linked table and real methos are linked at runtime and you cannot see them. It's interesting that in windows systems, processes and threads are managed in a completelly differettn way in compare to linux systems: the windows kernel simply works only on thread and processes are not actually used (see more in details by yourself). it is interesting also to notice that it can create and manage pipes and named

> pises that may be used for IPC or for shell pipes. Notice that in windows things are way too strange and heavy, and some imports are autocatically being listed but possibly are not used
>
> obviously we have many strange imports like ShellExecuteExA, and for each of them go to the microsoft documentation. possibly you should do it also to the other imports, but consider that sometimes you have to be quick, so for now focus on imports that really looks like malicious..

> **Correction**
>
> "processes are not actually used" is too strong. As seen in sections 3.6.1 and 9.7.3, the Windows scheduler runs threads, but processes still exist as the containers that hold the address space, handles and threads. The note is kept with its original wording; this nuance is the reading to use.

[**Addition, not from the course material:** The "last numbers" are imports **by ordinal**: the module is named but the functions are identified by number, so their names are not in the file. For Winsock, the usual mapping is 115 `WSAStartup`, 23 `socket`, 4 `connect`, 19 `send`, 16 `recv`, 3 `closesocket`, 9 `htons` and 11 `inet_addr`.]

An **import** identifies functionality that an executable obtains from another module. Imports help you infer what a program may be capable of doing. They are not a transcript of functions actually called, and legitimate applications use the same APIs.

The PDF points to **`ShellExecuteExA`**, a Windows Shell API that can perform an operation on a target, including opening or executing a file through the shell. Its presence makes process launching or shell-mediated execution a useful hypothesis. The A suffix identifies the ANSI version of the interface (section 9.5.1) and does not imply any malicious technique.

> **Simone's note (original wording)**
>
> see here the documentation `https://learn.microsoft.com/en-us/windows/win32/api/shellapi/nf-shellapi-shellexecuteexa`:

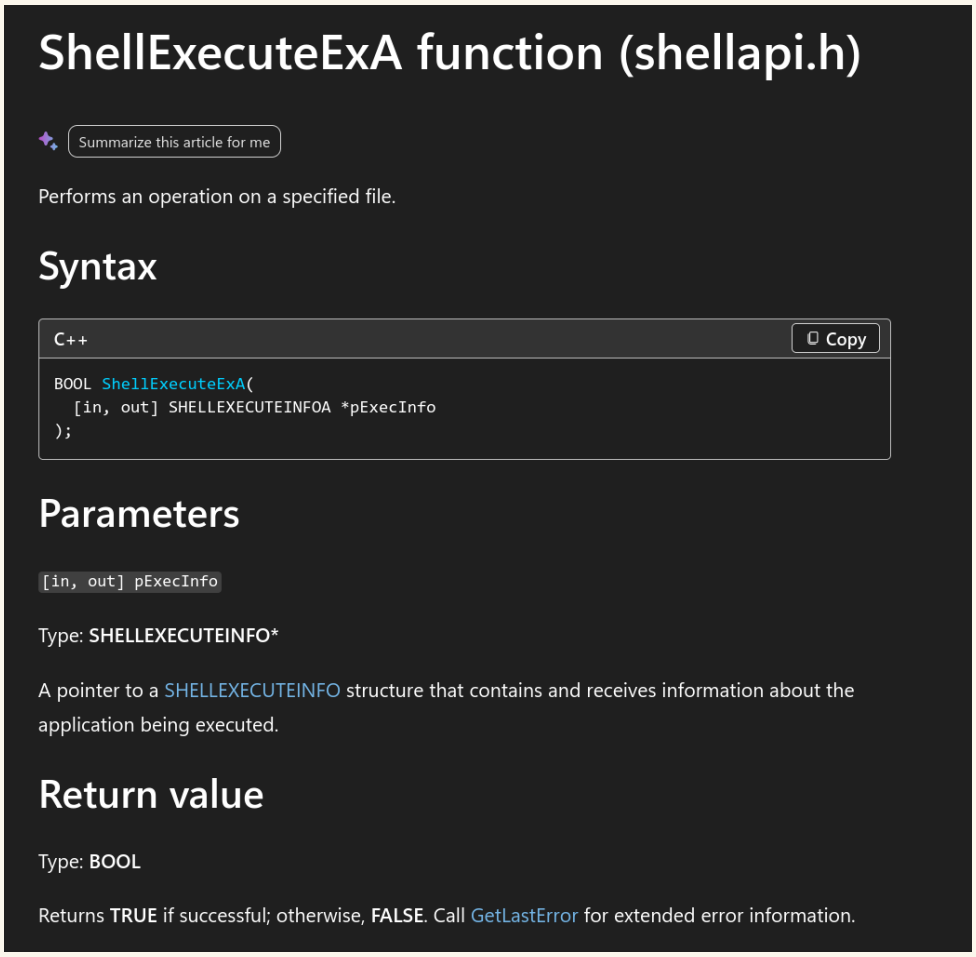


Figure 13.7: The Microsoft documentation of `ShellExecuteExA` (screenshot by Simone).

The page shows the signature `BOOL ShellExecuteExA([in, out] SHELLEXECUTEINFOA *pExecInfo);`. The function returns `TRUE` on success, otherwise `FALSE`, with details from `GetLastError`.

> **Simone's note (original wording)**
>
> not a lot of info. let's check that struckt type:

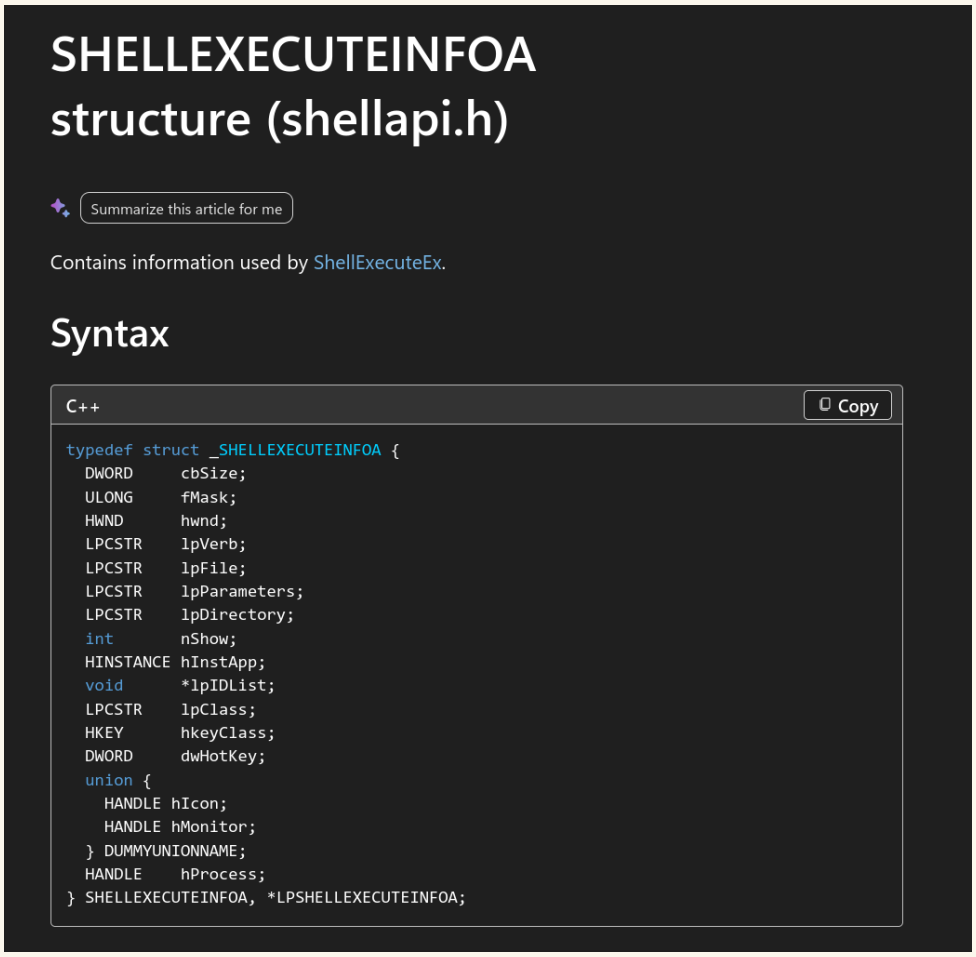


Figure 13.8: The SHELLEXECUTEINFOA structure in the documentation (screenshot by Simone).

The structure contains the fields `cbSize`, `fMask`, `hwnd`, `lpVerb`, `lpFile`, `lpParameters`, `lpDirectory`, `nShow`, `hInstApp`, `lpIDList`, `lpClass`, `hkeyClass`, `dwHotKey`, a union of `hIcon` and `hMonitor`, and `hProcess`.

> **Simone's note (original wording)**
>
> those are important because when we will start to reverse engeener something, we may be interested in analyse how the data structure is being filled. you can check each of them below the site, but don't loose into details: you have to be quick.

The hint directs attention to the shell as a way to start or open a target. The exact result depends on the operation, target, parameters and environment, and a target may be an executable or another file handled by shell associations. During dynamic analysis, inspect the launched image, the full command line, the

timing and the parent-child relationship. The import establishes a capability to investigate, while its arguments and the resulting activity establish how it is used.

The following groups are **examples to look for if present**, not an inventory of this sample.

Import group	Possible capability	Evidence needed
`ShellExecuteExA`, `CreateProcessA/W`	Launch a program or helper process	Process creation, executable path, arguments, parent-child relationship
`CreateFileA/W`, `WriteFile`, `CopyFileA/W`	Open, write or copy files	Exact path, operation result, written data or resulting file
`RegOpenKeyExA/W`, `RegSetValueExA/W`	Read configuration or change registry values	Key, value name, data, successful write, context
`socket`, `connect`, `send`, `recv`	Communicate through network sockets	Destination, timing, connection outcome, captured traffic
`LoadLibraryA/W`, `GetProcAddress`	Load modules and resolve functions at runtime	Which module and function names are resolved during execution
`OpenSCManagerA/W`, `CreateServiceA/W`	Interact with the service manager	Service configuration, referenced binary, installation and start outcome

Analyse **combinations**. File-writing APIs plus process-launching APIs suggest that the program might create and execute another component. Registry-writing APIs plus a startup-related path suggest possible persistence. These remain hypotheses until supported by context or execution.

A small import table does not establish that the program is simple: APIs may be resolved dynamically, hidden by packing or used by a loaded component. Conversely, an imported function may never be reached in the observed run.

A small behavioural map is more useful than a complete unannotated API list. Only the first API here is explicitly highlighted in the PDF; for the others, first verify that they are present.

```
Imported functionality -> possible capability -> runtime check
ShellExecuteExA        -> shell-mediated launch -> process
  tree and command line
File-writing APIs      -> file modification     -> successful
  writes and file state
Socket APIs            -> network communication -> connection
  attribution and packets
```

How to answer. For each selected import or group, explain the capability, propose a plausible use and name the observation that would confirm it. Avoid "it imports this function, therefore it definitely performs this action".

From the lab (9 October 2026, 40:05–68:35)

The sample imports 62 functions from 5 libraries. The number of functions per library is itself a clue: one from `user32.dll`, two from `shell32.dll`, and eight from `ws2_32.dll`, which the professor considers about what a single connection to a server needs. `msvcrt.dll` just reflects the Visual C++ compiler.

Separating signal from noise. Most of what a Windows program imports is standard runtime material. `CreateProcess` or `CreateThread` also appear in many benign programs, so the professor treats them as interesting only alongside stronger indicators. `ShellExecuteExA` is much rarer, which makes it worth following first. There is no manual for this skill: it comes from practice, and in the three hours of the exam you cannot afford to spend time on noise.

Reading the API name. `Ex` marks a newer version of a function that Microsoft kept alongside the older one. `A` means that text parameters are

expected in ASCII.

Correction

The professor says, hedging, that the Unicode variant ends in U. In the Windows API it ends in `W` (*wide*), as in `ShellExecuteExW`.

Correction

The professor describes `shell32.dll` as an interface to the shell, "the terminal". It is actually the library of the Windows graphical Shell. `ShellExecuteExA` opens or launches a target through that Shell, and it reaches a terminal only if the target is something like `cmd.exe`.

From the lab (9 October 2026, 40:05–68:35)

Following `ShellExecuteExA`. Its only parameter is a pointer to `SHELLEXECUTEINFOA`, so the useful documentation is the page of that structure. The fields that matter now are `lpVerb` (the action, such as `open` or `runas`), `lpFile` (the target) and `lpParameters` (its arguments). In later reverse engineering, you will see the disassembly filling these fields just before the call. Beyond this, the advice of the professor is not to get caught in the details: return to the documentation only when the analysis requires it.

The numbers at the end of the list. These are imports **by ordinal**: the function is identified by its number in the DLL instead of its name. It is a light obfuscation, because public tables map each number back to its function. A student summarizes them as initializing, opening, sending and receiving, which fits the mapping given earlier in this section (to verify before relying on it).

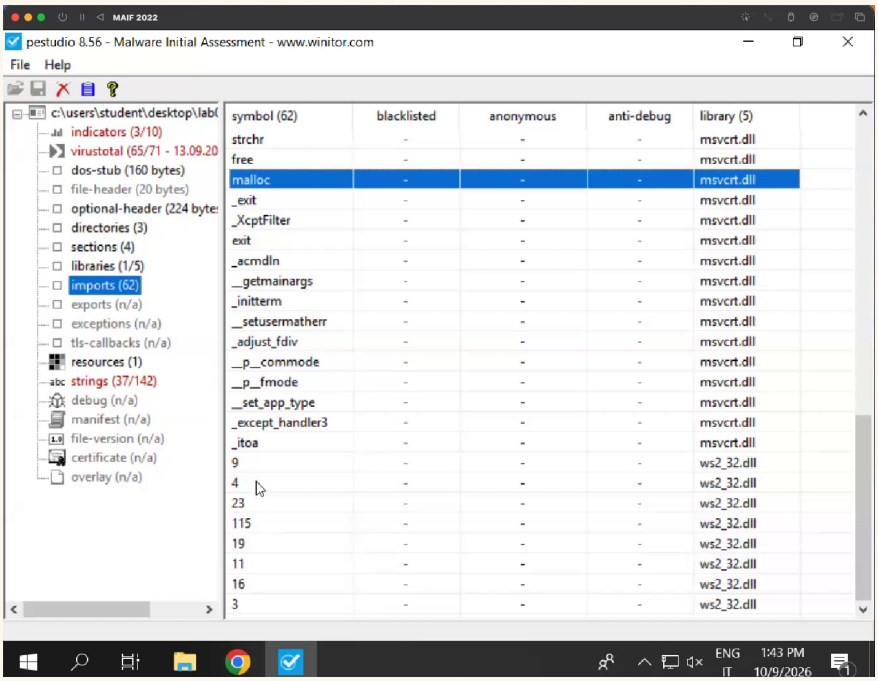


Figure 13.9: The imports by ordinal from `ws2_32.dll` at the end of the list (lab recording).

> **From the lab (9 October 2026, 40:05–68:35)**
>
> **Imports may be decoys.** An author can import functions that are never called, to add noise. A missing reference to the import table normally gives this away, but references can also be rebuilt at runtime. A student's hypothesis links the shell, pipes and network into a single flow. The professor calls it reasonable, while recalling that many other hypotheses fit the same imports and that the code must confirm them.

> **Correction**
>
> The professor says the Windows kernel "only works with threads" and that processes have no practical implication for it. That is an overstatement. Threads are indeed what gets scheduled, but a process is a real kernel object that owns the address space and the handles.

13.3.3 Question 3: strings

Hints in the PDF: look at the "strings" tab and notice `60.248.52.95:443`.

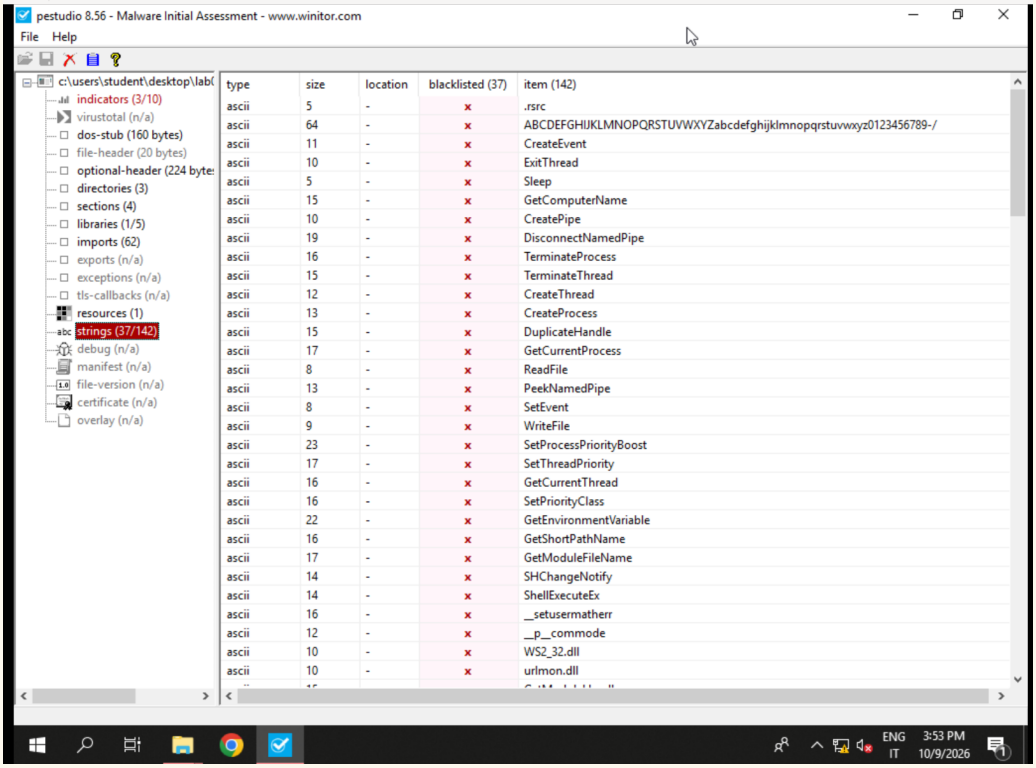


Figure 13.10: The strings tab, first part of the list (screenshot by Simone).

The visible part of the list shows `.rsrc`, a 64-character alphabet (`ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz 0123456789-/`), API names, `WS2_32.dll` and `urlmon.dll`. It does not show the address or `cmd.exe`, which appear further down the list, in the lab screenshots at the end of this section.

Strings are readable character sequences extracted from binary data (section 6.3). They can expose configuration, paths, command lines, domains, URLs, registry locations, messages and API names. Examine both ASCII strings and Windows UTF-16LE strings where the tool supports them.

> **Simone's note (original wording)**
>
> sometimes we can find a lot of mess numbers and chatcters: ignore them.

The PDF explicitly highlights `60.248.52.95:443`. This looks like an IPv4 address paired with a port number, so it may identify a destination used by the executable. Port `443` is commonly associated with HTTPS, but the number alone does not prove HTTPS, TLS, command-and-control traffic or even a successful connection. Programs can use arbitrary protocols on that port.

Possible interpretations include a configured server, a fallback destination, a test value or unused data. To distinguish them, look for whether the executable attempts a connection to that endpoint and what follows. The presence of the address is a **static indicator**, not yet an observed network behaviour. Preserve the value exactly, and do not contact the real endpoint to investigate it.

Other useful categories are executable names, temporary or system paths, startup-related registry paths, service names, URL paths, request headers and command-line switches. Classify strings by role, and pay attention to coherent sets, such as a destination plus request syntax or a file path plus a launch command. Explain why a string matters rather than reproducing a long list.

Consider false leads too: compiler runtime messages, standard library text and arbitrary bytes that happen to resemble text. If possible, record the offset or section of the string and whether code references it. Strings recovered after unpacking may be more informative than those in the original file.

Lack of readable strings does not imply lack of configuration, because data may be compressed, encoded, encrypted or constructed during execution. A string-heavy file is not automatically suspicious either.

> **Simone's note (original wording)**
>
> it could be interesting also the **cmd.exe** and we could guess of that program is being used in combination with other strings.
>
> The idea is to search those strings inside our malware when we will do reverse engineering of that.

How to answer. Give the exact string, its likely role, the reason it is noteworthy and the follow-up check. For the endpoint above, the follow-up is to correlate it with controlled network observations.

> **From the lab (9 October 2026, 68:35–88:58)**
>
> The `strings` functionality reads the file as a stream of bytes, without looking at its structure. That is why function names from the import table also appear here, and why many entries like `D$ RP` are just noise. The professor recommends ignoring the *blacklisted* column of PEStudio as

> unreliable.

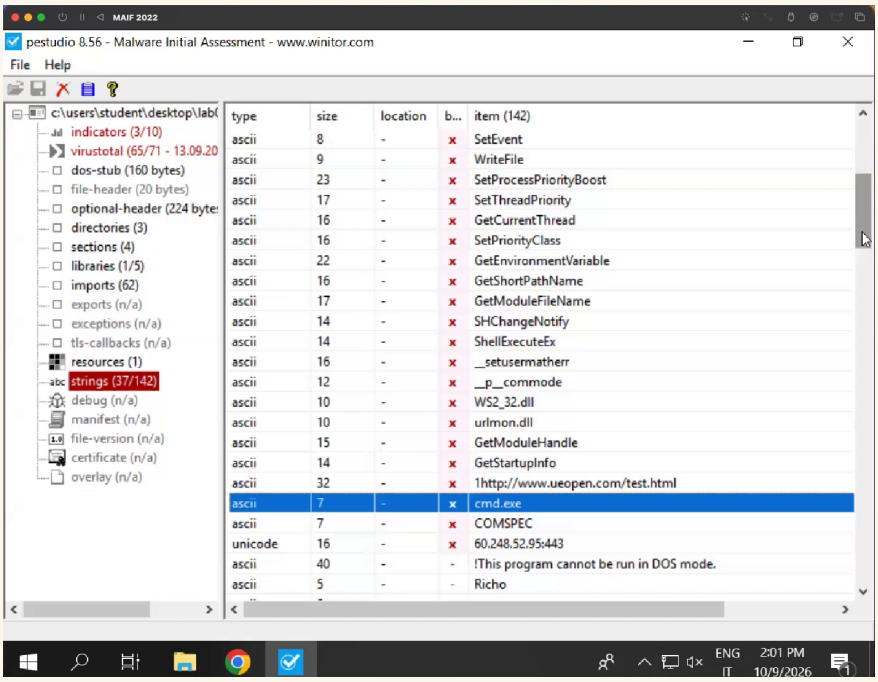


Figure 13.11: The strings tab in the lab, with `urlmon.dll`, the URL, `cmd.exe`, `COMSPEC` and the address (lab recording).

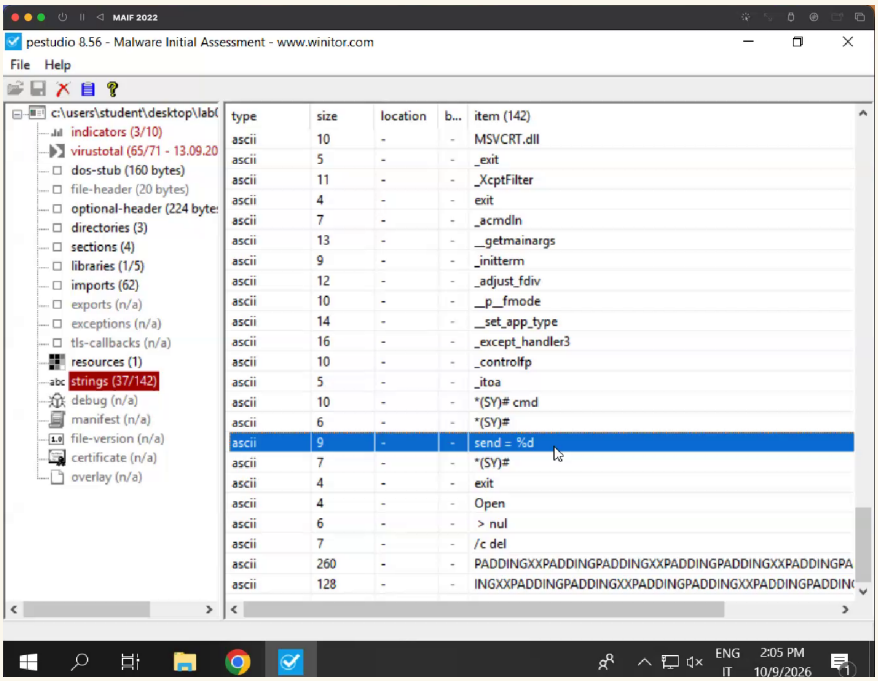


Figure 13.12: The strings tab further down, with `send = %d`, `> nul` and `/c del` (lab recording).

From the lab (9 October 2026, 68:35–88:58)

String	What the professor draws from it
`60.248.52.95:443`	Reinforces the network hypothesis from the imports and can be searched as an indicator of compromise. A student suggests that port 443 may mean a TLS connection
`urlmon.dll`	Another network clue. It is not in the imports, so it may be loaded at runtime, which basic analysis cannot check
`1http://www.ueopen.com/test.html`	A URL with an odd leading 1. The searches of the professor find nothing and the 1 stays unexplained
`cmd.exe`	A possible target of `ShellExecuteExA`, not a certainty. Opened normally it would show a prompt, so it must be used differently
`COMSPEC`	An environment variable pointing to the command interpreter, so possibly related
`/c del`	An argument for `cmd.exe` that deletes files and closes the prompt
`> nul`	Redirects output to nothing, like `/dev/null`
`send = %d, *(SY)#cmd`	Text with placeholders is typically used to build a string at runtime. Their meaning stays unclear

Why `cmd.exe` and `/c del` are separate strings. They would go into two different fields of the structure, `lpFile` and `lpParameters`. A third piece is missing, the file to delete, and no filename appears among the strings. It may be built at runtime. Only reverse engineering can settle this, by looking at the instructions that prepare the structure before the call.

Alphabet strings. A long string containing the whole alphabet usually implements an encoding such as Base64, or a quick obfuscation that shifts

characters. The professor does not commit to that here.

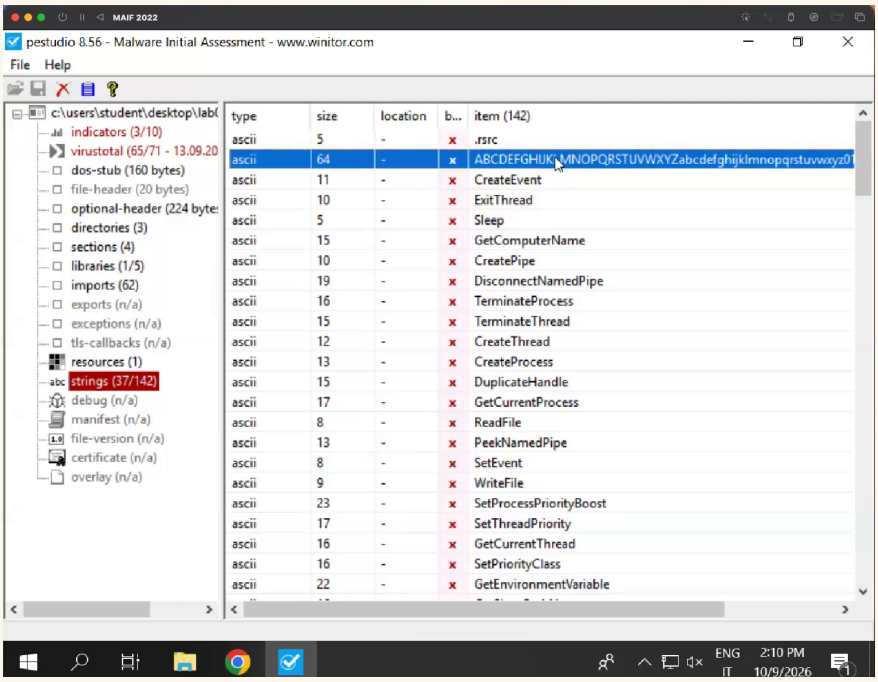


Figure 13.13: The 64-byte alphabet string selected in pestudio (lab recording).

Correction

The course note says that the length and order of this string fit the standard Base64 alphabet, reading only the part visible in this screenshot. The full string, in the screenshot of Simone at the start of this section, ends in `0123456789-/`. Standard Base64 ends in `+/`, so this is at most a variant with `-` in place of `+`. On-screen evidence alone does not show how the code uses it.

13.3.4 Question 4: running the sample

Hints in the PDF: use software to monitor the malware execution, for example ProcMon, and analyse the trace looking for actions; look for network activities and file-level activities.

Simone's note (original wording)

by clicking on that, it deletes itself: it explains why we have seen a string "del -r" or something similiar that could be used in combination with

> cmd.exe. this means that the malware wanted to hide tiself.

This behavioural observation comes from the note, not from the PDF, and the string is reported from memory ("or something similiar"). The exact string is `/c del`, as the lab screenshots above show. Before relying on the deletion, check the trace.

The question compares **predicted capability** with **observed behaviour**. Before execution, write a short hypothesis based on the imports and strings. Here the hints justify looking for shell-mediated execution and a possible attempt to use the highlighted endpoint, but they do not predict a confirmed outcome.

> **Step by step: A practical monitoring sequence**
>
> 1. Restore a clean snapshot and record the relevant environment.
> 2. Start Procmon and any required network capture before launching the sample.
> 3. Execute the sample through a controlled method and note the start time.
> 4. Observe its process tree and the relevant filesystem, registry and connection events.
> 5. Continue observing related processes for a stated interval.
> 6. Save the capture and collect resulting artifacts before resetting the machine.

During execution, record the process identity and command line, child processes, created or changed files, registry activity, network attempts, visible messages and exit behaviour. Include the observation interval and environment details such as Windows version and privilege level. The native Procmon trace preserves event details for later examination, and a screenshot can illustrate a finding but does not replace it.

Build a chronological account. A sequence such as "starts, writes a file, launches that file, then exits" explains more than a disconnected list of artifacts, and shows which process is responsible for each action. A program may show no

window while still working, and may terminate quickly after delegating work to a child, so keep monitoring related processes after the original exits.

Network activities. Search for connection attempts and data transfer. Procmon can correlate process activity and connection metadata, but it does not replace a packet capture for the contents. Distinguish name resolution, attempted connection, successful transport connection and meaningful application exchange. A program using a literal IP address may not need to resolve a domain. A quiet capture could mean that the path was not reached, the run ended too soon or the capture missed traffic, so it does not disprove network capability.

File-level activities. Inspect opens, reads, writes, renames, deletions and resulting files, and ask for each whether it succeeded and what changed. Link the role of a file to its later use: a file written and then executed is stronger evidence of a staged component than a file merely opened. Record the exact path, responsible process, time, size and hash. A component deleted quickly may still leave its path and lifecycle in the trace.

If expectations and observations differ, consider unreachable services, missing components, incorrect arguments, access restrictions, delays or a static clue that is never used. These are alternative explanations, not facts to assume.

How to answer. Describe the measured sequence, compare it with the initial hypothesis and state the boundary of the observation. “No successful connection was observed during a two-minute run” is more precise than “the malware has no networking”.

> **From the lab (9 October 2026, 88:58–91:46)**
>
> Someone in class reports that nothing happens, which the professor calls the common case with malware. What can be observed is that **the file deletes itself**. This fits `cmd.exe` with `/c del`, and it would also explain the missing filename: the sample may read its own name at runtime. Deleting itself matches a behavior seen in the lectures, hiding from the user. The hypothesis of the professor is a sample that stays active in memory and leaves no trace on disk, all still to be verified.

> **Not from the lab: other strings that fit**
>
> The strings also include `GetModuleFileName`, which returns the path of the running program, and `GetEnvironmentVariable`, which could read `COMSPEC`. Both fit the self-deletion hypothesis, but neither proves it.

13.3.5 Question 5: a Procmon filter

Hint in the PDF: filter out everything that is not directly or indirectly related to the EXE.

Process Monitor records Windows activity and lets you filter a large trace into a manageable view (section 8.2.1). A useful starting filter is the first one below, using the actual filename reported by the tool. A **PID** filter, the second one, distinguishes the instance you executed from another process with the same name.

```
Process Name | is | LAB_03-1.EXE | Include
PID | is | <observed PID> | Include
```

Filtering only the original process can hide behaviour delegated to children. Inspect the process tree and include relevant child PIDs. Where supported, use the process-tree inclusion option and verify what the resulting filter selects.

Direct activity is attributed to the process of the sample itself. **Indirect activity** may come from children, launched components or a service that carries out a requested action. Begin with the original process, then expand using process relationships, timing and matching paths or arguments. Attribution needs a plausible link, so do not include every background event just because it occurred during the run, and do not assume that every relevant event carries the name of the sample.

Further views may select a path or operations such as `WriteFile` and `RegSetValue`. Use them to answer a specific question, then return to the broader trace, and check that an additional condition does not accidentally remove relevant events.

Do not immediately exclude all failures. `NAME NOT FOUND` may reveal a searched-for configuration file, and `ACCESS DENIED` may explain an unsuccessful change. A failed attempt is useful evidence, though it does not prove the intended effect.

Prefer retaining a complete capture and using display filters during analysis. If a capture configuration discards filtered-out events, changing the filter afterwards cannot recover them, so verify the collection settings.

How to answer. Name the filter, explain the noise it reduces and identify its blind spot.

13.3.6 Question 6: host-based signatures

A **host-based indicator** is an observable artifact on the endpoint: a file, registry value, process, service or other local object associated with the sample. In this assignment, “signature” can mean such an indicator or a pattern built from several indicators. A formal detection rule requires further precision (section 2.1.2).

Look for files created or modified, distinctive command lines, registry writes, services and surviving components. Record exact paths, value names and data, file hashes and the responsible process. If the sample creates another executable, hash it separately from the original.

Distinguish **attempts** from **successful effects**. A `CreateFile` event can open an existing file, so the operation name alone does not establish creation: inspect its details and result, later writes and the resulting filesystem state. A successful registry read does not prove a modification either.

An indicator is more useful when it is specific. A common directory such as a temporary folder is weak evidence alone, while a distinctive hash, or an unusual service configuration combined with its executable path, is more discriminating.

Field	What to record
Artifact	File, registry value, process, service or other object
Exact identifier	Full path, hash, value name, service name or command line
Responsible process	PID, image path and relationship to the sample
Evidence	Tool event, operation result, resulting state, timestamp
Interpretation	Dropped component, configuration, persistence candidate or another role
Limitation	Possible benign overlap, environment dependence or incomplete attribution

How to answer. Report only artifacts actually found. If none were observed, specify the scope and duration of the run rather than claiming that none can exist.

13.3.7 Question 7: network-based signatures

A **network-based indicator** can be an address, domain, URL path, request pattern or distinctive application message. Separate the destination from the protocol and the content, because they answer different questions.

> **Step by step: Levels of evidence for `60.248.52.95:443`**
>
> 1. The endpoint appears in extracted strings.
> 2. The process attempts to connect to that endpoint.
> 3. A connection succeeds and data is exchanged.
> 4. The exchange contains evidence of a particular application protocol or role.

Each level supports a stronger conclusion than the one before. A connection attempt can be observed even when the remote service is unavailable. A

successful TCP connection still does not prove that the server issued commands or received stolen data.

Use process-associated connection observations for attribution, and a packet capture for the actual traffic. Record protocol, source and destination addresses, ports, times, outcome and visible application details. For encrypted traffic, a capture generally does not expose plaintext URLs or contents without additional authorized lab visibility.

Destinations can change, and an address can be shared or reassigned. Contextual patterns may have more detection value than an IP address alone. A destination in this historical lab is no evidence about its present-day owner or use.

How to answer. State whether an indicator comes from static inspection or a runtime observation, and report exactly what the evidence supports.

13.3.8 Question 8: what impeded the analysis

This question asks why your evidence may be incomplete, and what the controlled next step is.

Possible obstacle	Effect on analysis	Appropriate follow-up
Unreachable endpoint	The program may stop before revealing later behaviour	Observe connection attempts, and use controlled service simulation where appropriate
Missing runtime or DLL	Execution may fail during initialisation	Record the error and reproduce the required dependency in the lab
Insufficient privileges	Some file, registry or service changes may fail	Preserve failure evidence and compare with a justified privilege-level change
Wrong arguments or working directory	The intended code path may not execute	Inspect usage and configuration clues, and test a documented hypothesis
Delay or trigger requirement	A short run may miss activity	Extend observation, or test the suspected trigger in a fresh run
Packing or encoded strings	Static inspection may expose little	Identify the protection and inspect a recovered representation
Narrow filters	Child or delegated activity may be hidden	Revisit the complete capture and process relationships
Possible environment checks	The sample may behave differently in the lab	Seek evidence for the check before attributing inactivity to evasion

Change one relevant condition at a time, record it and restore a known baseline

between experiments. Otherwise you may not know which change caused the new behaviour.

How to answer. Describe the observed obstacle, the behaviour it prevented you from seeing and the experiment that could reduce the uncertainty. Do not use "anti-analysis" as a generic explanation for every quiet run (chapter 12).

13.3.9 Question 9: the purpose

Purpose is an inference drawn from the combined evidence. Common categories are the following, and a sample can have several roles.

- A **downloader** retrieves another component.
- A **dropper** releases an embedded component.
- A **backdoor** enables unauthorized remote control.
- A **stealer** collects information for export.

Distinguish a mechanism from a goal. Launching another process is a mechanism, while retrieving and executing a second stage is a possible role. Network communication is a mechanism, while command-and-control requires evidence of control-related interaction.

Construct a short argument: observation, inferred function, likely role and remaining uncertainty. An executable that is created and then launched supports a staging role. A network attempt with no meaningful exchange cannot yet be classified as command-and-control or exfiltration.

For this sample, the two explicit clues of the PDF alone support investigating **process execution** and **a possible configured network endpoint**. They do not establish a final classification.

How to answer. Give the narrowest purpose supported by the evidence and explain your confidence. If the later stage never ran, include that limitation.

From the lab (9 October 2026, 91:50–93:04)

Questions 6–9 belong to the following lab, because they need dynamic analysis. The professor gives only a first answer:

- **Question 6:** `cmd.exe` and `urlmon.dll`.
- **Question 7:** the URL, the IP address and the port.
- **Question 8:** the self-deletion, which is why the sample was not executed until the end.
- **Question 9:** too early to say.

13.4 LAB_03-2.EXE

The questions are the same as for the first sample, except that question 1 becomes: “What is the md5sum? What of interest does VirusTotal Report?”. The method of section 13.3 applies, so only what changes is described here. Repeat every step independently: similar questions do not mean similar behaviour.

13.4.1 Questions 1–3: hash, imports, strings

An **MD5 sum** is a 128-bit digest of the bytes of a file, usually displayed as 32 hexadecimal characters (section 6.2). Here it identifies the sample for comparison with existing reports. It does not describe behaviour or establish maliciousness. Compute it from the original executable, for example with the commands below; the filenames are illustrative.

```
Get-FileHash -Path '.\LAB_03-2.EXE' -Algorithm MD5
Get-FileHash -Path '.\LAB_03-2.EXE' -Algorithm SHA256

md5sum LAB_03-2.EXE
sha256sum LAB_03-2.EXE
```

The first two commands are for PowerShell, the last two for Linux. Preserve SHA-256 as well, because MD5 is unsuitable for collision-resistant security

decisions. A hash changes when the bytes change, including after unpacking or editing, so distinguish the hashes of original and derived files.

VirusTotal supports looking up existing reports by hash. Begin with a lookup, since uploading a file is a separate disclosure action that depends on whether the material may be shared. Interesting details include detection names and disagreements, file type, structural metadata, relationships to other files and behavioural observations. Note the report date: detection counts change, and another sandbox may produce different behaviour.

How to answer question 1. Give the measured MD5, preferably SHA-256 too, then summarise the relevant report evidence and its date. If no report is found, say so. No report, or no detections, does not establish that the sample is benign. The PDF supplies neither the digest nor a verdict.

Imports (question 2). Select groups that explain meaningful capabilities, and associate each with a testable hypothesis. Service-management imports justify inspecting service changes, but only a successful installation or configuration change supports a service-related finding. If the visible imports mostly resolve other functions, the table may expose a loader rather than the full program, so compare it with runtime modules and observed activity. List only functions observed in this executable, and keep external reputation separate from your own evidence.

Strings (question 3). Look for destinations, paths, commands, services, registry locations and object names, and for relationships among them. A service name beside a binary path suggests a configuration to investigate, not an installation. Do not carry `60.248.52.95:443` over from the first sample unless it appears here: shared course context is not shared evidence. A string seen both statically and in a runtime artifact is a correlation worth recording. A string never seen at runtime may belong to an unexecuted branch, an error message or unused data. Preserve spelling, and state whether runtime evidence supports the role.

13.4.2 Questions 4–5: execution and filter

Form a fresh prediction from this sample. Execute it from a clean baseline with monitoring active, then describe the observed sequence and all related processes. A reputation report that predicts a behaviour is a comparison point, not a substitute for your run. A difference may reflect OS compatibility, privileges, input, network availability, timing or different submitted bytes.

Be attentive to behaviour that moves beyond the initial process: a launched executable or installed service may continue after the original exits. Compare the initial hypothesis, any external report and your own measured behaviour, without forcing the trace to fit a preselected label.

For the filter, start with the actual process name or PID and add related processes when the tree links them. A focused operation filter can then investigate a suspected mechanism such as registry modification:

```
Operation | is | RegSetValue | Include
```

Check how it combines with your existing rules. It finds writes, but cannot say whether the sample also writes files or communicates. State the exact filter, its purpose and scope, and how you checked for activity outside that view.

13.4.3 Questions 6–7: signatures

Host. Collect successful file changes, registry writes, process launches, services and other distinctive local artifacts. Compare the state before and after execution, then tie the differences to the trace (section 8.6). A state comparison says that something changed, and the trace helps establish when, how and by which process. A generic filename or a coincidental background change should not become a definitive indicator without attribution. For services, record the exact name, configured executable, start configuration and installation outcome: the ability to create a service and a successful creation are different levels of evidence.

Network. Inspect destinations, name-resolution requests, connections and visible application messages for this executable and its components. Correlate

timing and process identity rather than attributing every packet in the VM to the sample. A remote endpoint may support a download, a connectivity check, reporting, command retrieval or another function, and protocol content and action sequence help distinguish these. A reproducible lab-side response can reveal whether later behaviour depends on communication (section 8.8.4). Document it as simulated, not as a genuine response of the original service.

13.4.4 Questions 8–9: obstacles and purpose

Identify the specific point at which progress stopped. Was the executable unable to initialise, unable to write a target, waiting for input or trying an unavailable service? Look for errors and failed operations before proposing explanations. Use targeted experiments: a longer interval for suspected delay, a compatible lab OS for a compatibility issue, a controlled network response for a demonstrated dependency. If basic analysis cannot resolve the obstacle, state what deeper investigation would need to establish. Recognising a boundary is part of a useful report.

For the purpose, combine static clues, your own execution evidence and external reporting, keeping their provenance clear, and base the final role on the observed chain of actions. Receiving and executing a second component supports a downloader or loader role. A persistent process alone does not prove remote control. Contact with a server alone does not prove theft of data. Include several roles only when each has support.

13.5 LAB_03-3.EXE

The assignment asks eight questions:

1. Are there any indications that this malware is packed? What are they? What is it packed with?
2. Are you able to unpack it? Why or why not?
3. What are a few strings that stick out to you and why?

4. What happens when you run this malware? Is it what you expected and why?
5. Are there any host-based signatures? (Files, registry keys, processes or services, etc). If so, what are they?
6. Are there any network-based signatures? (URLs, packet contents, etc) If so, what are they?
7. Is there anything that impeded your analysis? How so? How might you overcome this?
8. What do you think is the purpose of this malware?

13.5.1 Question 1: indications of packing

Packing transforms a program into a representation that is restored or decoded before its main logic runs (section 6.4). Often a small **unpacking stub** executes first and reconstructs code or data in memory. Packing can reduce size and obscure static analysis, and legitimate programs can be packed too.

Look for several mutually supporting clues:

- unusual section names or section layout;
- high entropy in sections with data that might be compressed or encrypted;
- a sparse import table focused on loading, resolving or allocating memory;
- few readable strings despite substantial executable content;
- an entry point associated with a small decoding stub;
- a packer identification reported by a file-inspection tool.

Entropy measures the distribution of byte values, with a maximum of 8 bits per byte for byte-oriented data. High values are compatible with compressed or encrypted data, but also occur in legitimate resources, so entropy is supporting evidence, not a diagnosis.

Names such as UPX0 and UPX1 can suggest UPX when present, but section names can be altered or imitated. Treat the label of a detector as a hypothesis, to corroborate with structure or with a successful packer-aware inspection.

How to answer. List the observed signs, explain their combined significance and distinguish a suspected packer from a confirmed one. The PDF does not name the packer of this sample.

13.5.2 Question 2: unpacking

> **Simone's note (original wording)**
>
> before doing it we could possibly use "Detect it easy v3" which is a tool to check if an known packer is being used and actually it is modified because the attacker modifies upx1=lol1 for instance and that's bad because we cannot use upx to unpack it. What is the Entropy when we check it inside pestudio? "how clode those bytes are to look like random?" if it is high, well it could possibly mean that we have an encryption. In detect it easy we can check the Entropy

The objective is to obtain a representation that exposes more of the real code and data. Preserve the original, work on a separate copy, and record hashes for both. If evidence identifies a supported UPX-packed executable, an example workflow is:

```
upx -t LAB_03-3.EXE
upx -d -o LAB_03-3-unpacked.exe LAB_03-3.EXE
```

These commands are conditional examples, not evidence that UPX applies here. The test concerns the integrity of the packed data of UPX, and it is not a malware-safety check. The explicit output path keeps the original separate from the derived file.

Validate the result by checking its PE structure, import visibility, strings, entry point and usefulness to the analysis. A larger output or a successful tool message alone does not establish that every layer was recovered. The recovered program remains potentially malicious.

Unpacking can fail because the format is unsupported, the packer was misidentified, headers were modified, the file is damaged or several layers exist. When static recovery fails, deeper analysis may observe the reconstruction at runtime and recover memory after unpacking (chapter 11). Turning that memory into a useful PE can require rebuilding imports and other metadata.

How to answer. Identify the method, tool output and validation result. If it failed, report the specific error and the hypothesis it supports, rather than asserting that the file cannot be unpacked.

13.5.3 Questions 3–4: strings and execution

Inspect strings in both the original and any recovered representation. The packed file may expose mostly stub text, while the unpacked image reveals configuration, commands, paths or networking details. Label each finding by provenance: original file, unpacked file or runtime memory. A recovered string was not necessarily visible in the original on-disk sample. Even after unpacking, individual values can remain encrypted or be constructed later, so few strings do not automatically mean that unpacking failed.

For the primary behavioural run, observe the **original** sample from a clean snapshot. If you also run a recovered version, label it as a separate experiment, because recovery can change loading behaviour or omit dependencies on the original layout.

Packing may hide the main code on disk while its eventual actions remain visible in dynamic analysis, so do not judge the program by its sparse original import table. A run may also fail before the protected code is reconstructed: record startup errors or early termination, and do not equate that with absence of malicious behaviour. Compare the observed sequence with hypotheses from before and after unpacking, keeping results for different file versions separate.

13.5.4 Questions 5–6: signatures

Host. Identify artifacts attributable to the executed sample. A packed program can reconstruct its payload in memory without writing an unpacked EXE to disk, so the absence of such a file does not disprove unpacking.

> **Do not confuse**
>
> An **analyst-generated unpacked file** is not a **file dropped by the malware**. Only the latter is an artifact of the execution. In the same way, the packed hash and the derived unpacked hash identify different byte sequences and must not be interchanged in a detection report.

Network. Observe whether the unpacked logic communicates, and correlate traffic with the process that performs it. Packing can hide a destination from static extraction without preventing it from appearing at runtime. If a recovered string matches a connection, that is a useful correlation, but the purpose of the traffic needs separate evidence. When packet content is unavailable or encrypted, document the metadata rather than inventing request paths or commands. If no activity appears, retain the observation interval and any errors that may have prevented it. State whether the endpoints were visible before or after unpacking.

13.5.5 Questions 7–8: obstacles and purpose

Explain which layer of the investigation was blocked. Packing may restrict static visibility yet leave dynamic behaviour observable. An unavailable remote dependency may permit unpacking but prevent later functionality. These obstacles require different next steps. If recovery fails, preserve the original, document the tool error and investigate whether the protected code becomes available during execution. If the program runs but does little, test demonstrated dependencies before assuming environment detection. Note when the task moves beyond basic analysis into debugging or memory reconstruction.

Classify the role of the underlying program from its capabilities and behaviour.

“Packed executable” describes its representation, not its purpose, and packing alone supports no particular classification. Use recovered imports and strings for hypotheses, and runtime evidence to distinguish retrieving another component, installing one, communicating for control, collecting information or another function. If only the unpacking mechanism was understood, state that the purpose of the payload remains unresolved.

13.6 LAB_03-4.EXE

The first six questions match those of the third sample. The seventh is new: “Is the malware performing actions to persist in the target systems? Which strategy does it use?”. It is also the end of the PDF, which does not provide the strategy.

13.6.1 Questions 1–2: packing

> **Simone’s note (original wording)**
>
> here we will see that it is again packed, but possibly it is not being modified (it can be but it looks like it is not), and it looks like that it has been packet with UPX. so simply we use upx to decompress it: for that purpose, create a backup of that file because upx actually overwrites the original file. once it finishes it, you upload it into pestudio and you check it.

The note reports what was seen in class. Still, inspect this executable independently, and do not assume the same packer as the previous sample merely because the questions are similar. Combine section structure, entropy, imports, strings, entry-point context and detector output. Also distinguish whole-image packing from a program containing a compressed embedded payload: a readable outer executable can still carry protected inner content. If evidence only establishes protected content, avoid naming an unverified packer.

Choose the unpacking method from this evidence and preserve the original. The `-o` option seen in section 13.5.2 avoids overwriting it, and the backup in the note reaches the same goal. If a standard unpacker is appropriate, record its exact

result and validate the recovered structure. If protection is layered, note which layer was removed and which remains.

For an embedded payload, recovery may expose a separate component rather than turn the outer executable into an ordinary unpacked image. Keep the original, recovered executable, embedded component and memory dump clearly distinguished, each with its own identifier where applicable. State what was recovered, how you verified its usefulness and what is still protected. A partial recovery can be useful without being complete.

13.6.2 Questions 3–4: strings and execution

Inspect the original and recovered content for behavioural clues and for **persistence-related candidates**: startup paths, service configuration text, task names or registry locations. The final question makes persistence an explicit target, but does not supply a finding. A registry path or service name is only a candidate until correlated with a successful action, because it may appear in a check, cleanup routine, error message or unused branch.

Capture ordinary execution, and inspect whether the sample establishes a way to run again. Relevant changes may appear during the initial run even though the automatic execution requires a later login, reboot or another trigger. Record the process sequence and the state left behind. If the initial executable exits but a service or component continues, identify which process remains and why. Continued execution in the current session is not proof of persistence across restart. If testing a trigger, preserve evidence from the initial run, document the trigger and keep monitoring the later phase.

13.6.3 Questions 5–6: signatures

Host. Record all attributed local artifacts, with attention to linked components. A registry startup entry and the file it references form a more explanatory finding than either alone. Verify successful writes and resulting state: configuration pointing to a nonexistent or invalid target may be a failed attempt rather than a working mechanism.

Mechanism	What to record
Service	Exact name, binary path, arguments, account, startup configuration where available
Scheduled task	Task path, action and trigger
Startup file	Location and target

Network. Inspect traffic both during the initial execution and during any tested later trigger. A persistent component might communicate immediately, at a delayed interval or only after it starts again. Record the responsible process, and distinguish installation-time communication from communication by the surviving component. Persistence can exist even when communication fails, so a persistent program need not use command-and-control.

13.6.4 Question 7: persistence

Persistence is a mechanism that allows a malicious component to execute again after the initial launch or across a lifecycle event, such as a new login or a restart (section 8.7). A long-running process is not sufficient evidence. Investigate both the **configuration** and the **target it launches**.

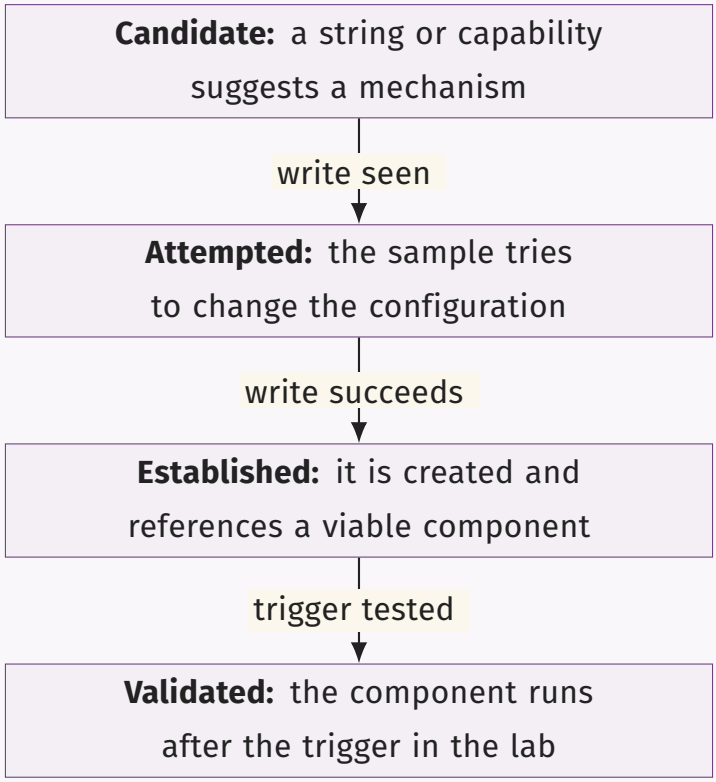
Candidate	What to inspect	Why it enables future execution
Registry Run	Successful value creation, executable path, user or machine context	A configured command can run at user login
Registry RunOnce	Exact value and its lifecycle	Can arrange execution at a later login, normally once
Startup folder	Created executable or shortcut, and its target	The startup item can be launched at login
Windows service	Service configuration, payload path, startup type, trigger conditions	The service manager can start the configured component
Scheduled task	Registered action, trigger, account, resulting state	The task scheduler can launch the component under its conditions

These are examples, not findings about LAB_03-4.EXE. The answer must come from its trace and resulting state. For registry-based candidates, locations often examined include:

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
HKLM\Software\Microsoft\Windows\CurrentVersion\Run
HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce
HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnce
```

A path in the strings, or a registry read targeting it, does not establish persistence. Look for a successful write, the exact value data, a viable referenced component and the relevant trigger. User context, privileges and registry view can affect the interpretation.

Flowchart: A graded conclusion on persistence



Each step needs stronger evidence than the one before. The strongest explanation links three elements: **what was configured**, **which payload it launches**, and **when or how that launch occurs**. For example, report the exact startup value and command, the identity of the target executable and whether execution at the next login was observed. If only installation was confirmed, say that activation remains untested.

How to answer. Name the strategy actually observed, give the supporting configuration and payload details, and distinguish an attempted mechanism from a successfully established and tested one.

13.7 Technical references for the lab

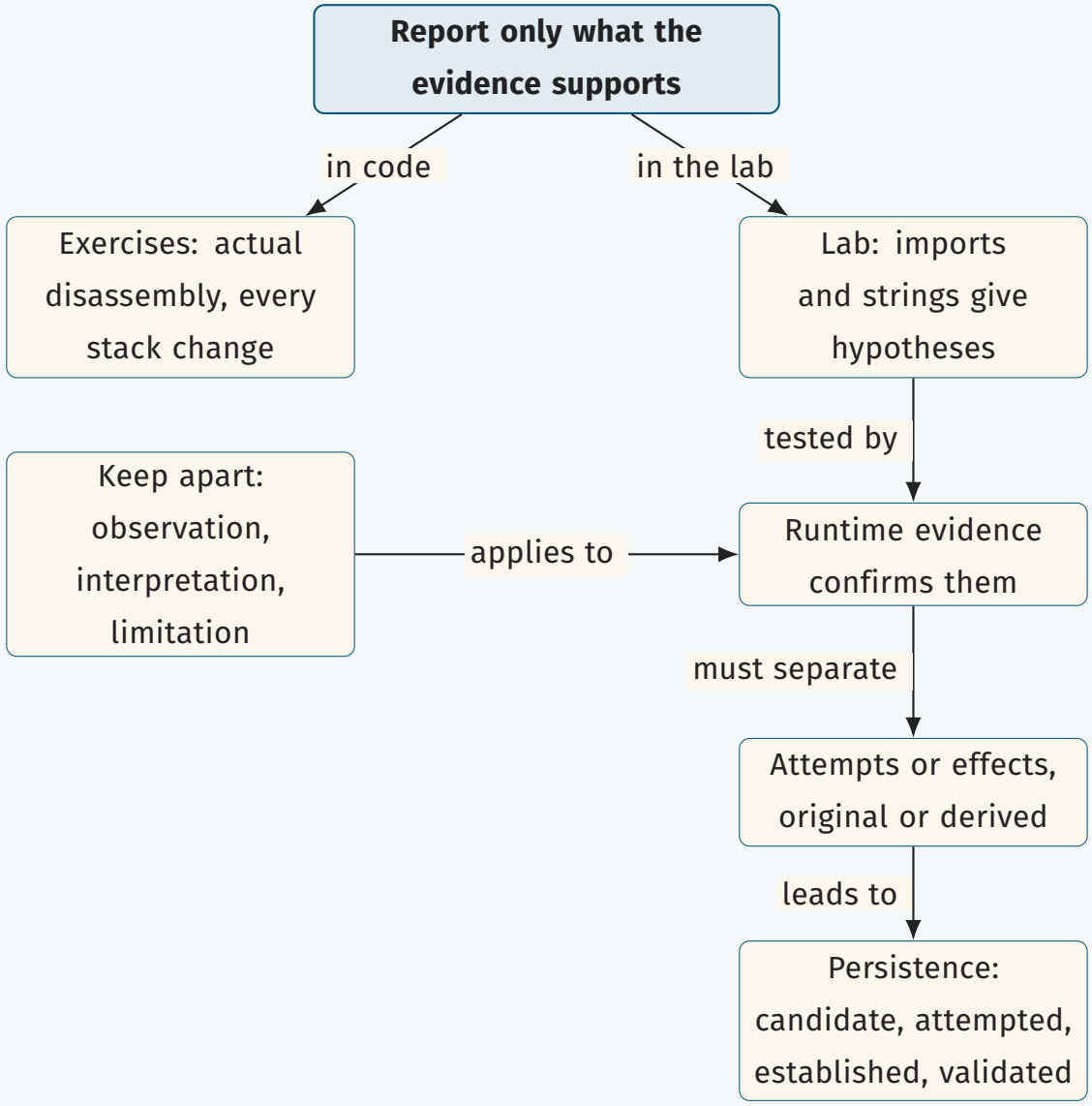
The PDF supplies the exercise structure and the two explicit clues for the first sample. The course notes cite these primary references for selected clarifications. They are not reports about the four executables.

- Microsoft, PE Format: `https://learn.microsoft.com/en-us/`

windows/win32/debug/pe-format

- Microsoft, ShellExecuteExA: https://learn.microsoft.com/en-us/windows/win32/api/shellapi/nf-shellapi-shellexecuteexa
- Microsoft Sysinternals, Process Monitor: https://learn.microsoft.com/en-us/sysinternals/downloads/procmon
- VirusTotal, Searching: https://docs.virustotal.com/docs/searching
- UPX, official documentation: https://github.com/upx/upx/blob/devel/doc/upx-doc.html
- Microsoft, Run and RunOnce Registry Keys: https://learn.microsoft.com/en-us/windows/win32/setupapi/run-and-runonce-registry-keys
- Microsoft, Service Programs: https://learn.microsoft.com/en-us/windows/win32/services/service-programs
- Microsoft, Task Scheduler: https://learn.microsoft.com/en-us/windows/win32/taskschd/task-scheduler-start-page

Concept map: how the ideas connect



The same rule holds in both parts: report only what the evidence supports. In the exercises, this means working from the actual disassembly and accounting for every stack change. In the lab, imports and strings give hypotheses, and only runtime evidence confirms them. Evidence must separate attempts from effects and original files from derived ones. This is what makes the four grades of persistence possible.

In short

- In the assembly exercises, work from the actual disassembly and account for every stack change, including return addresses.

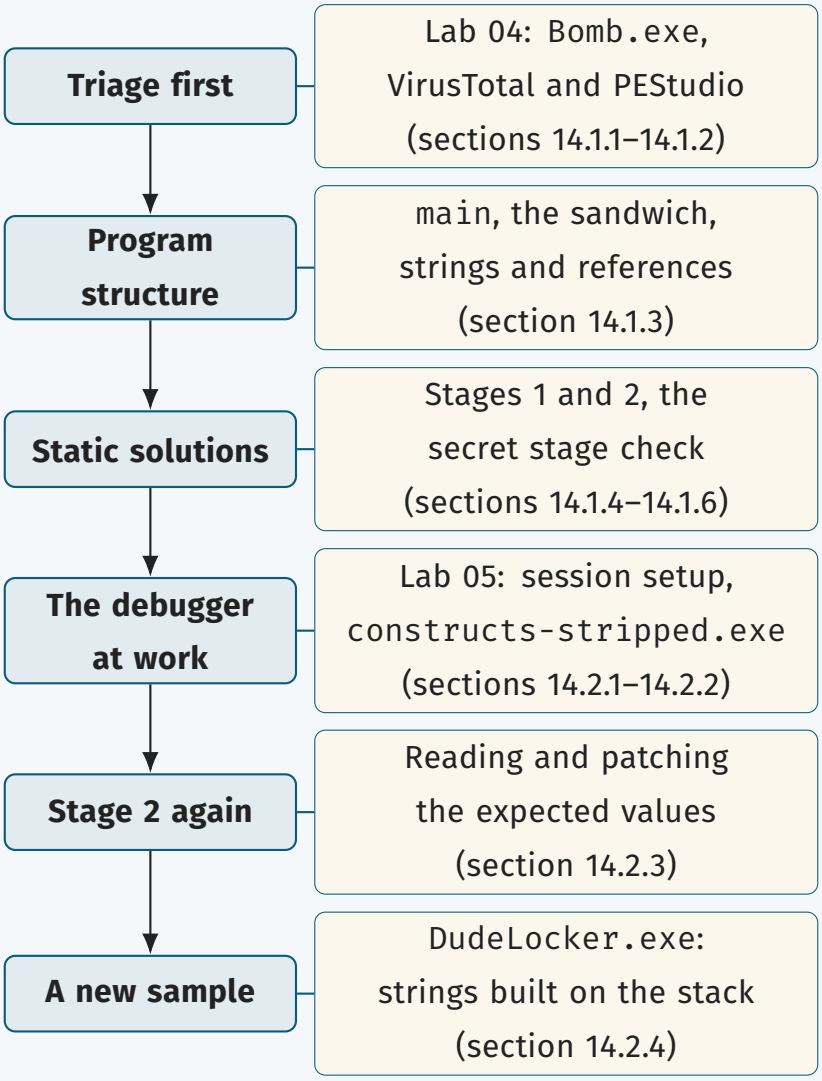
- In the lab, keep observation, interpretation and limitation separate.
- Imports and strings produce hypotheses, and only runtime evidence confirms them.
- Distinguish attempts from successful effects, and original files from derived ones.
- Persistence has four grades: candidate, attempted, established and validated.

Chapter 14

Labs 04 and 05: reversing and debugging

> **Before you start**
>
> **Central question:** how do you take an unknown binary apart in IDA, and what changes when you can also run it under a debugger?
>
> **Prerequisites:** IDA (chapter 5), Windows APIs and C constructs (chapters 9 and 10), the debugger (chapter 11).
>
> **Where it leads:** `DudeLocker.exe`, started here, continues in the next lab.

Chapter structure: what comes, and in which order



The chapter follows the two labs in order. Lab 04 solves the first stages of a puzzle by reading the code only. Lab 05 goes back to the same puzzle with the debugger, so you can compare the two ways of working on the same code. It then starts a new sample.

Remember

What the material contains. This chapter comes entirely from the recordings of the labs of 30 October and 6 November 2026, because the slides of these labs are not in the material. Addresses and values are as shown on screen, and the comments in the screenshots were typed by the professor during the lab.

14.1 Lab 04: the logic bomb

This lab applies the workflow of chapters 9 and 10 to `Bomb.exe`: a harmless command-line puzzle where each stage expects the right input, or the "bomb" blows up. It is a Windows variant of the logic bomb from Carnegie Mellon University. The goal is to show that apparently complex code can be dissected statically with IDA. The recording covers the basic triage, the program structure, stages 1 and 2, and the secret stage check.

> **Remember**
>
> **Try it yourself first.** The professor asks students not to look for write-ups or ask AI to solve it, because that teaches little. Stages 1 and 2 are solved below, so try them first if you want the exercise. Stage 3 onward is left to you.

14.1.1 Setup

The archive `Lab04-reverse_code` is on Classroom, and its password is `infected`. Its headers are encrypted, so the preview looks empty. The sample is not malicious, but the professor recommends opening a command prompt, moving to the folder and starting with a simple run, to see how it takes input.

14.1.2 Basic static analysis first

VirusTotal. Searching the hash finds no detections from 72 products.

> **Remember**
>
> **Search the hash, do not upload the file.** When working on real malware, uploading tells the attacker that their file reached you, since they can look up the hash. Anyone with a subscription can also download what you upload, which is a serious problem in a sensitive organization.

PEStudio gives these clues.

Where	Observation	What it suggests
Compiler stamp	2010, 32-bit	Can be forged, so it is only a hint
Manifest and libraries	Visual C++ runtime `MSVCR90` (Visual Studio 2008)	An old compiler, which can give leaner executables
Imports	35 from the C runtime, 13 from `kernel32.dll`: `fgets`, `sscanf`, `atoi`, `isspace`, `fopen`, `getenv`, `printf`	Reads and parses text. No network, crypto or process creation
`IsDebugger Present`	Present	Not suspicious: nearly every Windows program imports it
Strings	Path to a `.pdb` file under `Documents and Settings\user\Desktop`	A leftover from the build. In malware this can reveal usernames or paths
Strings	`Usage: %s [<input_file>]`	The input may come from an optional file
Strings	Stage messages, `%d %c %d`, a format with six `%d`, a "secret stage", `austinpowers`	The structure of the program and the expected inputs

The professor first calls this triage inconclusive, then changes opinion thanks to the strings: they give the structure of the whole program.

14.1.3 Program structure in IDA

Finding `main`. Here IDA finds it by itself. To verify, start from `start`, skip `__security_init_cookie`, which sets up the stack protection, and follow the jump to the C runtime startup. The two paths at the end lead to `exit`, so `main` is one of the last calls before them, and it is the one with three pushed arguments (compare section 5.4.1). When opening the file, answer no to the search for the PDB, because you do not have it.

The argument check. `main` compares `argc` with 1 (no arguments) and then with 2 (one file, opened with `fopen`). `argv+4` is the first real argument, since each pointer is 4 bytes. Three or more arguments print the usage, and a missing file prints an error. Each hypothesis can be tested by running the program with those arguments.

The big block. Scanning the single long block in `main` shows the strings found earlier, in order.

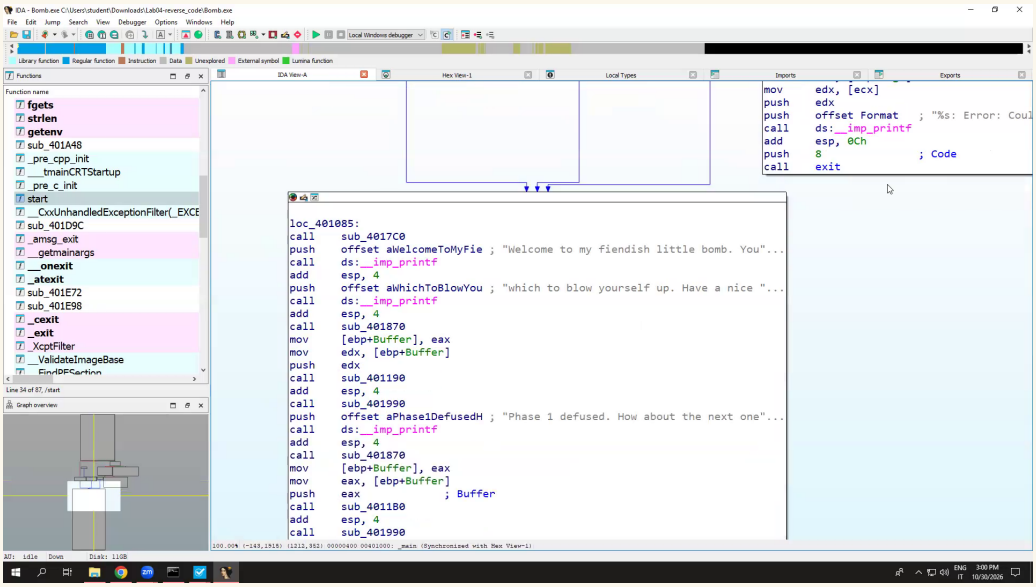


Figure 14.1: The long block of `main` in `Bomb.exe`, with the welcome messages and the first stage (lab recording, 30 October 2026).

The sandwich. Clicking a name highlights its other occurrences. `sub_401870` and another function each appear 6 times, around a function that changes every time. So each stage has the form: read the input, check it with a stage-specific function, then call a common function.

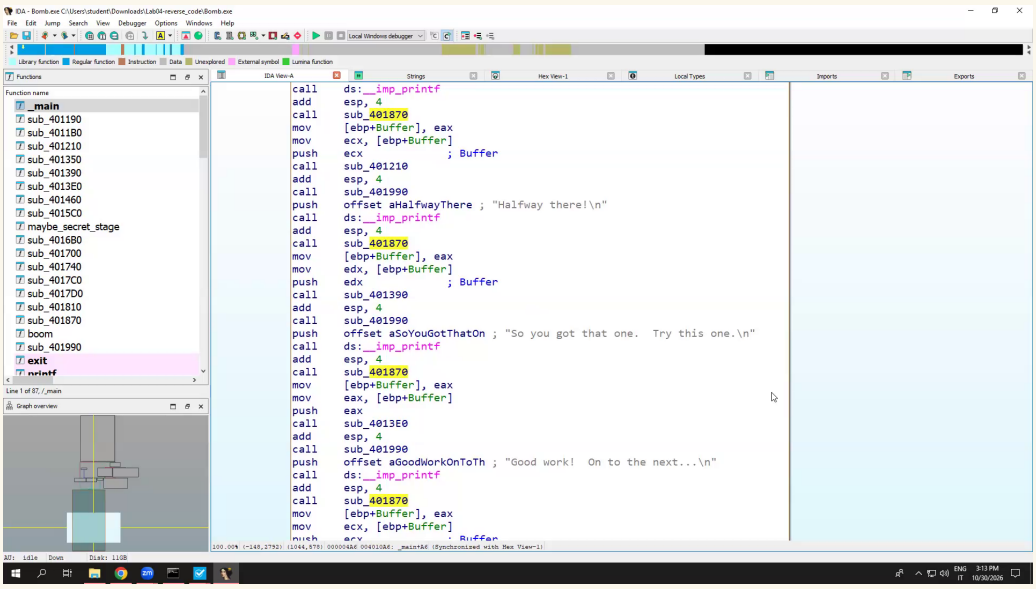


Figure 14.2: The repeated calls highlighted in `main`: each stage is a sandwich around its own checker (lab recording).

The value returned by `sub_401870` is pushed as the argument of the checker, and IDA guesses it is a buffer. So `sub_401870` becomes `getinput`. The professor uses the decompiler only to confirm this, not for the logic.

Starting from a string. Open *View* → *Open subviews* → *Strings*, click a string, then select its name and press X for the references. Press X on the name, not on the address.

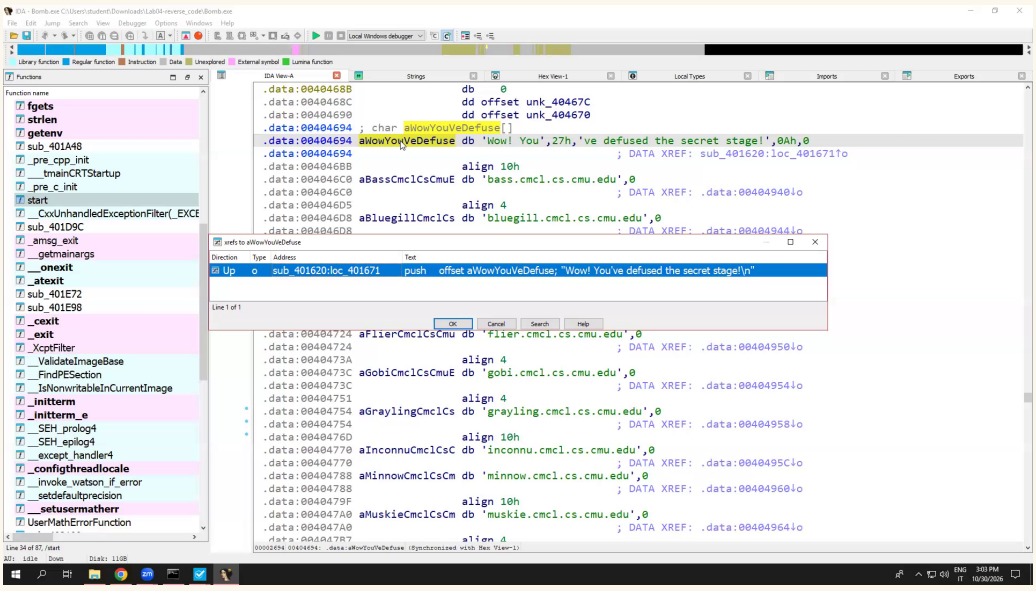


Figure 14.3: The cross-references of a string in the secret stage (lab recording).

Right-click can also show a graph of references, but the professor finds the list

better, because it shows the instruction, so you can see whether it reads or writes. Here the string about the secret stage leads to `sub_401620`.

A call with no continuation. In `sub_401620` a block ends with a call and no edge leaves it, even though other code follows at the next address. IDA recognized that the called function never returns. It is the function that makes the bomb explode, so the professor renames it `boom`, and from then on every `call boom` marks a wrong path.

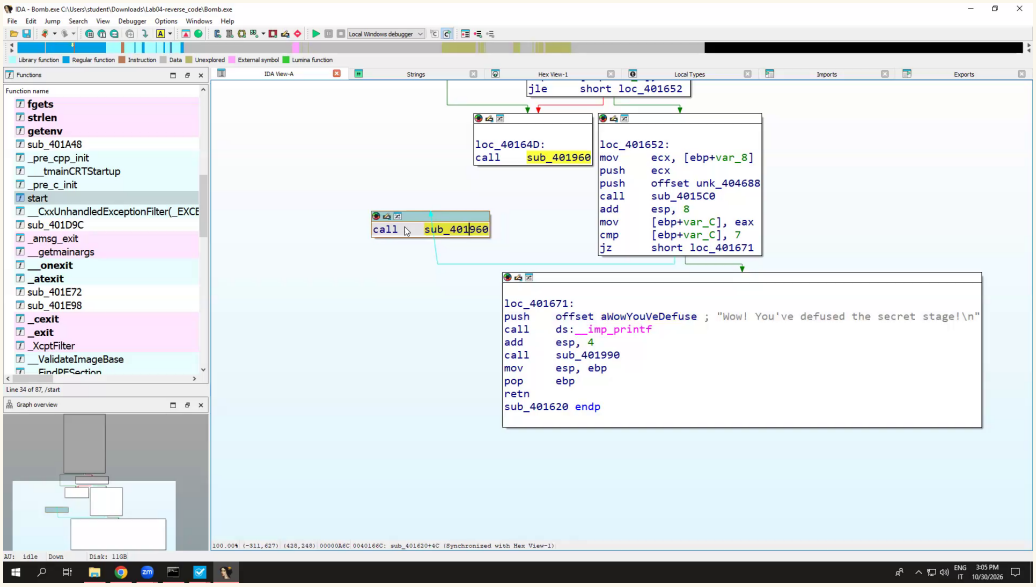


Figure 14.4: A block that ends in a call with no outgoing edge: the function never returns (lab recording).

The functions as renamed in the lab are the following.

Address	Name	Role
401190	stage1	Checks stage 1
4011B0	stage2	Checks stage 2
401620	maybe_secret_stage	Probably the secret stage, not analyzed in the recording
4016B0	not renamed	Parses six integers, used by stage2
401700	mystrlen	String length
401740	check_stage1_eqstr	Compares two strings
401870	getinput	Reads a line and returns its buffer
401990	checkForSecretStage	Common function called after each stage
404C04	gateToSecretStage	Global counter (this one is a variable)

14.1.4 Stage 1

stage1 passes the input and a fixed string to a function, then wants 0 as a result, otherwise boom. add esp, 8 after the call shows two arguments, removed by the caller as in cdecl (section 5.4).

Reading strings in the data. IDA shows the string at 404550 as text thanks to its heuristics. *Undefine* shows the raw bytes, as you would see them without help, and pressing A turns them back into an ASCII string.

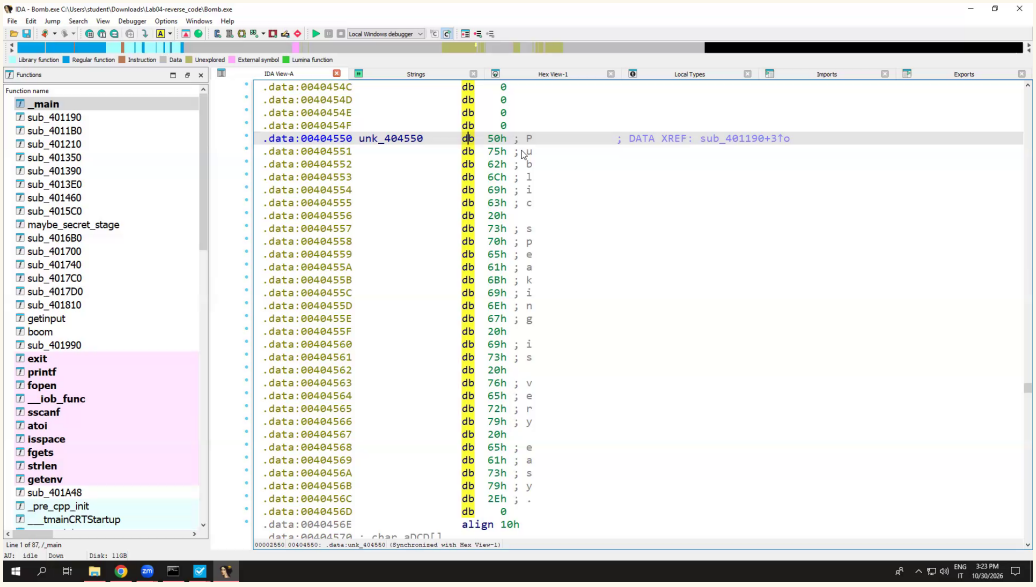


Figure 14.5: The string at `404550` after *Undefine*: one byte per line, with the characters on the right (lab recording).

A shortcut from the field. The professor recalls a friend, a malware analyst, who often just runs an API and inspects registers and memory. Here this means trying the string. With real malware, it requires a snapshot of the running VM to go back to.

Confirming from the code. `check_stage1_eqstr` calls the same function on both strings and compares the results kept in `ESI`, a register the callee must preserve. So the lengths must match, and the function is a `strlen`.

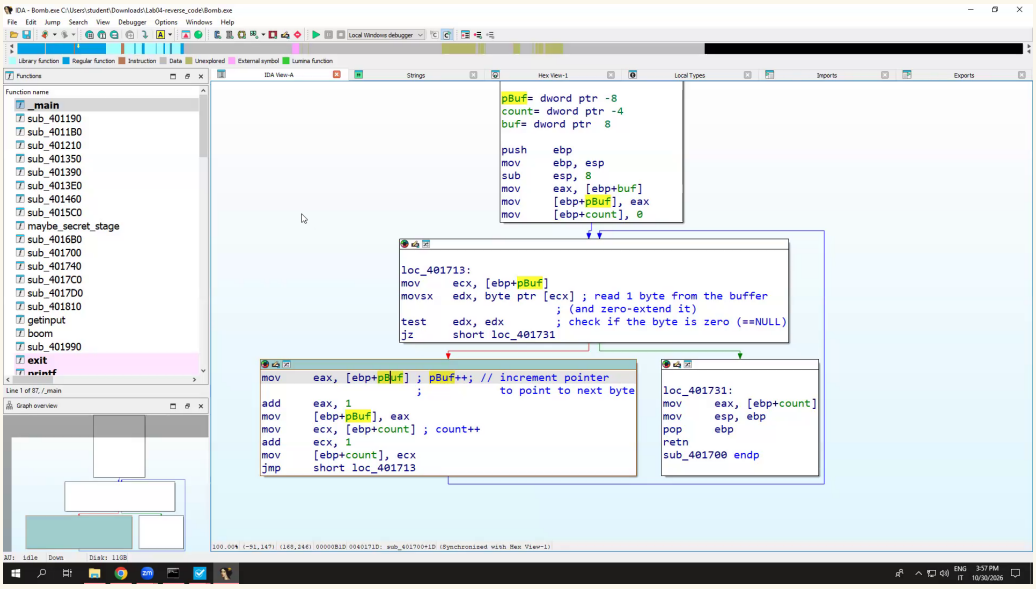


Figure 14.6: `mystrlen`: read a byte, test it for the terminator, increment pointer and counter (lab recording).

The loop reads a byte, tests it for the terminator, then increments both the pointer and the counter. The counter is the return value, since it ends in EAX.

> **Correction**
>
> The instruction on screen is `movsx`, which **sign-extends**. The professor reads the documentation of `movzx` during the lab, and the comment says "zero-extend". For testing whether a byte is zero it makes no difference. According to the professor, the extension is not strictly needed: the compiler, at this optimization level, prefers to work with 32 bits.

After the length check, a loop reads one byte from each string and compares them, returning 1 at the first difference. Returning 0 is the success case.

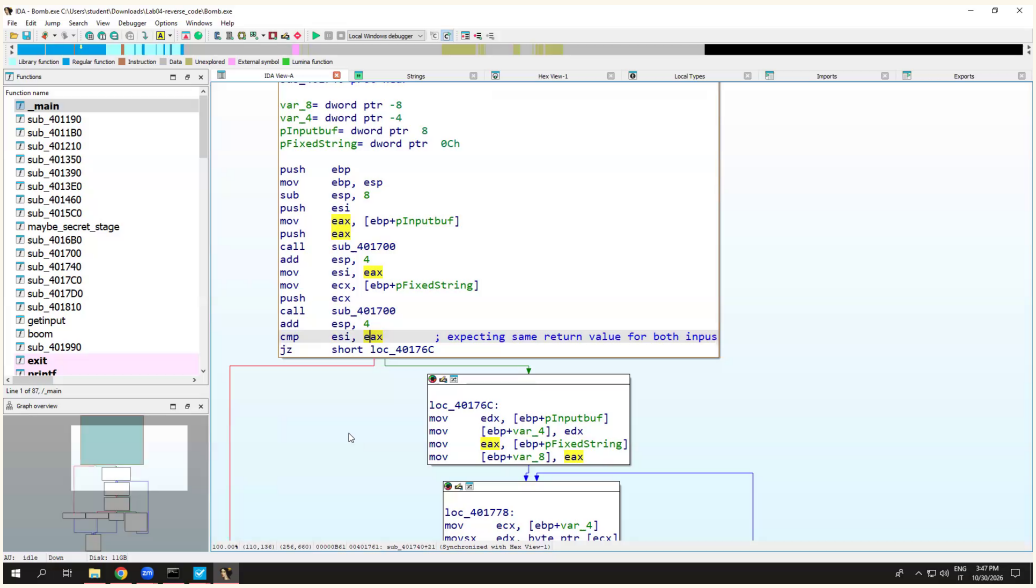


Figure 14.7: `check_stage1_eqstr`: the length check, then the comparison byte by byte (lab recording).

> **Remember**
>
> **Stage 1 answer.** `Public speaking is very easy.`, including the final period.

14.1.5 The secret stage check

`checkForSecretStage` runs after every stage, but does anything only when a global variable equals 6.

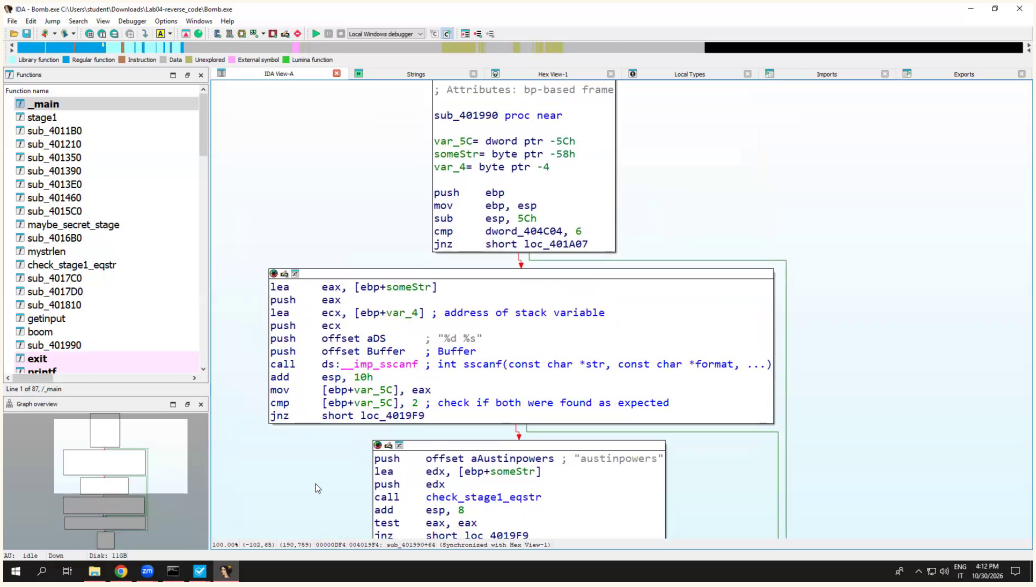


Figure 14.8: `checkForSecretStage`: the comparison with 6, then `sscanf` and the string `austinpowers` (lab recording).

- `sscanf` parses a global buffer with the format `"%d %s"`, and its return value must be 2, the number of items assigned.
- The string must equal `austinpowers`, checked with the same comparison function as stage 1.
- If it does, the program mocks you (“finding it and solving it are quite different”) and calls `maybe_secret_stage`.
- In any case it congratulates you for defusing the bomb, so you would not notice having missed the secret stage.

> **Correction**
>
> The professor reads the string as “Austin Powers” and the function as `scanf`. On screen they are `austinpowers`, in lowercase with no space, and `sscanf`.

Who updates the variable? Pressing X on it shows that all references read it, except one in `getinput`. So the professor names it `gateToSecretStage`.

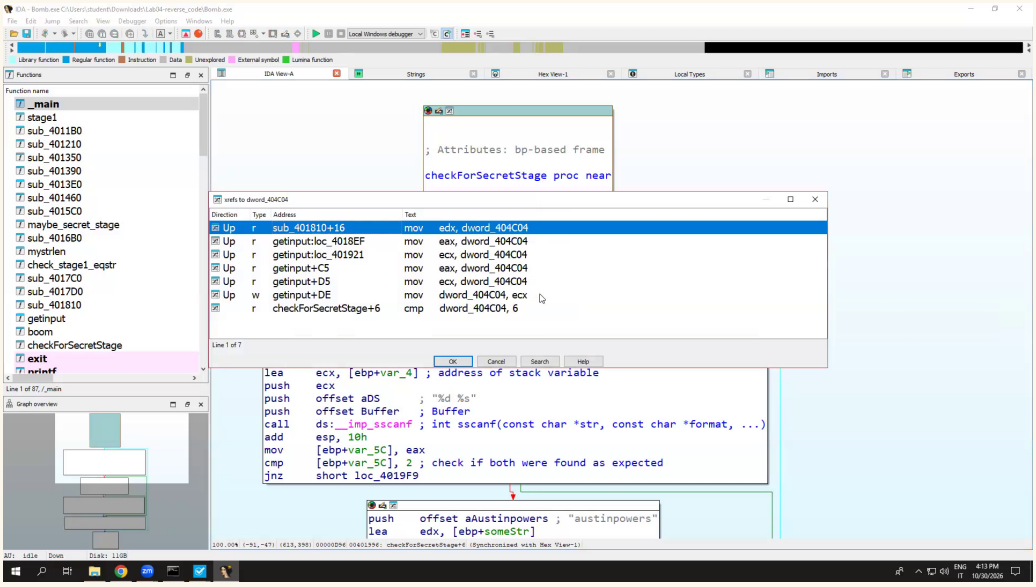


Figure 14.9: The references to the global variable: only one, in `getinput`, writes it (lab recording).

`getinput` increments it at every call, so it reaches 6 only when the input of stage 6 has been read. The professor assumes it starts at 0, though the view of `.data` in IDA does not show that clearly, and proposes verifying it with the debugger.

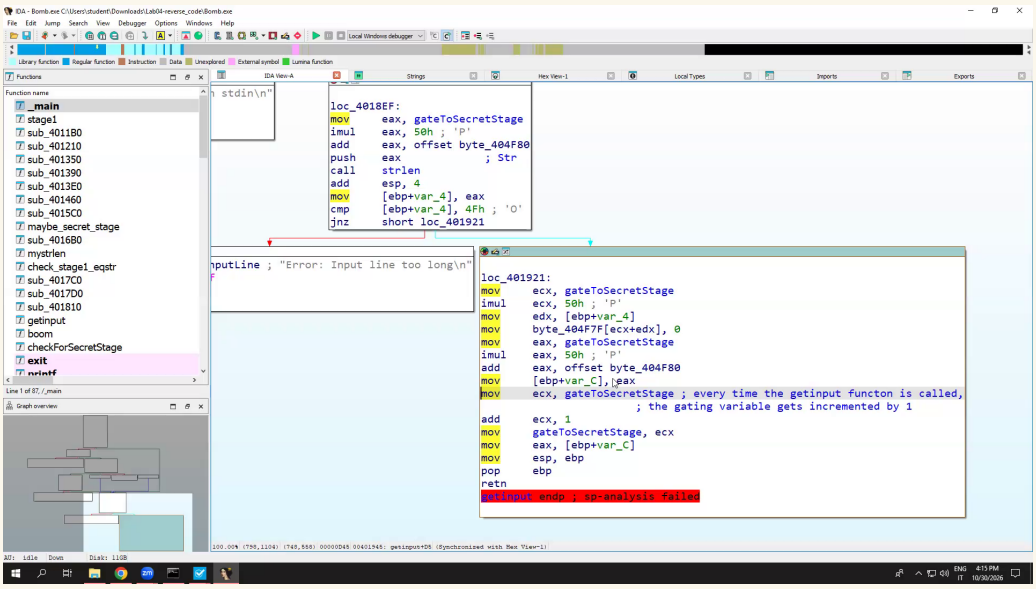


Figure 14.10: The end of `getinput`, where the gating variable is incremented (lab recording).

> **Not from the lab: the variable is also an index**
>
> In this screenshot the same variable is multiplied by `50h` and added to `byte_404F80`, and the length is compared with `4Fh`. So each line seems to occupy an 80-byte slot in a global array, indexed by this counter. This is a reading of the screenshot, not something said in the lab.

14.1.6 Stage 2

`stage2` passes the input and the address of a local variable to `sub_4016B0`, then wants that variable to be 1. `sub esp, 1Ch` reserves 28 bytes, an unusual number, so the professor suspects an array.

Parsing the numbers. `sub_4016B0` pushes that address plus 20, 16, 12, 8, 4 and 0, then the format with six `%d` and the buffer, and it calls `sscanf`. This confirms it is an array of 6 integers, which is why the variable is renamed `arrInt`. Arguments are pushed in reverse, so `arrInt[0]` is the first number typed. The return value must be at least 6, otherwise `boom`.

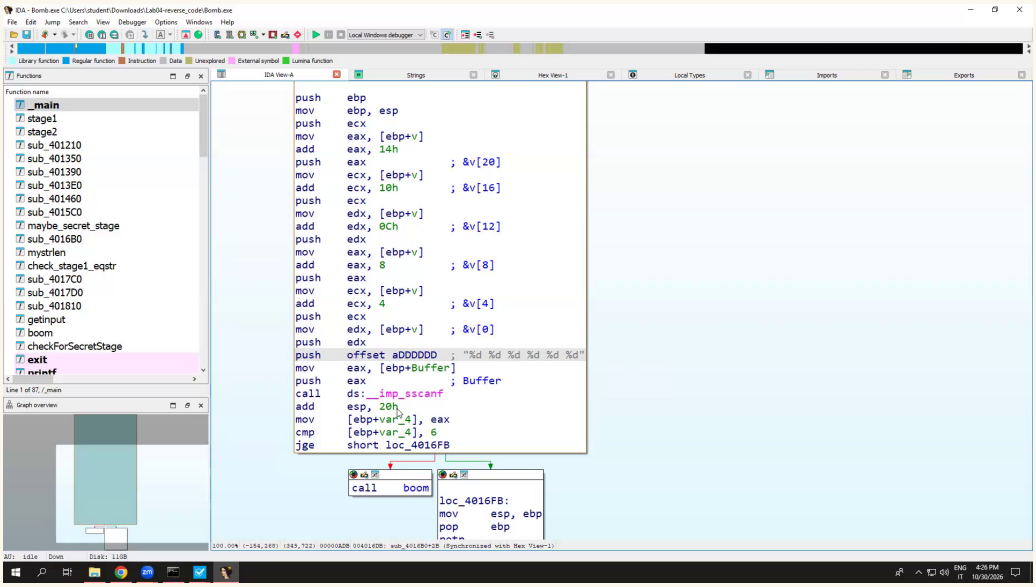


Figure 14.11: `sub_4016B0`: six addresses pushed in reverse order, then the format and the buffer (lab recording).

The loop. `count` goes from 1 to 5, since `arrInt[0]` is checked beforehand. Each cycle multiplies and then compares.

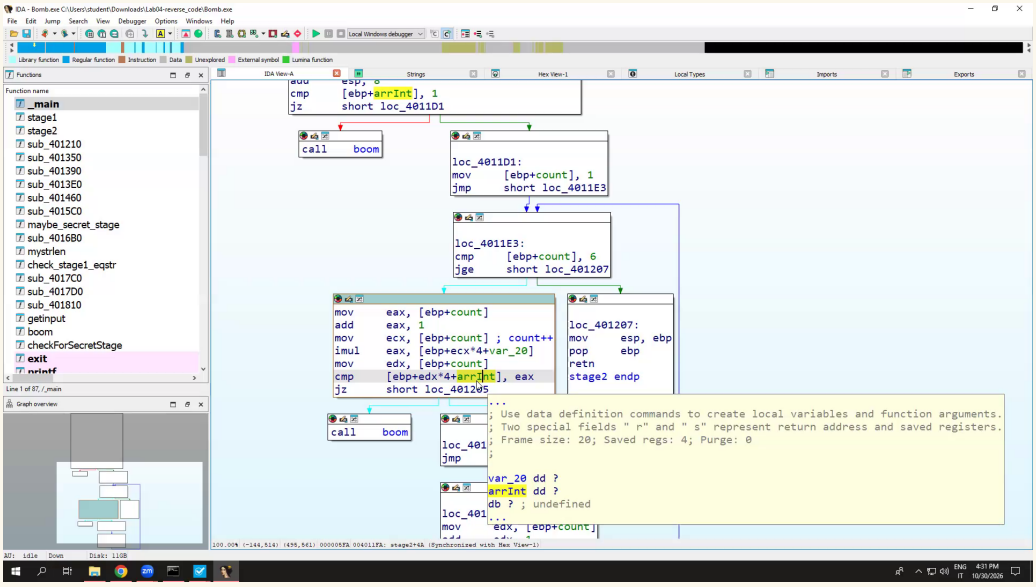


Figure 14.12: The loop of `stage2`, with the tooltip showing that `var_20` sits just before `arrInt` (lab recording).

The tooltip on `arrInt` shows why `var_20` matters: it sits 4 bytes before `arrInt`. So `[ebp+ecx*4+var_20]` is `arrInt[count-1]`, and `[ebp+edx*4+arrInt]` is `arrInt[count]`. `imul` with two operands multiplies the register by the other operand and stores the result in the register.

Decompiler. After simulating a few iterations by hand, the professor uses F5. In this case that is acceptable, because it saves a tremendous headache, and with a debugger it would be easy too.

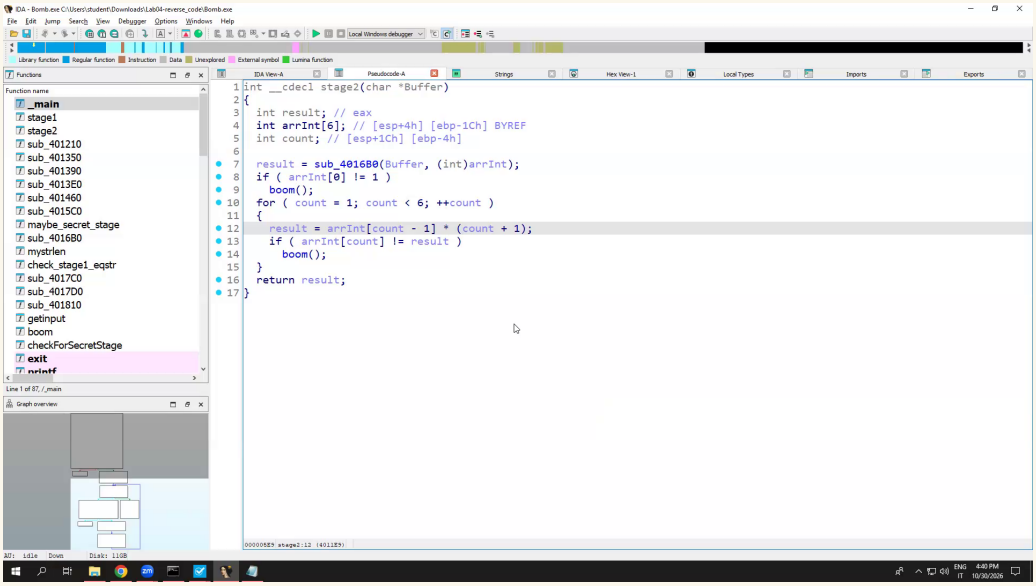


Figure 14.13: The decompiled `stage2` (lab recording).

$$arrInt[0] = 1, \qquad arrInt[count] = arrInt[count - 1] \times (count + 1)$$

This is the factorial sequence, not Fibonacci as first guessed in class.

> **Remember**
>
> **Stage 2 answer.** `1 2 6 24 120 720`

Saving your answers. The answers can go in a file passed as an argument, so you do not retype them. Do not add extra spaces: they can make a string comparison fail.

The following stages get increasingly complex and were left as an exercise. The professor believes solving a couple more without dynamic analysis is very good practice, and notes they become faster with the debugger (section 14.2.3).

14.2 Lab 05: debugging

This lab puts chapter 11 into practice with the debugger of IDA. It goes back to `constructs-stripped.exe` from section 5.3 and to stage 2 of Lab 04, then starts on `DudeLocker.exe`, a Flare-On challenge that will continue in the next lab.

14.2.1 Setting up a session

Find `main` first. A breakpoint on `main` skips the whole loading chain from `start`. In the stripped binary, follow the call from `start`, get past the setup and environment code, and take the last call before the exit: here `sub_401598`, then rename it with `N`.

Add a breakpoint. Use *Debugger* → *Breakpoints* → *Add breakpoint*, or right-click the instruction. An active breakpoint is red. Disabling it makes it green, so you can keep it without deleting it.

> **Remember**
>
> **Starting the process is the risky step.** IDA warns about the risk of unauthorized code execution, and with malware you should take it seriously. A sample may run code before your breakpoint, for example through TLS callbacks, and retaliate if it detects the debugger. So keep the network off and take a snapshot.

Some useful interface steps:

- Right-click the toolbar and enable *Debugger commands* to get the stepping buttons. In the free version you must do this at every new session.
- If you break the layout, use *Windows* → *Reset desktop*.
- G jumps in the view that has focus, so click the Hex View first, or you lose the disassembly.
- Right-click a register in the Hex View and use *Synchronize with* to follow it, then disable it, otherwise the view moves whenever the register changes.
- Hovering over an operand shows its computed address and content.
- Terminate with the stop button. Names and comments stay in the database, and when you close IDA you can keep them for next time.

14.2.2 `constructs-stripped.exe`

Watching instructions take effect. Registers changed by the last instruction turn blue. Stepping through the prologue shows it:

- `push ebp` changes ESP (from 3C to 38) and EIP, and the old EBP appears on top of the stack.
- `mov ebp, esp` changes EBP and EIP, and no memory.
- `and esp, 0FFFFFFF0h` moves ESP from 38 to 30, a multiple of 16.

`Step into` and `step over` behave the same until a `call`. The first `call` here is the compiler stub already seen in chapter 5, so step over it.

Little-endian in memory. Go to `407074` in the Hex View and step through the three `mov`. They write `1Eh`, `15h` and `3`, and you see `1E 00 00 00`: the least significant byte is at the lowest address. The professor then renames the globals `globA`, `globB` and `globC`.

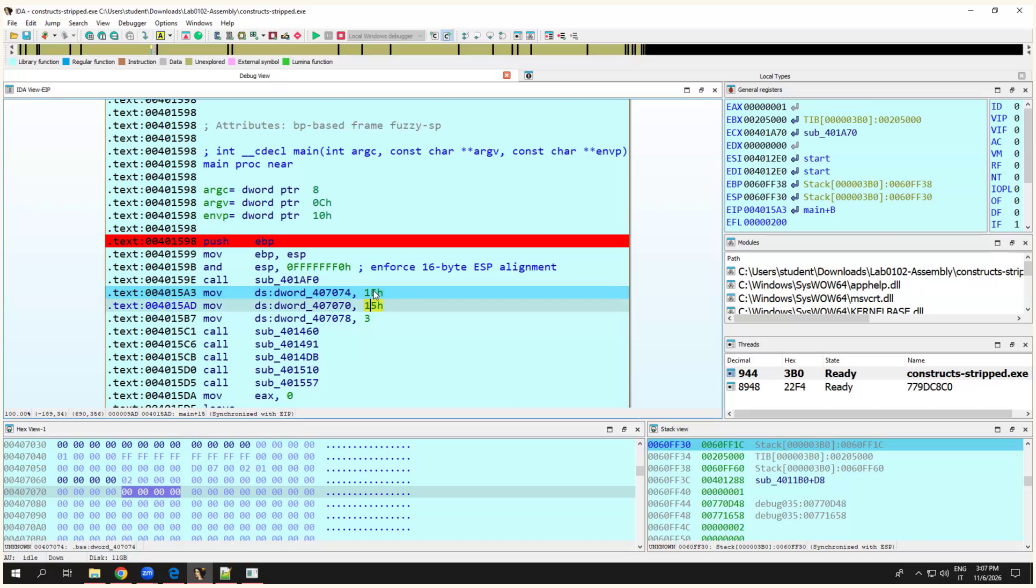


Figure 14.14: The debugger on `main`, with the writes to the globals visible in the Hex View (lab recording, 6 November 2026).

A safety net before each call. You do not know how long the next function is, and *run until return* is a heuristic that can fail if you are too deep inside. So the recommendation of the professor is to **place a breakpoint right after the call before stepping in**. If the function is boring, just continue.

Seeing a branch being decided. In `sub_401460` (`fun_if`), `cmp` changes only the flags. `ZF` turns blue and becomes 1, because `A` and `var_4` are both `1Eh`. The arrow of the branch to be taken blinks, here towards `loc_401484`, because `jge` also covers the equal case.

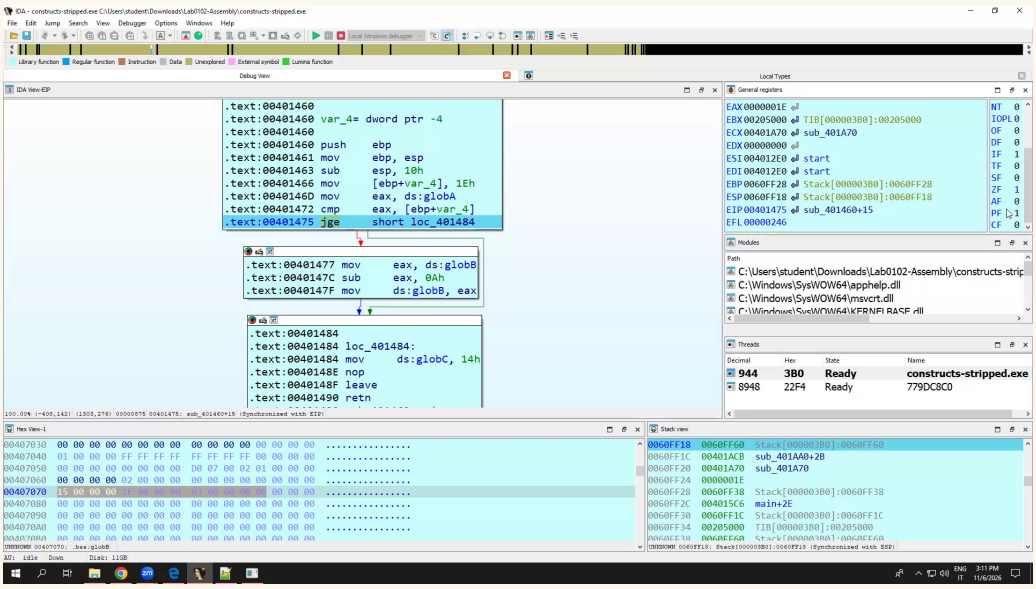


Figure 14.15: `fun_if` just before the `jge`: `ZF` is 1 and the arrow of the taken branch blinks (lab recording).

Forcing a branch. Right-click the skipped block and choose *Set IP*. The current instruction turns purple to show the flow was forced. So `sub eax, 0Ah` runs even though the program would have skipped it.

This is **forced execution**, and it is not sound in general: a block may depend on values set elsewhere. It is fine for an evasion check, or when you know that the code only changes a message.

The lab then shows the cost. In `sub_4014DB` (`fun_while`) the professor expected a loop, but it exits immediately, a consequence of the forced branch. On screen `globB` is `10h`, and `10h AND 3` is `0`.

> **Not from the lab: why 10h**
>
> Reading the values on screen: `globB` started at `15h`, the forced block subtracted `0Ah` (`0Bh`), and `fun_if_else` then added `5` (`10h`). Without forcing, the value would have been `1Ah`, and `1Ah AND 3` is `2`, so the loop would have run.

Getting out of a loop. In `sub_401510` (`fun_do_while`), `var_4` starts at `0Ah`, drops by 1 at each cycle and is compared with `0`, so the professor renames it `count` and hovers over it to watch 9, 8, 7. Rather than pressing the button again and again, place a breakpoint right after the loop, disable the one inside it, and

continue. A conditional breakpoint is another option, left for later.

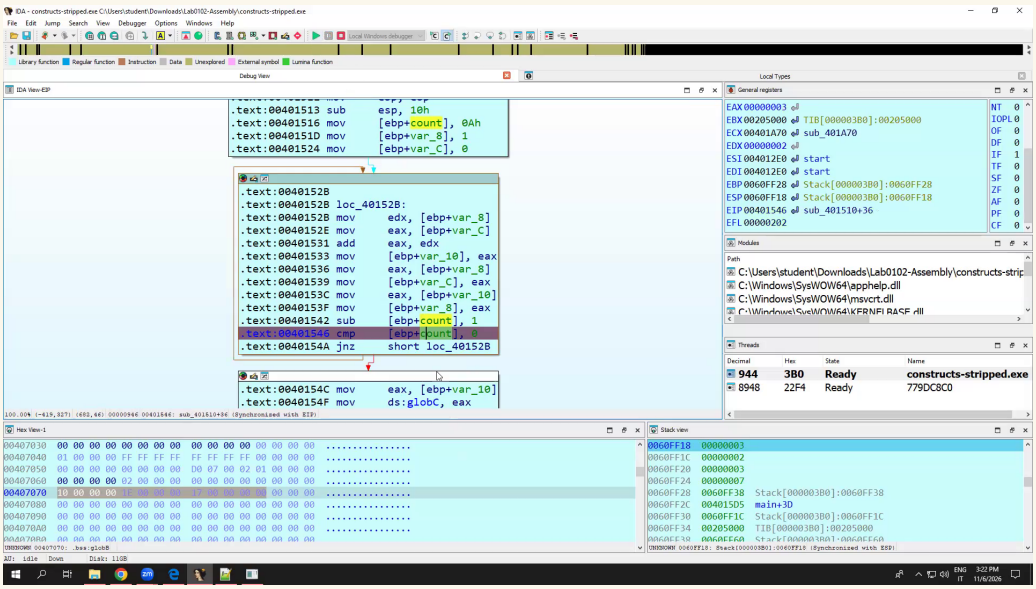


Figure 14.16: `fun_do_while`, with the counter renamed `count` (lab recording).

`ret` pops the return address: after it, `EIP` goes to `main+offset`, and the old value is still visible on the stack, simply no longer valid.

> **Correction**
>
> The professor reads the result as “59”, without saying the base. In memory it is `59 00 00 00`, so `0x59`, which is 89 in decimal.

> **Not from the lab: this is Fibonacci**
>
> The block computes `var_10 = var_C + var_8`, then `var_C = var_8` and `var_8 = var_10`, starting from 0 and 1, for 10 cycles. That gives 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, which matches `0x59`.

14.2.3 Bomb.exe, stage 2

Open the executable and load the saved database to keep your names and comments.

Where the I/O of the program goes. A console window opens in the background. If a step seems to hang and no line is highlighted, the thread may be blocked waiting for something from the system, here your input. Go to the console and

type it. The professor had stepped over `getinput`, so the debugger regained control by itself as soon as the input arrived.

Stepping into a library call by mistake. It happens easily with `printf`. Use *Run until return*, or go back and set a breakpoint after the call.

Skipping the math. Stage 1 is the known string. For stage 2 the professor types `1 2 3 4 5 6`, which are distinct and easy to spot in memory. Placing a breakpoint on the `boom` call and in the loop shows each check.

At the third value, hovering shows the operand: `arrInt[2]` holds 3, while EAX, computed by the program, holds 6. So the debugger gives away the expected number, with no algorithm to reconstruct.

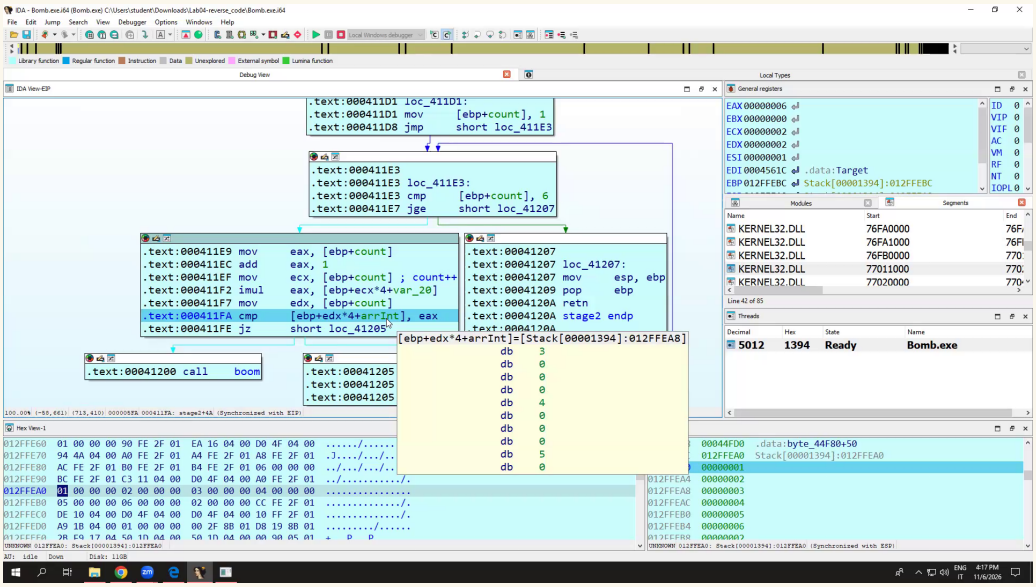


Figure 14.17: Hovering over the operand of `cmp`: `arrInt[2]` holds 3, while EAX holds the expected 6 (lab recording).

The addresses here start with `000411`, not `004011` as in Lab 04.

> **Not from the lab: this is ASLR**
>
> `4011FA` in Lab 04 and `411FA` here have the same offset `11FA` from the base of the module (section 11.7).

Patching memory to continue. Select the 4 bytes in the Hex View, choose *Edit*, type the new bytes, then *Apply changes*. The colour returns to normal when the change is applied. The comparison covers the whole dword, so all 4 bytes matter.

What happens if you force the jump instead. At the next value, 18h, the professor does not fix memory and uses *Set IP* past the check. The following round then expects a wrong value (14h), because the formula reads the **previous** array element, which was never corrected. The recovery works only because this block writes nothing to memory:

1. Patch the correct 18h into the array.
2. Set IP back to the start of the block.
3. Re-execute, since `count` was not modified, so the registers are rebuilt.

The expected value becomes 78h, as it should. This is a concrete example of why forcing a branch can be unsound.

Input position	Register (hex)	Decimal
2	2	2
3	6	6
4	18	24
5	78	120
6	2D0	720

Mind the byte order. A register reads as humans write numbers, but memory is little-endian. To store 2D0, slice the 8 hex digits two at a time and reverse them: `D0 02 00 00`. The screenshot shows the trap: `D0 20 00 00`, not yet applied, would be 20D0.

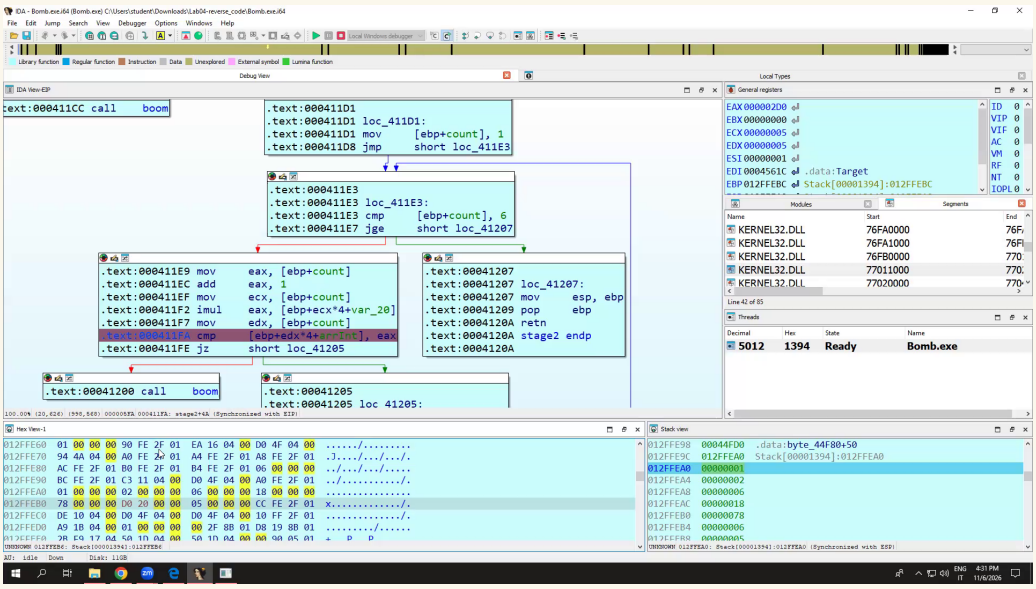


Figure 14.18: The Hex View during the patch of the array, with the wrong byte order not yet applied (lab recording).

Once you have the values, they are the answer for a clean run (section 14.1.6).

14.2.4 DudeLocker.exe

> **Remember**
>
> **A ransomware-like sample.** The challenge from Flare-On encrypts specific files, and `BusinessPapers` is a document it has already encrypted. The goal is to analyze it in order to recover that document, and in principle to write a decryptor. According to the professor it will not wipe your machine and works only under certain conditions, but treat it like malware. The password is `infected`, and many write-ups exist online. Only the start was covered here, and the rest continues next week.

Different compilation. There are very few subroutines, and it was built with a custom toolchain, to avoid the usual bloat.

The series of `mov`. The first question is what `sub_4019A0` is doing with five `mov` from `403028`–`403038` to the stack, 4 bytes at a time. Statically the constants look meaningless.

Stepping through them answers at once. Note that `sub esp, 1096` reserves

about a kilobyte, so the destination is far from the top of the stack: the computed address `19FF40` is what you need for the Hex View.

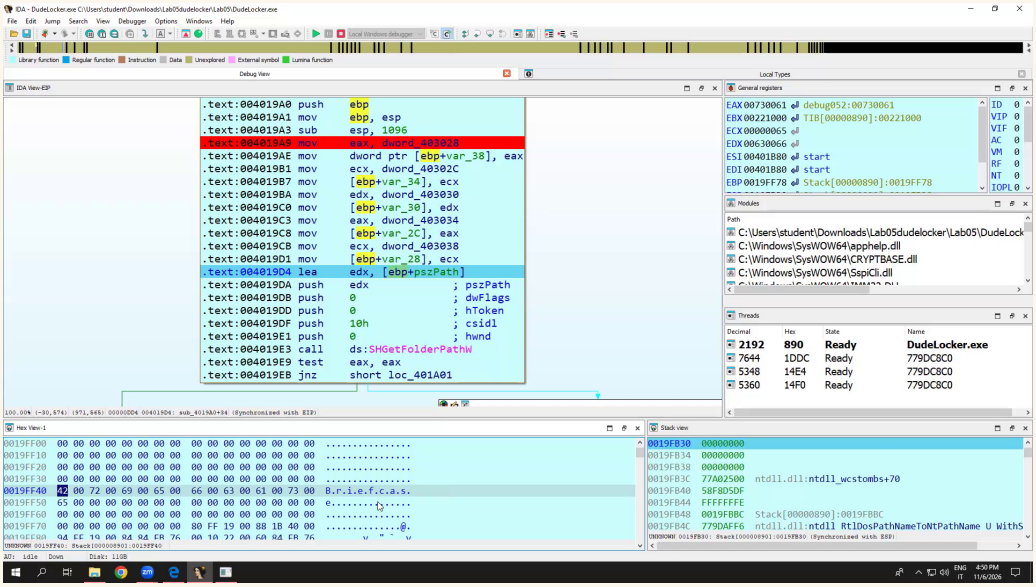


Figure 14.19: The series of `mov` in `sub_4019A0`, with the destination in the Hex View (lab recording).

The result is `Briefcase` in wide characters. The last dword also holds the two-byte terminator, so the string takes exactly 20 bytes. IDA did not recognize it as a string because of the zeros between characters.

Seeing it statically too. Go to `403028`, select the area, and choose *Undefine*: IDA now shows individual bytes rather than dwords. Select them all, including the final zeros, and the right-click menu suggests `"Briefcase"`, as the shortcut A.

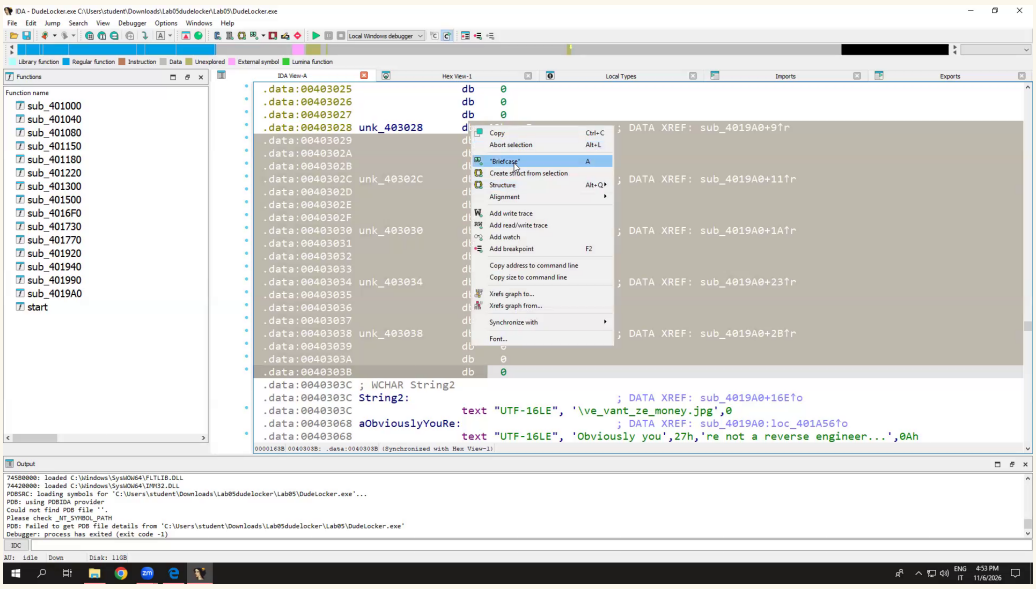


Figure 14.20: The data at `403028` after *Undefine*, with the suggested string in the menu (lab recording).

The strings nearby are worth reading too, such as `\ve_vant_ze_money.jpg` and `Obviously you're not a reverse engineer...`. You want to avoid the path leading to the latter.

The folder path. `SHGetFolderPathW` looks up folders whose location varies between machines. The `csidl` parameter here is `10h`, and `lea` passes the address of a local buffer. To check the result, execute the `lea` that puts the address of the buffer in EAX, then synchronize the Hex View with EAX.

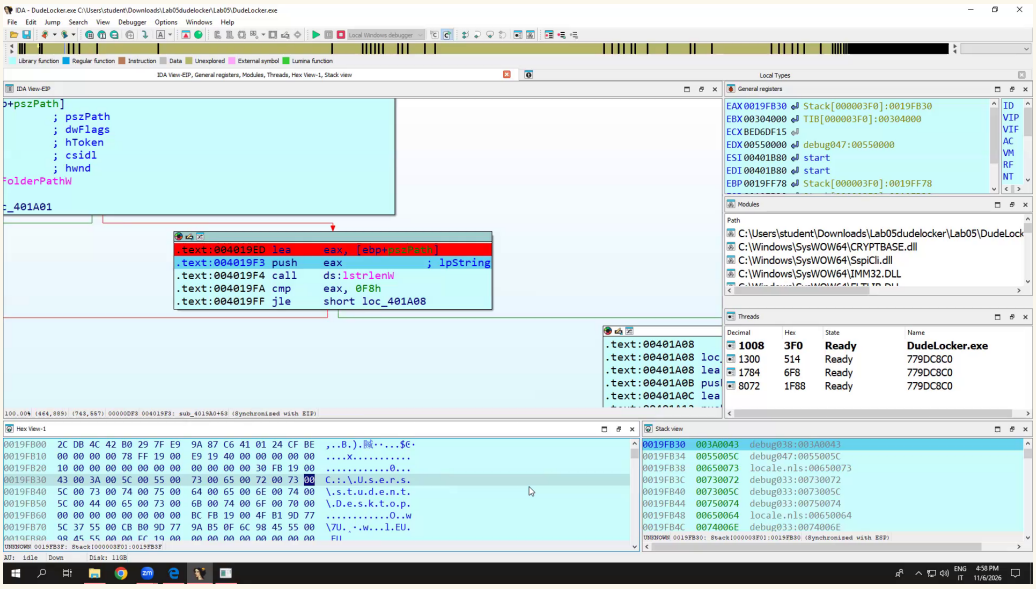
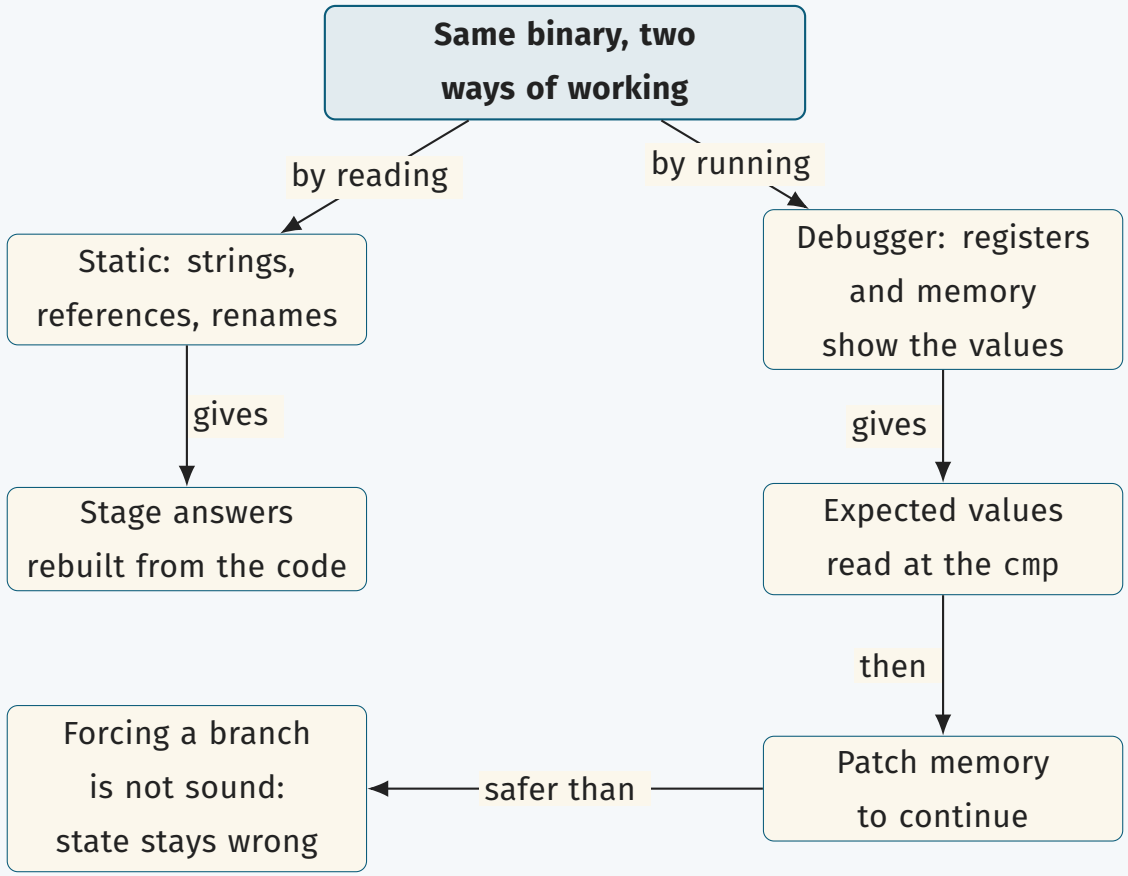


Figure 14.21: The buffer filled by `SHGetFolderPathW`, with the Hex View synchronized on EAX (lab recording).

The result is `C:\Users\student\Desktop`, again in wide characters. The rest of the function does string copies and concatenations, a `CreateFile`, and a call to study, left for next time. The failure paths on the left lead to the exit, so all these steps must succeed. Fixing the expectations of the sample, including by patching code, is the topic of the next lab.

Concept map: how the ideas connect



Lab 04 reaches the answers by reading: strings and references give the structure, and renaming makes the code readable. Lab 05 reaches the same values by running the code and reading them at the comparison. Keeping the program going then requires a choice. Patching memory keeps the state coherent, while forcing a branch leaves it wrong for what follows. The same debugging habits start to reveal `DudeLocker.exe`, whose strings do not appear until they are built.

In short

- Search the hash on VirusTotal, and do not upload a sample from a real

case.

- Strings and their references can give the structure of a program before you read its logic.
- A call with no outgoing edge is a function that never returns: renaming it marks every wrong path.
- Addresses pushed in sequence, plus a format string, reveal an array of arguments to `sscanf`.
- The debugger gives away the expected value at the comparison, with no algorithm to reconstruct.
- Place a breakpoint after a call before stepping in, and after a loop to leave it.
- Forced execution leaves the state wrong. Patching memory respects it, but mind the little-endian byte order.
- Constants moved to the stack can be a wide-character string that IDA does not recognize.

Review: try to answer without looking

1. Why search the hash of a sample instead of uploading the file to VirusTotal?
2. How does the control flow graph show that a function never returns?
3. In `stage2`, what does `[ebp+ecx*4+var_20]` address, and why?
4. How do you store the value `2D0` in a 4-byte array element?
5. Why did the expected value become `14h` after a forced jump?
6. Why did IDA not recognize `Briefcase` as a string?

Answers, with the section where to find them

1. The upload tells the attacker that the file reached you, and

subscribers can download it (section 14.1.2).

2. The block ends with the call and no edge leaves it (section 14.1.3).
3. `arrInt[count-1]`, because `var_20` sits 4 bytes before `arrInt` (section 14.1.6).
4. `D0 02 00 00`, because memory is little-endian (section 14.2.3).
5. The formula reads the previous element, which had not been corrected in memory (section 14.2.3).
6. It is in wide characters, with zeros between the letters, and it is moved in dwords (section 14.2.4).

Review sheets

These sheets collect what is hardest to keep in mind. They do not replace the chapters.

Which tool answers which question

Tool	Question it answers	Chapter
PEview, pestudio	How is this PE file structured?	6
Strings	Which readable text does the file contain?	6
PEiD, Detect It Easy	Is a known packer recognised?	6, 13
Dependency Walker	Which modules and functions does it import?	6
Process Monitor	What did the process do to files, registry and processes?	8
Process Explorer	Which processes run, and what is loaded in them?	8
Regshot	What changed in the registry between two moments?	8
Wireshark	What traffic passed on the network?	8
INetSim, ApateDNS	What does the sample do if the network seems to answer?	8
IDA disassembly	What does this code do, read without running it?	5
IDA debugger	What is the state at this exact instruction?	11

Calling conventions

Convention	Arguments	Stack cleanup	Chapter
`cdecl`	On the stack	Caller, with `add esp, N`	4
`stdcall`	On the stack	Callee, with `ret N`	4
`fastcall`	First two in ECX and EDX, then on the stack	Callee	4
`thiscall`	`this` in ECX, the others on the stack	Callee	4

Breakpoints

Kind	How it works	Chapter
Software	Overwrites an instruction byte with `0xCC` (`INT 3`), so it changes memory	11
Hardware	Uses the debug registers of the CPU, without changing the code	11
Conditional	A breakpoint that stops only if an expression holds	11

Words not to confuse

Pair	The difference	Chapter
`lea` and `mov` with brackets	`lea` computes the address, `mov` reads the memory at that address	4
Raw offset and RVA	Position in the file versus position relative to the load address	6
Virtual size and raw size	Size in memory versus size on disk	6
Process and thread	The container versus the unit that the scheduler runs	9
Packer and payload	The representation versus the logic it protects	6
Dropped file and unpacked copy	Made by the sample versus made by the analyst	13
Import and call	A capability the file declares versus a call that really happened	13
Attempt and effect	What the sample tried versus what actually changed	13
Running long and persistence	Still alive now versus able to run again after a login or restart	13
Forcing a branch and patching memory	Moving `EIP` past a check versus correcting the data the code reads later	14

How far the evidence goes

Evidence	What it supports	Chapter
Imports	Possible capabilities	13
Strings	Possible configuration	13
Execution	Behaviour under the tested conditions	13
Successful state changes	Artifacts	13
Trigger testing	Working persistence	13

Glossary

Each entry gives a short definition and points to the main explanation.

Anti-debugging Techniques with which a program detects or hinders a debugger (chapter 12).

Anti-disassembly Code arranged so that a disassembler shows the wrong instructions (chapter 4).

API (Application Programming Interface) The set of functions a system or library offers to programs (chapter 9).

ASLR (Address Space Layout Randomization) Loading modules at varying addresses between runs or restarts (chapter 11).

Behavioural analysis Studying a sample by observing what it does when it runs (chapter 7).

Breakpoint A point at which a debugger stops the execution (chapter 11).

Calling convention The rules on how arguments, return value and stack cleanup are handled in a call (chapter 4).

COM (Component Object Model) A Windows interface standard through which software components call each other (chapter 9).

Debugger A tool that controls the execution of a program and shows its state (chapter 11).

Disassembly Translating machine code into assembly (chapter 3).

DLL (Dynamic Link Library) A PE module that other programs load to reuse its code (chapter 9).

Dynamic analysis Examining a sample while it runs (chapter 2).

Endianness The order in which the bytes of a value are stored in memory (chapter 3).

Entropy A measure of how evenly byte values are distributed; high values fit compressed or encrypted data (chapter 13).

Evasion Behaviour with which malware avoids detection or analysis (chapter 12).

Export A function that a module makes available to others (chapter 6).

Forced execution Moving the instruction pointer into a block the program would have skipped; not sound in general (chapter 14).

Handle An opaque reference to an object opened through the operating system (chapter 9).

Hash A fixed-size digest that identifies the bytes of a file (chapter 6).

Hive A top-level section of the Windows Registry (chapter 8).

Import A function that an executable obtains from another module (chapter 6).

Indicator, host-based or network-based An observable artifact linked to a sample, on the endpoint or in its traffic (chapters 2 and 13).

ISA (Instruction Set Architecture) The instructions, registers and rules that a processor family exposes (chapter 3).

Kill chain A model of the stages of an attack (chapter 1).

Mutex A kernel object for mutual exclusion, also used by malware to run only once (chapter 9).

Native API The lower-level Windows interface exposed through `ntdll.dll` (chapter 9).

Ordinal A number that identifies an exported function in place of its name (chapter 13).

Packing Transforming a program so that its real code is restored only when it runs (chapter 6).

PE (Portable Executable) The file format of Windows executables and DLLs (chapter 6).

Persistence A mechanism that lets malware run again after a login or a restart (chapter 8).

Process The container of an address space, handles and threads (chapter 9).

Registry The hierarchical configuration database of Windows (chapter 8).

RVA (Relative Virtual Address) An address expressed relative to the load address of the image (chapter 6).

Sandbox An automated environment that runs a sample and reports its behaviour (chapter 7).

SEH (Structured Exception Handling) The Windows mechanism for dispatching exceptions to handlers (chapter 9).

Semantic gap The distance between the raw state seen from outside a VM and its meaning for the guest (chapter 12).

Service A program managed by the Windows service manager, which can start without a user (chapter 9).

Signature A pattern used to recognise malware on a host or on the network (chapter 2).

Snapshot A saved state of a VM, to return to a clean machine (chapter 7).

Stack frame The stack area of a single function call (chapter 4).

Static analysis Examining a sample without running it (chapter 2).

Thread The unit of execution scheduled by the operating system (chapter 3).

VMI (Virtual Machine Introspection) Observing a guest from outside, at the hypervisor level (chapter 12).

Win32 The main documented Windows API for applications (chapter 9).

Analytic index

addressing mode, 52
AddressOfEntryPoint, 132
Al-Khaser, 273
Alternate Data Streams (ADS), 195
anti-debugging, 296
anti-disassembly, 91
ApateDNS, 177
API tracing, 149
array in assembly, 228
ASLR, 247
ASLR (Address Space Layout Randomization), 52

beaconing, 31
blue pill, 275
breakpoint, 239

call $+5 pattern, 91
call and ret, 69
caller-save and callee-save registers, 77
calling convention, 74
CARO naming scheme, 154
cdecl, 75
CFF Explorer, 249
cld and std, 89
CLSID and IID, 215
COFF file header, 130
COM hijacking, 216
command and control (C2), 31

Component Object Model (COM), 215
conditional breakpoint, 243
cpuid, 268
CreateProcess, 207

D'Elia, Daniele Cono, 6
data directory, 133
debug registers, 241
debugger, 236
decompiler, 37, 104
DEP (Data Execution Prevention), 52
Dependency Walker, 139
dereferencing, 44
DF (Direction Flag), 89
diffing, 149
DllMain, 206
DRAKVUF, 294
dual use, 8
DWARF, 99
dynamic analysis, 24, 147
dynamic linking, 139

e_lfanew, 129
EBP, 45
eBPF, 290
EFLAGS, 44
EIP, 43
endianness, 40
enter, 82
entropy, 344
Equifax breach, 10
ESP, 45
EternalBlue, 11
Ether, 291
Event Tracing for Windows, 290
Ex, A and W suffixes, 198

exception
    and debuggers, 244
exfiltration, 31
exploit, 11
exports, 143

fastcall, 78
Ferrie, Peter, 296
fetch-decode-execute cycle, 47
file mapping, 194
fingerprinting, 31
first-chance exception, 245
flags (CF, OF, SF, ZF), 64
forced execution, 373
function (assembly), 66
Furtim, 283

Garfinkel, Tal, 291
global variable, 220

handle, 194
hardware breakpoint, 241
hash, 122
Heaven's Gate, 70
Hungarian notation, 193
hypervisor bit, 268

IA-32, 38
IDA, 98
    debugger, 250
IMAGE_SECTION_HEADER, 134
ImageBase, 132
import hash (imphash), 123
imports, 138
in-guest agent, 290
indicator of compromise (IoC), 8
INetSim, 179

instruction set architecture (ISA), 38
instrumentation, 151
INT 3, 240

jcc (conditional jump), 65
jump table, 227

kill chain, 13

LEA (Load Effective Address), 83
leave, 82
local variable, 220
LockBit, 12
Lockheed Martin, 13
logic bomb, 358
loop instruction, 90

memory breakpoint, 243
mutex, 210

Native API, 191
ncat, 178
nop, 62
NOP sled, 63
NtGlobalFlag, 297

OLE, 216
optional header, 130
overlay, 136

packer, 126
padding, 62, 231
PatchGuard, 190, 290
patching, 258
PDB, 99
PE (Portable Executable), 127
PEB, 287
PEB (Process Environment Block), 49

PEiD, 127
persistence, 28, 173
    in the lab, 350
Practical Malware Analysis (PMA), 7
Process Explorer, 166
Process Monitor (procmon), 164
process replacement, 168
prologue and epilogue, 81
protection rings, 189

Querzoni, Leonardo, 6

ransomware as a service (RaaS), 12
rdtsc, 271
ReactOS, 192
rebasing, 247
red pill, 267
register, 43
registry hive, 171
registry key and value, 170
Regshot, 171
REP prefix and string instructions, 85
Resource Hacker, 138
ret, 69
retf, 70
retn, 70
rootkit, 190
Rosenblum, Mendel, 291
rundll32, 163
runtime linking, 139
RVA (relative virtual address), 133

sandbox, 154
second-chance exception, 245
self-modifying code, 92
semantic gap, 293

Service Control Manager (SCM), 212
ServiceDll, 214
Shadow Brokers, 10
signature, 22
snapshot, 159
software breakpoint, 240
stack frame, 67
static analysis, 24
static linking, 139
stdcall, 75
stepping, 237
string, 124
struct in assembly, 229
Structured Exception Handling (SEH), 209
svchost.exe, 175
switch, 222
system monitoring, 149

tail call, 71
TEB (Thread Environment Block), 49
TeslaCrypt, 276
thiscall, 78
thread, 50
TimeDateStamp, 138
timing attack, 284
trap flag, 244
triple extortion, 12

Vectored Exception Handling, 210
virtual machine (VM), 7
virtual machine introspection, 290
VirusTotal, 121
VM exit, 271

WannaCry, 11
wear-and-tear artifacts, 284

Windows Registry, 170
Windows services, 212
WinINet, 204
Winsock, 203
Wireshark, 176
WMI, 288

x86-64, 38

Yahoo
    breach, 10

zero-day vulnerability, 11